

Reinforcement Learning Algorithms Set 2

Table of Contents

Towards Understanding Asynchronous Advantage Actor-critic: Convergence and Linear Speedup	2
The Implementation of Asynchronous Advantage Actor-Critic with Stigmergy in Network-assisted Multi-agent System	45
Dropout Q-Functions for Doubly Efficient Reinforcement Learning	51
Randomized Ensembled Double Q-Learning: Learning Fast Without a Model	81
TD-MPC2: Scalable, Robust World Models for Continuous Control	103
MENTOR: Mixture-of-Experts Network with Task-Oriented Perturbation for Visual Reinforcement Learning	129
END	145

Towards Understanding Asynchronous Advantage Actor-critic: Convergence and Linear Speedup

Han Shen

Rensselaer Polytechnic Institute

SHENH5@RPI.EDU

Kaiqing Zhang

Massachusetts Institute of Technology

KAIQING@MIT.EDU

Mingyi Hong

University of Minnesota, Minneapolis

MHONG@UMN.EDU

Tianyi Chen

Rensselaer Polytechnic Institute

CHENT18@RPI.EDU

Abstract

Asynchronous and parallel implementation of standard reinforcement learning (RL) algorithms is a key enabler of the tremendous success of modern RL. Among many asynchronous RL algorithms, arguably the most popular and effective one is the asynchronous advantage actor-critic (A3C) algorithm. Although A3C is becoming the workhorse of RL, its theoretical properties are still not well-understood, including its non-asymptotic analysis and the performance gain of parallelism (a.k.a. linear speedup). This paper revisits the A3C algorithm and establishes its non-asymptotic convergence guarantees. Under both i.i.d. and Markovian sampling, we establish the local convergence guarantee for A3C in the general policy approximation case and the global convergence guarantee in softmax policy parameterization. Under i.i.d. sampling, A3C obtains sample complexity of $\mathcal{O}(\epsilon^{-2.5}/N)$ per worker to achieve ϵ accuracy, where N is the number of workers. Compared to the best-known sample complexity of $\mathcal{O}(\epsilon^{-2.5})$ for two-timescale AC, A3C achieves *linear speedup*, which justifies the advantage of parallelism and asynchrony in AC algorithms theoretically for the first time. Numerical tests on synthetic environment, OpenAI Gym environments and Atari games have been provided to verify our theoretical analysis.

1. Introduction

Reinforcement learning (RL) has achieved impressive performance in many domains such as robotics (Lillicrap et al., 2016; Mnih et al., 2015) and video games (Mnih et al., 2016). However, these empirical successes are often at the expense of significant computation. To unlock high computation capabilities, the state-of-the-art RL approaches rely on sampling data from massive parallel simulators on multiple machines (Nair et al., 2015; Mnih et al., 2016; Assran et al., 2019; Chen et al., 2018). Empirically, these approaches can significantly *reduce training time* when implemented in an *asynchronous* manner. One popular method that achieves the state-of-art performance is the asynchronous variant of the actor-critic (AC) algorithm, referred to as A3C (Mnih et al., 2016).

A3C builds on the original AC algorithm (Konda, 2002). At a high level, AC simultaneously performs policy optimization (a.k.a. the actor step) using the policy gradient (PG) method (Sutton et al., 2000) and policy evaluation (a.k.a. the critic step) using the temporal difference learning (TD) algorithm (Sutton, 1988). To ensure scalability to large state-action spaces, both actor and critic steps can combine with various function approximation techniques. To ensure stability, AC is often implemented in a two time-scale fashion, where the actor step runs in the slow timescale and the

critic step runs in the fast timescale. Similar to other on-policy RL algorithms, AC uses samples generated from the target policy. Thus, data sampling is entangled with the learning procedure, which generates significant *overhead*. To speed up the sampling process of AC, A3C introduces multiple workers with a shared policy, and each worker has its own simulator to perform data sampling. The shared policy can be then updated using samples collected from multiple workers.

Despite the empirical success achieved by A3C, to the best of our knowledge, its theoretical property is not well-understood. The following *theoretical* questions remain unclear: **Q1**) Under what assumption does A3C converge? If so, does it converge to the global optimal solution? **Q2**) What is its convergence rate? **Q3**) Can A3C obtain benefit (or linear speedup) using parallelism and asynchrony?

For **Q3**), we are interested in the *training time linear speedup* with N workers, which is the ratio between the training time using a single worker and that using N workers. Since asynchronous parallelism mitigates the effect of stragglers and keeps workers busy, the training time speedup can be measured roughly by the sample complexity (i.e., computational) linear speedup (Lian et al., 2016):

$$\text{Speedup}(N) = \frac{\text{sample complexity with one worker}}{\text{average sample complexity per worker with } N \text{ workers}}. \quad (1)$$

If $\text{Speedup}(N) = \Theta(N)$, the speedup is linear, and the training time roughly reduces linearly as the number of workers increases. This paper aims to answer this question, towards the goal of providing theoretical justification for the empirical successes of parallel and asynchronous RL.

1.1. Related works

The PG method and its global convergence. The global optimality of the stationary points of policy optimization problems has been shown in (Bhandari and Russo, 2019). Then the finite-time convergence rate for exact PG method with softmax policy was established in (Agarwal et al., 2020) by utilizing a gradient-dominance type result under relative entropy regularized objective function. Later, (Mei et al., 2020) extended this result to the entropy regularized setting and established linear convergence rate for exact PG method under softmax parameterization. Later, (Bhandari and Russo, 2021) has proved linear convergence rate for general class PG methods. In the stochastic setting, (Zhang et al., 2019) has established local optimal convergence for stochastic PG with unbiased rollout and increasing step sizes, and (Wang et al., 2019) has established the global convergence of stochastic neural PG with increasing batch of i.i.d. samples. Later, (Zhang et al., 2021) proved that the minibatch version of PG achieves global convergence with the help of relative entropy regularization. But none of them consider the global convergence of the AC method. On the application side, the PG method has been broadly applied in various settings; see e.g. (Chai and Lau, 2019, 2020; El-Laham and Bugallo, 2021; Cervino et al., 2021). In (Chai and Lau, 2019), the actor critic method was used to jointly optimize the trajectory, transmission and caching content delivery of the unmanned aerial vehicles. In (Chai and Lau, 2020), the policy gradient method was used to jointly optimize the streaming rate and transmission power. In (El-Laham and Bugallo, 2021), the PG method was used to help the learning of a distribution adaptation strategy. In (Cervino et al., 2021), the PG method is used in a multitask learning algorithm which seeks to improve generalization to new tasks.

Analysis of AC algorithm. AC method was first proposed by (Borkar and Konda, 1997; Konda, 2002), with asymptotic convergence guarantees provided in (Borkar and Konda, 1997; Konda, 2002; Bhatnagar et al., 2009). It was not until recently that the *non-asymptotic* analyses of AC have

been established. The finite-sample guarantee for the batch AC algorithm has been established in (Yang et al., 2018; Kumar et al., 2019; Fu et al., 2020) with i.i.d. sampling. Later, in (Qiu et al., 2019), the finite-sample analysis was established for the double-loop nested AC algorithm under the Markovian setting. An improved analysis for the Markovian setting with minibatch updates has been presented in (Xu et al., 2020a) for the nested AC method. More recently, (Xu et al., 2020b; Wu et al., 2020) have provided the first finite-time analyses for the two-timescale AC algorithms under Markov sampling, with both $\tilde{O}(\epsilon^{-2.5})$ sample complexity, which is the best-known sample complexity for two-timescale AC. Through the lens of bi-level optimization, (Hong et al., 2020) has provided finite-sample guarantees for two-timescale AC, when a *natural* policy gradient step is used in the actor. Recently, (Fu et al., 2020) also analyzed the single-timescale AC algorithm under an exact critic oracle. On a less relevant line of research, AC-based multi-agent RL has been studied in (Zhang et al., 2018; Christianos et al., 2020; Qu et al., 2020). However, none of the existing works has analyzed the effect of the asynchronous and parallel updates in AC.

Parallel and distributed RL methods. In (Mnih et al., 2016), the original A3C method was proposed and became the workhorse in empirical RL. Later, (Babaeizadeh et al., 2017) has provided a GPU-version of A3C which significantly decreases training time. Recently, the A3C algorithm is further optimized in modern computers by (Stooke and Abbeel, 2019), where a large batch variant of A3C with improved efficiency is also proposed. In (Espeholt et al., 2018), an importance weighted distributed AC algorithm IMPALA has been developed to solve a collection of problems with one single set of parameters. A gossip-based distributed AC algorithm has been proposed in (Assran et al., 2019) which achieves performance competitive to A3C. Additionally, distributed RL is closely related to the multi-agent RL, both of which have a broad range of applications (Kar et al., 2013; Sadeghi et al., 2018; Wu et al., 2021). In (Kar et al., 2013), a distributed algorithm based on Q-learning was proposed and was shown to achieve convergence under a sparse communication network. In (Sadeghi et al., 2018), an asynchronous caching approach which utilized PG to find an optimal caching policy was developed. A robust decentralized TD learning method was proposed in (Wu et al., 2021) to defend against malicious agents in a multi-agent network.

Asynchronous stochastic optimization. For solving general optimization problems, asynchronous stochastic methods have received much attention recently. Due to the possible speedup that can be achieved by asynchronous optimization, it has also been extensively applied to various machine learning areas including RL (Mnih et al., 2016; Sadeghi et al., 2018) and distributed learning (Wu et al., 2018). The study of asynchronous stochastic methods can be traced back to 1980s (Bertsekas and Tsitsiklis, 1989). With the batch size M , (Agarwal and Duchi, 2011) analyzed asynchronous SGD (async-SGD) for convex functions, and derived a convergence rate of $\mathcal{O}(K^{-\frac{1}{2}}M^{-\frac{1}{2}})$ if delay K_0 is bounded by $\mathcal{O}(K^{\frac{1}{4}}M^{-\frac{3}{4}})$. This result implies linear speedup. (Feyzmahdavian et al., 2015) extended the analysis of (Agarwal and Duchi, 2011) to smooth convex with nonsmooth regularization and derived a similar rate. Recent studies by (Lian et al., 2016) improved upper bound of K_0 to $\mathcal{O}(K^{\frac{1}{2}}M^{-\frac{1}{2}})$. However, all these works have focused on the single-timescale SGD with a single variable, which cannot capture the stochastic recursion of the AC and A3C algorithms. To best of our knowledge, non-asymptotic analysis of asynchronous two-timescale SGD has remained unaddressed, and its speedup analysis is an uncharted territory.

1.2. This work

In this context, we revisit A3C with TD(0) for the critic update. The goal is to provide *non-asymptotic* guarantee and *linear speedup* justification for this popular algorithm.

Our contributions. Compared to the existing literature on both the AC algorithms and the async-SGD, our contributions can be summarized as follows.

c1) We revisit two-timescale A3C and establish its convergence rates with both i.i.d. and Markovian sampling. We first prove the local convergence rate for A3C in the general function approximation case, and then prove that A3C achieves global convergence for the softmax policy parameterization. To the best of our knowledge, this is the first non-asymptotic convergence result for *asynchronous parallel* AC algorithms.

c2) We characterize the sample complexity of A3C. In the i.i.d. setting, A3C achieves a sample complexity of $\mathcal{O}(\epsilon^{-2.5}/N)$ per worker, where N is the number of workers. Compared to the best-known complexity of $\mathcal{O}(\epsilon^{-2.5})$ for i.i.d. two-timescale AC (Hong et al., 2020), A3C achieves *linear speedup*, thanks to the parallelism and asynchrony. In the Markovian setting, if delay is bounded, the sample complexity of A3C matches the order of the non-parallel AC algorithm (Wu et al., 2020).

c3) We test A3C on a synthetic environment to verify our theoretical guarantees with both i.i.d. and Markovian sampling. We also test A3C on the classic control tasks and Atari Games.

Technical challenges. Compared to the recent analysis of nonparallel two-timescale AC in (Wu et al., 2020; Xu et al., 2020b; Hong et al., 2020), several new challenges arise due to the parallelism and asynchrony.

Markovian noise coupled with asynchrony and delay. The analysis of two-timescale AC algorithm is non-trivial because of the Markovian noise coupled with both the actor and critic steps. Different from the nonparallel AC that only involves a single Markov chain, A3C introduces multiple Markov chains (one per worker) that mix at different speeds. This is because at a given iteration, workers collect different number of samples and thus their chains mix to different degrees. As we will show later, the worker with the slowest mixing chain will determine the convergence.

Linear speedup for SGD with two coupled sequences. Parallel async-SGD has been shown to achieve linear speedup recently (Lian et al., 2016; Sun et al., 2017). Different from async-SGD, asynchronous AC is a two-timescale stochastic *semi-gradient* algorithm for solving the more challenging *bilevel* optimization problem (see (Hong et al., 2020)). The errors induced by asynchrony and delay are intertwined with both the actor and critic updates via a nested structure, which makes the sharp analysis more challenging. Our linear speedup analysis should be also distinguished from that of mini-batch async-SGD (Lian et al., 2017), where the speedup is a result of *variance reduction* thanks to the larger batch size generated by parallel workers.

2. Preliminaries

2.1. Markov decision process and policy gradient

A Markov decision process (MDP) can be described by $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma\}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s'|s, a)$ is the probability of transitioning to $s' \in \mathcal{S}$ from state $s \in \mathcal{S}$ and action $a \in \mathcal{A}$, $r(s, a, s')$ is the reward associated with the transition (s, a, s') , and $\gamma \in [0, 1)$ is a discount factor. Throughout the paper, we assume the reward r is upper-bounded by a constant $r_{\max}$. A policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ is defined as a mapping from the state space $\mathcal{S}$ to the probability distribution over the action space $\mathcal{A}$.

Considering discrete time t in an infinite horizon, a policy π can generate a trajectory $(s_0, a_0, \dots)$ with $a_t \sim \pi(\cdot|s_t)$ and $s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)$. Given a policy π , we define the state and state action value

functions as

$$\begin{aligned} V_\pi(s) &:= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1}) \mid s_0 = s \right], \\ Q_\pi(s, a) &:= \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, s_{t+1}) \mid s_0 = s, a_0 = a \right] \end{aligned} \quad (2)$$

where $\mathbb{E}$ is taken over the trajectory $(s_0, a_0, s_1, a_1, \dots)$ generated under policy π . With the above definitions, the advantage function is $A_\pi(s, a) := Q_\pi(s, a) - V_\pi(s)$. With η denoting the initial state distribution, the discounted state visitation measure induced by policy π is defined as $d_\pi(s) := (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid s_0 \sim \eta, \pi)$. We also overload the notation and define the state-action visitation distribution $d_\pi(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s \mid s_0 \sim \eta, \pi) \pi(a \mid s)$. In the case where π is parameterized by θ , we use d_θ as shorthand notations for d_{π_θ} .

The goal of RL is to find an optimal policy π^* defined as $\pi^* \in \arg \max_{\pi} J(\pi) := (1 - \gamma) \mathbb{E}_{s \sim \eta} [V_\pi(s)]$, with the optimal return defined as $J^* := \max_{\pi} J(\pi)$. When the state and action spaces are large, finding the optimal policy π becomes computationally intractable. To overcome the inherent difficulty of learning a function, the policy gradient methods search the best performing policy over a class of parameterized policies. We parameterize the policy with parameter $\theta \in \mathbb{R}^d$, and solve the optimization problem as

$$\max_{\theta \in \mathbb{R}^d} J(\theta) \quad \text{with} \quad J(\theta) := (1 - \gamma) \mathbb{E}_{s \sim \eta} [V_{\pi_\theta}(s)]. \quad (3)$$

To maximize $J(\theta)$ with respect to θ , one can update θ using the policy gradient (Sutton et al., 2000)

$$\nabla J(\theta) = \mathbb{E}_{s, a \sim d_\theta} [A_{\pi_\theta}(s, a) \psi_\theta(s, a)], \quad (4)$$

where $\psi_\theta(s, a) := \nabla \log \pi_\theta(a \mid s)$. Since computing $\mathbb{E}$ in (4) is expensive if not impossible, popular policy gradient-based algorithms iteratively update θ using stochastic estimate of (4) such as REINFORCE (Williams, 1992) and G(PO)MDP (Baxter and Bartlett, 2001).

It is also a common practice to adopt regularization and augment the objective function to

$$J_\lambda(\theta) := J(\theta) - \lambda \mathbb{E}_{s \sim \eta_p} [D_{KL}(\pi_p(\cdot \mid s) \mid \pi_\theta(\cdot \mid s))] \quad (5)$$

with a regularization constant $\lambda \geq 0$. Here η_p is a prior distribution of states, π_p is a prior policy. The regularization term encourages π_θ to imitate π_p , incorporating prior knowledge into training process. When π_p and η_p are set as uniform distributions, the regularization term is reduced to the relative-entropy regularization widely analyzed in the literature (Agarwal et al., 2020; Bhandari and Russo, 2019; Zhang et al., 2021). Moreover, the regularization prevents degenerate solutions that can lead to the pitfall of certain policy parametrization (Bhandari and Russo, 2019). Given π_p and η_p , we use $R(\theta)$ as a shorthand notation of $-\mathbb{E}_{s \sim \eta_p} [D_{KL}(\pi_p(\cdot \mid s) \mid \pi_\theta(\cdot \mid s))]$.

2.2. Actor-critic with value function approximation

Both REINFORCE and G(PO)MDP-based policy gradient algorithms rely on a Monte-Carlo estimate of the value function $V_{\pi_\theta}(s)$ and thus $\nabla J(\theta)$ by generating a trajectory per iteration. However, policy gradient methods based on Monte-Carlo estimate typically suffer from high variance and large

sampling cost. An alternative way is to recursively refine the estimate of $V_{\pi_\theta}(s)$. For a policy π_θ , it is known that $V_{\pi_\theta}(s)$ satisfies the Bellman equation (Sutton and Barto, 2018), that is

$$V_{\pi_\theta}(s) = \mathbb{E}_{a \sim \pi_\theta(\cdot|s), s' \sim \mathcal{P}(\cdot|s,a)} [r(s, a, s') + \gamma V_{\pi_\theta}(s')]. \quad (6)$$

In practice, when the state space $\mathcal{S}$ is prohibitively large, one cannot afford the computational and memory complexity of computing $V_{\pi_\theta}(s)$ and $A_{\pi_\theta}(s, a)$. To overcome this curse-of-dimensionality, a popular method is to approximate the value function using function approximation techniques. Given the state feature mapping $\phi(\cdot) : \mathcal{S} \rightarrow \mathbb{R}^{d'}$ for some $d' > 0$, we approximate the value function linearly as $V_{\pi_\theta}(s) \approx \hat{V}_\omega(s) := \phi(s)^\top \omega$, where $\omega \in \mathbb{R}^{d'}$ is the critic parameter.

Given π_θ , the task of finding the best ω such that $V_{\pi_\theta}(s) \approx \hat{V}_\omega(s)$ is usually addressed by TD learning (Sutton, 1988). Given π_θ , the task of finding the best ω such that $V_{\pi_\theta}(s) \approx \hat{V}_\omega(s)$ is usually addressed by TD learning (Sutton, 1988). Formally, we first define

$$A_{\theta,\phi} := \mathbb{E}_{s \sim \mu_{\pi_\theta}, s' \sim \mathcal{P}_{\pi_\theta}} [\phi(s)(\gamma \phi(s') - \phi(s))^\top], \quad (7a)$$

$$b_{\theta,\phi} := \mathbb{E}_{s \sim \mu_{\pi_\theta}, a \sim \pi_\theta} [r(s, a, s') \phi(s)]. \quad (7b)$$

where $\mathcal{P}_{\pi_\theta}(s'|s) := \sum_a \mathcal{P}(s'|s, a) \pi_\theta(a|s)$, and μ_{π_θ} is the stationary distribution of the Markov chain with transition distribution $\mathcal{P}$ and policy π_θ . Then given a policy π_θ , the *exact* TD update takes the following form:

$$\omega_{k+1} = \omega_k + \beta (A_{\theta,\phi} \omega_k + b_{\theta,\phi}). \quad (8)$$

When analyzing TD, the following standard assumption is often made:

Assumption 1 *For all $s \in \mathcal{S}$, the feature vector $\phi(s)$ is normalized so that $\|\phi(s)\|_2 \leq 1$. For all eligible θ , the symmetric part of $A_{\theta,\phi}$, denoted as $(A_{\theta,\phi} + A_{\theta,\phi}^\top)/2$, is negative definite and has a largest eigenvalue upper bounded by $-\lambda$.*

Assumption 1 is common in analyzing TD with linear function approximation; see e.g., (Bhandari et al., 2018; Wu et al., 2020, 2021). In fact, as shown in (Bhandari et al., 2018), when redundant features are removed such that the feature covariance matrix is full-rank, this assumption is satisfied. With this assumption, $A_{\theta,\phi}$ is full-rank, thus the update in (8) admits a unique stationary point $\omega^*(\theta) = -A_{\theta,\phi}^{-1} b_{\theta,\phi}$. Moreover, there exists a constant $R_\omega := \frac{r_{\max}}{\sqrt{\lambda}(1-\gamma)^{\frac{3}{2}}}$ such that $\|\omega^*(\theta)\|_2 \leq R_\omega$.

We often use the stochastic approximation of the TD update in (8). With k th transition defined as $x_k := (s_k, a_k, s_{k+1})$, the corresponding TD target is

$$\hat{\delta}(x_k, \omega_k) := r(s_k, a_k, s_{k+1}) + \gamma \phi(s_{k+1})^\top \omega_k - \phi(s_k)^\top \omega_k \quad (9)$$

and the critic gradient $g(x_k, \omega_k) := \hat{\delta}(x_k, \omega_k) \nabla \hat{V}_{\omega_k}(s_k)$. We update the parameter ω via

$$\omega_{k+1} = \Pi_{R_\omega} (\omega_k + \beta g(x_k, \omega_k)), \quad (10)$$

where β is the critic step size, and Π_{R_ω} is a projection operator that projects a vector to a l_2 norm ball with radius R_ω . The projection step is often used to control the norm of gradient. In AC, it prevents the actor and critic updates from going too far in the ‘wrong’ direction; see e.g., (Konda, 2002; Wu et al., 2020; Xu et al., 2020b; Zou et al., 2019).

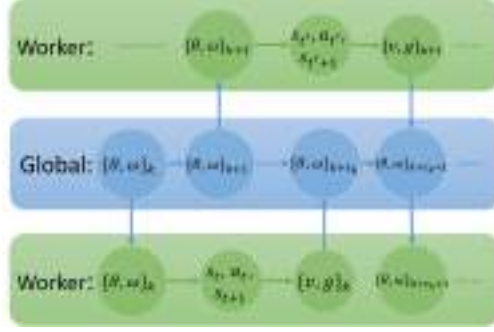


Figure 1: Implementation of A3C with two workers.

Using the definition that $A_{\pi_\theta}(s, a) = \mathbb{E}_{s' \sim \mathcal{P}}[r(s, a, s') + \gamma V_{\pi_\theta}(s')] - V_{\pi_\theta}(s)$, we can also rewrite (4) as $\nabla J(\theta) = \mathbb{E}_{s, a \sim d_\theta, s' \sim \mathcal{P}}[(r(s, a, s') + \gamma V_{\pi_\theta}(s') - V_{\pi_\theta}(s)) \psi_\theta(s, a)]$. Leveraging the value function approximation, we can then approximate the regularized policy gradient as

$$\hat{\nabla} J_\lambda(\theta) = \hat{\nabla} J(\theta) + \lambda \hat{\nabla} R(\theta) = \underbrace{\hat{\delta}(x, \omega) \psi_\theta(s, a)}_{v(x, \theta, \omega)} + \lambda \psi_\theta(x^p). \quad (11)$$

where $v(x, \theta, \omega)$ is an estimator of $\nabla J(\theta)$, and $x^p := (s^p \sim \eta_p, a^p \sim \pi_p(\cdot | s_p))$. Then it is easy to check that $\psi_\theta(x^p)$ is an unbiased estimator of $\nabla R(\theta)$. This gives rise to the policy update

$$\theta_{k+1} = \theta_k + \alpha (v(x_k, \theta_k, \omega_k) + \lambda \psi_\theta(x^p)), \quad (12)$$

where α is the stepsize for the actor update. To ensure convergence when simultaneously performing critic and actor updates, the stepsizes α and β often decay at two different rates, which is referred to the two-timescale AC (Konda, 2002; Wu et al., 2020).

3. A3C Implementation

To speed up the training process, AC can be implemented over N workers in a shared memory setting *without* coordinating among workers (Mnih et al., 2016). Each worker has its own simulator to perform sampling, and then collaboratively updates the shared policy π_θ using AC updates. As there is no synchronization after each update, the policy used by workers to generate samples may be outdated, which introduces staleness.

Notations on samples. Subscription t in x_t and x_t^p indicates the sample is generated in t th local iteration of a worker. When Markovian sampling is used, subscription t in $x_t = (s_t, a_t, s_{t+1})$ also indicates that it is the t th transition of the local Markov chain. We use k to denote the global counter (or iteration), which increases by one whenever a worker finishes the actor and critic updates in the shared memory. We use subscription (k) in $(s_{(k)}, a_{(k)}, s'_{(k)})$ and $(s_{(k)}^p, a_{(k)}^p)$ to indicate the samples used in the k th update.

Algorithm flow. Specifically, we initialize θ_0, ω_0 in the shared memory. Each worker will initialize the simulator with initial state s_0 . Without coordination, workers will load θ, ω in the shared memory. The worker then generates samples with either i.i.d. or Markovian sampling method. In Markovian sampling case, we maintain separate Markov chains for actor and critic. For critic, we

Algorithm 1 A3C: each worker’s view.

-
- 1: **Global initialize:** Global counter $k=0$, initial θ_0, ω_0 in the shared memory.
 - 2: **Worker initialize:** Counter $t=0$. Sample $s_0 \sim \eta, \hat{s}_0 \sim \eta$.
 - 3: **for** $t = 0, 1, 2, \dots$ **do**
 - 4: Read θ, ω in the shared memory.
 - 5: **option 1 (i.i.d. sampling):**
 - 6: $x_t = (s_t \sim \mu_{\pi_{\theta_t}}, a_t \sim \pi_{\theta_t}(\cdot|s_t), s'_t \sim \mathcal{P}(\cdot|s_t, a_t)).$
 - 7: $\hat{x}_t = (\hat{s}_t \sim d_{\pi_{\theta_t}}, \hat{a}_t \sim \pi_{\theta_t}(\cdot|\hat{s}_t), \hat{s}'_t \sim \mathcal{P}(\cdot|\hat{s}_t, \hat{a}_t)).$
 - 8: **option 2 (Markovian sampling):**
 - 9: $x_t = (s_t, a_t \sim \pi_{\theta}(\cdot|s_t), s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)).$
 - 10: $\hat{x}_t = (\hat{s}_t, \hat{a}_t \sim \pi_{\theta}(\cdot|\hat{s}_t), \hat{s}'_{t+1} \sim \mathcal{P}(\cdot|\hat{s}_t, \hat{a}_t)).$
 - 11: With probability γ : $\hat{s}_{t+1} = \hat{s}'_{t+1}$; Otherwise: $\hat{s}_{t+1} \sim \eta$.
 - 12: Compute $g(x_t, \omega) = \hat{\delta}(x_t, \omega) \nabla_{\omega} \hat{V}_{\omega}(s_t)$.
 - 13: Compute $v(\hat{x}_t, \theta, \omega) = \hat{\delta}(\hat{x}_t, \omega) \psi_{\theta}(\hat{s}_t, \hat{a}_t)$.
 - 14: Compute $\psi_{\theta}(x_t^p)$ with $x_t^p = (s_t^p \sim \eta_p, a_t^p \sim \pi_p(\cdot|s_t^p)).$
 - 15: In the shared memory, perform update (13).
 - 16: **end for**
-

generate samples following the original transition kernel $\mathcal{P}$. While the actor’s chain can be viewed as evolving under a transition kernel $\hat{\mathcal{P}} = \gamma\mathcal{P} + (1 - \gamma)\eta$. At each iteration, we have a probability of $1 - \gamma$ to reset the chain, thus taking the initial state distribution into account. If the actor’s chain evolves under $\mathcal{P}$ like critic, asymptotically the initial distribution η is forgotten, which will introduce an asymptotic error. Once samples are obtained, each worker locally computes the gradients, and then updates the parameters in shared memory asynchronously by

$$\omega_{k+1} = \Pi_{R_{\omega}} (\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k})) \quad (13a)$$

$$\theta_{k+1} = \theta_k + \alpha (v(\hat{x}_{(k)}, \theta_{k-\tau_k}, \omega_{k-\tau_k}) + \lambda \psi_{\theta_{k-\tau_k}}(x_{(k)}^p)) \quad (13b)$$

where τ_k is the delay in the k th actor and critic updates. See A3C in Algorithm 1 and Figure 1.

Parallel sampling. The AC update (10) and (12) uses samples generated “on-the-fly” from the target policy π_{θ} , which brings overhead. Compared with (10) and (12), the A3C update (13) allows parallel sampling from N workers, which is the key to linear speedup. We consider the case where only one worker can update parameters in the shared memory at the same time and the update cannot be interrupted. In practice, (13) can also be performed in a mini-batch fashion.

Separate sampling protocols. In Algorithm 1, we maintain separate sampling protocols for actor and critic. This is due to the mismatch between the actor and critic sampling distribution. As indicated by (7), the desired sampling distribution of critic is $\mu_{\pi_{\theta}}$. The policy gradient (4) requires sampling from $d_{\pi_{\theta}}$. However, $d_{\pi_{\theta}}$ and $\mu_{\pi_{\theta}}$ are in general different, and the difference is non-diminishing. Therefore, if one uses the same samples for actor and critic, either the actor or the critic update will have a non-diminishing bias.

To mitigate the asymptotic bias, it is just natural to choose different sampling protocols for actor and critic. Our theoretical analysis justifies this choice by proving that such sampling method gives unbiased stochastic gradients asymptotically. We also provide experiments to demonstrate the superiority of the separated sampling methods.

4. Convergence Analysis of A3C

In this section, we analyze the convergence of A3C in both i.i.d. and Markovian settings. Throughout this section, $\mathcal{O}(\cdot)$ contains constants that are independent of N and K_0 .

To analyze the performance of A3C, we make the following assumptions.

Assumption 2 *There exists K_0 such that the delay at each iteration is bounded by $\tau_k \leq K_0, \forall k$.*

Assumption 2 ensures the viability of analyzing the asynchronous update; see the same assumption in e.g., (Lian et al., 2016; Assran et al., 2019; Wu et al., 2018). In practice, the delay usually scales as the number of workers, that is $K_0 = \Theta(N)$.

Assumption 3 *For any $\theta, \theta' \in \mathbb{R}^d$, $s \in \mathcal{S}$ and $a \in \mathcal{A}$, there exist constants C_ψ, L_ψ, L_π such that: i) $\|\psi_\theta(s, a)\|_2 \leq C_\psi$; ii) $\|\psi_\theta(s, a) - \psi_{\theta'}(s, a)\|_2 \leq L_\psi \|\theta - \theta'\|_2$; iii) $|\pi_\theta(a|s) - \pi_{\theta'}(a|s)| \leq L_\pi \|\theta - \theta'\|_2$.*

Assumption 3 is common in analyzing policy gradient-type algorithms which has also been made by e.g., (Agarwal et al., 2020; Zhang et al., 2019). This assumption holds for many policy parameterization methods such as tabular softmax policy (Agarwal et al., 2020), Gaussian policy (Doya, 2000) and Boltzmann policy (Konda and Borkar, 1999).

Assumption 4 *For any θ , assume the Markov chains with transition kernels $\mathcal{P}$ and $\hat{\mathcal{P}}$ are irreducible and aperiodic under policy π_θ . Then there exist constants $\kappa > 0$ and $\rho \in (0, 1)$ such that*

$$\sup_{s \in \mathcal{S}} d_{TV}(\mathbb{P}(s_t \in \cdot | s_0 = s, \pi_\theta), \mu_{\pi_\theta}) \leq \kappa \rho^t, \quad (14a)$$

and

$$\sup_{s \in \mathcal{S}} d_{TV}(\mathbb{P}(\hat{s}_t \in \cdot | \hat{s}_0 = s, \pi_\theta), d_{\pi_\theta}) \leq \kappa \rho^t. \quad (14b)$$

where s_t is the t th state of the Markov chain with transition kernel $\mathcal{P}$, and $\hat{s}_t$ is the t th state of the Markov chain with transition kernel $\hat{\mathcal{P}}$.

Assumption 4 assumes the Markov chain mixes at a geometric rate. This assumption has also been made by other analysis on Markovian sampling; see e.g. (Bhandari et al., 2018; Wu et al., 2020). It is worth noting that the second part of our assumption, that is (14b), holds as long as $\gamma < 1$.

We define the critic approximation error as

$$\epsilon_{\text{app}} := \max_{\theta \in \mathbb{R}^d} \sqrt{\mathbb{E}_{s \sim \mu_\theta} |V_{\pi_\theta}(s) - \hat{V}_{\omega_\theta^*}(s)|^2} \quad (15)$$

where μ_θ is the stationary distribution under π_θ and $\mathcal{P}$. This error captures the quality of the critic function approximation; see also (Qiu et al., 2019; Wu et al., 2020, 2021). When the MDP is tabular and the feature matrix is full-rank, the value function V_{π_θ} is in the span of the features. In this case, we have $\epsilon_{\text{app}} = 0$.

We first give the convergence result of critic update.

Theorem 1 (Critic convergence) *Suppose Assumptions 1–4 hold. Consider Algorithm 1 with i.i.d. sampling and $\hat{V}_\omega(s) = \phi(s)^\top \omega$. Select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_{\theta_k}^*\|_2^2 = \mathcal{O}\left(\frac{K_0^2}{K^{\frac{4}{5}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{3}{5}}}\right) + \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right). \quad (16)$$

Given the critic convergence, we can present the convergence result of actor update.

Theorem 2 (Actor convergence) *Under the same assumptions of Theorem 1, select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right) + \mathcal{O}\left(\frac{K_0^2}{K^{\frac{4}{5}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{3}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (17)$$

If $K_0 = \Theta(N) = \mathcal{O}(K^{\frac{1}{5}})$, then it holds that

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}(K^{-\frac{2}{5}}) + \mathcal{O}(\epsilon_{\text{app}}) \quad (18)$$

where $\mathcal{O}(\cdot)$ contains constants independent of N and K_0 .

Corollary 1 (Linear speedup) *To reach ϵ -accuracy in (18), the required number of iterations is $\mathcal{O}(\epsilon^{-2.5})$. Since each iteration of A3C only uses one sample (one transition), the sample complexity is $\mathcal{O}(\epsilon^{-2.5})$, which matches the state-of-the-art sample complexity of two-timescale AC running on one worker. Then under A3C, the average sample complexity per worker is $\mathcal{O}(\epsilon^{-2.5}/N)$ which indicates linear speedup in (1). The negative effect of parameter staleness introduced by parallel asynchrony vanishes asymptotically with the step size. Vanished staleness allows for parallel computing from workers to speedup the training process.*

Remark 1 (Comparison to async-SGD analysis) *Different from async-SGD (e.g., (Lian et al., 2016)), the optimal critic parameter ω_θ^* is constantly drifting as θ changes at each iteration. This necessitates setting the actor update to be at a faster time scale than the critic. In this sense, the policy is static relative to the critic asymptotically. In actor update, the gradient $v(x, \theta, \omega)$ is biased because of inexact value function. The bias introduced by the critic optimality gap and the function approximation error correspond to the last two terms in (17).*

4.1. Convergence result with Markovian sampling

Theorem 3 (Critic convergence) *Suppose Assumptions 1–4 hold. Consider Algorithm 1 with Markovian sampling and $\hat{V}_\omega(s) = \phi(s)^\top \omega$. Select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 = \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right) + \mathcal{O}\left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}}\right) + \mathcal{O}\left(\frac{K_0 \log K}{K^{\frac{2}{5}}}\right). \quad (19)$$

The following theorem gives the convergence rate of actor update in Algorithm 1.

Theorem 4 (Actor convergence) *Under the same assumptions of Theorem 3, select step size $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. Then it holds that*

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}}\right) + \mathcal{O}\left(\frac{K_0 \log K}{K^{\frac{2}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (20)$$

If we further assume $K_0 = \Theta(N) = \mathcal{O}(K^{\frac{1}{5}})$. It holds that

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \tilde{\mathcal{O}}\left(K_0 K^{-\frac{2}{5}}\right) + \mathcal{O}(\epsilon_{\text{app}}) \quad (21)$$

where $\tilde{\mathcal{O}}(\cdot)$ hides constants and the logarithmic order of K .

Different from i.i.d. sampling, the stochastic gradients $g(x, \omega)$ and $v(x, \theta, \omega)$ are biased for Markovian sampling, and the bias decreases as the chain mixes. The mixing time corresponds to the logarithmic terms $\log K$ in (19) and (20). Because of asynchrony, at a given iteration, workers collect different number of samples and their chains mix to different degrees. The worker with the slowest mixing chain will determine the rate of convergence. The product of K_0 and $\log K$ in (19) and (20) appears due to the slowest mixing chain. As the last term in (19) dominates other terms asymptotically, the convergence rate degrades as the number of workers increases. While the theoretical linear speedup is difficult to establish in the Markovian setting, we will empirically test it in Section 5.

Remark 2 (Challenges compared to AC analysis) *Unlike synchronous AC, A3C introduces asynchrony and delay in both the actor and critic updates. At each iteration k , the delayed parameters will introduce extra error in $g(x, \omega_{k-\tau_k}) - g(x, \omega_k)$ and $v(x, \theta_{k-\tau_k}, \omega_{k-\tau_k}) - v(x, \theta_k, \omega_k)$. Furthermore, it also causes delays in sampling since samples are drawn from the delayed policy $\pi_{\theta_{k-\tau_k}}$ instead of π_{θ_k} . This delay will get amplified as every state on the Markov chain is generated by policies with different delays. At local counter t (t th transition on local Markov chain), we compare the chain transition in synchronous and asynchronous settings:*

$$\begin{aligned} \text{sync} : & s_t \xrightarrow{\theta_t} a_t \xrightarrow{\mathcal{P}} s_{t+1} \xrightarrow{\theta_{t+1}} a_{t+1} \cdots ; \\ \text{async} : & s_t \xrightarrow{\theta_{k-\tau_k}} a_t \xrightarrow{\mathcal{P}} s_{t+1} \xrightarrow{\theta_{k+d_t-\tau_{k+d_t}}} a_{t+1} \cdots \end{aligned}$$

where k is the global counter at which the local Markov chain takes t th transition, τ_k is the delay of policy used to generate t th local transition, d_t is the number of global updates between two local transitions. Clearly, the parameter delay makes the Markov chain more difficult to analyze.

4.2. Global convergence under structured problem

A3C is a gradient ascent type algorithm, thus can only achieve local convergence under a generally non-concave objective function $J_\lambda(\theta)$ w.r.t. θ . However, under some special structured problem, A3C can be shown to achieve global convergence. In this section, we consider the class of MDP which has finite state space and action space. Suppose the policy is parameterized by the softmax function:

$$\pi_\theta(a|s) = \frac{\exp(\theta_{s,a})}{\sum_{s,a} \exp(\theta_{s,a})} \quad (22)$$

where $\theta \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ and $\theta_{s,a}$ is the policy parameter corresponds to pair (s, a) . The softmax policy class cannot represent deterministic policies with finite θ . To avoid driving θ to infinity, it is crucial to penalize the deterministic policies with the regularization term introduced in (5). To do so, we set

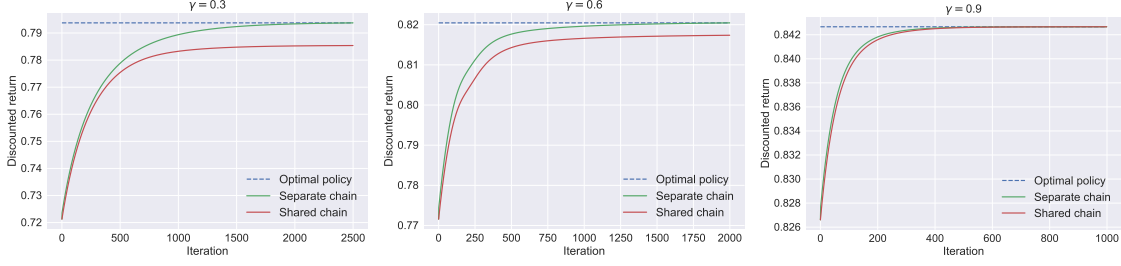


Figure 2: Algorithm 1 with separate chain sampling (option 2) vs shared chain sampling (setting $\hat{x}_t = x_t$ in the algorithm). The asymptotic error roughly scales proportionally to $1 - \gamma$. With a smaller γ , the objective function J becomes more shortsighted, and thus initial state distribution (restarting the chain) plays a more important role. If the actor shares the sample with critic, then a lack of chain restarting will introduce an unavoidable asymptotic error that grows larger as γ becomes smaller. Separate chain sampling works thanks to the random restarting with a probability scaling with γ .

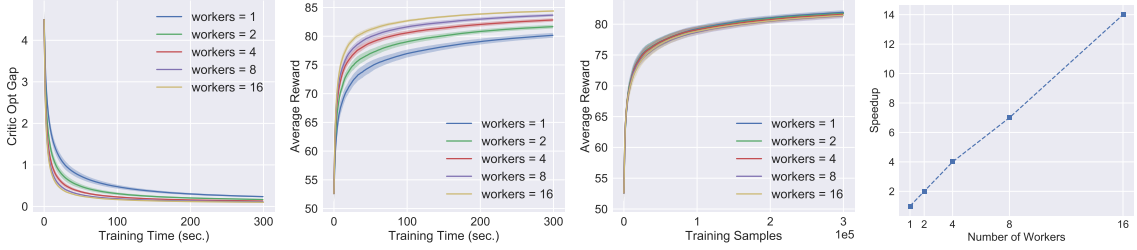


Figure 3: Convergence results of A3C with i.i.d. sampling in synthetic environment.

the priors η_p and π_p as uniform distribution on state and action space, then the objective function can be rewritten as

$$J_\lambda(\theta) = J(\theta) + \frac{\lambda}{|\mathcal{S}||\mathcal{A}|} \sum_{s,a} \log \pi_\theta(a|s) + \lambda \log |\mathcal{A}|. \quad (23)$$

Define the state feature matrix $\Phi' := [\phi(s^1), \phi(s^2), \dots, \phi(s^{|\mathcal{S}|})]^\top \in \mathbb{R}^{|\mathcal{S}| \times d'}$ of which rows are features. We make the following assumption on Φ' .

Assumption 5 For any eligible θ , there exists $\omega_\theta \in \mathbb{R}^{d'}$ such that $\Phi' \omega_\theta = V_{\pi_\theta}$.

This assumption assumes that the value function V_{π_θ} can be accurately approximated by linear functions. For the assumption to hold, it suffices to select a squared full-rank feature matrix Φ' . It is worth noting that when this assumption does not hold, our result in Theorem 5 holds with an extra error term, which is the function approximation error ϵ_{app} .

To establish global convergence, a gradient-dominance type condition was proven in (Agarwal et al., 2020):

Lemma 1 With softmax policy parameterization and uniform priors, if $\|\nabla J_\lambda(\theta)\|_2 \leq \frac{\lambda}{2|\mathcal{S}||\mathcal{A}|}$, then $J^* - J(\theta) \leq \epsilon_\lambda := \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty$.

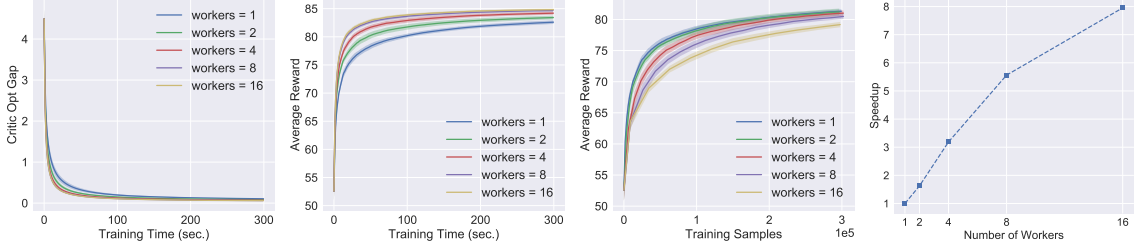


Figure 4: Convergence results of A3C with Markovian sampling in synthetic environment.

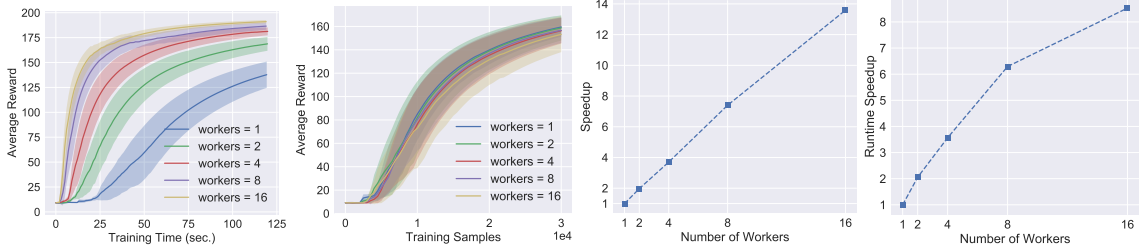


Figure 5: Speedup of A3C in OpenAI gym classic control task (Carpole).

For an arbitrary accuracy ϵ , if we set $\lambda = \frac{(1-\gamma)\epsilon}{2\|\frac{d\pi^*}{\eta}\|_\infty}$, then we have $\epsilon_\lambda = \epsilon$. Note that in order for $\|\frac{d\pi^*}{\eta}\|_\infty$ to be finite, we need $\eta(s) > 0$ for any $s \in \mathcal{S}$, which can be assumed without loss of generality. In the case where $\eta > 0$ does not hold, one can start with an exploratory initial state distribution $\eta' > 0$ like in (Agarwal et al., 2020), and our result still holds. This lemma allows us to establish connection between the gradient norm and optimality gap, giving rise to the following theorem.

Theorem 5 Suppose Assumptions 1, 2 and 4–5 hold. Consider Algorithm 1 with softmax policy and linear critic function $\hat{V}_\omega(s) = \phi(s)^\top \omega$. Select step size $\alpha = K^{-\frac{3}{5}}$, $\beta = K^{-\frac{2}{5}}$ and let $K_0 = \Theta(N) = \mathcal{O}(K^{\frac{1}{5}})$, then it holds for i.i.d. sampling

$$J^* - \frac{1}{K} \sum_{k=1}^K \mathbb{E}[J(\theta_k)] = \mathcal{O}\left(\lambda^{-2} K^{-\frac{2}{5}}\right) + \epsilon_\lambda, \quad (24a)$$

and for Markovian sampling

$$J^* - \frac{1}{K} \sum_{k=1}^K \mathbb{E}[J(\theta_k)] = \tilde{\mathcal{O}}\left(\lambda^{-2} K_0 K^{-\frac{2}{5}}\right) + \epsilon_\lambda. \quad (24b)$$

5. Numerical Experiments

We test the impact of separate sampling and the speedup property of A3C in both synthetically generated and Gym environments. The tests on synthetic environment were performed in a 16-core CPU computer, and those on Atari games were run in a 4 GPU computer.

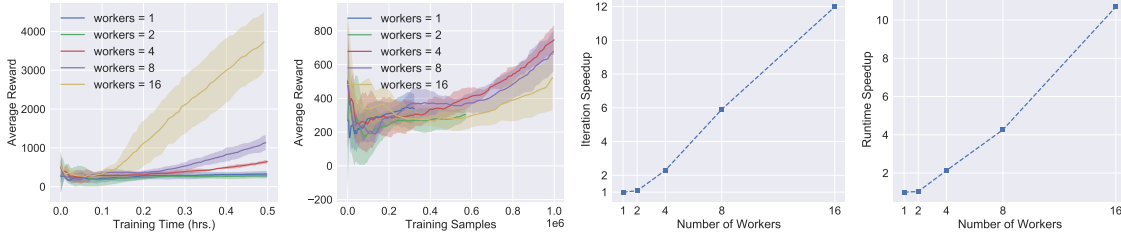


Figure 6: Speedup of A3C in OpenAI Gym Atari game (Beamrider).

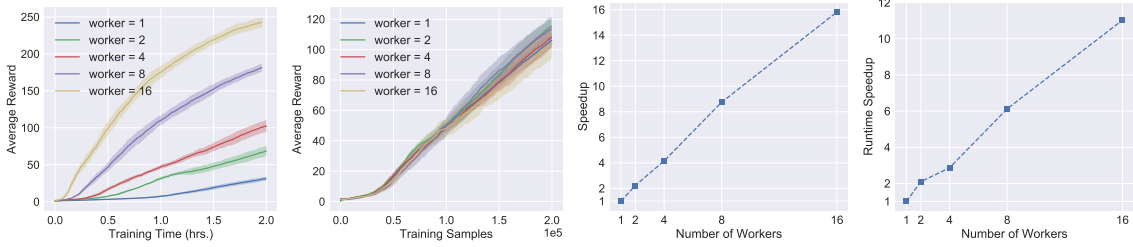


Figure 7: Speedup of A3C in OpenAI Gym Atari game (Breakout).

5.1. Separate sampling protocol

We compare the separate chain sampling method in Algorithm 1 with the shared chain sampling method. The shared chain method is simply using the same sample for both actor and critic, i.e., setting $\hat{x}_t = x_t$ in Algorithm 1.

To clearly demonstrate the impact of sampling, we mitigate the impact from other sources such as delay and MDP non-ergodicity by considering a synthetic environment with 1 worker. In this test, we use the tabular softmax policy parameterization. The synthetic MDP has a state space $|\mathcal{S}| = 10$, an discrete action space of $|\mathcal{A}| = 4$. State features each has a dimension of 10. Elements of the transition matrix, the reward and the state features are randomly sampled from a uniform distribution over $(0, 1)$.

It can be clearly observed from Figure 2 that using the same sample for actor and critic leads to an asymptotic error scaled with choice of γ . The intuitive explanation is in the caption of Figure 2. Although when $\gamma \rightarrow 1$, the error is small (but still exists), we want our algorithm design to not restrict the choice of γ , and thus adopt the separate chain sampling method.

5.2. Linear speedup

Experiment settings. For the synthetic environment, we used linear value function approximation and tabular softmax policy (Agarwal et al., 2020). For CartPole, we used a 3-layer MLP with 128 neurons and sigmoid activation function in each layer. The first two layers are shared for both actor and critic network. For the Atari games, we used a convolution-LSTM network. For network details, see (Dgriff, 2018).

For the separate sampling protocol test, we have $\alpha = 0.6$ and critic step size $\beta = 0.7$, along with $\lambda = 0.3$. For the speedup tests in synthetic environment, we set actor step size $\alpha_k = \frac{0.05}{(1+k)^{0.6}}$ and critic step size $\beta_k = \frac{0.05}{(1+k)^{0.4}}$. In tests of CartPole, we run Algorithm 1 with a minibatch of 20 samples. We update the actor network with a step size of $\alpha_k = \frac{0.01}{(1+k)^{0.6}}$ and critic network with a step size of $\beta_k = \frac{0.01}{(1+k)^{0.4}}$. See Table 1 for hyper-parameters in Atari game tests.

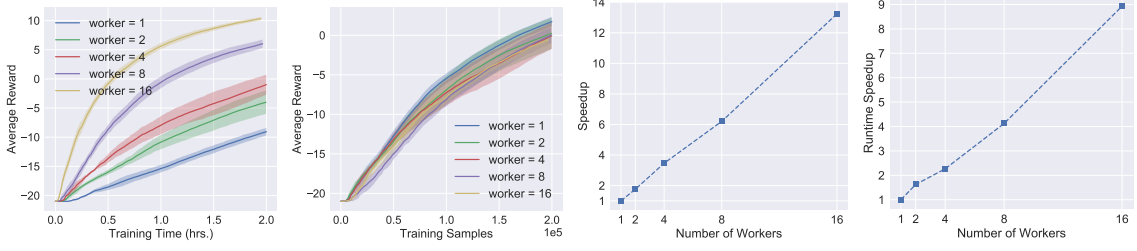


Figure 8: Speedup of A3C in OpenAI Gym Atari game (Pong).

Hyper-parameters	Value
Number of workers	1,2,4,8,16
Optimizer	Adam
Step size	0.00015
Batch size	20
Discount factor	0.99
Entropy coefficient	0.01
Frame size	80×80
Frame skip rate	4
Grayscale	Yes
Training reward clipping	$[-1,1]$

Table 1: Hyper-parameters of A3C in the Atari games.

Synthetic environment. We first test the speedup property of A3C in a synthetic environment with $|S| = 100$, $|A| = 5$ and state feature with dimension 10. The reward and transition matrix of the MDP are randomly generated in the same way as that in section 5.1. We evaluate the convergence of actor in terms of the average reward and the critic in terms of the gap $\|\omega_k - \omega_{\theta_k}^*\|_2$.

Figures 3 and 4 show the training time and sample complexity of running A3C with i.i.d. sampling and Markovian sampling respectively. The speedup plot is measured by the number of samples needed to achieve a target running average reward under different number of workers. All the results are average over 10 Monte-Carlo runs. Figure 3 shows that the sample complexity of A3C stays the same with different number of workers under i.i.d. sampling. Also, it can be observed from the speedup plot of Figure 3 that the A3C achieves roughly linear speedup, which is consistent with Corollary 1. The speedup of A3C with Markovian sampling shown in Figure 4 is roughly linear when number of workers is small.

OpenAI Gym environments. We also test the speedup property of A3C with neural network parametrization in the classic control (Carpole) and the Atari (Breakout and Pong) environments. In Figures 5-8, each curve was averaged over 5 Monte-Carlo runs with 95% confidence interval. Figures 5-8 show the speedup of A3C under different number of workers, where the average reward is computed by taking the running average of test rewards. The speedup is respectively measured by the number of samples and training time needed per worker to achieve a target average reward. Although not justified theoretically, Figures 5-8 suggest that the sample complexity speedup is roughly linear, and the runtime speedup slightly degrades when the number of workers increases.

This is partially due to hardware limit. Similar observation was obtained in async-SGD (Lian et al., 2016, Fig. 4).

6. Conclusions

This paper revisits the A3C algorithm. With linear value function approximation, the convergence of the A3C algorithm has been established under both i.i.d. and Markovian sampling settings. Under i.i.d. sampling, A3C achieves linear speedup compared to the best-known sample complexity of AC, theoretically justifying the benefit of parallelism and asynchrony for the first time. Under Markov sampling, such a linear speedup can be observed in most benchmark tasks. One limitation of this paper is that theoretical linear speedup cannot be established in the Markovian setting. This motivates two interesting directions: i) developing new tools of analyzing two-timescale SGD with Markov sampling; and, ii) designing better algorithms than A3C to achieve better speedup.

References

- A. Agarwal and J. C. Duchi. Distributed delayed stochastic optimization. In *Proceedings of Advances in Neural Information Processing Systems*, 2011.
- A. Agarwal, S. M. Kakade, J. D. Lee, and G. Mahajan. Optimality and approximation with policy gradient methods in markov decision processes. In *Proceedings of Thirty Third Conference on Learning Theory*, 2020.
- M. Assran, J. Romoff, N. Ballas, J. Pineau, and M. Rabbat. Gossip-based actor-learner architectures for deep reinforcement learning. In *Proceedings of Advances in Neural Information Processing Systems*, 2019.
- M. Babaeizadeh, I. Frosio, S. Tyree, J. Clemons, and J. Kautz. Reinforcement learning through asynchronous advantage actor-critic on a gpu. In *Proceedings of International Conference on Learning Representations*, 2017.
- J. Baxter and P. L. Bartlett. Infinite-horizon policy-gradient estimation. *J. Artificial Intelligence Res.*, 15:319–350, 2001.
- D. P. Bertsekas and J. N. Tsitsiklis. *Parallel and distributed computation: numerical methods*. Prentice-Hall, 1989.
- J. Bhandari and D. Russo. Global optimality guarantees for policy gradient methods. *arXiv preprint:1906.01786*, 2019.
- J. Bhandari and D. Russo. On the linear convergence of policy gradient methods for finite mdps. In *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, 2021.
- J. Bhandari, D. Russo, and R. Singal. A finite time analysis of temporal difference learning with linear function approximation. In *Proceedings of Conference on Learning Theory*, 2018.
- S. Bhatnagar, R. S. Sutton, M. Ghavamzadeh, and M. Lee. Natural actor critic algorithms. *Automatica*, 45:2471–2482, 2009.

- V. Borkar and V. Konda. The actor-critic algorithm as multi-time-scale stochastic approximation. *Sadhana*, 22(4):525–543, 1997.
- J. Cervino, J. A. Bazerque, M. Calvo-Fullana, and A. Ribeiro. Multi-task reinforcement learning in reproducing kernel hilbert spaces via cross-learning. *IEEE Transactions on Signal Processing*, 69: 5947–5962, 2021.
- S. Chai and V. K. N. Lau. Joint rate and power optimization for multimedia streaming in wireless fading channels via parametric policy gradient. *IEEE Transactions on Signal Processing*, 67(17): 4570–4581, 2019.
- S. Chai and V. K. N. Lau. Online trajectory and radio resource optimization of cache-enabled uav wireless networks with content and energy recharging. *IEEE Transactions on Signal Processing*, 68:1286–1299, 2020.
- T. Chen, K. Zhang, G. B. Giannakis, and T. Başar. Communication-efficient distributed reinforcement learning. *arXiv preprint: 1812.03239*, 2018.
- F. Christianos, L. Schäfer, and S. Albrecht. Shared experience actor-critic for multi-agent reinforcement learning. In *Proceedings of Advances in Neural Information Processing Systems*, 2020.
- Dgriff. Pytorch implementation of a3c. https://github.com/dgriff777/rl_a3c_pytorch, 2018.
- K. Doya. Reinforcement learning in continuous time and space. *Neural Computation*, 12(1):219–245, 2000.
- Y. El-Laham and M. F. Bugallo. Policy gradient importance sampling for bayesian inference. *IEEE Transactions on Signal Processing*, 69:4245–4256, 2021.
- L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. *arXiv preprint:1802.01561*, 2018.
- H. R. Feyzmahdavian, A. Aytekin, and M. Johansson. An asynchronous mini-batch algorithm for regularized stochastic optimization. *arXiv preprint:1505.04824*, 2015.
- Z. Fu, Z. Yang, and Z. Wang. Single-timescale actor-critic provably finds globally optimal policy. *arXiv preprint:2008.00483*, 2020.
- M. Hong, H.-T. Wai, Z. Wang, and Z. Yang. A two-timescale framework for bilevel optimization: Complexity analysis and application to actor-critic. *arXiv preprint:2007.05170*, 2020.
- S. Kar, J. Moura, and V. Poor. QD-learning: A collaborative distributed strategy for multi-agent reinforcement learning through Consensus+Innovations. *IEEE Transactions on Signal Processing*, 61(7):1848–1862, 2013.
- V. Konda. *Actor-critic algorithms*. PhD thesis, Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, 2002.

Supplementary Material

Appendix A. Preliminary Lemmas

A.1. Geometric mixing

The operation $p \otimes q$ denotes the product between two distributions $p(x)$ and $q(y)$, i.e. $(p \otimes q)(x, y) = p(x) \cdot q(y)$.

Lemma 2 Suppose Assumption 4 holds. For any $\theta \in \mathbb{R}^d$, we have

$$\sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}((s_t, a_t, s_{t+1}) \in \cdot | s_0, \pi_\theta), \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \leq \kappa \rho^t. \quad (25a)$$

and

$$\sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}((\hat{s}_t, \hat{a}_t, s'_{t+1}) \in \cdot | s_0, \pi_\theta), d_\theta \otimes \pi_\theta \otimes \mathcal{P}) \leq \kappa \rho^t. \quad (25b)$$

where (s_t, a_t, s_{t+1}) is the t th transition on the Markov chain with transition kernel $\mathcal{P}$. $(\hat{s}_t, \hat{a}_t)$ is the t th state-action pair on Markov chain with transition kernel $\hat{\mathcal{P}}$, and $s'_{t+1} \sim \mathcal{P}(\cdot | \hat{s}_t, \hat{a}_t)$.

Proof We start with

$$\begin{aligned} & \sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}((s_t, a_t, s_{t+1}) \in \cdot | s_0, \pi_\theta), \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \\ &= \sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}(s_t = \cdot | s_0, \pi_\theta) \otimes \pi_\theta \otimes \mathcal{P}, \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \\ &= \sup_{s_0 \in \mathcal{S}} \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_t = ds | s_0, \pi_\theta) \pi_\theta(a | s) \mathcal{P}(ds' | s, a) - \mu_\theta(ds) \pi_\theta(a | s) \mathcal{P}(ds' | s, a)| \\ &= \sup_{s_0 \in \mathcal{S}} \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds | s_0, \pi_\theta) - \mu_\theta(ds)| \sum_{a \in \mathcal{A}} \pi_\theta(a | s) \int_{s' \in \mathcal{S}} \mathcal{P}(ds' | s, a) \\ &= \sup_{s_0 \in \mathcal{S}} d_{TV}(\mathbb{P}(s_t \in \cdot | s_0, \pi_\theta), \mu_\theta) \\ &\leq \kappa \rho^t, \end{aligned} \quad (26)$$

Inequality (25a) along with the fact that the stationary distribution of the Markov chain with transition probability $\hat{\mathcal{P}}$ and policy π_θ is simply d_θ immediately implies (25b). This completes the proof. ■

For the use in the later proof, given $K > 0$, we first define m_K as:

$$m_K := \min \{ m \in \mathbb{N}^+ \mid \kappa \rho^{m-1} \leq \min\{\alpha, \beta\} \}, \quad (27)$$

where κ and ρ are constants defined in (4). m_K is the minimum number of samples needed for the Markov chain to approach the stationary distribution so that the bias incurred by the Markovian sampling is small enough.

A.2. Auxiliary Markov chain

The auxiliary Markov chain is a virtual Markov chain with no policy drifting — a technique developed in (Zou et al., 2019) to analyze stochastic approximation algorithms in non-stationary settings. We provide an analysis here for completeness.

Lemma 3 *Under Assumption 2 and Assumption 3, consider the update (13) in Algorithm 1 with Markovian sampling. For a given number of samples m , consider the Markov chain of the worker that contributes to the k th update:*

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_{m-1}}} a_{t-m+1} \cdots s_{t-1} \xrightarrow{\theta_{k-d_1}} a_{t-1} \xrightarrow{\mathcal{P}} s_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\mathcal{P}} s_{t+1},$$

where $(s_t, a_t, s_{t+1}) = (s_{(k)}, a_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$.

Given $(s_{t-m}, a_{t-m}, s_{t-m+1})$ and θ_{k-d_m} , we construct its auxiliary Markov chain by repeatedly applying $\pi_{\theta_{k-d_m}}$:

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\mathcal{P}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\mathcal{P}} \tilde{s}_{t+1}.$$

Then we have:

$$\begin{aligned} d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot | \theta_{k-d_m}, s_{t-m+1}), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot | \theta_{k-d_m}, s_{t-m+1})) \\ \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} [\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta_{k-d_m}, s_{t-m+1}]. \end{aligned} \quad (28)$$

Proof Throughout the lemma, all expectations and probabilities are conditioned on θ_{k-d_m} and s_{t-m+1} . We omit this condition for convenience.

With $\tilde{x}_t := (\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1})$, first we have

$$\begin{aligned} d_{TV}(\mathbb{P}(s_{t+1} \in \cdot), \mathbb{P}(\tilde{s}_{t+1} \in \cdot)) \\ = \frac{1}{2} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_{t+1} = ds') - \mathbb{P}(\tilde{s}_{t+1} = ds')| \\ = \frac{1}{2} \int_{s' \in \mathcal{S}} \left| \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds') \right| \\ \leq \frac{1}{2} \int_{s' \in \mathcal{S}} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds')| \\ = \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds')| \\ = d_{TV}(\mathbb{P}(x_t \in \cdot), \mathbb{P}(\tilde{x}_t \in \cdot)), \end{aligned} \quad (29)$$

where the second last equality is due to Tonelli's theorem. Next we have

$$\begin{aligned}
 & d_{TV}(\mathbb{P}(x_t \in \cdot), \mathbb{P}(\tilde{x}_t \in \cdot)) \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \int_{s' \in \mathcal{S}} |\mathbb{P}(s_t = ds, a_t = a, s_{t+1} = ds') - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a, \tilde{s}_{t+1} = ds')| \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds, a_t = a) - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a)| \int_{s' \in \mathcal{S}} \mathcal{P}(s_{t+1} = ds' | s_t = ds, a_t = a) \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds, a_t = a) - \mathbb{P}(\tilde{s}_t = ds, \tilde{a}_t = a)| \\
 &= d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)). \tag{30}
 \end{aligned}$$

Due to the fact that $\theta_{k-\tau_k}$ is dependent on s_t , we need to write $\mathbb{P}(s_t, a_t)$ as

$$\begin{aligned}
 \mathbb{P}(s_t, a_t) &= \int_{\theta_{k-\tau_k} \in \mathbb{R}^d} \mathbb{P}(s_t, \theta_{k-\tau_k}, a_t) \\
 &= \int_{\theta \in \mathbb{R}^d} \mathbb{P}(s_t) \mathbb{P}(\theta_{k-\tau_k} = d\theta | s_t) \pi_{\theta_{k-\tau_k}}(a_t | s_t) \\
 &= \mathbb{P}(s_t) \mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t | s_t) | s_t]. \tag{31}
 \end{aligned}$$

Then we have

$$\begin{aligned}
 & d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds) \mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) | s_t = ds] - \mathbb{P}(\tilde{s}_t = ds) \pi_{\theta_{k-d_m}}(\tilde{a}_t = a | \tilde{s}_t = ds)| \\
 &\leq \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds) \mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) | s_t = ds] - \mathbb{P}(s_t = ds) \pi_{\theta_{k-d_m}}(a_t = a | s_t = ds)| \\
 &\quad + \frac{1}{2} \int_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} |\mathbb{P}(s_t = ds) \pi_{\theta_{k-d_m}}(\tilde{a}_t = a | \tilde{s}_t = ds) - \mathbb{P}(\tilde{s}_t = ds) \pi_{\theta_{k-d_m}}(\tilde{a}_t = a | \tilde{s}_t = ds)| \\
 &= \frac{1}{2} \int_{s \in \mathcal{S}} \mathbb{P}(s_t = ds) \sum_{a \in \mathcal{A}} |\mathbb{E}[\pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) | s_t = ds] - \pi_{\theta_{k-d_m}}(a_t = a | s_t = ds)| \\
 &\quad + \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds) - \mathbb{P}(\tilde{s}_t = ds)|. \tag{32}
 \end{aligned}$$

Using Jensen's inequality, we have

$$\begin{aligned}
 & d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) \\
 &\leq \frac{1}{2} \int_{s \in \mathcal{S}} \mathbb{P}(s_t = ds) \sum_{a \in \mathcal{A}} \mathbb{E} \left[\left| \pi_{\theta_{k-\tau_k}}(a_t = a | s_t = ds) - \pi_{\theta_{k-d_m}}(a_t = a | s_t = ds) \right| \middle| s_t = ds \right] \\
 &\quad + \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds) - \mathbb{P}(\tilde{s}_t = ds)| \\
 &\leq \frac{1}{2} \int_{s \in \mathcal{S}} \mathbb{P}(s_t = ds) \sum_{a \in \mathcal{A}} \mathbb{E} [\|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 | s_t = ds] + \frac{1}{2} \int_{s \in \mathcal{S}} |\mathbb{P}(s_t = ds) - \mathbb{P}(\tilde{s}_t = ds)| \\
 &= \frac{1}{2} |\mathcal{A}| L_\pi \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 + d_{TV}(\mathbb{P}(s_t \in \cdot), \mathbb{P}(\tilde{s}_t \in \cdot)) \tag{33}
 \end{aligned}$$

where the last inequality follows Assumption 3.

Now we start to prove (28). First we have

$$\begin{aligned}
& d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) \\
& \stackrel{(33)}{\leq} d_{TV}(\mathbb{P}(s_t \in \cdot), \mathbb{P}(\tilde{s}_t \in \cdot)) + \frac{1}{2} |\mathcal{A}| L_\pi \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 \\
& \stackrel{(29)}{\leq} d_{TV}(\mathbb{P}(x_{t-1} \in \cdot), \mathbb{P}(\tilde{x}_{t-1} \in \cdot)) + \frac{1}{2} |\mathcal{A}| L_\pi \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 \\
& \stackrel{(30)}{=} d_{TV}(\mathbb{P}((s_{t-1}, a_{t-1}) \in \cdot), \mathbb{P}((\tilde{s}_{t-1}, \tilde{a}_{t-1}) \in \cdot)). \tag{34}
\end{aligned}$$

Since $d_{TV}(\mathbb{P}((s_{t-m}, a_{t-m}) \in \cdot), \mathbb{P}((\tilde{s}_{t-m}, \tilde{a}_{t-m}) \in \cdot)) = 0$, recursively applying (34) for $\{t-1, \dots, t-m\}$ gives

$$\begin{aligned}
d_{TV}(\mathbb{P}((s_t, a_t) \in \cdot), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot)) & \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{j=0}^m \mathbb{E} \|\theta_{k-d_j} - \theta_{k-d_m}\|_2 \\
& \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2, \tag{35}
\end{aligned}$$

which completes the proof. $\blacksquare$

A.3. Lipschitz continuity of actor and critic

We first give a proposition regarding the L_λ -Lipschitz continuity of the regularized policy gradient under proper assumptions, which has been shown by (Zhang et al., 2019; Agarwal et al., 2020).

Proposition 1 *Suppose Assumption 3 hold. For any $\theta, \theta' \in \mathbb{R}^d$, we have $\|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \leq L_\lambda \|\theta - \theta'\|_2$, where L_λ is a positive constant.*

We provide a justification for Lipschitz continuity of ω_θ^* in the next proposition.

Proposition 2 *Suppose Assumption 1, 3 and 4 hold. For any $\theta_1, \theta_2 \in \mathbb{R}^d$, we have*

$$\|\omega_{\theta_1}^* - \omega_{\theta_2}^*\|_2 \leq L_\omega \|\theta_1 - \theta_2\|_2,$$

where $L_\omega := 2r_{\max} |\mathcal{A}| L_\pi (\lambda^{-1} + \lambda^{-2}(1 + \gamma))(1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$.

Proof We use A_1, A_2, b_1 and b_2 as shorthand notations of $A_{\pi_{\theta_1}}, A_{\pi_{\theta_2}}, b_{\pi_{\theta_1}}$ and $b_{\pi_{\theta_2}}$ respectively. By Assumption 1, $A_{\theta, \phi}$ is invertible for any $\theta \in \mathbb{R}^d$, so we can write $\omega_\theta^* = -A_{\theta, \phi}^{-1} b_{\theta, \phi}$. Then we have

$$\begin{aligned}
\|\omega_1^* - \omega_2^*\|_2 &= \|-A_1^{-1} b_1 + A_2^{-1} b_2\|_2 \\
&= \|-A_1^{-1} b_1 - A_1^{-1} b_2 + A_1^{-1} b_2 + A_2^{-1} b_2\|_2 \\
&= \|-A_1^{-1} (b_1 - b_2) - (A_1^{-1} - A_2^{-1}) b_2\|_2 \\
&\leq \|A_1^{-1} (b_1 - b_2)\|_2 + \|(A_1^{-1} - A_2^{-1}) b_2\|_2 \\
&\leq \|A_1^{-1}\|_2 \|b_1 - b_2\|_2 + \|A_1^{-1} - A_2^{-1}\|_2 \|b_2\|_2 \\
&= \|A_1^{-1}\|_2 \|b_1 - b_2\|_2 + \|A_1^{-1} (A_2 - A_1) A_2^{-1}\|_2 \|b_2\|_2 \\
&\leq \|A_1^{-1}\|_2 \|b_1 - b_2\|_2 + \|A_1^{-1}\|_2 \|A_2^{-1}\|_2 \|b_2\|_2 \|A_2 - A_1\|_2 \\
&\leq \lambda^{-1} \|b_1 - b_2\|_2 + \lambda^{-2} r_{\max} \|A_1 - A_2\|_2, \tag{36}
\end{aligned}$$

where the last inequality follows Assumption 1, and the fact that

$$\|b_2\|_2 = \|\mathbb{E}[r(s, a, s')\phi(s)]\|_2 \leq \mathbb{E}\|r(s, a, s')\phi(s)\|_2 \leq \mathbb{E}[\|r(s, a, s')\|\|\phi(s)\|_2] \leq r_{\max}.$$

Denote (s^1, a^1, s'^1) and (s^2, a^2, s'^2) as samples drawn with θ_1 and θ_2 respectively, i.e. $s^1 \sim \mu_{\theta_1}$, $a^1 \sim \pi_{\theta_1}$, $s'^1 \sim \mathcal{P}$ and $s^2 \sim \mu_{\theta_2}$, $a^2 \sim \pi_{\theta_2}$, $s'^2 \sim \mathcal{P}$. Then we have

$$\begin{aligned} \|b_1 - b_2\|_2 &= \|\mathbb{E}[r(s^1, a^1, s'^1)\phi(s^1)] - \mathbb{E}[r(s^2, a^2, s'^2)\phi(s^2)]\|_2 \\ &\leq \sup_{s, a, s'} \|r(s, a, s')\phi(s)\|_2 \|\mathbb{P}((s^1, a^1, s'^1) \in \cdot) - \mathbb{P}((s^2, a^2, s'^2) \in \cdot)\|_{TV} \\ &\leq r_{\max} \|\mathbb{P}((s^1, a^1, s'^1) \in \cdot) - \mathbb{P}((s^2, a^2, s'^2) \in \cdot)\|_{TV} \\ &= 2r_{\max} d_{TV}(\mu_{\theta_1} \otimes \pi_{\theta_1} \otimes \mathcal{P}, \mu_{\theta_2} \otimes \pi_{\theta_2} \otimes \mathcal{P}) \\ &\leq 2r_{\max} |\mathcal{A}| L_{\pi} (1 + \log_{\rho} \kappa^{-1} + (1 - \rho)^{-1}) \|\theta_1 - \theta_2\|_2, \end{aligned} \quad (37)$$

where the first inequality follows the definition of total variation (TV) norm, and the last inequality follows Lemma A.1. in (Wu et al., 2020). Similarly we have:

$$\begin{aligned} \|A_1 - A_2\|_2 &\leq 2(1 + \gamma) d_{TV}(\mu_{\theta_1} \otimes \pi_{\theta_1}, \mu_{\theta_2} \otimes \pi_{\theta_2}) \\ &= (1 + \gamma) |\mathcal{A}| L_{\pi} (1 + \log_{\rho} \kappa^{-1} + (1 - \rho)^{-1}) \|\theta_1 - \theta_2\|_2. \end{aligned} \quad (38)$$

Substituting (37) and (38) into (36) completes the proof. $\blacksquare$

Appendix B. Proof of Main Theorems

B.1. Proof of Theorem 1

We first define the exact TD update as:

$$\bar{g}(x, \omega) := \mathbb{E}_{s \sim \mu_{\theta}, a \sim \pi_{\theta}, s' \sim \mathcal{P}} [g(x, \omega)]. \quad (39)$$

We also define constant $C_{\delta} := r_{\max} + (1 + \gamma) \max\{\frac{r_{\max}}{1 - \gamma}, R_{\omega}\}$, and we immediately have

$$\|g(x, \omega)\|_2 \leq |r(x) + \gamma \phi(s')^{\top} \omega - \phi(s)^{\top} \omega| \leq r_{\max} + (1 + \gamma) R_{\omega} \leq C_{\delta} \quad (40)$$

and likewise, we have $\|\bar{g}(x, \omega)\|_2 \leq C_{\delta}$.

The critic update in Algorithm 1 can be written as:

$$\omega_{k+1} = \Pi_{R_{\omega}}(\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k})), \quad (41)$$

where τ_k is the delay of the parameters used in evaluating the k th stochastic gradient, and $x_{(k)} := (s_{(k)}, a_{(k)}, s'_{(k)})$ is the sample used to evaluate the stochastic gradient at k th update.

Proof Using ω_k^* as shorthand notation of $\omega_{\theta_k}^*$, we start with the optimality gap

$$\begin{aligned}
& \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 \\
&= \|\Pi_{R_\omega}(\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k})) - \omega_{k+1}^*\|_2^2 \\
&\leq \|\omega_k + \beta g(x_{(k)}, \omega_{k-\tau_k}) - \omega_{k+1}^*\|_2^2 \\
&= \|\omega_k - \omega_k^*\|_2^2 + 2\beta \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle + 2\langle \omega_k - \omega_k^*, \omega_k^* - \omega_{k+1}^* \rangle \\
&\quad + \|\omega_k^* - \omega_{k+1}^* + \beta g(x_{(k)}, \omega_{k-\tau_k})\|_2^2 \\
&= \|\omega_k - \omega_k^*\|_2^2 + 2\beta \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle \\
&\quad + 2\langle \omega_k - \omega_k^*, \omega_k^* - \omega_{k+1}^* \rangle + 2\|\omega_k^* - \omega_{k+1}^*\|_2^2 + 2C_\delta^2 \beta^2.
\end{aligned} \tag{42}$$

The second term in (42) can be decomposed as

$$\begin{aligned}
\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle &= \langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) \rangle + \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) - g(x_{(k)}, \omega_k) \rangle \\
&\quad + \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle.
\end{aligned} \tag{43}$$

We first bound $\langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) \rangle$ in (43) as

$$\begin{aligned}
\langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) \rangle &= \langle \omega_k - \omega_k^*, \bar{g}(\theta_k, \omega_k) - \bar{g}(\theta_k, \omega_k^*) \rangle \\
&= \left\langle \omega_k - \omega_k^*, \mathbb{E} \left[(\gamma \phi(s') - \phi(s))^\top (\omega_k - \omega_k^*) \phi(s) \right] \right\rangle \\
&= \left\langle \omega_k - \omega_k^*, \mathbb{E} \left[\phi(s) (\gamma \phi(s') - \phi(s))^\top (\omega_k - \omega_k^*) \right] \right\rangle \\
&= \left\langle \omega_k - \omega_k^*, A_{\pi_{\theta_k}} (\omega_k - \omega_k^*) \right\rangle \\
&\leq -\lambda \|\omega_k - \omega_k^*\|_2^2,
\end{aligned} \tag{44}$$

where the first equality is due to $\bar{g}(\theta, \omega_\theta^*) = A_{\theta, \phi} \omega_\theta^* + b = 0$, and the last inequality follows Assumption 1.

We then bound the term $\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) - g(x_{(k)}, \omega_k) \rangle$ in (43) as

$$\begin{aligned}
\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) - g(x_{(k)}, \omega_k) \rangle &= \left\langle \omega_k - \omega_k^*, \left(\gamma \phi(s'_{(k)}) - \phi(s_{(k)}) \right)^\top (\omega_{k-\tau_k} - \omega_k) \phi(s_{(k)}) \right\rangle \\
&\leq (1 + \gamma) \|\omega_k - \omega_k^*\|_2 \|\omega_{k-\tau_k} - \omega_k\|_2 \\
&\leq (1 + \gamma) \|\omega_k - \omega_k^*\|_2 \left\| \sum_{i=k-\tau_k}^{k-1} (\omega_{i+1} - \omega_i) \right\|_2 \\
&\leq (1 + \gamma) \|\omega_k - \omega_k^*\|_2 \sum_{i=k-\tau_k}^{k-1} \beta \|g(x_i, \omega_{i-\tau_i})\|_2 \\
&\leq 2C_\delta K_0 \beta \|\omega_k - \omega_k^*\|_2,
\end{aligned} \tag{45}$$

where the last inequality follows the definition of C_δ in (40). Substituting (45) and (44) into (43) gives

$$\begin{aligned}
\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_{k-\tau_k}) \rangle &\leq -\lambda \|\omega_k - \omega_k^*\|_2^2 + 2C_\delta K_0 \beta \|\omega_k - \omega_k^*\|_2 \\
&\quad + \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle.
\end{aligned} \tag{46}$$

Next we jointly bound the third and fourth term in (42) as

$$\begin{aligned}
 & \langle \omega_k - \omega_k^*, \omega_k^* - \omega_{k+1}^* \rangle + \|\omega_k^* - \omega_{k+1}^*\|_2^2 \\
 & \leq \|\omega_k - \omega_k^*\|_2 \|\omega_k^* - \omega_{k+1}^*\|_2 + \|\omega_k^* - \omega_{k+1}^*\|_2^2 \\
 & \leq 2L_\omega \|\omega_k - \omega_k^*\|_2 \|\theta_k - \theta_{k+1}\|_2 + 2L_\omega^2 \|\theta_k - \theta_{k+1}\|_2^2 \\
 & \leq 2L_\omega C_p \alpha \|\omega_k - \omega_k^*\|_2 + 2L_\omega^2 C_p^2 \alpha^2,
 \end{aligned} \tag{47}$$

where constant $C_p := C_\delta C_\psi + \lambda C_\psi$. The second inequality is due to the L_ω -Lipschitz continuity of ω_θ^* shown in Proposition 2, and the last inequality follows the fact that

$$\|\theta_k - \theta_{k+1}\|_2 = \alpha \|\hat{\delta}(x_{(k)}, \omega_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) + \lambda \psi_{\theta_{k-\tau_k}}(x_{(k)}^p)\|_2 \leq \alpha C_p. \tag{48}$$

Substituting (46) and (47) into (42), and taking expectation on both sides yield

$$\begin{aligned}
 \mathbb{E} \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 & \leq (1 - 2\lambda\beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 2\beta(C_1 \frac{\alpha}{\beta} + C_2 K_0 \beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2 \\
 & \quad + 2\beta \mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle + C_q \beta^2,
 \end{aligned} \tag{49}$$

where $C_1 := L_\omega C_p$, $C_2 := 2C_\delta$ and $C_q := 2C_\delta^2 + 2L_\omega^2 C_p^2 \frac{\alpha^2}{\beta^2}$.

For brevity, we use $x \sim \theta$ to denote $s \sim \mu_\theta$, $a \sim \pi_\theta$ and $s' \sim \mathcal{P}$ in this proof. Consider the third term in (49) conditioned on $\theta_k, \omega_k, \theta_{k-\tau_k}$. We bound it as

$$\begin{aligned}
 & \mathbb{E} [\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle | \theta_k, \omega_k, \theta_{k-\tau_k}] \\
 & = \left\langle \omega_k - \omega_k^*, \mathbb{E}_{x_{(k)} \sim \theta_{k-\tau_k}} [g(x_{(k)}, \omega_k) | \omega_k] - \bar{g}(\theta_k, \omega_k) \right\rangle \\
 & = \left\langle \omega_k - \omega_k^*, \bar{g}(\theta_{k-\tau_k}, \omega_k) - \bar{g}(\theta_k, \omega_k) \right\rangle \\
 & \leq \|\omega_k - \omega_k^*\|_2 \|\bar{g}(\theta_{k-\tau_k}, \omega_k) - \bar{g}(\theta_k, \omega_k)\|_2 \\
 & \leq 2R_\omega \left\| \mathbb{E}_{x \sim \theta_{k-\tau_k}} [g(x, \omega_k)] - \mathbb{E}_{x \sim \theta_k} [g(x, \omega_k)] \right\|_2 \\
 & \leq 2R_\omega \sup_x \|g(x, \omega_k)\|_2 \left\| \mu_{\theta_{k-\tau_k}} \otimes \pi_{\theta_{k-\tau_k}} \otimes \mathcal{P} - \mu_{\theta_k} \otimes \pi_{\theta_k} \otimes \mathcal{P} \right\|_{TV} \\
 & \leq 4R_\omega C_\delta d_{TV}(\mu_{\theta_{k-\tau_k}} \otimes \pi_{\theta_{k-\tau_k}} \otimes \mathcal{P}, \mu_{\theta_k} \otimes \pi_{\theta_k} \otimes \mathcal{P}),
 \end{aligned} \tag{50}$$

where second last inequality follows the definition of TV norm and the last inequality uses the definition of C_δ in (40).

Define constant $C_3 := 2R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$. Then by following the third item in (Wu et al., 2020, Lemma A.1), we can write (50) as

$$\begin{aligned}
 & \mathbb{E} [\langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle | \theta_k, \omega_k, \theta_{k-\tau_k}] \\
 & \leq 4R_\omega C_\delta d_{TV}(\mu_{\theta_{k-\tau_k}} \otimes \pi_{\theta_{k-\tau_k}} \otimes \mathcal{P}, \mu_{\theta_k} \otimes \pi_{\theta_k} \otimes \mathcal{P}) \\
 & \leq C_3 \|\theta_{k-\tau_k} - \theta_k\|_2 \\
 & \leq C_3 \sum_{i=k-\tau_k}^{k-1} \alpha \|g(x_i, \omega_{i-\tau_i})\|_2 \\
 & \leq C_3 C_\delta K_0 \alpha,
 \end{aligned} \tag{51}$$

Taking total expectation on both sides of (51) and substituting it into (49) yield

$$\begin{aligned} \mathbb{E}\|\omega_{k+1} - \omega_{k+1}^*\|_2^2 &\leq (1 - 2\lambda\beta)\mathbb{E}\|\omega_k - \omega_k^*\|_2^2 + 2\beta\left(C_1\frac{\alpha}{\beta} + C_2K_0\beta\right)\mathbb{E}\|\omega_k - \omega_k^*\|_2 \\ &\quad + 2C_3C_\delta K_0\beta\alpha + C_q\beta^2. \end{aligned} \quad (52)$$

which along with the fact $\alpha = \frac{1}{(K+1)^{\frac{3}{5}}}$ and $\beta = \frac{1}{(K+1)^{\frac{2}{5}}}$ implies

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E}\|\omega_k - \omega_k^*\|_2^2 = \mathcal{O}\left(\frac{K_0^2}{K^{\frac{4}{5}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{3}{5}}}\right) + \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right). \quad (53)$$

This completes the proof. ■

B.2. Proof of Theorem 2

We first define the ‘optimal’ TD target as:

$$\delta(x, \theta) := r(s, a, s') + \gamma V_{\pi_\theta}(s') - V_{\pi_\theta}(s).$$

The update in Algorithm 1 can be written as:

$$\theta_{k+1} = \theta_k + \alpha(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k})\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda\psi_{\theta_{k-\tau_k}}(x_{(k)}^p)). \quad (54)$$

For brevity, we use ω_k^* as shorthand notation of $\omega_{\theta_k}^*$ in this proof. Then we are ready to give the convergence proof.

Proof From L_λ -Lipschitz continuity of regularized policy gradient shown in Proposition 1, we have:

$$\begin{aligned} J_\lambda(\theta_{k+1}) &\geq J_\lambda(\theta_k) + \langle \nabla J_\lambda(\theta_k), \theta_{k+1} - \theta_k \rangle - \frac{L_\lambda}{2}\|\theta_{k+1} - \theta_k\|_2^2 \\ &= J_\lambda(\theta_k) + \alpha \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\quad + \alpha \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda\psi_{\theta_{k-\tau_k}}(x_{(k)}^p) \right\rangle - \frac{L_\lambda}{2}\|\theta_{k+1} - \theta_k\|_2^2 \\ &\geq J_\lambda(\theta_k) + \alpha \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\quad + \alpha \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda\psi_{\theta_{k-\tau_k}}(x_{(k)}^p) \right\rangle - \frac{L_\lambda}{2}C_p^2\alpha^2, \end{aligned}$$

where the last inequality follows the definition of C_p in (48).

Taking expectation on both sides of the last inequality yields

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1} \\ &\quad + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda\psi_{\theta_{k-\tau_k}}(x_{(k)}^p) \right\rangle}_{I_2} - \frac{L_\lambda}{2}C_p^2\alpha^2. \end{aligned} \quad (55)$$

where we used the fact that $\mathbb{E}[\psi_{\theta_{k-\tau_k}}(x_{(k)}^p)|\theta_{k-\tau_k}] = \nabla R(\theta_{k-\tau_k})$.

We first decompose I_1 as

$$\begin{aligned} I_1 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &= \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1^{(1)}} \\ &\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1^{(2)}}. \end{aligned} \quad (56)$$

We bound $I_1^{(1)}$ as

$$\begin{aligned} I_1^{(1)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top (\omega_{k-\tau_k} - \omega_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \|\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)})\|_2 \|\omega_k - \omega_{k-\tau_k}\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\ &\geq -(1+\gamma) C_\psi \mathbb{E} [\|\nabla J_\lambda(\theta_k)\|_2 \|\omega_k - \omega_{k-\tau_k}\|_2] \\ &\geq -(1+\gamma) C_\psi C_\delta K_0 \beta \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2, \end{aligned} \quad (57)$$

where the last inequality follows

$$\begin{aligned} \|\omega_k - \omega_{k-\tau_k}\|_2 &= \left\| \sum_{i=k-\tau_k}^{k-1} (\omega_{i+1} - \omega_i) \right\|_2 \\ &\leq \sum_{i=k-\tau_k}^{k-1} \|\beta g(x_i, \omega_{i-\tau_i})\|_2 \\ &\leq \beta K_0 C_\delta, \end{aligned} \quad (58)$$

where the second inequality is due to (40).

Then we bound $I_1^{(2)}$ as

$$\begin{aligned} I_1^{(2)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &= -\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top (\omega_k^* - \omega_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \|\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)})\|_2 \|\omega_k - \omega_k^*\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\ &\geq -(1+\gamma) C_\psi \mathbb{E} [\|\nabla J_\lambda(\theta_k)\|_2 \|\omega_k - \omega_k^*\|_2]. \end{aligned} \quad (59)$$

Collecting lower bounds of $I_1^{(1)}$ and $I_1^{(2)}$ gives

$$\begin{aligned} I_1 &\geq -2C_\psi \mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 (C_\delta K_0 \beta + \|\omega_k - \omega_k^*\|_2) \right] \\ &\geq -\frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 2C_\psi^2 \mathbb{E} \left[(C_\delta K_0 \beta + \|\omega_k - \omega_k^*\|_2)^2 \right] \\ &\geq -\frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \beta^2 - 4C_\psi^2 \mathbb{E} \|\omega_k - \omega_k^*\|_2^2, \end{aligned} \quad (60)$$

where the the second and third inequality follow Young's inequality.

Now we consider I_2 . We first decompose I_2 as

$$\begin{aligned}
I_2 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) \right\rangle \\
&= \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_2^{(1)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_2^{(2)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle}_{I_2^{(3)}} + \|\nabla J_\lambda(\theta_k)\|_2^2.
\end{aligned} \tag{61}$$

We bound $I_2^{(1)}$ as

$$\begin{aligned}
I_2^{(1)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
&= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top (\omega_k^* - \omega_{k-\tau_k}^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
&\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \left\| \left(\gamma \phi(\hat{s}'_{(k)}) - \phi(\hat{s}_{(k)}) \right)^\top \right\|_2 \|\omega_k^* - \omega_{k-\tau_k}^*\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\
&\geq -L_V C_\psi (1 + \gamma) \mathbb{E} \|\omega_k^* - \omega_{k-\tau_k}^*\|_2 \\
&\geq -L_V L_\omega C_\psi (1 + \gamma) \mathbb{E} \|\theta_k - \theta_{k-\tau_k}\|_2 \\
&\geq -L_V L_\omega C_\psi C_p (1 + \gamma) K_0 \alpha,
\end{aligned} \tag{62}$$

where $L_V := \frac{r_{\max}}{1-\gamma} C_\psi + \lambda C_\psi$ is the trivial upper bound of $\|\nabla J_\lambda(\theta)\|_2$ and $\|\nabla J(\theta)\|_2$. The second last inequality follows from Proposition 2 and the last inequality uses (48) as

$$\begin{aligned}
\|\theta_k - \theta_{k-\tau_k}\|_2 &\leq \sum_{i=k-\tau_k}^{k-1} \|\theta_{i+1} - \theta_i\|_2 \\
&= \sum_{i=k-\tau_k}^{k-1} \alpha \|\hat{\delta}(\hat{x}_i, \omega_{i-\tau_i}) \psi_{\theta_{i-\tau_i}}(\hat{s}_i, \hat{a}_i)\|_2 \\
&\leq \sum_{i=k-\tau_k}^{k-1} \alpha C_p \leq C_p K_0 \alpha.
\end{aligned} \tag{63}$$

We bound $I_2^{(2)}$ as

$$\begin{aligned}
 I_2^{(2)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\
 &\geq -\mathbb{E} \left[\|\nabla J_\lambda(\theta_k)\|_2 \left| \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right| \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \right] \\
 &\geq -L_V C_\psi \mathbb{E} \left| \hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}^*) - \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \right| \\
 &= -L_V C_\psi \mathbb{E} \left| \gamma \left(\phi(\hat{s}'_{(k)})^\top \omega_{k-\tau_k}^* - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right) + V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) - \phi(\hat{s}_{(k)})^\top \omega_{k-\tau_k}^* \right| \\
 &\geq -L_V C_\psi \left(\gamma \mathbb{E} \left| \phi(\hat{s}'_{(k)})^\top \omega_{k-\tau_k}^* - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right| + \mathbb{E} \left| V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) - \phi(\hat{s}_{(k)})^\top \omega_{k-\tau_k}^* \right| \right) \\
 &\geq -L_V C_\psi \left(\gamma \sqrt{\mathbb{E} \left| \phi(\hat{s}'_{(k)})^\top \omega_{k-\tau_k}^* - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right|^2} + \sqrt{\mathbb{E} \left| V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) - \phi(\hat{s}_{(k)})^\top \omega_{k-\tau_k}^* \right|^2} \right) \\
 &\geq -L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}.
 \end{aligned} \tag{64}$$

We bound $I_2^{(3)}$ as

$$\begin{aligned}
 I_2^{(3)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle \\
 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \mathbb{E} \left[\delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle \\
 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \mathbb{E}_{\substack{\hat{s}_{(k)} \sim d_{\theta_{k-\tau_k}} \\ \hat{a}_{(k)} \sim \pi_{\theta_{k-\tau_k}}}} \left[A_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right] + \lambda \nabla R(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle,
 \end{aligned} \tag{65}$$

where we used the fact that

$$\begin{aligned}
 &\mathbb{E} \left[\delta(\hat{x}_{(k)}, \theta_{k-\tau_k}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{\substack{\hat{s}_{(k)}, \hat{a}_{(k)} \sim d_{\theta_{k-\tau_k}} \\ \hat{s}'_{(k)} \sim \mathcal{P}}} \left[\left(r(\hat{s}_{(k)}, \hat{a}_{(k)}, \hat{s}'_{(k)}) + \gamma V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{d_{\theta_{k-\tau_k}}} \left[\left(\mathbb{E}_{\hat{s}'_{(k)} \sim \mathcal{P}} \left[r(\hat{s}_{(k)}, \hat{a}_{(k)}, \hat{s}'_{(k)}) + \gamma V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}'_{(k)}) \right] - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{d_{\theta_{k-\tau_k}}} \left[\left(Q_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - V_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right] \\
 &= \mathbb{E}_{d_{\theta_{k-\tau_k}}} \left[A_{\pi_{\theta_{k-\tau_k}}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \middle| \theta_{k-\tau_k}, \theta_k \right].
 \end{aligned}$$

Continuing from (65),

$$\begin{aligned}
 I_2^{(3)} &= \left\langle \nabla J_\lambda(\theta_k), \nabla J_\lambda(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k) \right\rangle \\
 &\geq -\|\nabla J_\lambda(\theta_k)\|_2 \|\nabla J_\lambda(\theta_{k-\tau_k}) - \nabla J_\lambda(\theta_k)\|_2 \\
 &\geq -L_V L_\lambda \|\theta_{k-\tau_k} - \theta_k\|_2 \geq -L_V L_\lambda C_p K_0 \alpha,
 \end{aligned} \tag{66}$$

where the second last inequality is due to L_λ -Lipschitz continuity of policy gradient shown in Proposition 1, and the last inequality follows (63).

Collecting lower bounds of $I_2^{(1)}$, $I_2^{(2)}$ and $I_2^{(3)}$ gives

$$I_2 \geq -D_1 K_0 \alpha - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \quad (67)$$

where constant $D_1 := L_V L_\omega C_\psi C_p (1 + \gamma) + L_V L_\lambda C_p$.

Substituting (60) and (67) into (55) yields

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \frac{\alpha}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 \alpha \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \alpha \beta^2 \\ &\quad - 2L_V C_\psi \epsilon_{\text{app}} \alpha - (D_1 K_0 + \frac{L_\lambda}{2} C_p^2) \alpha^2. \end{aligned} \quad (68)$$

After telescoping, we have

$$\begin{aligned} \sum_{k=1}^K \frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 &\leq \frac{1}{\alpha} (J^* - J_\lambda(\theta_{K_0})) + 4C_\psi^2 \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 4K C_\psi^2 C_\delta^2 K_0^2 \beta^2 \\ &\quad + 2KL_V C_\psi \epsilon_{\text{app}} + K(D_1 K_0 + \frac{L_\lambda}{2} C_p^2) \alpha. \end{aligned} \quad (69)$$

Select $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$, we have

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{1}{K^{\frac{2}{5}}}\right) + \mathcal{O}\left(\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2\right) + \mathcal{O}\left(\frac{K_0^2}{K^{\frac{4}{5}}}\right) + \mathcal{O}\left(\frac{K_0}{K^{\frac{3}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (70)$$

This completes the proof. $\blacksquare$

B.3. Proof of Theorem 3

Given the definition in Section B.1, we now give the convergence proof of critic update in Algorithm 1 with linear function approximation and Markovian sampling.

Proof By following the derivation of (49), we have

$$\begin{aligned} \mathbb{E} \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 &\leq (1 - 2\lambda\beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 2\beta \left(C_1 \frac{\alpha}{\beta} + C_2 K_0 \beta \right) \mathbb{E} \|\omega_k - \omega_k^*\|_2 \\ &\quad + 2\beta \mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle + C_q \beta^2, \end{aligned} \quad (71)$$

where $C_1 := C_p L_\omega$, $C_2 := C_\delta (1 + \gamma)$ and $C_q := 2C_\delta^2 + 2L_\omega^2 C_p^2 \max_{(k)} \frac{\alpha^2}{\beta^2}$.

Now we consider the third item in the last inequality. For some $m \in \mathbb{N}^+$, we define $M := (K_0 + 1)m + K_0$. Following Lemma 4 (to be presented in Section C.1), for some $d_m \leq M$ and positive constants C_4, C_5, C_6, C_7 , we have

$$\begin{aligned} &\mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle \\ &\leq C_4 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 + C_5 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 + C_6 \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2 + C_7 \kappa \rho^{m-1} \\ &\leq C_4 \sum_{i=k-d_m}^{k-1} \mathbb{E} \|\theta_{i+1} - \theta_i\|_2 + C_5 \sum_{i=\tau_k}^{d_m-1} \sum_{j=k-d_m}^{k-i-1} \mathbb{E} \|\theta_{j+1} - \theta_j\|_2 + C_6 \sum_{i=k-d_m}^{k-1} \mathbb{E} \|\omega_{i+1} - \omega_i\|_2 + C_7 \kappa \rho^{m-1} \\ &\leq C_4 d_m C_p \alpha + C_5 (d_m - \tau_k)^2 C_p \alpha + C_6 d_m C_\delta \beta + C_7 \kappa \rho^{m-1} \\ &\leq (C_4 M + C_5 M^2) C_p \alpha + C_6 M C_\delta \beta + C_7 \kappa \rho^{m-1}, \end{aligned} \quad (72)$$

where the last inequality is due to $\tau_k \geq 0$ and $d_m \leq M$.

Further letting $m = m_K$ which is defined in (27) yields

$$\begin{aligned} & \mathbb{E} \langle \omega_k - \omega_k^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle \\ &= (C_4 M_K + C_5 M_K^2) C_p \alpha + C_6 C_\delta M_K \beta + C_7 \kappa \rho^{m_K-1} \\ &\leq (C_4 M_K + C_5 M_K^2) C_p \alpha + C_6 C_\delta M_K \beta + C_7 \alpha, \end{aligned} \quad (73)$$

where $M_K = (K_0 + 1)m_K + K_0$, and the last inequality follows the from $m_K = \mathcal{O}(\log K)$.

Substituting (73) into (71) gives

$$\begin{aligned} \mathbb{E} \|\omega_{k+1} - \omega_{k+1}^*\|_2^2 &\leq (1 - 2\lambda\beta) \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 + 2\beta \left(C_1 \frac{\alpha}{\beta} + C_2 K_0 \beta \right) \mathbb{E} \|\omega_k - \omega_k^*\|_2 \\ &\quad + 2\beta \left((C_4 M_K + C_5 M_K^2) C_p \alpha + C_6 C_\delta M_K \beta + C_7 \alpha \right) + C_q \beta^2. \end{aligned} \quad (74)$$

Select $\alpha = K^{-\frac{3}{5}}$ and $\beta = K^{-\frac{2}{5}}$. After telescoping, we have

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 = \mathcal{O} \left(\frac{1}{K^{\frac{2}{5}}} \right) + \mathcal{O} \left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}} \right) + \mathcal{O} \left(\frac{K_0 \log K}{K^{\frac{2}{5}}} \right).$$

This completes the proof. ■

B.4. Proof of Theorem 4

Given the definition in section B.2, we now give the convergence proof of actor update in Algorithm 1 with linear value function approximation and Markovian sampling method.

Proof By following the derivation of (55), we have

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_{k-\tau_k}) - \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle}_{I_1} \\ &\quad + \underbrace{\alpha \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(\hat{x}_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) \right\rangle}_{I_2} - \frac{L_\lambda}{2} C_p^2 \alpha^2. \end{aligned} \quad (75)$$

The item I_1 can be bounded by following (60) as

$$I_1 \geq -\frac{1}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \beta^2 - 4C_\psi^2 \mathbb{E} \|\omega_k - \omega_k^*\|_2^2. \quad (76)$$

Next we consider I_2 . We first decompose it as

$$\begin{aligned}
I_2 &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \hat{\delta}(x_{(k)}, \omega_k^*) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) + \lambda \nabla R(\theta_{k-\tau_k}) \right\rangle \\
&= \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(x_{(k)}, \omega_k^*) - \delta(x_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) \right\rangle}_{I_2^{(1)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(x_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) - \nabla J(\theta_k) \right\rangle}_{I_2^{(2)}} \\
&\quad + \underbrace{\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \lambda \nabla R(\theta_{k-\tau_k}) - \lambda \nabla R(\theta_k) \right\rangle}_{I_2^{(3)}} + \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2. \tag{77}
\end{aligned}$$

For some $m \in \mathbb{N}^+$, define $M := (K_0 + 1)m + K_0$. Following Lemma 5, for some $d_m \leq M$ and positive constants $D_2, D_3, D_4, D_5, I_2^{(1)}$ can be bounded as

$$\begin{aligned}
I_2^{(1)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(x_{(k)}, \omega_k^*) - \delta(x_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) \right\rangle \\
&\geq -D_2 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_3 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_4 \sum_{i=k-d_m}^{k-\tau_k} \mathbb{E} \|\theta_i - \theta_{k-d_m}\|_2 \\
&\quad - D_5 \kappa \rho^{m-1} - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}} \\
&\geq -D_2 (d_m - \tau_k) C_p \alpha - D_3 d_m C_p \alpha - D_4 (d_m - \tau_k)^2 C_p \alpha \\
&\quad - D_5 \kappa \rho^{m-1} - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}}, \tag{78}
\end{aligned}$$

where the derivation of the last inequality is similar to that of (72). By setting $m = m_K$ in (78), and following the fact that $d_{m_K} \leq M_K$ and $\tau_k \geq 0$, we have

$$\begin{aligned}
I_2^{(1)} &\geq -D_2 M_K C_p \alpha - D_3 M_K C_p \alpha - D_4 M_K^2 C_p \alpha - D_5 \kappa \rho^{m_K-1} - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}} \\
&= -((D_2 + D_3) C_p M_K + D_4 C_p M_K^2) \alpha - D_5 \kappa \rho^{m_K-1} - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}} \\
&\geq -((D_2 + D_3) C_p M_K + D_4 C_p M_K^2) \alpha - D_5 \alpha - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}}, \tag{79}
\end{aligned}$$

where the last inequality is due to the definition of m_K .

Following Lemma 6, for some positive constants D_6, D_7 and D_8 , we bound $I_2^{(2)}$ as

$$\begin{aligned}
I_2^{(2)} &= \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(x_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(s_{(k)}, a_{(k)}) - \nabla J(\theta_k) \right\rangle \\
&\geq -D_6 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_7 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_8 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_9 \kappa \rho^{m-1}.
\end{aligned}$$

Similar to the derivation of (79), we have

$$I_2^{(2)} \geq -(D_6 C_p M_K + D_7 C_p M_K + D_8 C_p M_K^2) \alpha - D_9 \alpha. \tag{80}$$

Term $I_2^{(3)}$ can be bounded as

$$\begin{aligned} I_2^{(3)} &\geq -\lambda L_V \|\nabla R(\theta_k) - \nabla R(\theta_{k-\tau_k})\|_2 \geq -\lambda L_V L_\psi \|\theta_k - \theta_{k-\tau_k}\|_2 \\ &\geq -\lambda L_V L_\psi K_0 C_p \alpha. \end{aligned} \quad (81)$$

Collecting the lower bounds of $I_2^{(1)}$, $I_2^{(2)}$ and $I_2^{(3)}$ yields

$$I_2 \geq -D_K \alpha - (1 + \gamma) L_V C_\psi \epsilon_{\text{app}} + \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2, \quad (82)$$

where we define $D_K := C_p(D_4 + D_8)M_K^2 + C_p(D_2 + D_3 + D_6 + D_7)M_K + \lambda L_V L_\psi K_0 C_p + D_5 + D_9$ for brevity.

Substituting (76) and (82) into (75) yields

$$\begin{aligned} \mathbb{E}[J_\lambda(\theta_{k+1})] &\geq \mathbb{E}[J_\lambda(\theta_k)] + \frac{\alpha}{2} \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 - 4C_\psi^2 \alpha \mathbb{E} \|\omega_k - \omega_k^*\|_2^2 - 4C_\psi^2 C_\delta^2 K_0^2 \alpha \beta^2 \\ &\quad - 2L_V C_\psi \epsilon_{\text{app}} \alpha - D_K \alpha^2. \end{aligned} \quad (83)$$

Choose step size $\alpha = K^{-\frac{3}{5}}$, $\beta = K^{-\frac{2}{5}}$. With $D_K = \mathcal{O}(M_K^2) = \mathcal{O}(K_0^2 \log^2 K)$, the last inequality implies

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|_2^2 = \mathcal{O}\left(\frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\omega_k - \omega_k^*\|_2^2\right) + \mathcal{O}\left(\frac{K_0^2 \log^2 K}{K^{\frac{3}{5}}}\right) + \mathcal{O}(\epsilon_{\text{app}}). \quad (84)$$

This completes the proof. ■

B.5. Proof of Theorem 5

Proof We define an event E_k as $\|\nabla J_\lambda(\theta_k)\| \leq \frac{\lambda}{2|\mathcal{S}||\mathcal{A}|}$ and its complement E_k^c as $\|\nabla J_\lambda(\theta_k)\| > \frac{\lambda}{2|\mathcal{S}||\mathcal{A}|}$. We use $\mathbf{1}_{E_k}$ to indicate whether the event happens or not, i.e. $\mathbf{1}_{E_k} = 1$ if E_k happens and $\mathbf{1}_{E_k} = 0$ if E_k^c happens. Then we have for the optimality gap:

$$\begin{aligned} \sum_{k=1}^K \mathbb{E}[J^* - J(\theta_k)] &= \sum_{k=1}^K \mathbb{E}[(J^* - J(\theta_k))\mathbf{1}_{E_k}] + \sum_{k=1}^K \mathbb{E}[(J^* - J(\theta_k))\mathbf{1}_{E_k^c}] \\ &\leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k}] + \sum_{k=1}^K \mathbb{E}[(J^* - J(\theta_k))\mathbf{1}_{E_k^c}] \\ &\leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k}] + J^* \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k^c}] \\ &\leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty K + J^* \sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k^c}], \end{aligned} \quad (85)$$

where the first inequality follows from Lemma 1.

Now it suffices to bound $\sum_{k=1}^K \mathbb{E}[\mathbf{1}_{E_k^c}]$.

$$\begin{aligned} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|^2 &\geq \sum_{k=1}^K \mathbb{E} [\|\nabla J_\lambda(\theta_k)\|^2 \mathbf{1}_{E_k^c}] \\ &\geq \sum_{k=1}^K \frac{\lambda^2}{4|\mathcal{S}|^2|\mathcal{A}|^2} \mathbb{E}[\mathbf{1}_{E_k^c}] \end{aligned} \quad (86)$$

Substituting the above inequality into (85) and dividing both sides by K give

$$\frac{1}{K} \sum_{k=1}^K \mathbb{E}[J^* - J(\theta_k)] \leq \frac{2\lambda}{1-\gamma} \left\| \frac{d_{\pi^*}}{\eta} \right\|_\infty + \frac{4|\mathcal{S}|^2|\mathcal{A}|^2}{\lambda^2} \frac{1}{K} \sum_{k=1}^K \mathbb{E} \|\nabla J_\lambda(\theta_k)\|^2 \quad (87)$$

It is known that the softmax policy satisfies Assumption 3, thus we immediately know that Theorem 2 and Theorem 4 hold. Furthermore, by assumption 5, we have $\phi(s)^\top \omega_\theta^* = V_{\pi_\theta}(s)$, leading to $\epsilon_{\text{app}} = 0$. Applying Theorem 2 and 4 to (87) completes the proof. $\blacksquare$

Appendix C. Supporting Lemmas

C.1. Supporting Lemmas for Theorem 3

Lemma 4 For any $m \geq 1$ and $k \geq (K_0 + 1)m + K_0 + 1$, we have

$$\begin{aligned} \mathbb{E} \langle \omega_k - \omega_{\theta_k}^*, g(x_{(k)}, \omega_k) - \bar{g}(\theta_k, \omega_k) \rangle &\leq C_4 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 + C_5 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 \\ &\quad + C_6 \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2 + C_7 \kappa \rho^{m-1}, \end{aligned}$$

where constant $d_m \leq (K_0 + 1)m + K_0$, and $C_4 := 2C_\delta L_\omega + 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$, $C_5 := 4R_\omega C_\delta |\mathcal{A}| L_\pi$ and $C_6 := 4(1 + \gamma)R_\omega + 2C_\delta$, $C_7 := 8R_\omega C_\delta$.

Proof Consider the collection of random samples $\{x_{(k-K_0-1)}, x_{(k-K_0)}, \dots, x_{(k)}\}$. Suppose $x_{(k)}$ is sampled by worker n , then due to Assumption 2, $\{x_{(k-K_0-1)}, x_{(k-K_0)}, \dots, x_{(k-1)}\}$ will contain at least another sample drawn by worker n . Therefore, $\{x_{(k-(K_0+1)m)}, x_{(k-(K_0+1)m+1)}, \dots, x_{(k-1)}\}$ will contain at least m samples from worker n .

Consider the Markov chain formed by $m+1$ samples in $\{x_{(k-(K_0+1)m)}, x_{(k-(K_0+1)m+1)}, \dots, x_{(k)}\}$:

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_m-1}} a_{t-m+1} \cdots s_{t-1} \xrightarrow{\theta_{k-d_1}} a_{t-1} \xrightarrow{\mathcal{P}} s_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\mathcal{P}} s_{t+1},$$

where $(s_t, a_t, s_{t+1}) = (s_{(k)}, a_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$.

Suppose θ_{k-d_m} was used to generate the k_m th update, then we have $x_{t-m} = x_{(k_m)}$. Following Assumption 2, we have $\tau_{k_m} = k_m - (k - d_m) \leq K_0$. Since $x_{(k_m)}$ is in $\{x_{(k-(K_0+1)m)}, \dots, x_{(k)}\}$, we have $k_m \geq k - (K_0 + 1)m$. Combining these two inequalities, we have

$$d_m \leq (K_0 + 1)m + K_0. \quad (88)$$

Given $(s_{t-m}, a_{t-m}, s_{t-m+1})$ and θ_{k-d_m} , we construct an auxiliary Markov chain as Lemma 3:

$$s_{t-m} \xrightarrow{\theta_{k-d_m}} a_{t-m} \xrightarrow{\mathcal{P}} s_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\mathcal{P}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\mathcal{P}} \tilde{s}_{t+1}.$$

For brevity, we define

$$\Delta_1(x, \theta, \omega) := \langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle.$$

Throughout this proof, we use $\theta, \theta', \omega, \omega', x$ and $\tilde{x}$ as shorthand notations of $\theta_k, \theta_{k-d_m}, \omega_k, \omega_{k-d_m}, x_t$ and $\tilde{x}_t := (\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1})$, respectively.

First we decompose $\Delta_1(x, \theta, \omega)$ as

$$\begin{aligned} \Delta_1(x, \theta, \omega) &= \underbrace{\Delta_1(x, \theta, \omega) - \Delta_1(x, \theta', \omega)}_{I_1} + \underbrace{\Delta_1(x, \theta', \omega) - \Delta_1(x, \theta', \omega')}_{I_2} \\ &\quad + \underbrace{\Delta_1(x, \theta', \omega') - \Delta_1(\tilde{x}, \theta', \omega')}_{I_3} + \underbrace{\Delta_1(\tilde{x}, \theta', \omega')}_{I_4}. \end{aligned} \quad (89)$$

We bound I_1 in (89) as

$$\begin{aligned} \Delta_1(x, \theta, \omega) - \Delta_1(x, \theta', \omega) &= \langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle \\ &\leq |\langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle| \\ &\quad + |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle|. \end{aligned} \quad (90)$$

For the first term in (90), we have

$$\begin{aligned} |\langle \omega - \omega_\theta^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle| &= |\langle \omega_\theta^* - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle| \\ &\leq \|\omega_\theta^* - \omega_{\theta'}^*\|_2 \|g(x, \omega) - \bar{g}(\theta, \omega)\| \\ &\leq 2C_\delta \|\omega_\theta^* - \omega_{\theta'}^*\|_2 \\ &\leq 2C_\delta L_\omega \|\theta - \theta'\|_2, \end{aligned} \quad (91)$$

where the last inequality is due to Proposition 2.

We use $x \sim \theta'$ as shorthand notations to represent that $s \sim \mu_{\theta'}$, $a \sim \pi_{\theta'}$, $s' \sim \mathcal{P}$. For the second term in (90), we have

$$\begin{aligned} &|\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta, \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle| \\ &= |\langle \omega - \omega_{\theta'}^*, \bar{g}(\theta', \omega) - \bar{g}(\theta, \omega) \rangle| \\ &\leq \|\omega - \omega_{\theta'}^*\|_2 \|\bar{g}(\theta', \omega) - \bar{g}(\theta, \omega)\|_2 \\ &\leq 2R_\omega \|\bar{g}(\theta', \omega) - \bar{g}(\theta, \omega)\|_2 \\ &= 2R_\omega \|\mathbb{E}_{x \sim \theta'} [g(x, \omega)] - \mathbb{E}_{x \sim \theta} [g(x, \omega)]\|_2 \\ &\leq 2R_\omega \sup_x \|g(x, \omega)\|_2 \|\mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P} - \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}\|_{TV} \\ &\leq 2R_\omega C_\delta \|\mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P} - \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}\|_{TV} \\ &= 4R_\omega C_\delta d_{TV}(\mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}, \mu_\theta \otimes \pi_\theta \otimes \mathcal{P}) \\ &\leq 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1}) \|\theta - \theta'\|_2, \end{aligned} \quad (92)$$

where the third inequality follows the definition of TV norm, the second last inequality follows (40), and the last inequality follows (Wu et al., 2020, Lemma A.1).

Collecting the upper bounds of the two terms in (90) yields

$$I_1 \leq [2C_\delta L_\omega + 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})] \|\theta - \theta'\|_2. \quad (93)$$

Next we bound $\mathbb{E}[I_2]$ in (89) as

$$\begin{aligned} \mathbb{E}[I_2] &= \mathbb{E}[\Delta_1(x, \theta', \omega) - \Delta_1(x, \theta', \omega')] \\ &= \mathbb{E} \langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle - \langle \omega' - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle \\ &\leq \mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &\quad + \mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle - \langle \omega' - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle|. \end{aligned} \quad (94)$$

We bound the first term in (94) as

$$\begin{aligned} &\mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - \bar{g}(\theta', \omega) \rangle - \langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &= \mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega) - g(x, \omega') + \bar{g}(\theta', \omega') - \bar{g}(\theta', \omega) \rangle| \\ &\leq 2R_\omega (\mathbb{E} \|g(x, \omega) - g(x, \omega')\|_2 + \mathbb{E} \|\bar{g}(\theta', \omega') - \bar{g}(\theta', \omega)\|_2) \\ &\leq 2R_\omega (\mathbb{E} \|g(x, \omega) - g(x, \omega')\|_2 + \mathbb{E} \|\mathbb{E}_{x \sim \theta'}[g(x, \omega')] - \mathbb{E}_{x \sim \theta'}[g(x, \omega)]\|_2) \\ &= 2R_\omega (\mathbb{E} \|(\gamma \phi(s') - \phi(s))^\top (\omega - \omega')\|_2 + \mathbb{E} \|\mathbb{E}_{x \sim \theta'}[(\gamma \phi(s') - \phi(s))^\top] (\omega' - \omega)\|_2) \\ &\leq 2R_\omega ((1 + \gamma) \mathbb{E} \|\omega - \omega'\|_2 + (1 + \gamma) \mathbb{E} \|\omega - \omega'\|_2) \\ &= 4R_\omega (1 + \gamma) \mathbb{E} \|\omega - \omega'\|_2. \end{aligned} \quad (95)$$

We bound the second term in (94) as

$$\begin{aligned} &\mathbb{E} |\langle \omega - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle - \langle \omega' - \omega_{\theta'}^*, g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &= \mathbb{E} |\langle \omega - \omega', g(x, \omega') - \bar{g}(\theta', \omega') \rangle| \\ &\leq 2C_\delta \mathbb{E} \|\omega - \omega'\|_2. \end{aligned} \quad (96)$$

Collecting the upper bounds of the two terms in (94) yields

$$\mathbb{E}[I_2] \leq (4(1 + \gamma)R_\omega + 2C_\delta) \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2. \quad (97)$$

We first bound I_3 as

$$\begin{aligned} \mathbb{E}[I_3 | \theta', \omega', s_{t-m+1}] &= \mathbb{E} [\Delta_1(x, \theta', \omega') - \Delta_1(\tilde{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] \\ &\leq |\mathbb{E} [\Delta_1(x, \theta', \omega') | \theta', \omega', s_{t-m+1}] - \mathbb{E} [\Delta_1(\tilde{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}]| \\ &\leq \sup_x |\Delta_1(x, \theta', \omega')| \|\mathbb{P}(x \in \cdot | \theta', \omega', s_{t-m+1}) - \mathbb{P}(\tilde{x} \in \cdot | \theta', \omega', s_{t-m+1})\|_{TV} \\ &\leq 8R_\omega C_\delta d_{TV} (\mathbb{P}(x \in \cdot | \theta', s_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1})), \end{aligned} \quad (98)$$

where the second last inequality follows the definition of TV norm, and the last inequality follows

$$|\Delta_1(x, \theta', \omega')| \leq \|\omega' - \omega_{\theta'}^*\|_2 \|g(x, \omega') - \bar{g}(\theta', \omega')\|_2 \leq 4R_\omega C_\delta. \quad (99)$$

By following (28) in Lemma 3, we have

$$d_{TV}(\mathbb{P}(x \in \cdot | \theta', s_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1})) \leq \frac{1}{2} |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} [\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta', s_{t-m+1}].$$

Substituting the last inequality into (98), then taking total expectation on both sides yield

$$\mathbb{E}[I_3] \leq 4R_\omega C_\delta |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2. \quad (100)$$

Next we bound I_4 . Define $\bar{x} := (\bar{s}, \bar{a}, \bar{s}')$ where $\bar{s} \sim \mu_{\theta'}$, $\bar{a} \sim \pi_{\theta'}$ and $\bar{s}' \sim \mathcal{P}$. It is immediate that

$$\begin{aligned} \mathbb{E}[\Delta_1(\bar{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] &= \langle \omega' - \omega_{\theta'}^*, \mathbb{E}[g(\bar{x}, \omega') | \theta', \omega', s_{t-m+1}] - \bar{g}(\theta', \omega') \rangle \\ &= \langle \omega' - \omega_{\theta'}^*, \bar{g}(\theta', \omega') - \bar{g}(\theta', \omega') \rangle = 0. \end{aligned} \quad (101)$$

Then we have

$$\begin{aligned} \mathbb{E}[I_4 | \theta', \omega', s_{t-m+1}] &= \mathbb{E} [\Delta_1(\tilde{x}, \theta', \omega') - \Delta_1(\bar{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] \\ &\leq |\mathbb{E} [\Delta_1(\tilde{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}] - \mathbb{E} [\Delta_1(\bar{x}, \theta', \omega') | \theta', \omega', s_{t-m+1}]| \\ &\leq \sup_x |\Delta_1(x, \theta', \omega')| \|\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}) - \mathbb{P}(\bar{x} \in \cdot | \theta', s_{t-m+1})\|_{TV} \\ &\leq 8R_\omega C_\delta d_{TV}(\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}), \mathbb{P}(\bar{x} \in \cdot | \theta', s_{t-m+1})) \\ &= 8R_\omega C_\delta d_{TV}(\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}), \mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}), \end{aligned} \quad (102)$$

where the second inequality follows the definition of TV norm, and the third inequality follows (101).

The auxiliary Markov chain with policy $\pi_{\theta'}$ starts from initial state s_{t-m+1} , and $\tilde{s}_t$ is the $(m-1)$ th state on the chain. Following Lemma 2, we have:

$$\begin{aligned} d_{TV}(\mathbb{P}(\tilde{x} \in \cdot | \theta', s_{t-m+1}), \mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \\ = d_{TV}(\mathbb{P}((\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1}) \in \cdot | \theta', s_{t-m+1}), \mu_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \leq \kappa \rho^{m-1}. \end{aligned} \quad (103)$$

Substituting the last inequality into (102) and taking total expectation on both sides yield

$$\mathbb{E}[I_4] \leq 8R_\omega C_\delta \kappa \rho^{m-1}. \quad (104)$$

Taking total expectation on (89) and collecting bounds of I_1, I_2, I_3, I_4 yield

$$\begin{aligned} \mathbb{E}[\Delta_1(x, \theta, \omega)] &\leq C_4 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 + C_5 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 \\ &\quad + C_6 \mathbb{E} \|\omega_k - \omega_{k-d_m}\|_2 + C_7 \kappa \rho^{m-1}, \end{aligned} \quad (105)$$

where $C_4 := 2C_\delta L_\omega + 4R_\omega C_\delta |\mathcal{A}| L_\pi (1 + \log_\rho \kappa^{-1} + (1 - \rho)^{-1})$, $C_5 := 4R_\omega C_\delta |\mathcal{A}| L_\pi$, $C_6 := 4(1 + \gamma)R_\omega + 2C_\delta$ and $C_7 := 8R_\omega C_\delta$. $\blacksquare$

C.2. Supporting Lemmas for Theorem 4

Lemma 5 For any $m \geq 1$ and $k \geq (K_0 + 1)m + K_0 + 1$, we have

$$\begin{aligned} \mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle &\geq -D_2 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 \\ &\quad - D_3 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_4 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_5 \kappa \rho^{m-1} - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \end{aligned}$$

where $D_2 := 2L_V L_\psi C_\delta$, $D_3 := (2C_\delta C_\psi L_\lambda + L_V C_\psi (L_\omega + L_V)(1 + \gamma))$, $D_4 := 2L_V C_\psi C_\delta |\mathcal{A}| L_\pi$ and $D_5 := 4L_V C_\psi C_\delta$.

Proof For the worker that contributes to the k th update, we construct its Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{P}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_m-1}} \hat{a}_{t-m+1} \cdots \hat{s}_{t-1} \xrightarrow{\theta_{k-d_1}} \hat{a}_{t-1} \xrightarrow{\hat{P}} \hat{s}_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\hat{P}} \hat{s}_{t+1},$$

where $(s_t, a_t, \hat{s}_{t+1}) = (\hat{s}_{(k)}, \hat{a}_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$. By (88) in Lemma 4, we have $d_m \leq (K_0 + 1)m + K_0$.

Given $(\hat{s}_{t-m}, \hat{a}_{t-m}, \hat{s}_{t-m+1})$ and θ_{k-d_m} , we construct an auxiliary Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{P}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\hat{P}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\hat{P}} \tilde{s}_{t+1}.$$

First we have

$$\begin{aligned} &\left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ &\quad + \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle. \end{aligned} \quad (106)$$

We first bound the first term in (106) as

$$\begin{aligned} &\left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ &\geq -\|\nabla J_\lambda(\theta_k)\|_2 \|\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k)\| \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -\|\nabla J_\lambda(\theta_k)\|_2 \left(\|\hat{\delta}(\hat{x}_{(k)}, \omega_k^*)\| + \|\delta(\hat{x}_{(k)}, \theta_k)\| \right) \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -L_V \left(\|\hat{\delta}(\hat{x}_{(k)}, \omega_k^*)\| + \|\delta(\hat{x}_{(k)}, \theta_k)\| \right) \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -2L_V C_\delta \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ &\geq -2L_V L_\psi C_\delta \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2, \end{aligned} \quad (107)$$

where the last inequality follows Assumption 3 and second last inequality follows

$$\begin{aligned} |\hat{\delta}(x, \omega_\theta^*)| &\leq |r(x)| + \gamma \|\phi(s')\|_2 \|\omega_\theta^*\|_2 + \|\phi(s)\|_2 \|\omega_\theta^*\|_2 \leq r_{\max} + (1 + \gamma) R_\omega \leq C_\delta, \\ |\delta(x, \theta)| &\leq |r(x)| + \gamma |V_{\pi_\theta}(s')| + |V_{\pi_\theta}(s)| \leq r_{\max} + (1 + \gamma) \frac{r_{\max}}{1 - \gamma} \leq C_\delta. \end{aligned}$$

Substituting (107) into (106) gives

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ & \geq -2L_V L_\psi C_\delta \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 + \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle. \end{aligned} \quad (108)$$

Then we start to bound the second term in (108). For brevity, we define

$$\Delta_2(x, \theta) := \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta_{k-d_m}}(s, a) \right\rangle.$$

In the following proof, we use $\theta, \theta', \omega_\theta^*, \omega_{\theta'}^*, x$ as shorthand notations for $\theta_k, \theta_{k-d_m}, \omega_k^*, \omega_{k-d_m}^*, x_t$ respectively. We also define $\bar{x} := (\bar{s}, \bar{a}, \bar{s}')$, where $\bar{s} \sim d_{\theta'}$, $\bar{a} \sim \pi_{\theta'}$ and $\bar{s}' \sim \mathcal{P}$. Define $\tilde{x} := (\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}, \tilde{s}'_{t-d_m})$ where $\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}$ are state and action on the auxiliary Markov chain, and $\tilde{s}'_{t-d_m} \sim \mathcal{P}(\cdot | \tilde{s}_{t-d_m}, \tilde{a}_{t-d_m})$ is a virtual sample. We decompose the second term in (108) as

$$\Delta_2(x, \theta) = \underbrace{\Delta_2(x, \theta) - \Delta_2(x, \theta')}_{I_1} + \underbrace{\Delta_2(x, \theta') - \Delta_2(\tilde{x}, \theta')}_{I_2} + \underbrace{\Delta_2(\tilde{x}, \theta') - \Delta_2(\bar{x}, \theta')}_{I_3} + \underbrace{\Delta_2(\bar{x}, \theta')}_{I_4}.$$

We bound the term I_1 as

$$\begin{aligned} I_1 &= \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta') \right) \psi_{\theta'}(s, a) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\ &\quad + \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta') \right) \psi_{\theta'}(s, a) \right\rangle. \end{aligned} \quad (109)$$

For the first term in I_1 , we have

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta) - \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\ &\geq -\|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \|\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta)\|_2 \|\psi_{\theta'}(s, a)\|_2 \\ &\geq -2C_\delta C_\psi \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\ &\geq -2C_\delta C_\psi L_\lambda \|\theta - \theta'\|_2, \end{aligned} \quad (110)$$

where the last inequality is due to the L_λ -Lipschitz continuity of policy gradient shown in Proposition 1.

For the second term in I_1 , we have

$$\begin{aligned}
& \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle - \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta') \right) \psi_{\theta'}(s, a) \right\rangle \\
&= \left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(x, \omega_\theta^*) - \hat{\delta}(x, \omega_{\theta'}^*) + \delta(x, \theta') - \delta(x, \theta) \right) \psi_{\theta'}(s, a) \right\rangle \\
&\geq -L_V C_\psi \left| \hat{\delta}(x, \omega_\theta^*) - \hat{\delta}(x, \omega_{\theta'}^*) + \delta(x, \theta') - \delta(x, \theta) \right| \\
&\geq -L_V C_\psi \left| \gamma \phi(s')^\top (\omega_\theta^* - \omega_{\theta'}^*) + \phi(s)^\top (\omega_{\theta'}^* - \omega_\theta^*) + \gamma V_{\pi_{\theta'}}(s') - \gamma V_{\pi_\theta}(s') + V_{\pi_\theta}(s) - V_{\pi_{\theta'}}(s) \right| \\
&\geq -L_V C_\psi (\gamma \|\omega_\theta^* - \omega_{\theta'}^*\|_2 + \|\omega_{\theta'}^* - \omega_\theta^*\|_2 + \gamma |V_{\pi_{\theta'}}(s') - V_{\pi_\theta}(s')| + |V_{\pi_\theta}(s) - V_{\pi_{\theta'}}(s)|) \\
&\geq -L_V C_\psi (\gamma L_\omega \|\theta - \theta'\|_2 + L_\omega \|\theta - \theta'\|_2 + \gamma L_V \|\theta - \theta'\|_2 + L_V \|\theta - \theta'\|_2) \\
&= -L_V C_\psi (L_\omega + L_V)(1 + \gamma) \|\theta - \theta'\|_2,
\end{aligned} \tag{111}$$

where the last inequality is due to the L_ω -Lipschitz continuity of ω_θ^* shown in Proposition 2 and L_V -Lipschitz continuity of $V_{\pi_\theta}(s)$. Collecting the upper bounds of I_1 yields

$$I_1 \geq -(2C_\delta C_\psi L_\lambda + L_V C_\psi (L_\omega + L_V)(1 + \gamma)) \|\theta - \theta'\|_2. \tag{112}$$

First we bound I_2 as

$$\begin{aligned}
\mathbb{E}[I_2 | \theta', \hat{s}_{t-m+1}] &= \mathbb{E} [\Delta_2(x, \theta') - \Delta_2(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}] \\
&\geq -\left| \mathbb{E} [\Delta_2(x, \theta') | \theta', \hat{s}_{t-m+1}] - \mathbb{E} [\Delta_2(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}] \right| \\
&\geq -\sup_x |\Delta_2(x, \theta')| \left| \mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}) - \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}) \right|_{TV} \\
&\geq -4L_V C_\psi C_\delta d_{TV} (\mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1})) \\
&= -4L_V C_\psi C_\delta d_{TV} (\mathbb{P}((\hat{s}_t, \hat{a}_t) \in \cdot | \theta', \hat{s}_{t-m+1}), \mathbb{P}((\tilde{s}_t, \tilde{a}_t) \in \cdot | \theta', \hat{s}_{t-m+1})) \\
&\geq -2L_V C_\psi C_\delta |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} [\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta', \hat{s}_{t-m+1}],
\end{aligned} \tag{113}$$

where the second inequality is due to the definition of TV norm, the last inequality follows (28) in Lemma 3, and the second last inequality follows the fact that

$$|\Delta_2(x, \theta')| \leq \|\nabla J_\lambda(\theta')\|_2 |\hat{\delta}(x, \omega_{\theta'}^*) - \delta(x, \theta')| \|\psi_{\theta'}(s, a)\|_2 \leq 2L_V C_\delta C_\psi. \tag{114}$$

Taking total expectation on both sides of (113) yields

$$\mathbb{E}[I_2] \geq -2L_V C_\psi C_\delta |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2. \tag{115}$$

Next we bound I_3 as

$$\begin{aligned}
\mathbb{E}[I_3 | \theta', \hat{s}_{t-m+1}] &= \mathbb{E} [\Delta_2(\tilde{x}, \theta') - \Delta_2(\bar{x}, \theta') | \theta', \hat{s}_{t-m+1}] \\
&\geq -\left| \mathbb{E} [\Delta_2(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}] - \mathbb{E} [\Delta_2(\bar{x}, \theta') | \theta', \hat{s}_{t-m+1}] \right| \\
&\geq -\sup_x |\Delta_2(x, \theta')| \left| \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}) - \mathbb{P}(\bar{x} \in \cdot | \theta', \hat{s}_{t-m+1}) \right|_{TV} \\
&\geq -4L_V C_\psi C_\delta d_{TV} (\mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}),
\end{aligned} \tag{116}$$

where the second inequality is due to the definition of TV norm, and the last inequality follows (114).

The auxiliary Markov chain with policy $\pi_{\theta'}$ starts from initial state $\hat{s}_{t-m+1}$, and $\tilde{s}_t$ is the $(m-1)$ th state on the chain. Following Lemma 2, we have:

$$\begin{aligned} d_{TV} \left(\mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \hat{\mathcal{P}} \right) &= d_{TV} \left(\mathbb{P}((\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1}) \in \cdot | \theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P} \right) \\ &\leq \kappa \rho^{m-1}. \end{aligned} \quad (117)$$

Substituting the last inequality into (116) and taking total expectation on both sides yield

$$\mathbb{E}[I_3] \geq -4L_V C_\psi C_\delta \kappa \rho^{m-1}. \quad (118)$$

We bound I_4 as

$$\begin{aligned} \mathbb{E}[I_4 | \theta'] &= \mathbb{E} \left[\left\langle \nabla J_\lambda(\theta'), \left(\hat{\delta}(\bar{x}, \omega_{\theta'}^*) - \delta(\bar{x}, \theta') \right) \psi_{\theta'}(s, a) \right\rangle \middle| \theta' \right] \\ &\geq -L_V C_\psi \mathbb{E} \left[\left\| \hat{\delta}(\bar{x}, \omega_{\theta'}^*) - \delta(\bar{x}, \theta') \right\| \middle| \theta' \right] \\ &= -L_V C_\psi \mathbb{E} \left[\left\| \gamma \left(\phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right) + V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right\| \middle| \theta' \right] \\ &\geq -L_V C_\psi \left(\gamma \mathbb{E} \left[\left| \phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right| \middle| \theta' \right] + \mathbb{E} \left[\left| V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right| \middle| \theta' \right] \right) \\ &\geq -L_V C_\psi \left(\gamma \sqrt{\mathbb{E} \left[\left| \phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right|^2 \middle| \theta' \right]} + \sqrt{\mathbb{E} \left[\left| V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right|^2 \middle| \theta' \right]} \right) \\ &= -L_V C_\psi \left(\gamma \sqrt{\mathbb{E}_{\bar{s}' \sim \mu_{\theta'}} \left| \phi(\bar{s}')^\top \omega_{\theta'}^* - V_{\pi_{\theta'}}(\bar{s}') \right|^2} + \sqrt{\mathbb{E}_{\bar{s} \sim \mu_{\theta'}} \left| V_{\pi_{\theta'}}(\bar{s}) - \phi(\bar{s})^\top \omega_{\theta'}^* \right|^2} \right) \\ &\geq -L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \end{aligned} \quad (119)$$

where the second last inequality follows Jensen's inequality.

Taking total expectation on both sides of (108), and collecting lower bounds of I_1 , I_2 , I_3 and I_4 yield

$$\begin{aligned} &\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \left(\hat{\delta}(\hat{x}_{(k)}, \omega_k^*) - \delta(\hat{x}_{(k)}, \theta_k) \right) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right\rangle \\ &\geq -D_2 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_3 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_4 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 \\ &\quad - D_5 \kappa \rho^{m-1} - L_V C_\psi (1 + \gamma) \epsilon_{\text{app}}, \end{aligned} \quad (120)$$

where $D_2 := 2L_V L_\psi C_\delta$, $D_3 := (2C_\delta C_\psi L_\lambda + L_V C_\psi (L_\omega + L_V)(1 + \gamma))$, $D_4 := 2L_V C_\psi C_\delta |\mathcal{A}| L_\pi$ and $D_5 := 4L_V C_\psi C_\delta$. $\blacksquare$

Lemma 6 For any $m \geq 1$ and $k \geq (K_0 + 1)m + K_0 + 1$, we have

$$\begin{aligned} &\mathbb{E} \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle \\ &\geq -D_6 \mathbb{E} \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_7 \mathbb{E} \|\theta_k - \theta_{k-d_m}\|_2 - D_8 \sum_{i=\tau_k}^{d_m} \mathbb{E} \|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_9 \kappa \rho^{m-1}, \end{aligned}$$

where $d_m \leq (K_0 + 1)m + K_0$, $D_6 := L_V C_\delta L_\psi$, $D_7 := C_p L_\lambda + (1 + \gamma) L_V^2 C_\psi + 2L_V L_\lambda$, $D_8 := L_V C_p |\mathcal{A}| L_\pi$ and $D_9 := 2L_V C_p$.

Proof For the worker that contributes to the k th update, we construct its Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{\mathcal{P}}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_{m-1}}} \hat{a}_{t-m+1} \cdots \hat{s}_{t-1} \xrightarrow{\theta_{k-d_1}} \hat{a}_{t-1} \xrightarrow{\hat{\mathcal{P}}} s_t \xrightarrow{\theta_{k-d_0}} a_t \xrightarrow{\hat{\mathcal{P}}} \hat{s}_{t+1},$$

where $(s_t, a_t, \hat{s}_{t+1}) = (\hat{s}_{(k)}, \hat{a}_{(k)}, s'_{(k)})$, and $\{d_j\}_{j=0}^m$ is some increasing sequence with $d_0 := \tau_k$. By (88) in Lemma 4, we have $d_m \leq (K_0 + 1)m + K_0$.

Given $(\hat{s}_{t-m}, \hat{a}_{t-m}, \hat{s}_{t-m+1})$ and θ_{k-d_m} , we construct an auxiliary Markov chain:

$$\hat{s}_{t-m} \xrightarrow{\theta_{k-d_m}} \hat{a}_{t-m} \xrightarrow{\hat{\mathcal{P}}} \hat{s}_{t-m+1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-m+1} \cdots \tilde{s}_{t-1} \xrightarrow{\theta_{k-d_m}} \tilde{a}_{t-1} \xrightarrow{\hat{\mathcal{P}}} \tilde{s}_t \xrightarrow{\theta_{k-d_m}} \tilde{a}_t \xrightarrow{\hat{\mathcal{P}}} \tilde{s}_{t+1}.$$

First we have

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle \\ &= \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ & \quad + \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle. \end{aligned} \quad (121)$$

We bound the first term in (121) as

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \left(\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) \right) \right\rangle \\ & \geq -\|\nabla J_\lambda(\theta_k)\|_2 \|\delta(\hat{x}_{(k)}, \theta_k)\|_2 \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ & \geq -L_V C_\delta \|\psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)})\|_2 \\ & \geq -L_V C_\delta L_\psi \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2, \end{aligned} \quad (122)$$

where the last inequality follows Assumption 3. Substituting (122) into (121) gives

$$\begin{aligned} & \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J_\lambda(\theta_k) \right\rangle \\ & \geq -L_V C_\delta L_\psi \|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 + \left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-d_m}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J_\lambda(\theta_k) \right\rangle. \end{aligned} \quad (123)$$

Then we start to bound the second term in (123). For brevity, we define

$$\Delta_3(x, \theta) := \left\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta_{k-d_m}}(s, a) - \nabla J(\theta) \right\rangle.$$

Throughout the following proof, we use θ, θ', x as shorthand notations of $\theta_k, \theta_{k-d_m}, x_t$ respectively. Define $\tilde{x} := (\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}, \tilde{s}'_{t-d_m})$ where $\tilde{s}_{t-d_m}, \tilde{a}_{t-d_m}$ are state and action on the auxiliary Markov chain, and $\tilde{s}'_{t-d_m} \sim \mathcal{P}(\cdot | \tilde{s}_{t-d_m}, \tilde{a}_{t-d_m})$ is a virtual sample.

We decompose $\Delta_3(x, \theta)$ as

$$\Delta_3(x, \theta) = \underbrace{\Delta_3(x, \theta) - \Delta_3(x, \theta')}_{I_1} + \underbrace{\Delta_3(x, \theta') - \Delta_3(\tilde{x}, \theta')}_{I_2} + \underbrace{\Delta_3(\tilde{x}, \theta')}_{I_3}.$$

We first bound I_1 as

$$\begin{aligned}
 |I_1| &= |\Delta_3(x, \theta) - \Delta_3(x, \theta')| \\
 &= |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta), \nabla J(\theta) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle + \langle \nabla J_\lambda(\theta'), \nabla J(\theta') \rangle| \\
 &\leq |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| + |\langle \nabla J_\lambda(\theta), \nabla J(\theta) \rangle - \langle \nabla J_\lambda(\theta'), \nabla J(\theta') \rangle| \\
 &\leq |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| + 2L_V L_\lambda \|\theta - \theta'\|_2, \quad (124)
 \end{aligned}$$

where the last inequality uses the fact that $\langle \nabla J_\lambda(\theta), \nabla J(\theta) \rangle$ is $2L_V L_\lambda$ -lipschitz continuous. We bound the first term in (124) as

$$\begin{aligned}
 &|\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &\leq |\langle \nabla J_\lambda(\theta), \delta(x, \theta) \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &\quad + |\langle \nabla J_\lambda(\theta), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle - \langle \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &= |\langle \nabla J_\lambda(\theta), (\delta(x, \theta) - \delta(x, \theta')) \psi_{\theta'}(s, a) \rangle| + |\langle \nabla J_\lambda(\theta) - \nabla J_\lambda(\theta'), \delta(x, \theta') \psi_{\theta'}(s, a) \rangle| \\
 &\leq L_V C_\psi |\delta(x, \theta) - \delta(x, \theta')| + C_p \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\
 &= L_V C_\psi |\gamma(V_{\pi_\theta}(s') - V_{\pi_{\theta'}}(s')) + V_{\pi_{\theta'}}(s) - V_{\pi_\theta}(s)| + C_p \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\
 &\leq L_V C_\psi (\gamma |V_{\pi_\theta}(s') - V_{\pi_{\theta'}}(s')| + |V_{\pi_{\theta'}}(s) - V_{\pi_\theta}(s)|) + C_p \|\nabla J_\lambda(\theta) - \nabla J_\lambda(\theta')\|_2 \\
 &\leq L_V C_\psi (\gamma L_V \|\theta - \theta'\|_2 + L_V \|\theta' - \theta\|) + C_p L_\lambda \|\theta - \theta'\|_2 \\
 &= (C_p L_\lambda + (1 + \gamma) L_V^2 C_\psi) \|\theta - \theta'\|_2. \quad (125)
 \end{aligned}$$

Substituting the above inequality into (124) gives the lower bound of I_1 :

$$I_1 \geq - (C_p L_\lambda + (1 + \gamma) L_V^2 C_\psi + 2L_V L_\lambda) \|\theta - \theta'\|_2.$$

First we bound I_2 as

$$\begin{aligned}
 \mathbb{E}[I_2 | \theta', \hat{s}_{t-m+1}] &= \mathbb{E}[\Delta_3(x, \theta') - \Delta_3(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}] \\
 &\geq -|\mathbb{E}[\Delta_3(x, \theta') | \theta', \hat{s}_{t-m+1}] - \mathbb{E}[\Delta_3(\tilde{x}, \theta') | \theta', \hat{s}_{t-m+1}]| \\
 &\geq -\sup_x |\Delta_3(x, \theta')| \|\mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}) - \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1})\|_{TV} \\
 &\geq -2L_V (C_p + L_V) d_{TV} (\mathbb{P}(x \in \cdot | \theta', \hat{s}_{t-m+1}), \mathbb{P}(\tilde{x} \in \cdot | \theta', \hat{s}_{t-m+1})) \\
 &\geq -L_V (C_p + L_V) |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E}[\|\theta_{k-i} - \theta_{k-d_m}\|_2 | \theta', \hat{s}_{t-m+1}], \quad (126)
 \end{aligned}$$

where the second inequality is due to the definition of TV norm, the last inequality is due to (28) in Lemma 3, and the second last inequality follows the fact that

$$|\Delta_3(x, \theta')| \leq \|\nabla J_\lambda(\theta)\|_2 (\|\delta(x, \theta) \psi_{\theta_{k-d_m}}(s, a)\|_2 + \|\nabla J(\theta)\|_2) \leq L_V (C_p + L_V). \quad (127)$$

Taking total expectation on both sides of (126) yields

$$\mathbb{E}[I_2] \geq -L_V (C_p + L_V) |\mathcal{A}| L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E}[\|\theta_{k-i} - \theta_{k-d_m}\|_2]. \quad (128)$$

Define $\bar{x} := (\bar{s}, \bar{a}, \bar{s}')$, where $\bar{s} \sim d_{\theta'}$, $\bar{a} \sim \pi_{\theta'}$ and $\bar{s}' \sim \mathcal{P}$. Then we can further decompose I_3 as

$$\mathbb{E}[I_3|\theta', \hat{s}_{t-m+1}] = \mathbb{E}[\Delta_3(\tilde{x}, \theta') - \Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] + \mathbb{E}[\Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}]. \quad (129)$$

The first term in (129) can be bounded as

$$\begin{aligned} & \mathbb{E}[\Delta_3(\tilde{x}, \theta') - \Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] \\ & \geq -|\mathbb{E}[\Delta_3(\tilde{x}, \theta')|\theta', \hat{s}_{t-m+1}] - \mathbb{E}[\Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}]| \\ & \geq -\sup_x |\Delta_3(x, \theta')| \|\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}) - \mathbb{P}(\bar{x} \in \cdot|\theta', \hat{s}_{t-m+1})\|_{TV} \\ & \geq -2L_V(C_p + L_V)d_{TV}(\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}), \mathbb{P}(\bar{x} \in \cdot|\theta', \hat{s}_{t-m+1})) \\ & = -2L_V(C_p + L_V)d_{TV}(\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \end{aligned} \quad (130)$$

where the second inequality follows the definition of TV -norm, and the third one follows (127).

The auxiliary Markov chain with policy $\pi_{\theta'}$ starts from initial state $\hat{s}_{t-m+1}$, and $\tilde{s}_t$ is the $(m-1)$ th state on the chain. Following Lemma 2, we have:

$$\begin{aligned} d_{TV}(\mathbb{P}(\tilde{x} \in \cdot|\theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) &= d_{TV}(\mathbb{P}((\tilde{s}_t, \tilde{a}_t, \tilde{s}_{t+1}) \in \cdot|\theta', \hat{s}_{t-m+1}), d_{\theta'} \otimes \pi_{\theta'} \otimes \mathcal{P}) \\ &\leq \kappa \rho^{m-1}. \end{aligned}$$

Substituting the last inequality into (130) yields

$$\mathbb{E}[\Delta_3(\tilde{x}, \theta') - \Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] \geq -2L_V(C_p + L_V)\kappa \rho^{m-1}. \quad (131)$$

The second term in (129) is simply

$$\mathbb{E}[\Delta_3(\bar{x}, \theta')|\theta', \hat{s}_{t-m+1}] = \mathbb{E}[\langle \nabla J_\lambda(\theta'), \delta(\bar{x}, \theta') \psi_{\theta'}(\bar{s}, \bar{a}) - \nabla J(\theta') \rangle |\theta', \hat{s}_{t-m+1}] = 0. \quad (132)$$

Substituting (131) and (132) into (129) yields

$$\mathbb{E}[I_3|\theta', \hat{s}_{t-m+1}] \geq -2L_V(C_p + L_V)\kappa \rho^{m-1}. \quad (133)$$

Taking total expectation on $\Delta_3(x, \theta)$ and collecting lower bounds of I_1, I_2, I_3 yield

$$\begin{aligned} \mathbb{E}[\Delta_3(x, \theta)] &\geq -(C_p L_\lambda + (1 + \gamma)L_V^2 C_\psi + 2L_V L_\lambda) \mathbb{E}\|\theta_k - \theta_{k-d_m}\|_2 \\ &\quad - L_V(C_p + L_V)|\mathcal{A}|L_\pi \sum_{i=\tau_k}^{d_m} \mathbb{E}\|\theta_{k-i} - \theta_{k-d_m}\|_2 - 2L_V(C_p + L_V)\kappa \rho^{m-1}. \end{aligned} \quad (134)$$

Taking total expectation on (123) and substituting the above inequality into it yield

$$\begin{aligned} & \mathbb{E}\left\langle \nabla J_\lambda(\theta_k), \delta(\hat{x}_{(k)}, \theta_k) \psi_{\theta_{k-\tau_k}}(\hat{s}_{(k)}, \hat{a}_{(k)}) - \nabla J(\theta_k) \right\rangle \\ & \geq -D_6 \mathbb{E}\|\theta_{k-\tau_k} - \theta_{k-d_m}\|_2 - D_7 \mathbb{E}\|\theta_k - \theta_{k-d_m}\|_2 - D_8 \sum_{i=\tau_k}^{d_m} \mathbb{E}\|\theta_{k-i} - \theta_{k-d_m}\|_2 - D_9 \kappa \rho^{m-1}, \end{aligned} \quad (135)$$

where $D_6 := L_V C_\delta L_\psi$, $D_7 := C_p L_\lambda + (1 + \gamma)L_V^2 C_\psi + 2L_V L_\lambda$, $D_8 := L_V(C_p + L_V)|\mathcal{A}|L_\pi$, $D_9 := 2L_V(C_p + L_V)$. $\blacksquare$

The Implementation of Asynchronous Advantage Actor-Critic with Stigmergy in Network-assisted Multi-agent System

Kun Chen*, Rongpeng Li*, Zhifeng Zhao[†], and Honggang Zhang*

*Zhejiang University, [†]Zhejiang Lab

Email: {beatther,lirongpeng, honggangzhang}@zju.edu.cn, zhaozf@zhejianglab.com

Abstract—Multi-agent system (MAS) needs to mobilize multiple simple agents to complete complex tasks. However, it is difficult to coherently coordinate distributed agents by means of limited local information. In this paper, we propose a decentralized collaboration method named as “stigmergy” in network-assisted MAS, by exploiting digital pheromones (DP) as an indirect medium of communication and utilizing deep reinforcement learning (DRL) on top. Correspondingly, we implement an experimental platform, where KHEPERA IV robots form targeted specific shapes in a decentralized manner. Experimental results demonstrate the effectiveness and efficiency of the proposed method. Our platform could be conveniently extended to investigate the impact of network factors (e.g., latency, data rate, etc) on the level of collective intelligence.

Index Terms—multi-agent system, stigmergy mechanism, digital pheromones, deep reinforcement learning, KHEPERA IV robots.

I. INTRODUCTION

With the development of artificial intelligence (AI) and the Internet, multi-agent system (MAS) has received the attention on many researchers and become a research hotspot in the field of control disciplines and AI [1]. MAS is a large-scale system composed of numerous intelligent agents with perception, communication, computation, and execution capabilities. Because of its autonomous and distributed characteristics [2], connected intelligent agents can collaborate to complete high-complexity tasks. Based on this property of MAS, it has been widely used in various scenarios such as swarm robots control, UAV collaboration and satellite formation [3], [4].

However, in a distributed MAS, due to the limited communication resources and computing capability, agents can neither communicate with each other in real-time regularly nor update the status of controller in a timely manner. Besides, the fact that actions performed by distributed agents inevitably change the surrounding environment, which further affects the decisions on other agents and produces a time-lasting impact. This interaction between agents and non-stationary environment makes it difficult to model the entire system.

On the other hand, there have been some efforts in a centralized controller, which receives information generated during the interaction between all distributed agents and the environment, such as the state of the environment observed by distributed agents at each moment decision-making, etc [5]. In this way, the centralized controller can be regarded as a

joint agent, and the environmental state that it receives at each moment is a collection of states observed by all agents, which can approximate the research on multi-agents by a simplified single-agent solution. Nevertheless, in practical scenarios such as intelligent transportation systems, smart grids and sensor networks, the existence of centralized controllers cannot be guaranteed [6]–[8]. In addition, for MAS with large action space and state space, the introduction of a centralized controller will suffer from the curse of dimensionality, significantly incurring the amount of calculation of a centralized controller and even being impossible to calculate.

Stigmergy is an indirect communication mechanism existing on social insect species, which relies on “pheromone” deposited in the environment to coordinate individual insects’ activities, and exhibit strong robustness with simple form [9]. Stigmergy mechanism is widely used in the implementation of collective intelligence. In order to seek the characteristic of “pheromone” in the engineering system, digital pheromone (DP) is employed to simulate the existence form of pheromone in the environment, acting as medium in stigmergy mechanism to enable indirect communication with distributed agents [10]. Agents judge the state of the surrounding environment, leave a certain number of digital pheromones at the current location, and generate a DP map by integrating the digital pheromones in the environment. As time goes by, the increasingly refined DP map enables all distributed agents to gather toward the destinations with high digital pheromones volume.

However, using stigmergy mechanism alone to guide the behavior of agents often fails to complete the task efficiently. In the field of AI, a generalized purpose of reinforcement learning (RL) is to continuously try and optimize its decision-making mechanism by interacting with agent and environment, so as to better adapt to the particular environment [11], [12]. Recently, with deep reinforcement learning (DRL) combining RL and deep neural networks (DNN) making a splash in various fields, many advanced DRL algorithms have greatly improved the efficiency with which agents learn from environment constantly [13], [14]. On top of DRL, multi-agent reinforcement learning (MARL) has manifested itself in many application scenarios and complex modeling [15], [16], which trains multiple agents in a specific manner to accomplish tricky mission collaboratively.

Meanwhile, inspired by federated learning which designs a

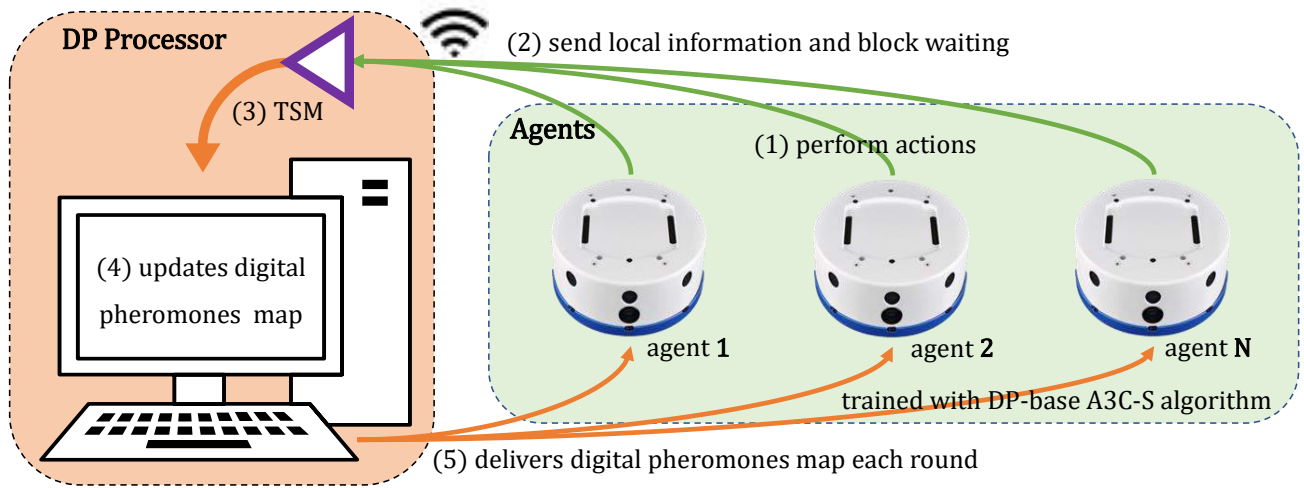


Fig. 1. The decentralized control framework with the DP-based A3C-S algorithm in network-assisted MAS.

virtual center to collaborate different parties without exchanging data [17], we propose a decentralized control framework based on stigmergy in network-assisted MAS by establishing a WLAN. Besides, in order to strengthen the sense of collaboration between agents, we combine it with asynchronous advantage actor-critic (A3C) algorithm, which is named as asynchronous advantage actor-critic with stigmergy (A3C-S) algorithm, to train agents to efficiently handle complex tasks in MAS [18].

To verify the validity of this framework, we design an experimental scenario where several randomly placed agents arrive at tracking points with as few rounds as possible in a certain area. After extensive simulation experiments to train and test, we exploit KHEPERA IV robots to implement the well-trained model in practical settings and introduce temporal synchronization mechanism (TSM) to overcome the problem of time asynchrony in robot information exchanges.

The remainder of this paper is mainly organized as follows. In Section II, we elaborate on the scenario and purpose of the experiment, and present the decentralized control framework in network-assisted MAS, including composition and execution procedure of system framework. In Section III, we mathematically discuss the details of the DP-based A3C-S Algorithm included DP in stigmergy and the A3C-S algorithm, and describe the design philosophy of TSM. In Section IV, the well-trained model is deployed on KHEPERA IV robots to verify the efficiency of system framework via extensive experiments. Finally, we conclude this paper with a summary.

II. SYSTEM MODEL

In the experimental scenario designed in this paper, the objective is that agents who are randomly placed within a certain area reach the target points in as few rounds as possible to form targeted specific shapes. Based on implicit information such as DP, agents can obtain the local environmental state through indirect communication. In contrast to the approach from planning a global course of action at the outset, we

expect that agents move in the general direction of a target point originally, and determine to the specific target point based on the judgment on their surroundings. Depending on the properties of DP in stigmergy, agents are capable to converge on a high probability towards a place with high digital pheromones volume in their favor.

However, directly applying DP in stigmergy to guide agents may cause agents that have occupied the target points become immobile, making other agents spend more rounds seeking unoccupied target points and incurring the system inefficiency. The intuitive solution is to keep all distributed agents in an “active” state at all times. In other words, even when agents reach a location with the highest pheromone volume in the observation range, they still need to choose to “move” or “stay” depending on the state of their surroundings.

In order to achieve this more flexible system, we propose a decentralized control framework with the DP-based A3C-S algorithm in network-assisted MAS, as shown in Fig. 1. It consists of two parts, that is, DP processor and agents, which form a WLAN to connect multiple distributed agents to establish communication. Furthermore, since agents and DP processor often communicate with each other out of synchronization in actual scenarios, we design and calibrate a TSM manner to align the agents’ time of performing actions and communication to overcome this conundrum.

- **DP processor.** DP processor collects local information on the procession of TSM, and calculates the digital pheromones of agents’ location based on the distance between agents and target points. With simulating the superposition, decay and diffusion properties of pheromone, DP processor generates a DP map that contains the distribution of digital pheromones for providing more relevant information of the state space in the whole activity area, which is delivered to agents each round to be foundation for decision-making.
- **Agents.** Each agent only acquires limited DP map around itself via interaction with DP processor, which determine

the location of next action without directly communicating with other agents. Each agent has a limited perception range of digital pheromones, and the ability of perception decreases along with the increase in the distance, which attracts agents to move towards at target points gradually because of the property that agents have a tendency to congregate to high pheromone volume places. Besides, distributed agents are trained with A3C-S algorithm to strengthen the collaboration and accomplish tasks efficiently.

Fig. 1 illustrates the whole execution procedure of the system framework.

- 1) Agents sense the state of surrounding environment based on DP map, compute the next decision and perform actions.
- 2) While agents are acting, they simultaneously upload local information (including the current location and the decision) to the DP processor and wait for the feedback information on DP processor in a blocking manner.
- 3) The information uploaded by agents is processed by TSM, and the entire information is delivered to the DP processor.
- 4) With simulating the superposition, decay and diffusion properties of pheromone, DP processor merges the collected information and updates DP map.
- 5) DP processor delivers the updated DP map to each agent, and repeats recursively until all agents reach the target points.

III. THE DP-BASED A3C-S ALGORITHM

This section elaborates on the problems of experimental scenario and the corresponding solutions, including DP in stigmergy, the A3C-S algorithm and TSM.

A. DP in Stigmergy

In the system model of this paper, DP is regarded as the medium in stigmergy mechanism and is constantly updated after agents perform actions, which is used as a bridge to instruct distributed agents with a more efficient manner to seek the target points. Inspired by the fact that the chemical pheromone left by different ants during swarm foraging can be superposed together to enhance the total influence, we model the accumulation of digital pheromones left by different agents as a linear superposition, which can be express as:

$$\mu_{(i,j)}(t) = \begin{cases} \mu_{(i,j)}(t-1) + a, & \text{if } (i,j) \text{ is target point,} \\ \mu_{(i,j)}(t-1) \cdot b, & \text{otherwise.} \end{cases} \quad (1)$$

where $\mu_{(i,j)}$ represents the amount of digital pheromones at the position (i,j) , a represents the fixed amount of digital pheromones left by an agent at a time, and $b \in (0, 1)$ denotes the decaying rate. After being superposed and updated, digital pheromones are diffused to the surrounding area at a fixed *diffusion rate*, and reduce the number of digital pheromones in the position already occupied by agents with a fixed *decay rate*. Similar to the ant colony searching for food, based on

the local state space, each agent needs to choose a direction that is most beneficial to itself from the direction of several potential candidates within its perception range (to gather at location with high pheromone volume).

B. The A3C-S Algorithm

During the experiment, we observe that modeling a DP map can make all distributed agents gather in the direction of high pheromone volume, but they frequently cannot cooperate well to accomplish relatively complex tasks efficiently. Therefore, we utilize A3C-S algorithm to train agents to efficiently handle complex tasks.

In the agents training phase, it is particularly essential to define the input of the neural network and the rewards. With the guidance of DP map obtained from the DP processor, each agent can perceive the number of digital pheromones within a certain range. We regard any block (unit area) filled with digital pheromones as an "attractor" in the local environment, which has an attractive effect on nearby agents to effectively observe the local state space. Such an attractor can be expressed as:

$$P_{(i,j),(m,n)} = \frac{\mu_{(m,n)} \cdot D(d_{(i,j),(m,n)})}{\sum_{(m,n) \in \zeta_{(i,j)}} \mu_{(m,n)} \cdot D(d_{(i,j),(m,n)})} \quad (2)$$

where $P_{(i,j),(m,n)}$ is the probability that agent chooses the attractor at position (m,n) when the agent is at position (i,j) . $\zeta_{(i,j)}$ is the set of attractors within the sensing range of agent. $d_{(i,j),(m,n)}$ is the Euclidean distance between the position of agent and attractor. $D(\cdot)$ is a monotonous function used to decrease the effect of digital pheromones as the inter-distance $d_{(i,j),(m,n)}$ increases [10]. The attractor of each unit block around the agent is calculated, which is encoded as the input of neural network to make corresponding decisions under various environmental conditions.

Based on the calculated attractor, distributed agents perform actions to move closer to the position of high digital pheromones volume. Since the purpose of experiment is that all agents reach the target points in as few rounds as possible, a reward function is needed to measure the global impact of the agents' action. We define the global minimum distance as the degree of proximity between the swarm of agents and target points. Therefore, when agents complete a round, a smaller global distance means that the swarm of agents is moving closer to the target points. Positive feedback is given to the agents to ensure them to learn the behavior that is beneficial to the global system. Conversely, the system is moving in a direction that is detrimental to the experimental purpose, and agents are given a penalty to prevent them from performing the actions later. The above process can be expressed as follows:

$$R(t) = \gamma \cdot \left(\min_{\alpha \in \Phi, \beta \in \Xi} d_{\alpha,\beta}(t-1) - \min_{\alpha \in \Phi, \beta \in \Xi} d_{\alpha,\beta}(t) \right) \quad (3)$$

where $R(t)$ represents the global reward after agents perform actions. Φ and Ξ belong to the set of the agents' and the target

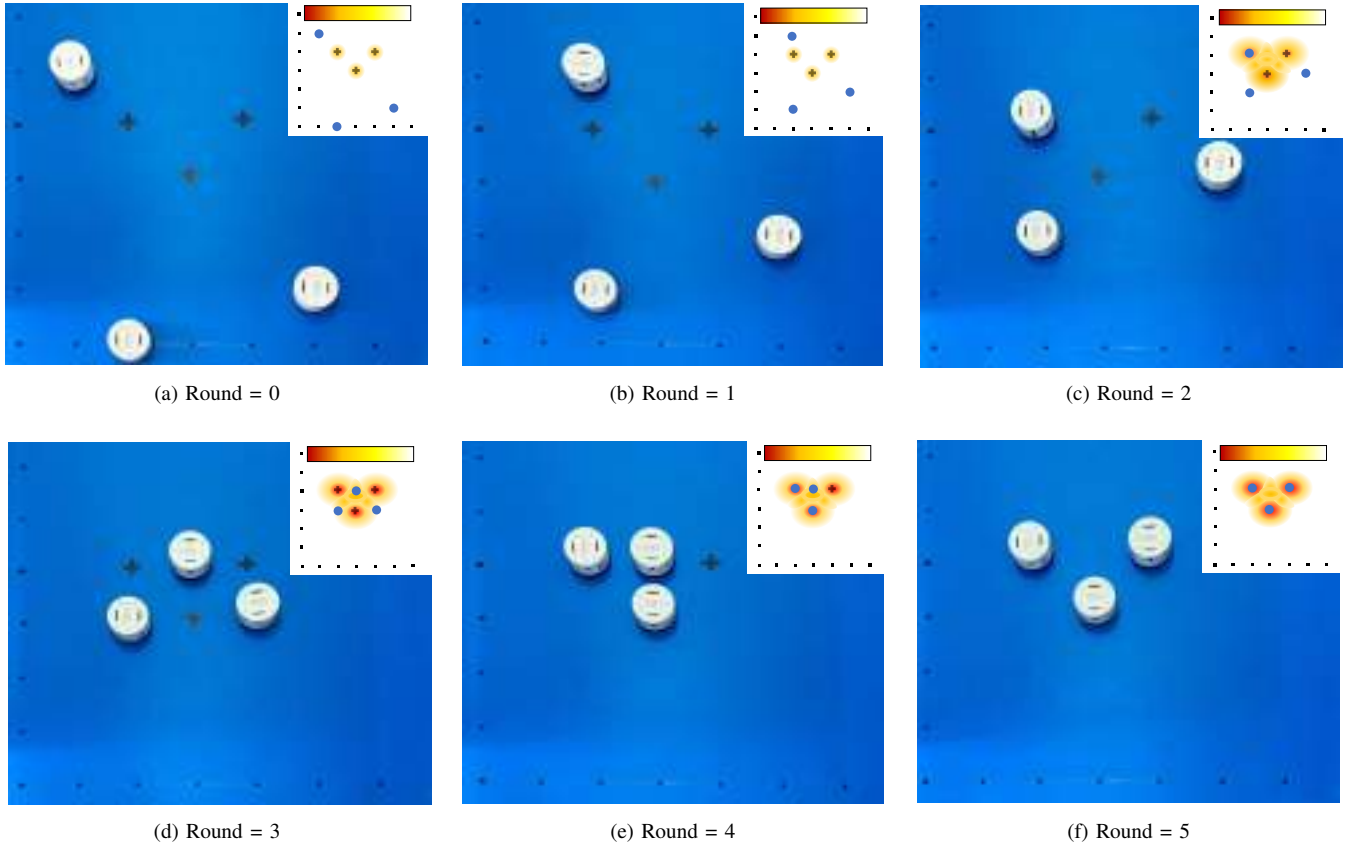


Fig. 2. The experimental platform of 3 KHEPERA IV robots with the system model. Marks "+" in the figure represents the target points. The upper right corner of each figure represents the DP map. The darker the color, the higher the digital pheromones volume.

points' position respectively. $d_{\alpha,\beta}$ is the Manhattan distance between agent and target point. γ is a constant matching coefficient, whose function is to calculate the reward value within a certain range. By training agents to adopt different strategies in different environments, the overall efficiency of decision-making is greatly improved compared with methods of only using DP maps to guide agents to seek target points (that is, enhancing agents' sense of cooperation).

C. Temporal Synchronization Mechanism

When deploying the system model to the real swarm robots, the timing of performing actions and communication varies each round, causing the updated DP map to be scrambled, which will result in sending the wrong DP map to agents to perform unanticipated actions.

To prevent information mismatch when agents communicate with DP processor, TSM is introduced to follow strict timing requirements to overcome the above problems when interact between distributed agents and DP processor. Similar to a buffer, DP processor stores the message received in the memory before receiving all local messages from agents, and process all messages uniformly when they have been received. DP processor also sets up a mechanism to discard and re-transmit timeouts to prevent system crashes in the event of packet loss. The role of TSM is that system sacrifices part

of the agents' moving time to coordinate the behavior of distributed agents, as well as reduces the probability of collision between the agents to some extent. We will investigate the impact on asynchronous methods of the future.

IV. EXPERIMENTAL RESULTS

In this part, we establish an experimental platform based on a DP processor and 3 KHEPERA IV robots, and deploy the system framework with the DP-based A3C-S algorithm to the swarm of KHEPERA IV robots. In experiment, a variety of models are also deployed on this experimental platform, and compared to the model of this paper to verify the efficiency of these models by evaluating performance in terms of degree of completion (DoC) and number of total steps. Our Demonstrator implementation is shown in Fig. 2 and the result of experiment is shown in Fig. 3 and Fig. 4. We specifically introduce involved experimental platform, deployment and result as follows.

A. Experimental Platform

We use a PC with WiFi module as the DP processor, and three KHEPERA IV robots as distributed agents. The user datagram protocol (UDP) is utilized to establish the connection between DP processor and distributed agents.

- **DP processor.** Being an implementer of DP processor does not require high performance, just the ability of communication and simple calculation. DP processor is employed as the server side of UDP to send updated DP maps to agents using the manner of multicast.
- **Agents.** The KHEPERA IV is a compact robot designed for any indoor lab application, which is as the client side of UDP to send the local status information to DP processor in system framework. Since the robots are not equipped with positioning modules, we employ odometry and leverage dead reckoning method to estimate the position [15]. By informing robots of initial position and obtaining encoder values of fixed time intervals, the robots' distance and angle for movement are calculated to enable positioning function. The CPU processor of each KHEPERA IV robot is Texas Instruments DaVinci DM3730 @ 800MHz.

B. Experimental Deployment

To verify the efficiency of decentralized control framework with DP-based A3C-S algorithm in network-assisted MAS, we design an experimental scenario where three randomly placed robots within a certain area reach three target points with as few rounds as possible, and the robots can only perform one action per round. We use a simulation software to simulate the behavior of the robots in a real environment, apply the system framework to train the robots centrally, and load the results of training to each KHEPERA IV robot distributedly. Fig. 2 shows a complete experiment with KHEPERA IV robots, which is described as follows.

First, we initialize the robots by placing them randomly within a certain area and informing the initial position. Similar to the real environment, we assume that a small number of digital pheromones exist at the target points, as shown in Fig. 2a. In the earlier phase, since the robots are randomly scattered and far away from the target points, the collaborative action between robots is not yet involved, but the general direction of the increase in digital pheromones is perceived, which motivates the robots to approach the target points more strongly, and attract more robots to approach the target points, as shown in Fig. 2b and Fig. 2c. As the number of rounds increase, distributed robots move into a narrow space closed to the target points, where each robot is trained with DP-based A3C-S algorithm to compute the optimal action under the current environmental conditions. In particular, during the experiment, even if server distributed robots have already reached the target points, in order to achieve the global optimality efficiently (reach the target points with as few rounds as possible), robots who have already reached the target points will continue to perform actions based on the state of environment, as shown in Fig. 2d and Fig. 2e. This design can not only improve the overall performance of the system, but also make the whole system more flexible. Fig. 2f shows the position of robots and distribution of DP map at the end of experiment.

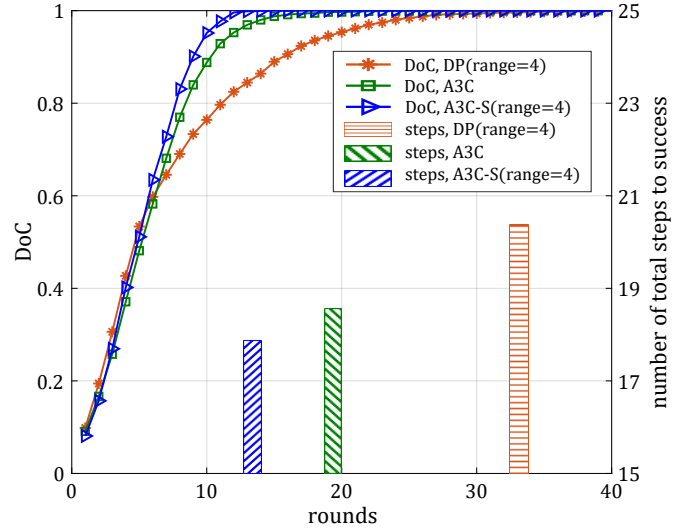


Fig. 3. The performance of DP-based A3C-S algorithm compared with the cases of adopting DP or A3C algorithm only. The performance of evaluating performance is in terms of DoC and number of total steps.

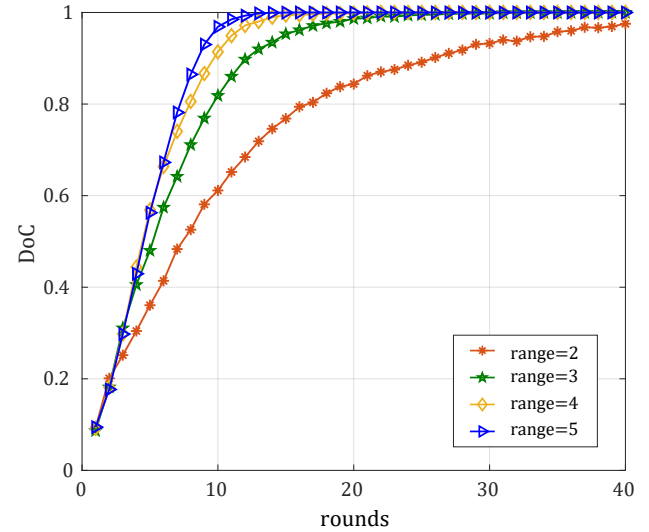


Fig. 4. The effect of changing the range of digital pheromones perception on experimental performance.

C. Experimental Results

We evaluate the performance of the aforementioned procedures by repeating the experiments above, and compare approaches using the DP-based A3C-S algorithm with the cases of adopting DP or A3C algorithm only, and Fig. 3 presents the numerical results in detail. When DoC arrives at 1, it means that all robots have reached the target points, and the histogram shows the number of rounds and total steps taken at completing the purpose. It can be observed from the figure that in the first few rounds of the experiment, applying the framework designed in this paper produce similar to the results as using only DP in stigmergy guidance, but as the distance

between robots became smaller, the presence or absence of swarm collaboration becomes particularly important.

The robot perceives a certain range of digital pheromones as foundation for decision making. In our experiments, we observed that changing the range of robots' perception has significant impact on the efficiency with which these distributed robots complete task, as shown in Fig. 4. When the perceptual range is short (e.g., range = 2), the system converges slowly and fails to complete the purpose even within the specified maximum number of steps; when the perceptual range increases above a certain threshold (e.g., range = 5), the performance of the system does not change significantly and may even attract more noisier information to interfere with the decision.

In summary, DP can attract robots to places with high pheromones volume at the beginning, and A3C algorithm can coordinate the behavior of agents when robots close to the target points. When selecting the appropriate perceptual range for the robot, we can conclude that compared to the cases of adopting DP or A3C algorithm only, the proposed DP-based A3C-S algorithm yields superior performance (shown in the Fig. 3 as the number of rounds when DoC first reached 1).

V. CONCLUSION AND DISCUSSIONS

In this paper, we have demonstrated the implementation means for achieving the decentralized control framework in network-assisted MAS. Furthermore, we have verified the effectiveness and efficiency of exploiting the DP-based A3C-S algorithm in a practical MAS with KHEPERA IV robots.

We boldly argue that the cooperation of collective intelligence implied in this paper could have significant merits in the future, especially in the field of wireless communications networks, and have a lot of issues to be addressed.

- As the working frequency becomes higher, the coverage of base stations gradually shrink. Along with the increasingly maturity of the internet of vehicles, it will be of great importance to coordinate the BSs and network entities by collective intelligence.
- How to develop the collective intelligence remains an issue? Ideas from brains could contribute a lot, as similar mechanism like stigmergy has been found and manifested [10].

Therefore, we hope this paper could act as a starting point to solicit interesting ideas and promote the development.

ACKNOWLEDGMENT

This work was supported in part by National Key R&D Program of China (2019YFB1802900), National Natural Science Foundation of China (No. 61701439, 61731002), Zhejiang Key R&D Plan (No. 2019C01002, 2019C03131), the Project sponsored by Zhejiang Lab (No. 2019LC0AB01), Zhejiang Provincial Natural Science Foundation of China (No. LY20F010016), the Huawei Cooperation Project.

REFERENCES

- [1] W. Ren, R. W. Beard, and E. M. Atkins, "Information consensus in multivehicle cooperative control," *IEEE Control Systems*, vol. 27, no. 2, pp. 71–82, 2007.
- [2] R. Li, Z. Zhao, X. Xu, F. Ni, and H. Zhang, "The collective advantage for advancing communications and intelligence," *IEEE Wireless Communications*, 2020.
- [3] L. Matignon, L. Jeanpierre, and A.-I. Mouaddib, "Coordinated multi-robot exploration under communication constraints using decentralized markov decision processes," in *Twenty-sixth AAAI Conference on Artificial Intelligence*, 2012.
- [4] A. K. Whitten, H.-L. Choi, L. B. Johnson, and J. P. How, "Decentralized task allocation with coupled constraints in complex missions," in *Proceedings of the 2011 American Control Conference*. IEEE, 2011, pp. 1642–1649.
- [5] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proceedings of the Tenth International Conference on Machine Learning*, 1993, pp. 330–337.
- [6] J. L. Adler and V. J. Blue, "A cooperative multi-agent transportation management and route guidance system," *Transportation Research Part C: Emerging Technologies*, vol. 10, no. 5–6, pp. 433–454, 2002.
- [7] D. S. Callaway and I. A. Hiskens, "Achieving controllability of electric loads," *Proceedings of the IEEE*, vol. 99, no. 1, pp. 184–199, 2010.
- [8] M. Rabbat and R. Nowak, "Distributed optimization in sensor networks," in *Proceedings of the 3rd International Symposium on Information Processing in Sensor Networks*, 2004, pp. 20–27.
- [9] H. V. D. Parunak, S. A. Brueckner, and J. Sauter, "Digital pheromones for coordination of unmanned vehicles," in *International Workshop on Environments for Multi-Agent Systems*. Springer, 2004, pp. 246–263.
- [10] X. Xu, Z. Zhao, R. Li, and H. Zhang, "Brain-inspired stigmergy learning," *IEEE Access*, vol. 7, pp. 54 410–54 424, 2019.
- [11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT press, 2018.
- [12] G. Tesauro, "Temporal difference learning and td-gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [13] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [14] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [15] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 2, pp. 156–172, 2008.
- [16] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Machine Learning Proceedings 1994*. Elsevier, 1994, pp. 157–163.
- [17] H. Brendan McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," *arXiv e-prints*, p. arXiv:1602.05629, Feb. 2016.
- [18] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. P. Lillicrap, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of the 33rd International Conference on Machine Learning - Volume 48*, ser. ICML'16. JMLR.org, 2016, p. 1928–1937.

DROPOUT Q-FUNCTIONS FOR DOUBLY EFFICIENT REINFORCEMENT LEARNING

Takuya Hiraoka^{1,2}, Takahisa Imagawa², Taisei Hashimoto^{2,3}, Takashi Onishi^{1,2},
Yoshimasa Tsuruoka^{2,3}

¹ NEC Corporation

² National Institute of Advanced Industrial Science and Technology

³ The University of Tokyo

{takuya-h1, takashi.onishi}@nec.com, imagawa.t@aist.go.jp,

{hashimoto-taisei388@g.ecc, tsuruoka@logos.t}.u-tokyo.ac.jp

ABSTRACT

Randomized ensembled double Q-learning (REDQ) (Chen et al., 2021b) has recently achieved state-of-the-art sample efficiency on continuous-action reinforcement learning benchmarks. This superior sample efficiency is made possible by using a large Q-function ensemble. However, REDQ is much less computationally efficient than non-ensemble counterparts such as Soft Actor-Critic (SAC) (Haarnoja et al., 2018a). To make REDQ more computationally efficient, we propose a method of improving computational efficiency called DroQ, which is a variant of REDQ that uses a small ensemble of dropout Q-functions. Our dropout Q-functions are simple Q-functions equipped with dropout connection and layer normalization. Despite its simplicity of implementation, our experimental results indicate that DroQ is doubly (sample and computationally) efficient. It achieved comparable sample efficiency with REDQ, much better computational efficiency than REDQ, and comparable computational efficiency with that of SAC.

1 INTRODUCTION

In the reinforcement learning (RL) community, improving **sample efficiency** of RL methods has been important. Traditional RL methods have been shown to be promising for solving complex control tasks such as dexterous in-hand manipulation (OpenAI et al., 2018). However, RL methods generally require millions of training samples to solve a task (Mendonca et al., 2019). This poor sample efficiency of RL methods is a severe obstacle to practical RL applications (e.g., applications on limited computational resources or in real-world environments without simulators). Motivated by these issues, many RL methods have been proposed to achieve higher sample efficiency. For example, Haarnoja et al. (2018a,b) proposed Soft Actor-Critic (SAC), which achieved higher sample efficiency than the previous state-of-the-art RL methods (Lillicrap et al., 2015; Fujimoto et al., 2018; Schulman et al., 2017).

Since 2019, RL methods that use a high **update-to-data (UTD)** ratio to achieve high sample efficiency have emerged. The UTD ratio is the number of updates taken by the agent compared to the number of actual interactions with the environment. A high UTD ratio promotes sufficiently training Q-functions within a few interactions, which leads to sample-efficient learning. Model-Based Policy Optimization (MBPO) (Janner et al., 2019) is a seminal RL method that uses a high UTD ratio of 20–40 and achieves significantly higher sample efficiency than SAC, which uses a UTD ratio of 1. Encouraged by the success of MBPO, many RL methods with high UTD ratios have been proposed (Shen et al., 2020; Lai et al., 2020).

With such methods, randomized ensembled double Q-learning (REDQ) proposed by Chen et al. (2021b) is currently the most sample-efficient method for the MuJoCo benchmark. REDQ uses a high UTD ratio and large ensemble of Q-functions. The use of a high UTD ratio increases an estimation bias in policy evaluation, which degrades sample-efficient learning. REDQ uses an ensemble of Q-functions to suppress the estimation bias and improve sample efficiency. Chen et al. (2021b) demonstrated that the sample efficiency of REDQ is equal to or even better than that of MBPO.

However, REDQ leaves room for improvement in terms of **computational efficiency**. REDQ runs 1.1 to 1.4 times faster than MBPO (Chen et al., 2021b) but is still less computationally efficient than non-ensemble-based RL methods (e.g., SAC) due to the use of large ensembles. In Section 4.2, we show that REDQ requires more than twice as much computation time per update, and much larger memory. Computational efficiency is important in several scenarios, e.g., in RL applications with much lighter on-device computation (e.g. mobile phones or other light-weight edge devices) (Chen et al., 2021a), or in situations in which rapid trials and errors are required for developing RL agents (e.g., hyperparameter tuning or proof of concept for RL applications with limited time resources). Therefore, RL methods that are superior not only in terms of sample efficiency but also in computational efficiency are preferable.

We propose a method of improving computational efficiency. Our method is called DroQ and is a REDQ variant that *uses a small ensemble of dropout Q-functions, in which dropout (Srivastava et al., 2014) and layer normalization (Ba et al., 2016) are used (Section 3)*. We experimentally show that DroQ is doubly (computationally and sample) efficient: (i) DroQ significantly improves computational efficiency over REDQ (Section 4.2) by more than two times and (ii) achieves sample efficiency comparable to REDQ (Section 4.1).

Although our primary contribution is **proposing a doubly efficient RL method**, we also make three significant contributions from other perspectives:

1. Simplicity of implementation. DroQ can be implemented by basically adding a few lines of readily available functions (dropout and layer normalization) to Q-functions in REDQ (and SAC). This simplicity enables one to easily replicate and extend it.

2. First successful demonstration of the usefulness of dropout in high UTD ratio settings. Previous studies incorporated dropout into RL (Gal & Ghahramani, 2016; Harrigan, 2016; Moerland et al., 2017; Gal et al., 2016; 2017; Kahn et al., 2017; Jaques et al., 2019; He et al., 2021) (see Section 5 for details). However, these studies focused on *low* UTD ratio settings (i.e., UTD ratio ≤ 1). Dropout approaches generally do not work as well as ensemble approaches (Osband et al., 2016; Ovadia et al., 2019; Lakshminarayanan et al., 2017; Durasov et al., 2021). For this reason, instead of dropout approaches, ensemble approaches have been used in RL with high UTD ratio settings (Chen et al., 2021b; Janner et al., 2019; Shen et al., 2020; Hiraoka et al., 2021; Lai et al., 2020). In Section 4, we argue that DroQ achieves almost the same or better bias reduction ability and sample/computationally efficiency compared with ensemble-based RL methods in high UTD ratio settings. This sheds light on dropout approaches once again and promotes their use as a reasonable alternative (or complement) to ensemble approaches in high UTD ratio settings.

3. Discovery of engineering insights to effectively apply dropout to RL. Specifically, we discovered that the following two engineering practices are effective in reducing bias and improving sample efficiency: (i) using the dropout and ensemble approaches together for constructing Q-functions (i.e., using *multiple* dropout Q-functions) (Section 4.1 and Appendix A.3); and (ii) introducing layer normalization into dropout Q-functions (Section 4.3 and Appendices D, E and F). These engineering insights were not revealed in previous RL studies and would be useful to practitioners who attempt to apply dropout to RL.

2 PRELIMINARIES

2.1 MAXIMUM ENTROPY REINFORCEMENT LEARNING (MAXIMUM ENTROPY RL)

RL addresses the problem of an agent learning to act in an environment. At each discrete time step t , the environment provides the agent with a state s_t , the agent responds by selecting an action a_t , and then the environment provides the next reward r_t and state s_{t+1} . For convenience, as needed, we use the simpler notations of r , s , a , s' , and a' to refer to a reward, state, action, next state, and next action, respectively.

We focus on *maximum entropy* RL, in which an agent aims to find its policy that maximizes the expected return with an entropy bonus: $\arg \max_{\pi} \sum_{t=0}^T \gamma^t \mathbb{E}_{s_t, a_t} [r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))]$. Here, $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, \infty)$ is a policy and $\mathcal{H}$ is entropy. Temperature α balances exploitation and exploration and affects the stochasticity of the policy.

Algorithm 1 REDQ

-
- 1: Initialize policy parameters θ , N Q-function parameters ϕ_i , $i = 1, \dots, N$, and empty replay buffer $\mathcal{D}$. Set target parameters $\bar{\phi}_i \leftarrow \phi_i$, for $i = 1, \dots, N$.
 - 2: **repeat**
 - 3: Take action $a_t \sim \pi_\theta(\cdot|s_t)$; Observe reward r_t , next state s_{t+1} ; $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r_t, s_{t+1})$.
 - 4: **for** G updates **do**
 - 5: Sample a mini-batch $\mathcal{B} = \{(s, a, r, s')\}$ from $\mathcal{D}$.
 - 6: Sample a set $\mathcal{M}$ of M distinct indices from $\{1, 2, \dots, N\}$.
 - 7: Compute the Q target y (same for all N Q-functions):

$$y = r + \gamma \left(\min_{i \in \mathcal{M}} Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a'|s') \right), \quad a' \sim \pi_\theta(\cdot|s')$$

- 8: **for** $i = 1, \dots, N$ **do**
- 9: Update ϕ_i with gradient descent using

$$\nabla_{\phi} \frac{1}{|\mathcal{B}|} \sum_{(s, a, r, s') \in \mathcal{B}} (Q_{\phi_i}(s, a) - y)^2$$

- 10: Update target networks with $\bar{\phi}_i \leftarrow \rho \bar{\phi}_i + (1 - \rho) \phi_i$.
- 11: Update θ with gradient ascent using

$$\nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \left(\frac{1}{N} \sum_{i=1}^N Q_{\phi_i}(s, a) - \alpha \log \pi_\theta(a|s) \right), \quad a \sim \pi_\theta(\cdot|s)$$

2.2 RANDOMIZED ENSEMBLED DOUBLE Q-LEARNING (REDQ)

REDQ (Chen et al., 2021b) is a sample-efficient model-free method for solving maximum-entropy RL problems (Algorithm 1). It has two primary components to achieve high sample efficiency.

1. High UTD ratio: It uses a high UTD ratio G , which is the number of updates (lines 4–10) taken by the agent compared to the number of actual interactions with the environment (line 3). The high UTD ratio promotes sufficient training of Q-functions within a few interactions. However, this also increases an overestimation bias in the Q-function training, which degrades sample-efficient learning (Chen et al., 2021b).

2. Ensemble of Q-functions: To reduce the overestimation bias, it uses an ensemble of N Q-functions for the target to be minimized (lines 6–7). Specifically, a random subset $\mathcal{M}$ of the ensemble is selected (line 6) then used for calculating the target (line 7). The size of the subset $\mathcal{M}$ is kept fixed and is denoted as M . In addition, each Q-function in the ensemble is randomly and independently initialized but updated with the same target (lines 8–9). Chen et al. (2021b) showed that using a large ensemble ($N = 10$) and small subset ($M = 2$) successfully reduces the bias.

Although using a large ensemble of Q-functions is beneficial for reducing bias and improving sample efficiency, this makes REDQ computationally intensive. In the next section, we discuss reducing the ensemble size.

3 INJECTING MODEL UNCERTAINTY INTO TARGET WITH DROPOUT Q-FUNCTIONS

In this section, we discuss replacing the large ensemble of Q-functions in REDQ with a small ensemble of dropout Q-functions. We start our discussion by reviewing what the ensemble in REDQ does from the viewpoint of model uncertainty injection. Then, we propose to use dropout for model uncertainty injection instead of the large ensemble. Specifically, we propose (i) the dropout Q-function that is a Q-function equipped with dropout and layer normalization, and (ii) DroQ, a variant of REDQ that uses a small ensemble of dropout Q-functions. Finally, we explain that the size of the ensemble can be smaller in DroQ than in REDQ.

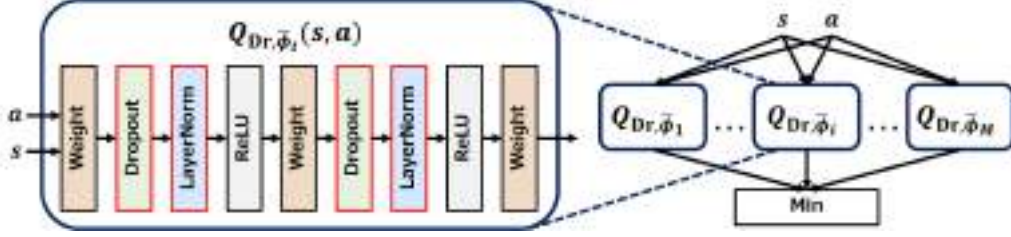


Figure 1: Dropout Q-function implementation (left part) and how dropout Q-functions are used in target (right part). **Dropout Q-function implementation:** Our dropout function is implemented by modifying that used by [Chen et al. \(2021b\)](#). Our modification (highlighted in red) is adding dropout (Dropout) and layer normalization (LayerNorm). “Weight” is a weight layer and “ReLU” is the activation layer of rectified linear units. Parameters $\bar{\phi}_i$ represent the weights and biases in weight layers. **How dropout Q-functions are used in target:** M dropout Q-functions are used to calculate the target value as $\min_{i=1, \dots, M} Q_{Dr, \bar{\phi}_i}(s, a)$.

We first explain our insight that, in REDQ, model (Q-function parameters’) uncertainty is injected into the target. In REDQ, the subset $\mathcal{M}$ of N Q-functions is used to compute the target value $\min_{i \in \mathcal{M}} Q_{\bar{\phi}_i}(s', a')$ (lines 6–7 in Algorithm 1). This can be interpreted as an approximation for $\mathbb{E}_{\bar{\phi}_1, \dots, \bar{\phi}_M} [\min_{i=1, \dots, M} Q_{\bar{\phi}_i}(s', a')]$, the expected target value with respect to model uncertainty:

$$\begin{aligned} \mathbb{E}_{\bar{\phi}_1, \dots, \bar{\phi}_M} \left[\min_{i=1, \dots, M} Q_{\bar{\phi}_i}(s', a') \right] &= \int \min_{i=1, \dots, M} Q_{\bar{\phi}_i}(s', a') \prod_{i=1, \dots, M} p(\bar{\phi}_i) d\bar{\phi}_i \\ &\approx \int \min_{i=1, \dots, M} Q_{\bar{\phi}_i}(s', a') \prod_{i=1, \dots, M} q_{\text{en}}(\bar{\phi}_i) d\bar{\phi}_i \approx \underbrace{\min_{i \in \mathcal{M}} Q_{\bar{\phi}_i}(s', a')}_{\text{(used in line 7 in Algorithm 1)}}. \end{aligned}$$

On the left hand side in the second line, the model distribution $p(\bar{\phi}_i)$ is replaced with a proposal distribution q_{en} , which is based on resampling from the ensemble in line 6 in Algorithm 1. On the right hand side (RHS) in the second line, the expected target value is further approximated by one sample average on the basis of q_{en} . The resulting approximation is used in line 7 in Algorithm 1. In the remainder of this section, we discuss another means to approximate the expected target value.

We use M dropout Q-functions Q_{Dr} for the target value approximation (Figure 1). Here, Q_{Dr} is a Q-function that has dropout connections ([Srivastava et al., 2014](#)). The left part of Figure 1 shows Q_{Dr} implementation by adding dropout layers to the Q-function implementation used by [Chen et al. \(2021b\)](#). Layer normalization ([Ba et al., 2016](#)) is applied after dropout for more effectively using dropout as with [Vaswani et al. \(2017\)](#); [Xu et al. \(2019\)](#). By using Q_{Dr} , the target value is approximated as

$$\begin{aligned} \mathbb{E}_{\bar{\phi}_1, \dots, \bar{\phi}_M} \left[\min_{i=1, \dots, M} Q_{\bar{\phi}_i}(s', a') \right] &\approx \int \min_{i=1, \dots, M} Q_{\bar{\phi}_i}(s', a') \prod_{i=1, \dots, M} q_{dr}(\bar{\phi}_i) d\bar{\phi}_i \\ &\approx \min_{i=1, \dots, M} Q_{Dr, \bar{\phi}_i}(s', a'). \end{aligned}$$

Instead of q_{en} , a proposal distribution q_{dr} based on the dropout is used on the RHS in the first line. The expected target value is further approximated by one sample average on the basis of q_{dr} in the second line. We use the resulting approximation $\min_{i=1, \dots, M} Q_{Dr, \bar{\phi}_i}(s', a')$ for injecting model uncertainty into the target value (the right part of Figure 1). For calculating $\min_{i=1, \dots, M} Q_{Dr, \bar{\phi}_i}(s', a')$, we use M dropout Q-functions, which have independently initialized and trained parameters $\bar{\phi}_1, \dots, \bar{\phi}_M$. Using M dropout Q-functions improves the performance of DroQ (our RL method described in the next paragraph), compared with that using a single dropout Q-function (further details are given in [A.3](#)).

We now explain DroQ, in which Q_{Dr} is used for considering model uncertainty. The algorithmic description of DroQ is shown in Algorithm 2. DroQ is a variant of REDQ, and the modified parts

¹We assume that $\forall_{i=1, \dots, N}$, $\bar{\phi}_i$ independently follow an identical distribution.

Algorithm 2 DroQ

- 1: Initialize policy parameters θ , M Q-function parameters ϕ_i , $i = 1, \dots, M$, and empty replay buffer $\mathcal{D}$. Set target parameters $\bar{\phi}_i \leftarrow \phi_i$, for $i = 1, \dots, M$.
- 2: **repeat**
- 3: Take action $a_t \sim \pi_\theta(\cdot|s_t)$. Observe reward r_t , next state s_{t+1} ; $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r_t, s_{t+1})$.
- 4: **for** G updates **do**
- 5: Sample a mini-batch $\mathcal{B} = \{(s, a, r, s')\}$ from $\mathcal{D}$.
- 6: Compute the Q target y for the dropout Q-functions:

$$y = r + \gamma \left(\min_{i=1, \dots, M} Q_{\text{Dr}, \bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a'|s') \right), \quad a' \sim \pi_\theta(\cdot|s')$$

- 7: **for** $i = 1, \dots, M$ **do**
- 8: Update ϕ_i with gradient descent using

$$\nabla_{\phi} \frac{1}{|\mathcal{B}|} \sum_{(s, a, r, s') \in \mathcal{B}} (Q_{\text{Dr}, \phi_i}(s, a) - y)^2$$

- 9: Update target networks with $\bar{\phi}_i \leftarrow \rho \bar{\phi}_i + (1 - \rho) \phi_i$.
- 10: Update θ with gradient ascent using

$$\nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \left(\frac{1}{M} \sum_{i=1}^M Q_{\text{Dr}, \phi_i}(s, a) - \alpha \log \pi_\theta(a|s) \right), \quad a \sim \pi_\theta(\cdot|s)$$

from REDQ are highlighted in red in the algorithm. In line 6, Q_{Dr} is used to inject model uncertainty into the target, as we discussed in the previous paragraph. In lines 8 and 10, M dropout Q-functions are used instead of N Q-functions ($N \geq M$) to make DroQ more computationally efficient.

The ensemble size of the dropout Q-functions for DroQ (i.e., M) should be smaller than that of Q-functions for REDQ (i.e., N). M for DroQ is equal to the subset size for REDQ, which is not greater than N . In practice, M is much smaller than N (e.g., $M = 2$ and $N = 10$ in [Chen et al. \(2021b\)](#)). This reduction in the number of Q-functions makes DroQ more computationally efficient. Specifically, DroQ is faster due to the reduction in the number of Q-function updates (line 8 in Algorithm 2). DroQ also requires less memory for holding Q-function parameters. In Section 4.2 we show that DroQ is computationally faster and less memory intensive than REDQ.

4 EXPERIMENTS

We conducted experiments to evaluate and analyse DroQ. In Section 4.1 we evaluate DroQ’s performance (sample efficiency and bias-reduction ability). In Section 4.2 we evaluate the computational efficiency of DroQ. In Section 4.3 we present an ablation study for DroQ.

4.1 SAMPLE EFFICIENCY AND BIAS-REDUCTION ABILITY OF DROQ

To evaluate the performances of DroQ, we compared DroQ with three baseline methods in MuJoCo benchmark environments ([Todorov et al., 2012](#); [Brockman et al., 2016](#)). Following [Chen et al. \(2021b\)](#); [Janner et al. \(2019\)](#), we prepared the following environments: **Hopper**, **Walker2d**, **Ant**, and **Humanoid**. In these environments, we compared the following four methods:

REDQ: Baseline method that follows the REDQ algorithm ([Chen et al., 2021b](#)) (Algorithm 1).

SAC: Baseline method that follows the SAC algorithm ([Haarnoja et al., 2018a,b](#)). To improve the sample efficiency of this method, delayed policy update and high UTD ratio $G = 20$ were used, as suggested by [Chen et al. \(2021b\)](#).

DroQ: Proposed method that follows Algorithm 2.

Double uncertainty value network (DUVN): Baseline method. This method is a DroQ variant that injects model uncertainty into a target in the same manner as the original DUVN ([Moerland et al., 2017](#)). The original DUVN uses the output of a single dropout Q-function $Q_{\text{Dr}, \bar{\phi}_1}(s', a')$ without

Table 1: Process times (in msec) per overall loop (e.g., lines 3–10 in Algorithm 2). Process times per Q-functions update (e.g., lines 4–9 in Algorithm 2) are shown in parentheses.

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
SAC	910 (870)	870 (835)	888 (848)	893 (854)
REDQ	2328 (2269)	2339 (2283)	2336 (2277)	2400 (2340)
DUVN	664 (636)	762 (731)	733 (700)	692 (660)
DroQ	948 (905)	933 (892)	954 (913)	989 (946)

layer normalization for target as $y = r + \gamma Q_{\text{Dr}, \bar{\phi}_1}(s', a')$. We introduce the original DUVN to DroQ by modifying target on line 6 in Algorithm 2 as $y = r + \gamma (Q_{\text{Dr}, \bar{\phi}_1}(s', a') - \alpha \log \pi_\theta(a'|s'))$. Here, layer normalization is not applied in $Q_{\text{Dr}, \bar{\phi}_1}(s', a')$. We use this method to investigate the performance of the existing *single* dropout Q-function (non-ensemble) method. Following Chen et al. (2021b), we set the hyperparameters as $G = 20$, $N = 10$, and $M = 2$ for all methods except DUVN ($M = 1$). More detailed hyperparameter settings are given in Appendix H.

The methods were compared on the basis of average return and estimation bias.

Average return: An average return over episodes. We regarded 1000 environment steps in Hopper and 3000 environment steps in the other environments as one epoch, respectively. After every epoch, we ran ten test episodes with the current policy and recorded the average return.

Average/std. bias: Average and standard deviation of the normalized estimation error (bias) of Q-functions Q_{ϕ_i} (Chen et al., 2021b). The error represents how significantly the Q-value estimate differs from the true one. Formally, the error is defined as $|Q^\pi(s, a) - \hat{Q}(s, a)| / \mathbb{E}_{\bar{s}, \bar{a} \sim \pi} [Q^\pi(\bar{s}, \bar{a})]$, where $Q^\pi(s, a)$ is the true Q-value under the current policy π and $\hat{Q}(s, a)$ is its estimate. In our experiment, $Q^\pi(s, a)$ was approximated by the discounted Monte Carlo return obtained with π in the test episodes. In addition, $\hat{Q}(s, a)$ was evaluated as $\hat{Q}(s, a) = 1/N \sum_{i=1}^N Q_{\phi_i}(s, a)$ for SAC and REDQ and as $\hat{Q}(s, a) = 1/M \sum_{i=1}^M Q_{\text{Dr}, \phi_i}(s, a)$ for DroQ and DUVN, respectively.

The comparison results (Figure 2) indicate that DroQ achieved almost the same level of performance as REDQ. Regarding the average return, REDQ and DroQ achieved almost the same sample efficiency overall. In Walker2d and Ant, their learning curves highly overlapped, and there was no significant difference between them. In Humanoid, REDQ was slightly better than DroQ. In Hopper, DroQ was better than REDQ. In all environments except Hopper, REDQ and DroQ improved their average return significantly earlier than SAC and DUVN. Regarding the bias, REDQ and DroQ consistently kept the value-estimation bias closer to zero than SAC and DUVN in all environments.

4.2 COMPUTATIONAL EFFICIENCY OF DROQ

We next evaluated the computational efficiency of DroQ. We compared DroQ with the baseline methods on the basis of the following criteria: (i) **Process time** required for executing methods; (ii) **Number of parameters** of each method; (iii) **Bottleneck memory consumption** suggested from the Pytorch profiler³. Bottleneck memory consumption is the maximum memory consumption recorded when running the methods. For evaluation, we ran each method on a machine equipped with two Intel(R) Xeon(R) CPU E5-2667 v4 and one NVIDIA Tesla K80.

Process times per update (numbers in parentheses in Table 1) indicate that DroQ runs more than two times faster than REDQ. DroQ (and SAC) requires process times in the 800–900-msec range. REDQ requires process times in the 2200–2300-msec range. Process times also show that learning Q-functions is dominant in an overall loop. This suggests that using compact (e.g., small numbers of) Q-functions is important for improving overall process times.

The number of parameters and bottleneck memory consumption of each method indicate that DroQ is more memory efficient than REDQ. Regarding the numbers of parameters (Table 2), we can see that those of DroQ (and SAC and DUVN) are about one-fifth those of REDQ. Note that the number of parameters of DroQ is equal to that of SAC since DroQ and SAC use the same number (two) of Q-functions. Regarding the bottleneck memory consumption (Table 3), we can see that that for DroQ (SAC and DUVN) is about one-third that for REDQ. We can also see that the bottleneck

³Harrigan (2016); He et al. (2021) also propose similar targets.

³https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html

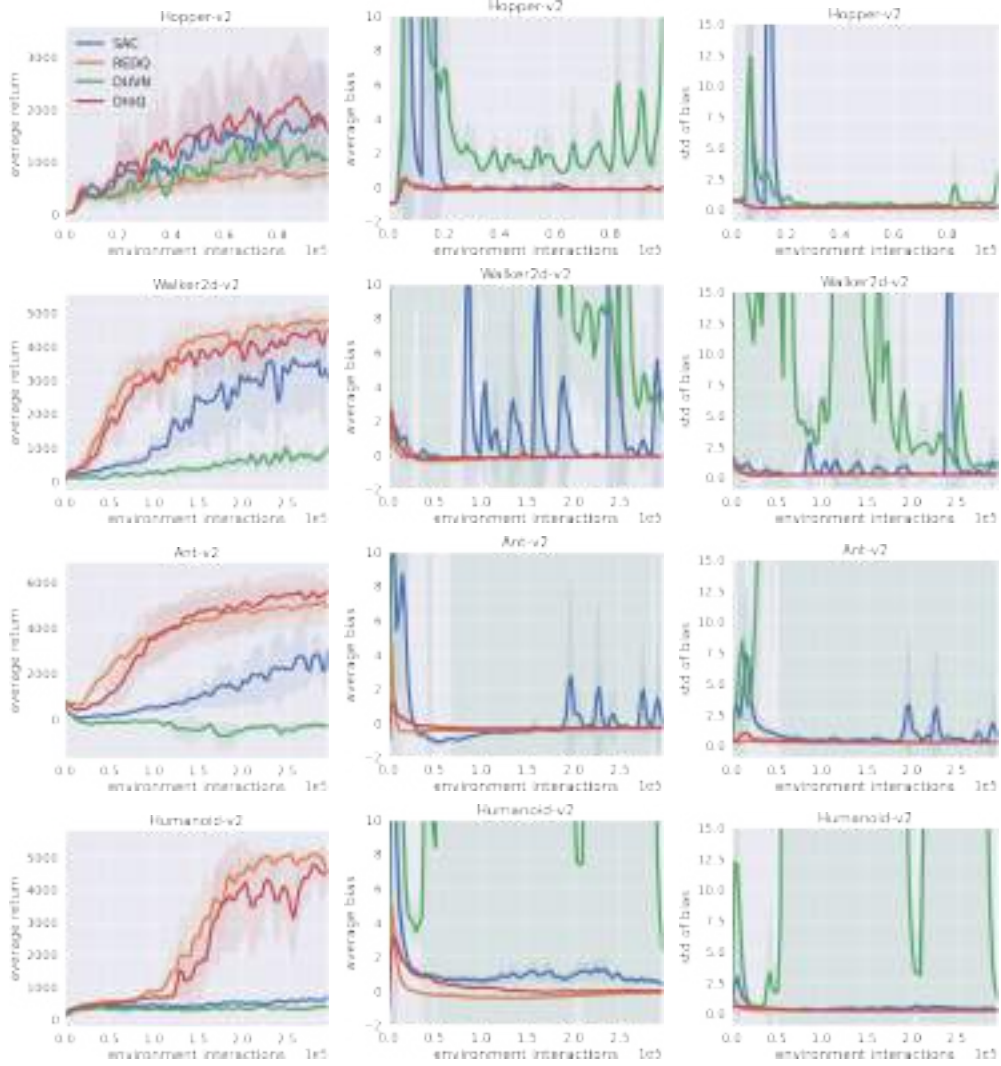


Figure 2: Average return and average/standard deviation of estimation bias for REDQ, SAC, DUVN, and DroQ. The horizontal axis represents the number of interactions with the environment (e.g., the number of executions of line 3 of Algorithm 2). For each method, average score of five independent trials are plotted as solid lines, and standard deviation across trials is plotted as transparent shaded region.

Table 2: Number of parameters

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
SAC	141,826	146,434	152,578	166,402
REDQ	698,890	721,930	752,650	821,770
DUVN	139,778	144,386	150,530	164,354
DroQ	141,826	146,434	152,578	166,402

memory consumption is almost independent of the environment. This is because one of the most memory-intensive parts is the ReLU activation at the hidden layers in Q-functions⁴.

⁴In our experiment, the number of units in hidden layer is invariant over environments (Appendix H).

Table 3: Bottleneck memory consumption (in megabytes) with methods. Three worst bottleneck memory consumptions are shown in form of “1st worst bottleneck memory consumption / 2nd worst bottleneck memory consumption / 3rd worst bottleneck memory consumption.”

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
SAC	73 / 64 / 62	73 / 64 / 62	73 / 64 / 62	73 / 65 / 62
REDQ	241 / 211 / 200	241 / 211 / 200	241 / 211 / 200	241 / 212 / 201
DUVN	72 / 71 / 51	72 / 71 / 51	72 / 71 / 51	72 / 71 / 52
DroQ	73 / 72 / 69	73 / 72 / 69	73 / 72 / 70	73 / 72 / 70

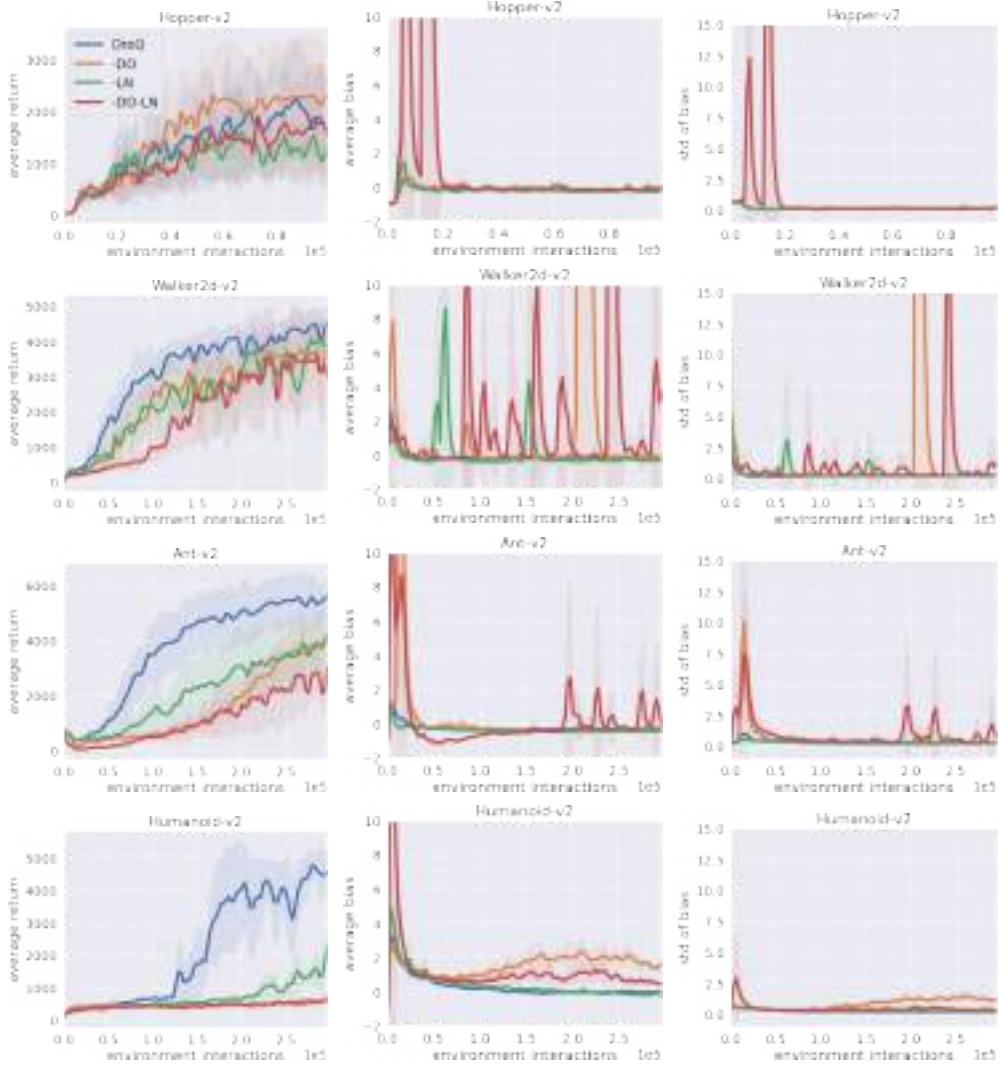


Figure 3: Ablation study results

4.3 ABLATION STUDY

As an ablation study of DroQ, we investigated the performance of DroQ variants that do not use either dropout, or layer normalization, or both. We refer to the one without dropout as **-DO**, that without layer normalization as **-LN**, and that without both as **-DO-LN**. The results (Figure 3) indicate that the synergic effect of using dropout and layer normalization is high, especially in complex environments (Ant and Humanoid). In these environments, DroQ significantly outperformed its ablation variants (-DO, -LN and -DO-LN) in terms of both average return and bias reduction.

Table 4: Comparison between related studies and ours. We classify related studies into five types (e.g., “Ensemble Q-functions”) on basis of three criteria (e.g. “Type of model uncertainty”).

Type of study	Method for injecting model uncertainty	Type of model uncertainty	Focus on high UTD ratio setting?
Ensemble Q-functions	Ensemble	Q-functions	Partially yes Chen et al. (2021b)
Ensemble transition models	Ensemble	Transition models	Partially yes e.g., Janner et al. (2019)
Dropout Q-functions	Dropout	Q-functions	No
Dropout transition models	Dropout	Transition models	No
Normalization	–	–	No
Our study	Dropout (with ensemble)	Q-functions	Yes

5 RELATED WORK

In this section, we review related studies and compare them with ours (Table 4).

Ensemble Q-functions: Ensembles of Q-functions have been used in RL to consider model uncertainty (Faußer & Schwenker, 2015; Osband et al., 2016; Anschel et al., 2017; Osband et al., 2018; Agarwal et al., 2020; Lee et al., 2021; Lan et al., 2020; Chen et al., 2021b). **Ensemble transition models:** Ensembles of transition (and reward) models have been introduced to model-based RL, e.g., (Chua et al., 2018; Kurutach et al., 2018; Janner et al., 2019; Shen et al., 2020; Yu et al., 2020; Lee et al., 2020; Hiraoka et al., 2021; Abraham et al., 2020). The methods proposed in the above studies use a large ensemble of Q-functions or transition models, thus are computationally intensive. DroQ does not use a large ensemble of Q-functions, and thus is computationally lighter.

Dropout transition models: Gal et al. (2016, 2017); Kahn et al. (2017) introduced dropout and its modified variant to transition models of model-based RL methods. **Dropout Q-functions:** Gal & Ghahramani (2016) introduced dropout to a Q-function at action selection for considering model uncertainty in exploration. Harrigan (2016); Moerland et al. (2017); Jaques et al. (2019); He et al. (2021) introduced dropout to policy evaluation in the same vein as us. However, there are three main differences between these studies and ours. (i) Target calculation: they introduced dropout to a *single* Q-function and used it for a target value, whereas we introduce dropout to *multiple* Q-functions and use the minimum of their outputs for the target value. (ii) Use of engineering to stabilize learning: their methods do not use engineering to stabilize the learning of dropout Q-functions, whereas DroQ uses layer normalization to stabilize the learning. (iii) RL setting: they focused on a low UTD ratio setting, whereas we focused on a high UTD ratio setting. A high UTD ratio setting increases a high estimation bias, and thus is a more challenging setting than a low UTD ratio setting. In Sections 4.1 and A.3, we showed that their methods do not perform successfully in a high UTD setting.

Normalization in RL: Normalization (e.g., batch normalization (Ioffe & Szegedy, 2015) or layer normalization (Ba et al., 2016)) has been introduced into RL. Batch normalization and its variant are introduced in deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015) and twin delayed DDPG (Fujimoto et al., 2018) (Bhatt et al., 2020). Layer normalization is introduced in the implementation of maximum a posteriori policy optimisation (Abdolmaleki et al., 2018; Hoffman et al., 2020). It is also introduced in SAC extensions (Ma et al., 2020; Zhang et al., 2021). Unlike our study, the above studies did not introduce dropout to consider model uncertainty. In Section 4.3, we showed that using layer normalization alone is not sufficient in high UTD settings. In Appendix E, we also show that using batch normalization does not contribute to performance improvement in high UTD settings.

6 CONCLUSION

We proposed, DroQ, an RL method based on a small ensemble of Q-functions that are equipped with dropout connection and layer normalization. We experimentally demonstrated that DroQ significantly improves computational and memory efficiencies over REDQ while achieving sample efficiency comparable with REDQ. In the ablation study, we found that using both dropout and layer normalization had a synergistic effect on improving sample efficiency, especially in complex environments (e.g., Humanoid).

A EFFECT OF DROPOUT RATE ON DROQ AND ITS VARIANTS

A.1 DROQ

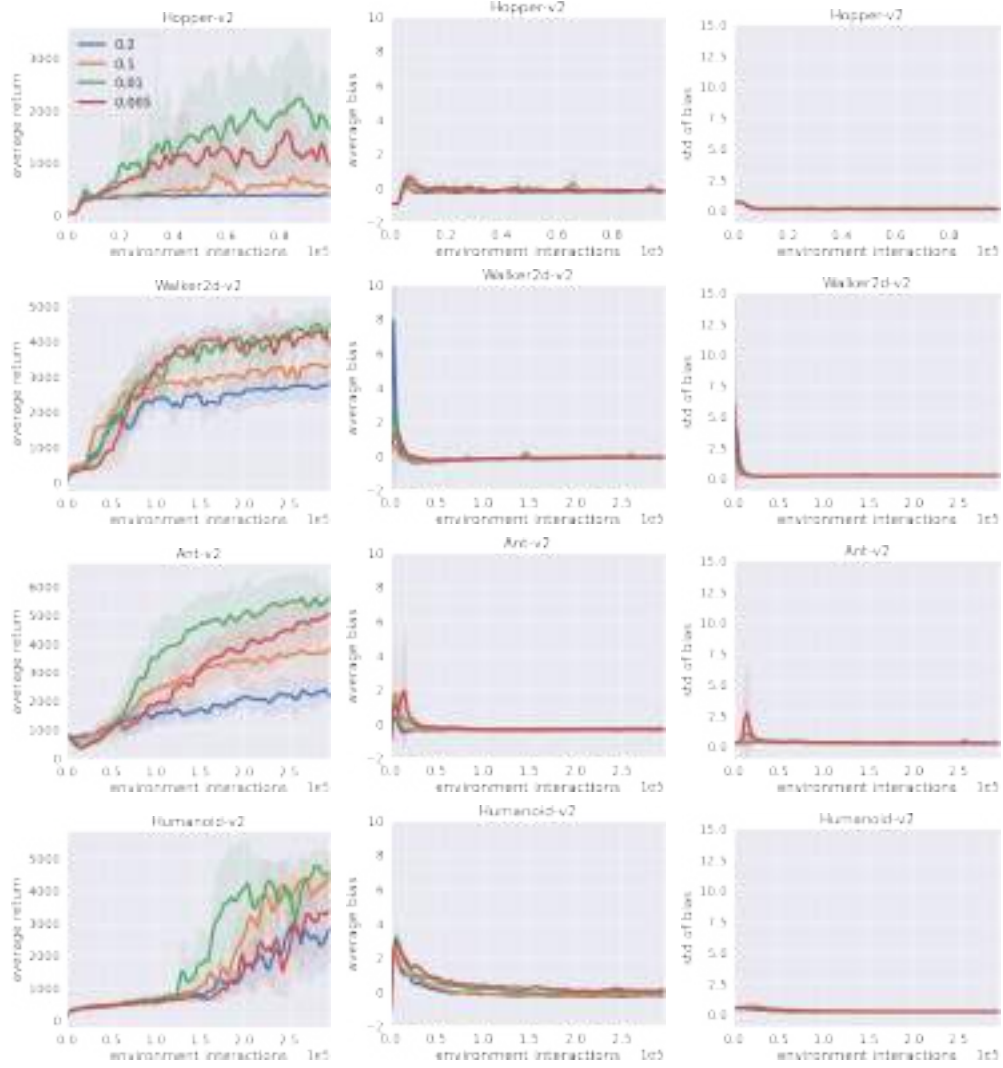


Figure 4: Average return and average/standard deviation of estimation bias for DroQ with different dropout rates. Scores for DroQ are plotted as solid lines and labelled in accordance with dropout rates (e.g., “0.2”).

A.2 DROQ WITHOUT LAYER NORMALIZATION

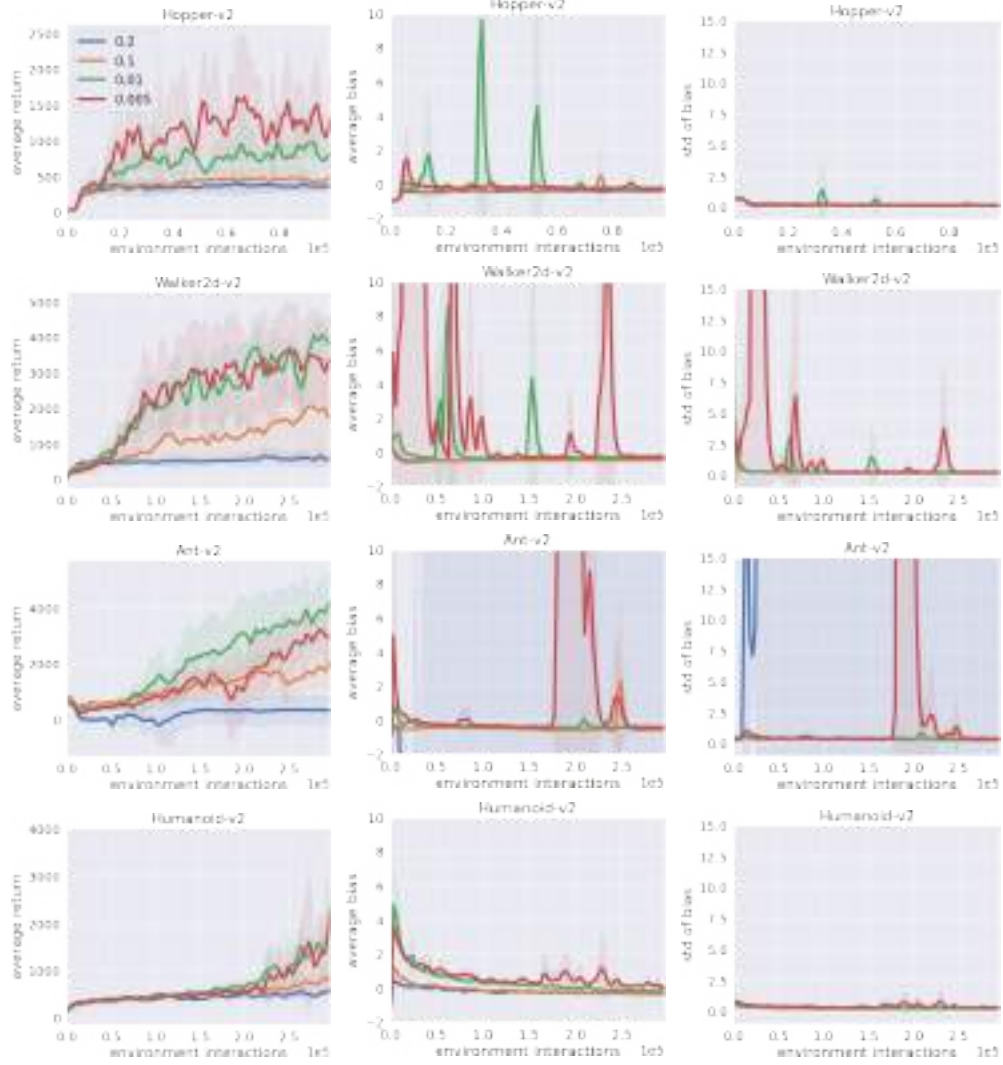


Figure 5: Average return and average/standard deviation of estimation bias for DroQ without layer normalization (i.e., -LN in Section 4.3) with different dropout rates

A.3 SIN-DROQ: DROQ VARIANT USING A SINGLE DROPOUT Q-FUNCTION

DroQ (Algorithm 2) uses an ensemble of multiple (M) dropout Q-functions. This raises the question: “Why not use a single dropout Q-function for DroQ?” To answer this question, we compared DroQ with **Sin-DroQ**, a variant of DroQ that uses a single dropout Q-function. Specifically, with Sin-DroQ, the target in line 6 in Algorithm 2 is calculated by evaluating $Q_{\text{Dr}\bar{\phi}_1}(s', a')$, a single dropout Q-function, M times:

$$y = r + \gamma \left(\underbrace{\min(Q_{\text{Dr}\bar{\phi}_1}(s', a'), \dots, Q_{\text{Dr}\bar{\phi}_1}(s', a'))}_{M \text{ evaluation results of } Q_{\text{Dr}\bar{\phi}_1}(s', a')} - \alpha \log \pi_\theta(a'|s') \right), \quad a' \sim \pi_\theta(\cdot|s').$$

Note that, the output of $Q_{\text{Dr}\bar{\phi}_1}(s', a')$ can differ in each evaluation due to the use of a dropout connection. It should be also noted that this target is the same as the one proposed in Jaques et al. (2019). The remaining part of Sin-DroQ is the same as DroQ.

From the comparison results of DroQ and Sin-DroQ (Figure 6), we can see that the average return of Sin-DroQ was lower than that of DroQ. This result indicates that using multiple dropout Q-functions is preferable to using a single dropout Q-function with DroQ.

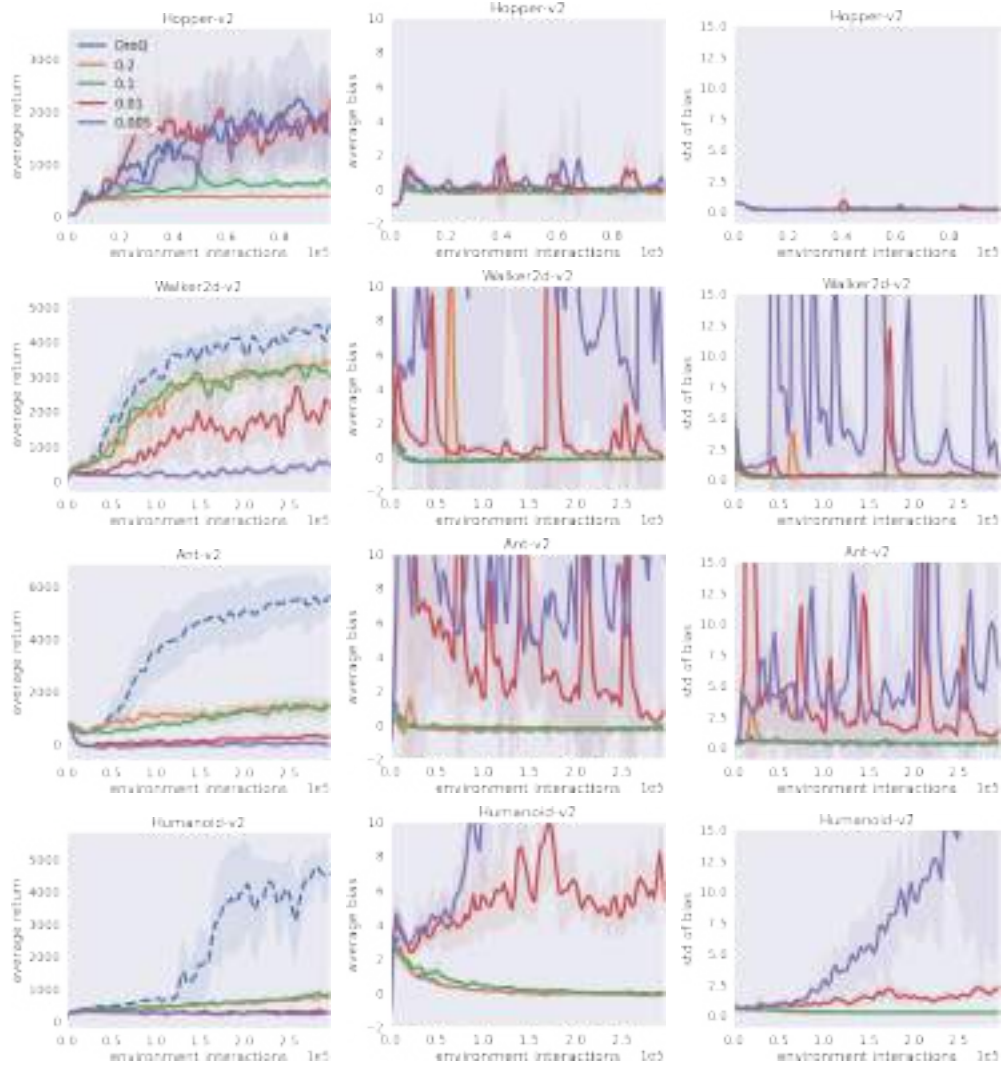


Figure 6: Average return and average/standard deviation of estimation bias for DroQ and Sin-DroQ. Scores for DroQ are plotted as dash lines. Scores for Sin-DroQ are plotted as solid lines and labelled in accordance with dropout rates (e.g., “0.2”).

B REDQ WITH DIFFERENT ENSEMBLE SIZE N

In Section 4, we discussed comparing DroQ with REDQ, which uses an ensemble size of 10 (i.e., $N=10$). To make a more detailed comparison, we compared DroQ and REDQ by varying the ensemble size for REDQ. We denote REDQ that uses an ensemble size of N as “REDQ N ” (e.g., REDQ5 for REDQ with an ensemble size of five).

Regarding average return (left part of Figure 7), overall, DroQ was superior to REDQ2–5. DroQ was comparable with REDQ2 and REDQ3 in Hopper but superior in more complex environments (Walker, Ant, and Humanoid). In addition, DroQ was somewhat better than REDQ5 in all environments. Regarding estimation bias (middle and right parts of Figure 7), overall, DroQ was significantly better than REDQ2 and comparable with REDQ3–10. Regarding the processing speed (Table 5), DroQ ran as fast as REDQ3 and faster than REDQ5 by 1.4 to 1.5 times. Regarding memory efficiency (Tables 6 and 7), DroQ was less memory intensive than REDQ3–10.

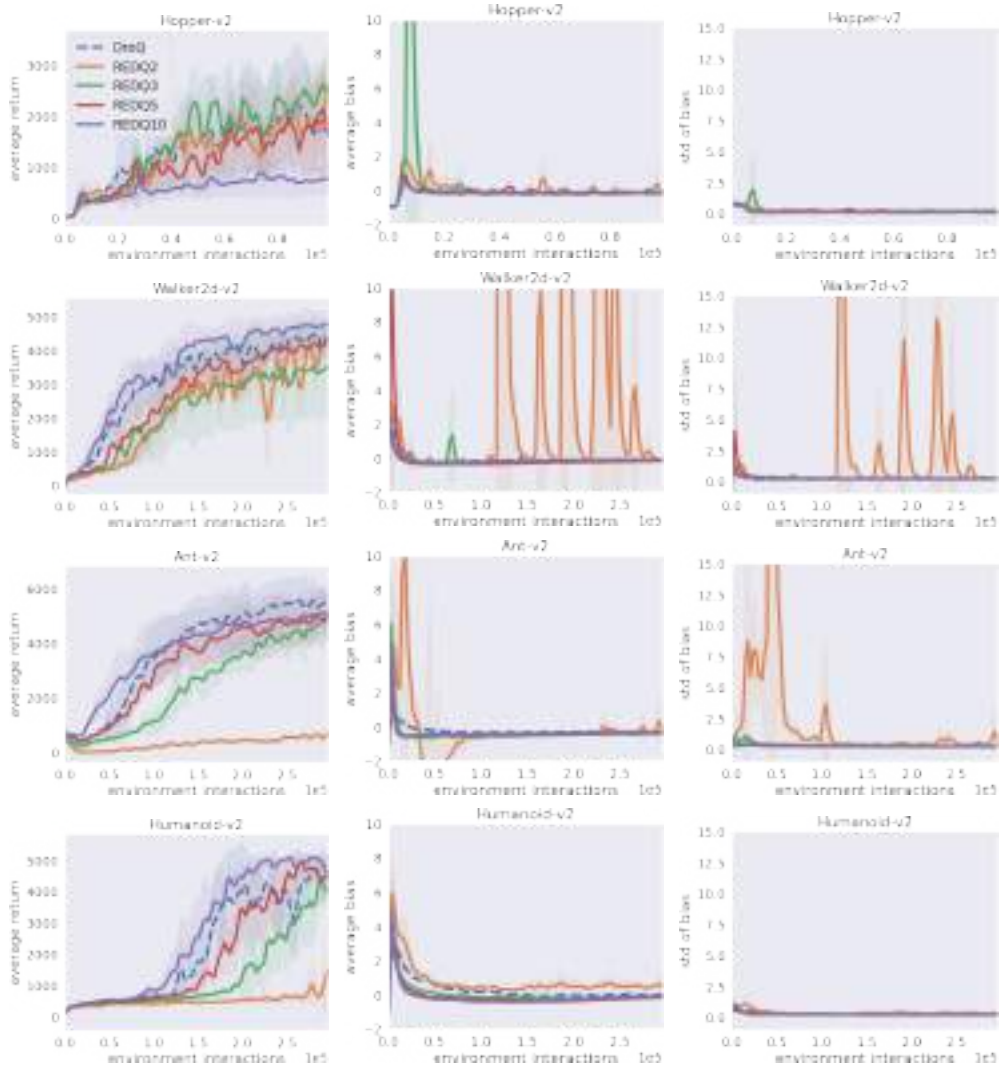


Figure 7: Average return and average/standard deviation of estimation bias for DroQ and REDQ. Scores for DroQ are plotted as dash lines. Scores for REDQ are plotted as solid lines and labelled as “REDQ N ”.

Table 5: Process times (in msec) per overall loop (e.g., lines 3–10 in Algorithm 2). Process times per Q-functions update (e.g., lines 4–9 in Algorithm 2) are shown in parentheses.

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
DroQ	948 (905)	933 (892)	954 (913)	989 (946)
REDQ2	832 (792)	802 (768)	675 (641)	820 (773)
REDQ3	1052 (1014)	876 (838)	919 (881)	950 (906)
REDQ5	1414 (1368)	1425 (1378)	1373 (1327)	1552 (1503)
REDQ10	2328 (2269)	2339 (2283)	2336 (2277)	2400 (2340)

Table 6: Number of parameters

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
DroQ	141,826	146,434	152,578	166,402
REDQ2	139,778	144,386	150,530	164,354
REDQ3	209,667	216,579	225,795	246,531
REDQ5	349,445	360,965	376,325	410,885
REDQ10	698,890	721,930	752,650	821,770

Table 7: Bottleneck memory consumption (in megabytes) with methods. Three worst bottleneck memory consumptions are shown in form of “1st worst bottleneck memory consumption / 2nd worst bottleneck memory consumption / 3rd worst bottleneck memory consumption.”

	Hopper-v2	Walker2d-v2	Ant-v2	Humanoid-v2
DroQ	73 / 72 / 69	73 / 72 / 69	73 / 72 / 70	73 / 72 / 70
REDQ2	73 / 51 / 51	73 / 51 / 51	73 / 51 / 51	73 / 52 / 52
REDQ3	94 / 64 / 62	94 / 64 / 62	94 / 64 / 62	94 / 65 / 62
REDQ5	136 / 106 / 100	136 / 106 / 100	136 / 106 / 100	136 / 107 / 101
REDQ10	241 / 211 / 200	241 / 211 / 200	241 / 211 / 200	241 / 212 / 201

C ADDITIONAL ABLATION STUDY OF DROQ

Dropout is introduced into three parts of the algorithm for DroQ (i.e., lines 6, 8 and 10 of Algorithm 2 in Section 3). In this section, we conducted an ablation study to answer the question “which dropout introduction contributes to overall performance improvement of DroQ?” We compared DroQ with its following variants:

-DO@TargetQ: A DroQ variant that does not use dropout in line 6. Specifically, dropout is not used in $Q_{\text{Dr}, \bar{\phi}_i}(s', a')$ in the following part in line 6.

$$y = r + \gamma \left(\min_{i=1, \dots, M} Q_{\text{Dr}, \bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a'|s') \right), \quad a' \sim \pi_\theta(\cdot|s')$$

-DO@CurrentQ: A DroQ variant that does not use dropout in line 8. Specifically, dropout is not used in $Q_{\text{Dr}, \phi_i}(s, a)$ in the following part in line 8.

$$\nabla_\phi \frac{1}{|\mathcal{B}|} \sum_{(s, a, r, s') \in \mathcal{B}} (Q_{\text{Dr}, \phi_i}(s, a) - y)^2$$

-DO@PolicyOpt: A DroQ variant that does not use dropout in line 10. Specifically, dropout is not used in $Q_{\text{Dr}, \phi_i}(s, a)$ in the following part in line 10.

$$\nabla_\theta \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \left(\frac{1}{M} \sum_{i=1}^M Q_{\text{Dr}, \phi_i}(s, a) - \alpha \log \pi_\theta(a|s) \right), \quad a \sim \pi_\theta(\cdot|s)$$

-DO: A DroQ variant that does not use dropout in lines 6, 8, and 10.

In this ablation study, we compared the methods on the basis of average return and estimation bias.

The comparison results (Figure 8) indicate that using dropout for both current and target Q-functions (i.e., $Q_{\text{Dr}, \bar{\phi}_i}(s', a')$ and $Q_{\text{Dr}, \phi_i}(s, a)$ in lines 6 and 8) is effective. Regarding average return, the DroQ variants that do not use dropout for either target Q-functions in line 6 or current Q-functions in line 8 perform significantly worse than that of DroQ. For example, in Ant, the variants that do not use dropout for target Q-functions (-DO@TargetQ and -DO) perform much worse than DroQ. In addition, in Humanoid, the variants that do not use dropout for current Q-functions (-DO@CurrentQ and -DO) perform much worse than DroQ. Regarding estimation bias, that of the variants that do not use dropout for target Q-functions (-DO@TargetQ and -DO) is significantly worse than that of DroQ in all environments.

These results complement the results presented in He et al. (2021). They show that using dropout for both current and target Q-functions improves the performance of the maximum entropy RL method (SAC) in low UTD settings. Our results presented in the previous paragraph implies that their insight (i.e., “using dropout for both current and target Q-functions improves the performance”) is valid, even in high UTD settings.

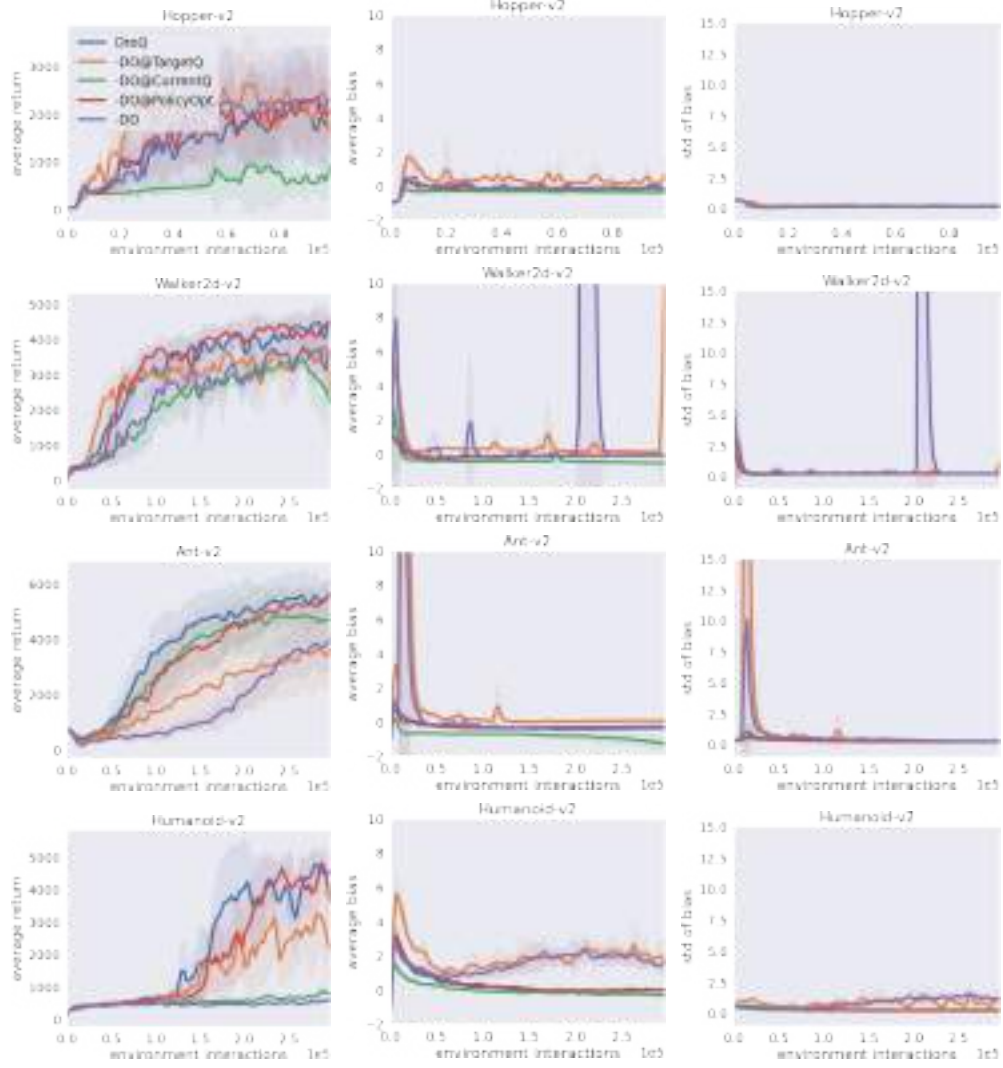


Figure 8: Additional ablation study result about dropout for DroQ

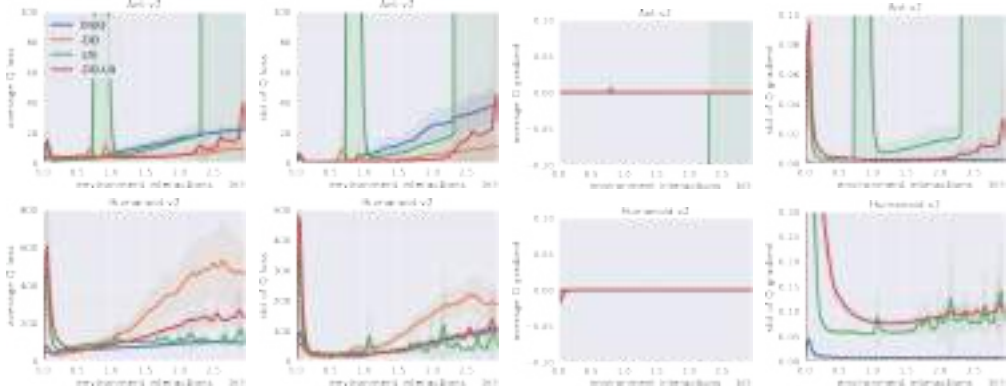


Figure 9: Average/standard deviation of the Q-function loss (“average Q loss” and “std of Q loss”) and its gradient with respect to parameters (“average Q gradient” and “std of Q gradient”). Q-function loss is calculated as $\frac{1}{M} \sum_{i=1}^M L_i$, where $L_i := (Q_{\phi_i}(s, a) - r - \gamma (\min_{i \in \mathcal{M}} Q_{\phi_i}(s', a') - \alpha \log \pi_{\theta}(a'|s')))^2$, and its gradient is calculated as $\frac{1}{M} \sum_{i=1}^M \frac{\partial L_i}{\partial \phi_i}$. The definition of method labels (e.g., “DroQ” or “-DO”) in the legend is the same as that used in Figure 3.

D WHY IS THE COMBINATION OF DROPOUT AND LAYER NORMALIZATION IMPORTANT?

Figure 3 in Section 4.3 showed that combining dropout and layer normalization improves overall performance, especially in complex environments (Ant and Humanoid).

The main reason for this performance improvement could be that layer normalization suppresses the learning instability caused by dropout. Using dropout destabilizes the Q-functions learning. Figure 9 shows that (i) the Q-function loss of the method using dropout (-LN) oscillates more significantly than that of the method not using dropout (-DO-LN), and that (ii) the variance of the gradient loss with respect to the Q-function parameters also oscillates accordingly. Layer normalization suppresses such learning instabilities. The right part of Figure 9 shows that the method using both dropout and layer normalization (DroQ) suppresses the oscillations of the gradient more significantly than the -LN. This stabilization of learning could enable better Q-value propagation and consequently improve overall performance.

E EFFECTS OF OTHER NORMALIZATION METHODS

In Section 4.3 and Appendix D, we demonstrated that combining dropout and layer normalization is effective in improving performance. In this section, we examine the effectiveness of normalization methods other than layer normalization.

We examine the effects of (i) batch normalization (Ioffe & Szegedy, 2015), (ii) group normalization (Wu & He, 2018), and (iii) layer normalization without variance re-scaling. Batch normalization is an off-the-shelf and popular normalization method, and has been introduced into RL in low UTD settings (Bhatt et al., 2020). We introduce this popular method to verify whether it is effective in high UTD settings. Group normalization is a method of normalization using the average and variance over the elements of a target input, which is similar to layer normalization⁵. We introduce this method to verify whether a method equipped with a similar mechanism to layer normalization works well in high UTD settings. Layer normalization without variance re-scaling is the layer normalization variant that does not re-scale the target input’s variance. Xu et al. (2019) pointed out that variance re-scaling is an important component for layer normalization to work well. We introduce this method to verify whether variance re-scaling is an important component also in our settings.

We compare six DroQ variants that use these three normalization methods. We denote -DO⁶ variants using batch normalization, group normalization, and layer normalization without variance re-scaling, instead of layer normalization, as +BN, +GN, and +LNwoVR, respectively. We also denote DroQ variants using batch normalization, group normalization, and layer normalization without variance re-scaling, instead of layer normalization, as +DO+BN, +DO+GN, and +DO+LNwoVR, respectively. We compare these six methods (+BN, +GN, +LNwoVR, +DO+BN, +DO+GN, and +DO+LNwoVR).

From the experimental results of the six methods (Figures 10 and 11), we get the following insights:

1. Batch normalization does not work well. Figure 10 shows that the methods using batch normalization (+BN and +DO+BN) do not significantly improve the average return and estimation bias in Humanoid and Ant. Further, Figure 11 shows that the Q-function learning with these methods is very unstable.
2. Group normalization, which has a similar mechanism to layer normalization, works well when combined with dropout. Figure 10 shows that the method using the group normalization and dropout (+DO+GN) successfully improves the average return and estimation bias in Humanoid and Ant. The figure also shows that the method that uses only group normalization (+GN) fails in improving the average return and estimation bias.
3. Layer normalization without variance re-scaling does not work well. Figure 10 shows that the methods that introduce this layer normalization variant (+LNwoVR and +DO+LNwoVR) do not significantly improve the average return and estimation bias well especially in Humanoid. Further, Figure 11 shows that the Q-function learning is unstable compared to +DO+GN (and DroQ in Figure 9) in both Ant and Humanoid. This result implies that one of the primal factors for the synergistic effect of dropout and layer normalization is the variance re-scaling.

⁵Layer normalization normalizes all input elements using their mean and variance. On the other hand, group normalization divides the input elements into subgroups (two groups in our setting) and normalizes each group using the intra-group mean and variance.

⁶DroQ without dropout, which is introduced in Sections 4.3 and D.

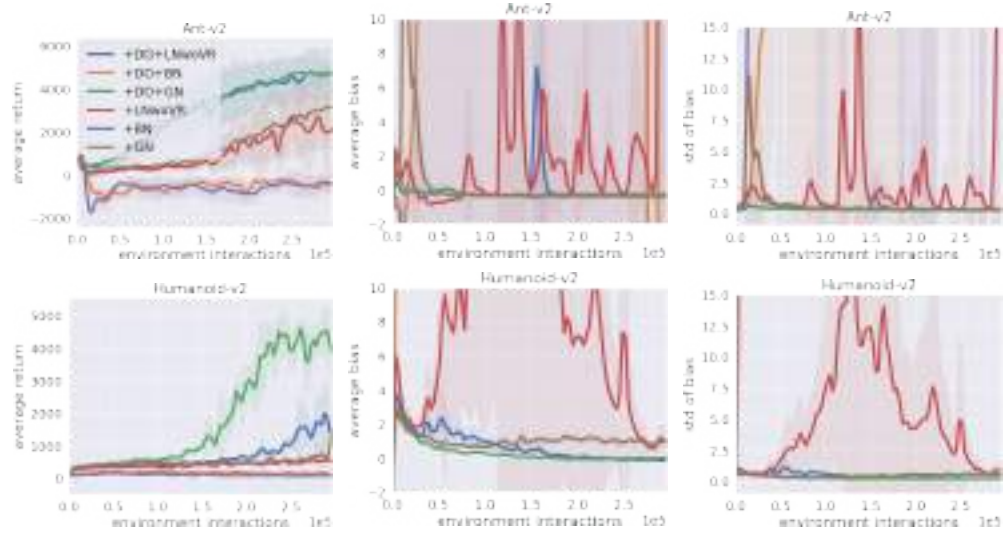


Figure 10: Average return and average/standard deviation of estimation bias for six methods using batch normalization, group normalization, and the variant of layer normalization.

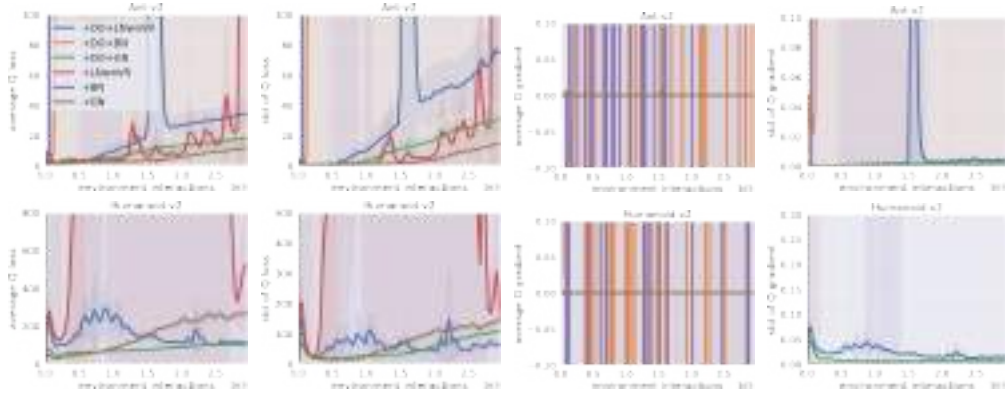


Figure 11: Average/variance of the Q-function loss (left part) and its gradient with respect to parameters (right part).

Algorithm 3 DroQ with ensemble of N dropout Q-functions (DroQN)

- 1: Initialize policy parameters θ , N Q-function parameters ϕ_i , $i = 1, \dots, N$, and empty replay buffer $\mathcal{D}$. Set target parameters $\bar{\phi}_i \leftarrow \phi_i$, for $i = 1, \dots, N$.
- 2: **repeat**
- 3: Take action $a_t \sim \pi_\theta(\cdot|s_t)$. Observe reward r_t , next state s_{t+1} ; $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r_t, s_{t+1})$.
- 4: **for** G updates **do**
- 5: Sample a mini-batch $\mathcal{B} = \{(s, a, r, s')\}$ from $\mathcal{D}$.
- 6: **Sample a set $\mathcal{M}$ of M distinct indices from $\{1, 2, \dots, N\}$.**
- 7: Compute the Q target y for the dropout Q-functions:

$$y = r + \gamma \left(\min_{i \in \mathcal{M}} Q_{\text{Dr}, \bar{\phi}_i}(s', a') - \alpha \log \pi_\theta(a'|s') \right), \quad a' \sim \pi_\theta(\cdot|s')$$

- 8: **for** $i = 1, \dots, N$ **do**
- 9: Update ϕ_i with gradient descent using

$$\nabla_{\phi} \frac{1}{|\mathcal{B}|} \sum_{(s, a, r, s') \in \mathcal{B}} (Q_{\text{Dr}, \phi_i}(s, a) - y)^2$$

- 10: Update target networks with $\bar{\phi}_i \leftarrow \rho \bar{\phi}_i + (1 - \rho) \phi_i$.
- 11: Update θ with gradient ascent using

$$\nabla_{\theta} \frac{1}{|\mathcal{B}|} \sum_{s \in \mathcal{B}} \left(\frac{1}{N} \sum_{i=1}^N Q_{\text{Dr}, \phi_i}(s, a) - \alpha \log \pi_\theta(a|s) \right), \quad a \sim \pi_\theta(\cdot|s)$$

F RELATION BETWEEN (1) ENSEMBLE SIZE AND (2) THE EFFECT OF LAYER NORMALIZATION AND DROPOUT ON OVERALL PERFORMANCE

In Section 4.3 and Appendix D, we showed that using layer normalization and dropout improves overall performance. Further, in Sections 4.1 and A.3, we showed that using multiple (two) dropout Q-functions improves overall performance. These results raise two new questions: (i) *what happens if we use more than three dropout Q-functions?* and (ii) *what the effect of layer normalization and dropout would be in such cases?* In this section, we conduct an additional experiment to answer questions (i) and (ii).

For our additional experiment, we introduce the **DroQ with ensemble of N dropout Q-functions** (DroQN) algorithm (Algorithm 3). This algorithm is a variant of DroQ algorithm (Algorithm 2) which uses bootstrapping of a subset $\mathcal{M}$ of N dropout Q-functions (line 6). This algorithm can also be regarded as a variant of REDQ algorithm (Algorithm 1) which uses N dropout Q-functions. With DroQN algorithm, the ensemble size can be changed by varying the value of N .

We compare the method based on the **DroQN** algorithm and its ablation variants:

DroQN-LN: the DroQN variant that does not use layer normalization for the dropout Q-function Q_{Dr, ϕ_i} .

DroQN-DO-LN: the DroQN variant that does not use layer normalization and dropout for Q_{Dr, ϕ_i} . Note that DroQN-DO-LN is identical with REDQN presented in Appendix B.

We evaluate the performance of these methods for the case of $N = 2, 3, 5, 10$.

Learning curves of the methods are shown in Figure 12.

For the question (i), in complex environments (Ant and Humanoid), increasing the size of the ensemble improves the sample efficiency. In Ant, DroQ5–10 are more sample efficient than DroQ2–3. In Humanoid, the sample efficiency is monotonically improved when the ensemble size of DroQN is increased as $N = 2, 3, 5, 10$.

For the question (ii), in the complex environments, the synergy of layer normalization and dropout is significant especially when the value of N is small ($N = 2, 3$). In Ant and Humanoid, DroQ2–3 significantly improves the sample efficiency compared to DroQ2–3-DO. On the other hand, the effect of layer normalization itself becomes dominant when the value of N is large ($N \geq 5$). For Ant and

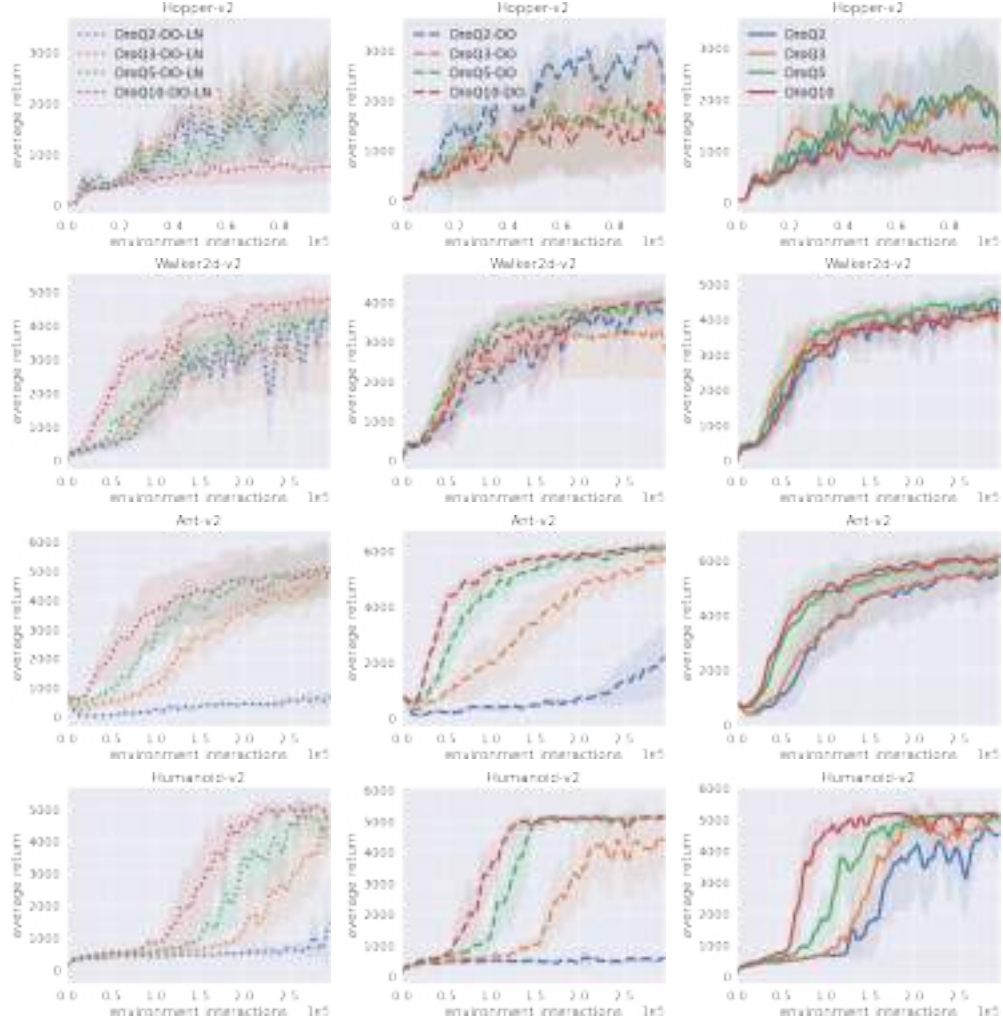


Figure 12: Additional ablation study result (average return). Left part (dotted lines): DroQ N -DO-LN, middle part (dashed lines): DroQ N -LN, right part (solid lines): DroQ N

Humanoid, we can see that DroQ5-10-DO significantly improves the sample efficiency compared to DroQ5-10-DO-LN. However, DroQ5-10 does not significantly improve the sample efficiency compared to DroQ5-10-DO.

Based on the above result, we suggest (1) using layer normalization and dropout together for small ensemble cases, and (2) using layer normalization alone for large ensemble cases.

G EFFECT OF DROPOUT Q-FUNCTIONS ON SAC

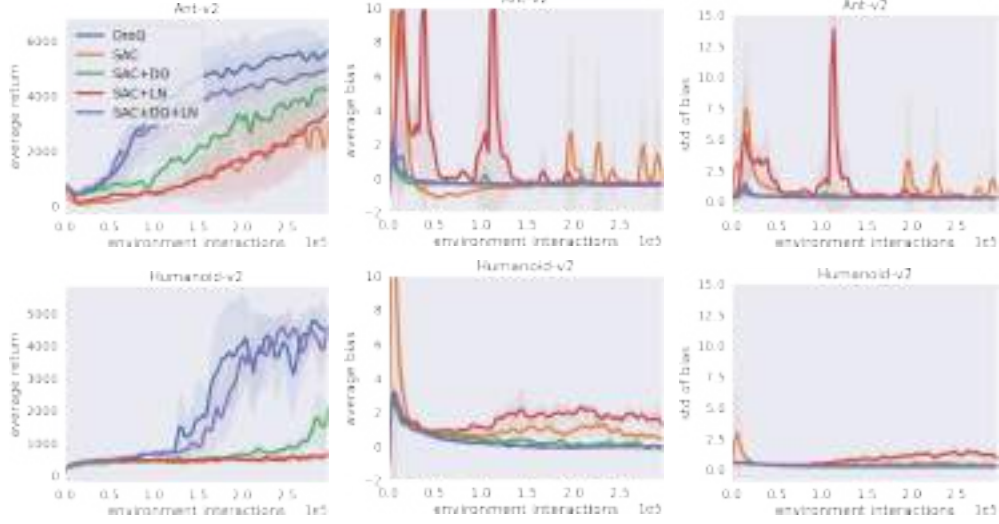


Figure 13: Average return and average/standard deviation of estimation bias for DroQ, SAC, and its variants. SAC+DO is a SAC variant using dropout, SAC+LN is a SAC variant using layer normalization, and SAC+DO+LN is a SAC variant using dropout and layer normalization. Note that the only difference between DroQ ($M = 2$) and SAC+DO+LN is the policy update part (line 10 in Algorithm 2). DroQ ($M = 2$) updates policy as $\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} \left(\frac{1}{2} \sum_{i=1}^2 Q_{\text{Dr}, \phi_i}(s, a) - \alpha \log \pi_{\theta}(a|s) \right)$. On the other hand, SAC+DO+LN updates policy as $\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} \left(\frac{1}{2} \min_{i=1}^2 Q_{\text{Dr}, \phi_i}(s, a) - \alpha \log \pi_{\theta}(a|s) \right)$.

H HYPERPARAMETER SETTINGS

The hyperparameter settings for each method in the experiments discussed in Section 4 are listed in Table 8. Parameter values, except for (i) dropout rate for DroQ and DUVN and (ii) M for DUVN, were set according to Chen et al. (2021b). The dropout rate (i) was set through line search, and M for DUVN (ii) was set according to Harrigan (2016); Moerland et al. (2017).

Table 8: Hyperparameter settings

Method	Parameter	Value
SAC, REDQ, DroQ, and DUVN	optimizer	Adam (Kingma & Ba, 2015)
	learning rate	$3 \cdot 10^2$
	discount rate (γ)	0.99
	target-smoothing coefficient (ρ)	0.005
	replay buffer size	10^6
	number of hidden layers for all networks	2
	number of hidden units per layer	256
	mini-batch size	256
	random starting data	5000
	UTD ratio G	20
REDQ and DroQ	in-target minimization parameter M	2
REDQ	ensemble size N	10
DroQ and DUVN	dropout rate	0.01
DUVN	in-target minimization parameter M	1

I EXPERIMENTS ON THE ORIGINAL REDQ CODEBASE

Experiments presented in the main content of this paper have been implemented on top of the SAC codebase (<https://github.com/ku2482/soft-actor-critic.pytorch>). In this section, we report the results of our experiments on the original REDQ codebase (<https://github.com/watchernyu/REDQ>). Our experiments are to replicate experiments in the main content of this paper. To evaluate the computational efficiency, we plot the wallclock time required to complete a certain number of interactions with the environment, instead of the standard deviation of bias. In addition, dropout rate values are re-tuned for each environment (Table 9). Figure 14 shows the replica of the results presented in Section 4.1. Figure 15 shows the replica of the results provided in Section 4.3. Figure 16 shows the replica of the results provided in Section A.1. Figure 17 shows the replica of the results provided in Section D. Figures 18 and 19 show the replica of the results provided in Section E. Figure 20 shows the replica of the results provided in Section F.

Table 9: Dropout rate setting for Appendix I

Environment	Value
Hopper-v2	0.0001
Walker2d-v2	0.005
Ant-v2	0.01
Humanoid-v2	0.1

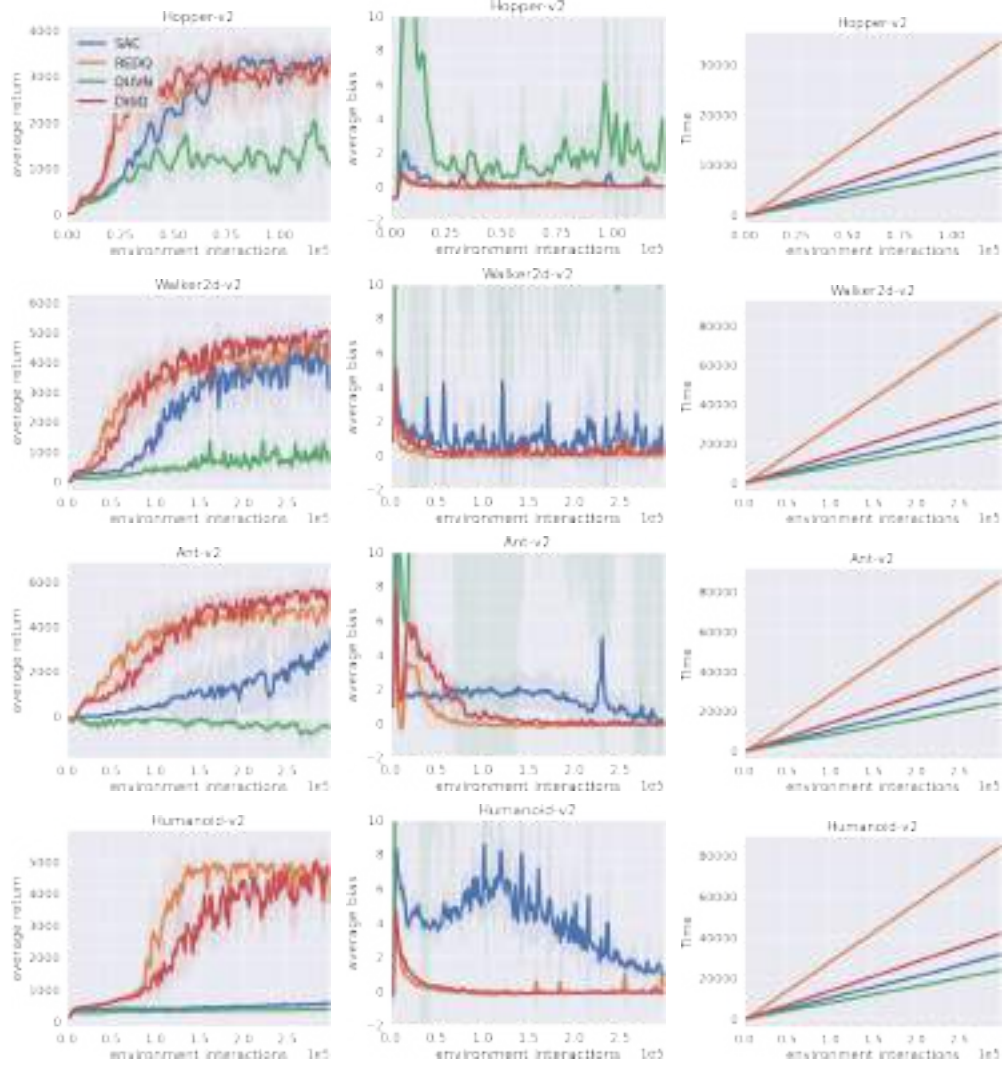


Figure 14: Average return, average of estimation bias, and wallclock time (second) for REDQ, SAC, DUVN, and DroQ. The horizontal axis represents the number of interactions with the environment (e.g., the number of executions of line 3 of Algorithm 2). For each method, average score of five independent trials are plotted as solid lines, and standard deviation across trials is plotted as transparent shaded region.

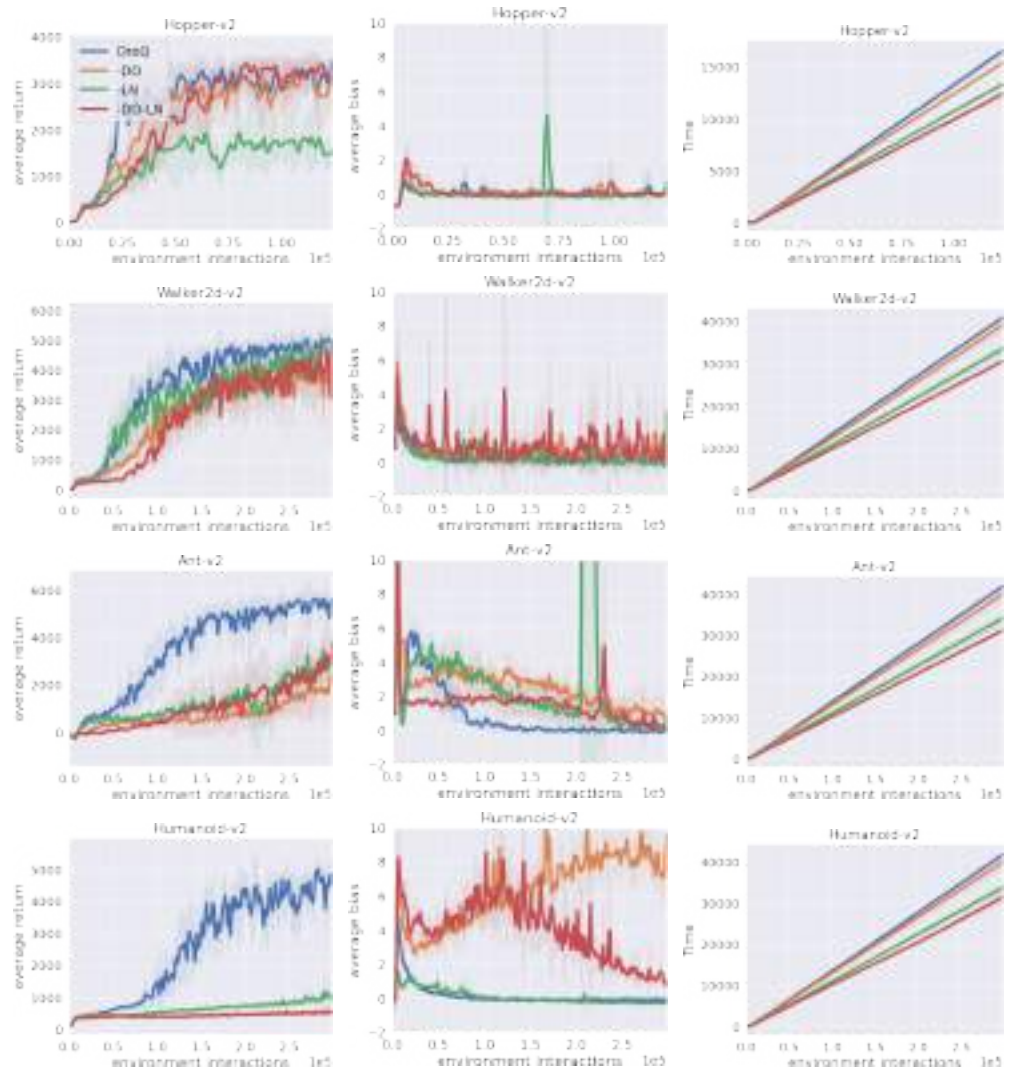


Figure 15: Ablation study results

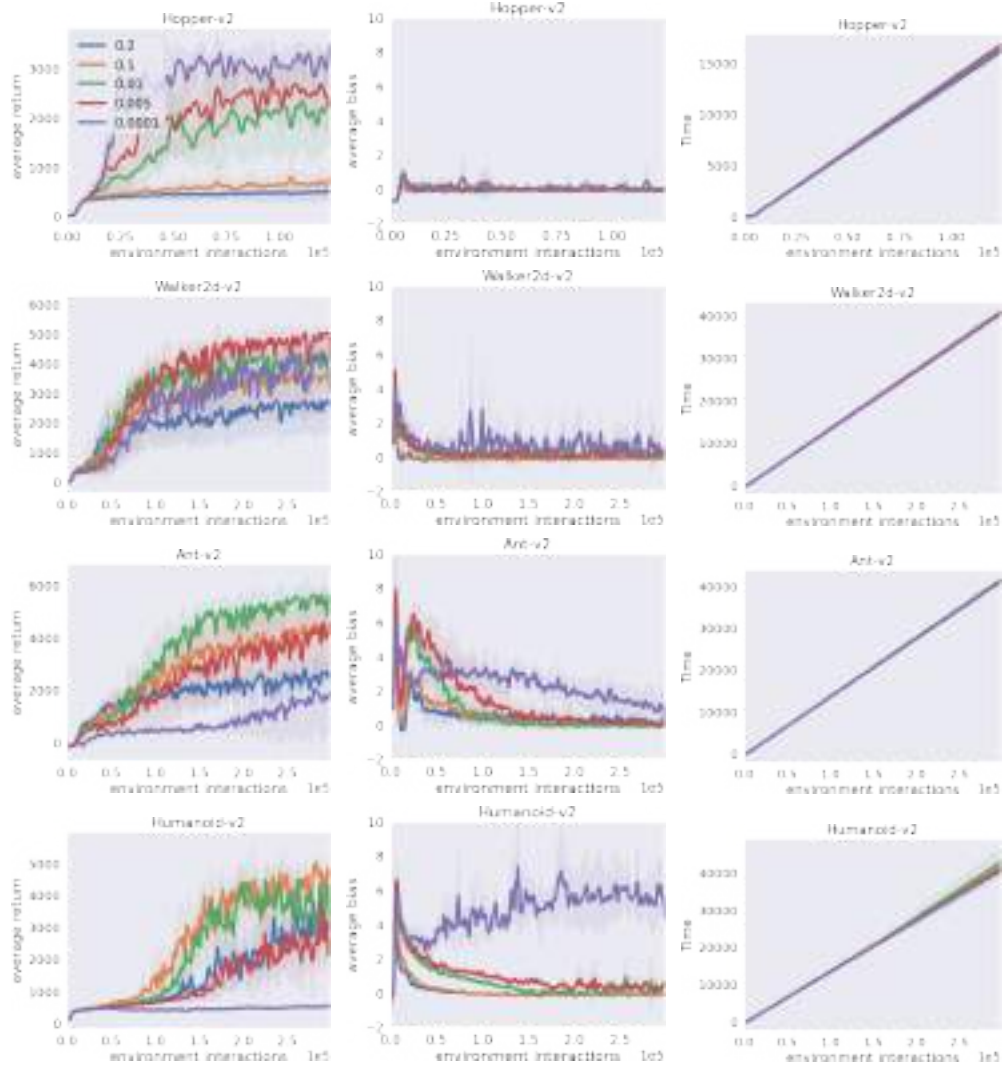


Figure 16: Average return, average of estimation bias, and wallclock time (second) for DroQ with different dropout rates. Scores for DroQ are plotted as solid lines and labelled in accordance with dropout rates (e.g., “0.2”).

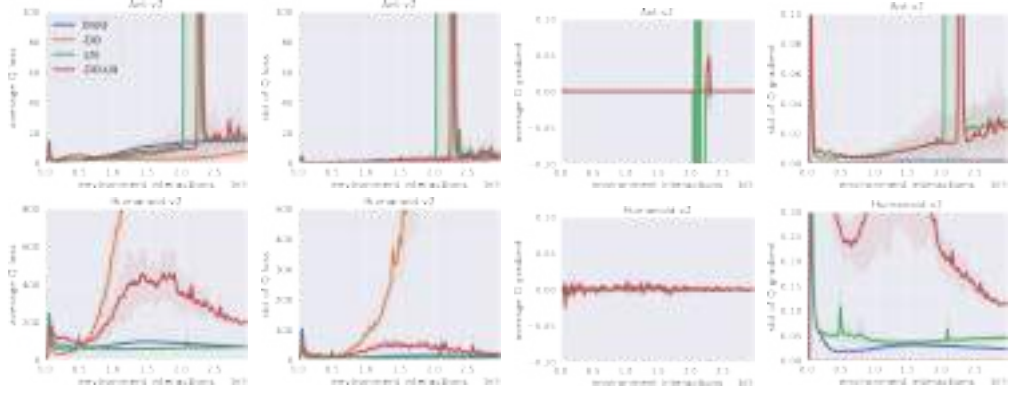


Figure 17: Average/standard deviation of the Q-function loss (“average Q loss” and “std of Q loss”) and its gradient with respect to parameters (“average Q gradient” and “std of Q gradient”). Q-function loss is calculated as $\frac{1}{M} \sum_{i=1}^M L_i$, where $L_i := (Q_{\phi_i}(s, a) - r - \gamma (\min_{i \in \mathcal{M}} Q_{\bar{\phi}_i}(s', a') - \alpha \log \pi_{\theta}(a'|s')))^2$, and its gradient is calculated as $\frac{1}{M} \sum_{i=1}^M \frac{\partial L_i}{\partial \phi_i}$.

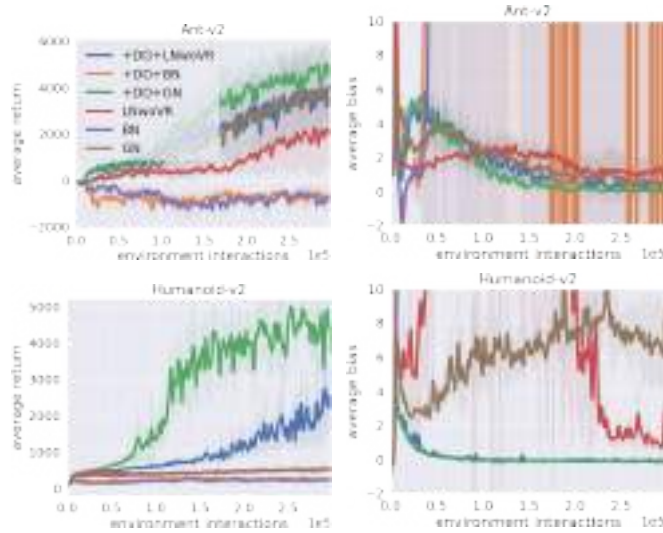


Figure 18: Average return and average estimation bias for six methods using batch normalization, group normalization, and the variant of layer normalization.

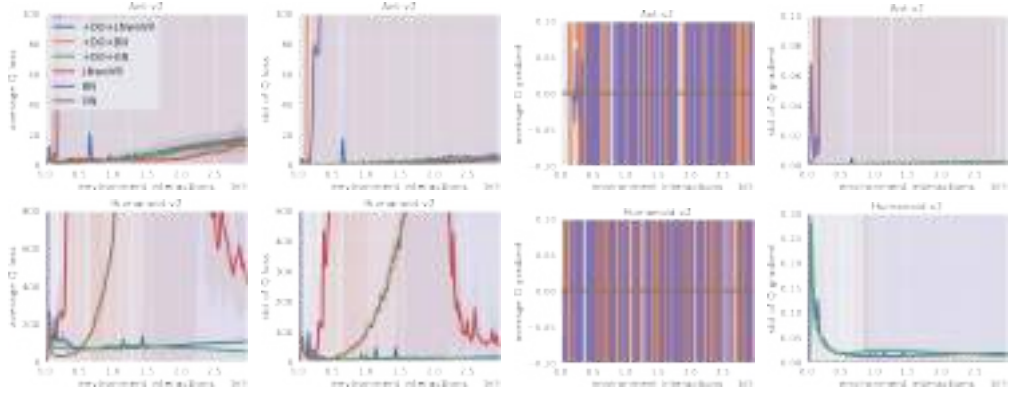


Figure 19: Average/variace of the Q-function loss (left part) and its gradient with respect to parameters (right part).

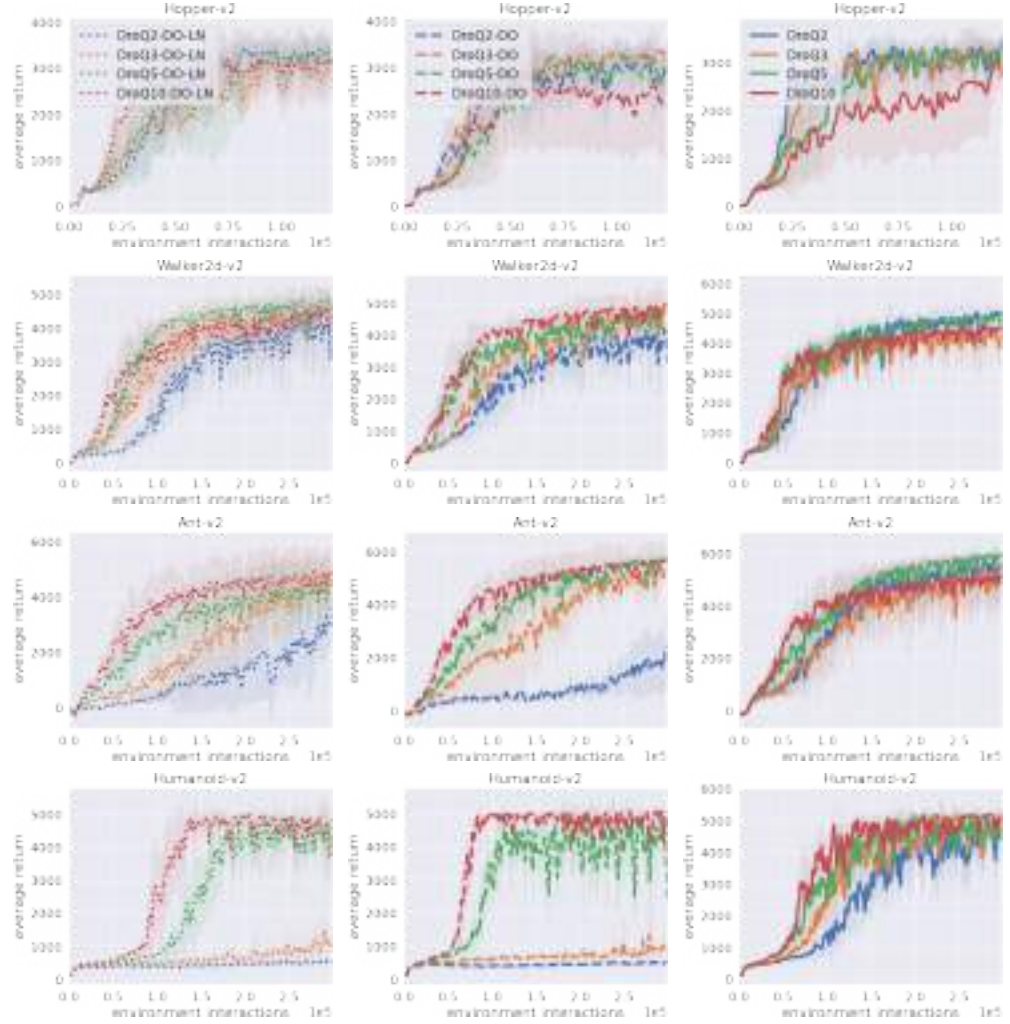


Figure 20: Additional ablation study result (average return). Left part (dotted lines): DroQ-DO-LN, middle part (dashed lines): DroQ-LN, right part (solid lines): DroQ

J OUR SOURCE CODE

Our source code is available at <https://github.com/TakuyaHiraoka/Dropout-Q-Functions-for-Doubly-Efficient-Reinforcement-Learning>

RANDOMIZED ENSEMBLED DOUBLE Q-LEARNING: LEARNING FAST WITHOUT A MODEL

Xinyue Chen^{1*}Che Wang^{1,2*}Zijian Zhou^{1*}Keith Ross^{1,2†}¹ New York University Shanghai² New York University

ABSTRACT

Using a high Update-To-Data (UTD) ratio, model-based methods have recently achieved much higher sample efficiency than previous model-free methods for continuous-action DRL benchmarks. In this paper, we introduce a simple model-free algorithm, Randomized Ensembled Double Q-Learning (REDQ), and show that its performance is just as good as, if not better than, a state-of-the-art model-based algorithm for the MuJoCo benchmark. Moreover, REDQ can achieve this performance using fewer parameters than the model-based method, and with less wall-clock run time. REDQ has three carefully integrated ingredients which allow it to achieve its high performance: (i) a UTD ratio $\gg 1$; (ii) an ensemble of Q functions; (iii) in-target minimization across a random subset of Q functions from the ensemble. Through carefully designed experiments, we provide a detailed analysis of REDQ and related model-free algorithms. To our knowledge, REDQ is the first successful model-free DRL algorithm for continuous-action spaces using a UTD ratio $\gg 1$.

1 INTRODUCTION

Recently, model-based methods in continuous action space domains have achieved much higher sample efficiency than previous model-free methods. Model-based methods often attain higher sample efficiency by using a high Update-To-Data (UTD) ratio, which is the number of updates taken by the agent compared to the number of actual interactions with the environment. For example, Model-Based Policy Optimization (MBPO) (Janner et al., 2019), is a state-of-the-art model-based algorithm which updates the agent with a mix of real data from the environment and “fake” data from its model, and uses a large UTD ratio of 20-40. Compared to Soft-Actor-Critic (SAC), which is model-free and uses a UTD of 1, MBPO achieves much higher sample efficiency in the OpenAI MuJoCo benchmark (Todorov et al., 2012; Brockman et al., 2016). This raises the question of whether it is also possible to achieve such high performance without a model?

In this paper, we introduce a simple model-free algorithm called Randomized Ensemble Double Q learning (REDQ), and show that its performance is just as good as, if not better than, MBPO. The result indicates, that at least for the MuJoCo benchmark, simple model-free algorithms can attain the performance of current state-of-the-art model-based algorithms. Moreover, REDQ can achieve this performance using fewer parameters than MBPO, and with less wall-clock run time.

Like MBPO, REDQ employs a UTD ratio that is $\gg 1$, but unlike MBPO it is model-free, has no roll outs, and performs all updates with real data. In addition to using a UTD ratio that is $\gg 1$, it has two other carefully integrated ingredients: an ensemble of Q functions; and in-target minimization across a *random subset* of Q functions from the ensemble.

Through carefully designed experiments, we provide a detailed analysis of REDQ. We introduce the metrics of average Q-function bias and standard deviation (std) of Q-function bias. Our results show that using ensembles with in-target minimization reduces the std of the Q-function bias to close to

*Equal contribution, in alphabetical order.

†Correspondence to: Keith Ross <keithwross@nyu.edu>.

zero for most of training, even when the UTD is very high. Furthermore, by adjusting the number of randomly selected Q-functions for in-target minimization, REDQ can control the average Q-function bias. In comparison with standard ensemble averaging and with SAC with a higher UTD, REDQ has much lower std of Q-function bias while maintaining an average bias that is negative but close to zero throughout most of training, resulting in significantly better learning performance. We perform an ablation study, and show that REDQ is very robust to choices of hyperparameters, and can work well with a small ensemble and a small number of Q functions in the in-target minimization. We also provide a theoretical analysis, providing additional insights into REDQ. Finally, we consider combining the REDQ algorithm with an online feature extractor network (OFENet) (Ota et al., 2020) to further improve performance, particularly for the more challenging environments Ant and Humanoid. We achieve more than 7x the sample efficiency of SAC to reach a score of 5000 for both Ant and Humanoid. In Humanoid, REDQ-OFE also greatly outperforms MBPO, reaching a score of 5000 at 150K interactions, which is 3x MBPO’s score at that point.

To ensure our comparisons are fair, and to ensure our results are reproducible (Henderson et al., 2018; Islam et al., 2017; Duan et al., 2016), we provide open source code¹. For all algorithmic comparisons, we use the same codebase (except for MBPO, for which we use the authors’ code).

2 RANDOMIZED ENSEMBLED DOUBLE Q-LEARNING (REDQ)

Janner et al. (2019) proposed Model-Based Policy Optimization (MBPO), which was shown to be much more sample efficient than popular model-free algorithms such as SAC and PPO for the MuJoCo environments. MBPO learns a model, and generates “fake data” from its model as well as “real data” through environment interactions. It then performs parameter updates using both the fake and the real data. One of the distinguishing features of MBPO is that it has a UTD ratio $\gg 1$ for updating its Q functions, enabling MBPO to achieve high sample efficiency.

We propose Randomized Ensembled Double Q-learning (REDQ), a novel model-free algorithm whose sample-efficiency performance is just as good as, if not better than, the state-of-the-art model-based algorithm for the MuJoCo benchmark. The pseudocode for REDQ is shown in Algorithm 1. REDQ can be used with any standard off-policy model-free algorithm, such as SAC (Haarnoja et al., 2018b), SOP (Wang et al., 2019), TD3 (Fujimoto et al., 2018), or DDPG (Lillicrap et al., 2015). For the sake of concreteness, we use SAC in Algorithm 1. REDQ has the following key components: (i) To improve sample efficiency, the UTD ratio G is much greater than one; (ii) To reduce the variance in the Q-function estimate, REDQ uses an ensemble of N Q-functions, with each Q-function randomly and independently initialized but updated with the same target; (iii) To reduce over-estimation bias, the target for the Q-function includes a minimization over a *random subset* $\mathcal{M}$ of the N Q-functions. The size of the subset $\mathcal{M}$ is kept fixed, and is denoted as M , and is referred to as the *in-target minimization parameter*. Since our default choice for M is $M = 2$, we refer to the algorithm as Randomized Ensembled *Double* Q-learning (REDQ).

REDQ shares some similarities with Maxmin Q-learning (Lan et al., 2020), which also uses ensembles and also minimizes over multiple Q-functions in the target. However, Maxmin Q-learning and REDQ have many differences, e.g., Maxmin Q-learning minimizes over the full ensemble in the target, whereas REDQ minimizes over a random subset of Q-functions. Unlike Maxmin Q-learning, REDQ controls over-estimation bias and variance of the Q estimate by separately setting M and N . REDQ has many possible variations, some of which are discussed in the ablation section.

REDQ has three key hyperparameters, G , N , and M . When $N = M = 2$ and $G = 1$, then REDQ simply becomes the underlying off-policy algorithm such as SAC. When $N = M > 2$ and $G = 1$, then REDQ is similar to, but not equivalent to, Maxmin Q-learning (Lan et al., 2020). In practice, we find $M = 2$ works well for REDQ, and that a wide range of values around $N = 10$ and $G = 20$ work well. To our knowledge, REDQ is the first successful model-free DRL algorithm for continuous-action spaces using a UTD ratio $G \gg 1$.

¹Code and implementation tutorial can be found at: <https://github.com/watchernyu/REDQ>

Algorithm 1 Randomized Ensembled Double Q-learning (REDQ)

-
- 1: Initialize policy parameters θ , N Q-function parameters $\phi_i, i = 1, \dots, N$, empty replay buffer $\mathcal{D}$. Set target parameters $\phi_{\text{targ},i} \leftarrow \phi_i$, for $i = 1, 2, \dots, N$
 - 2: **repeat**
 - 3: Take one action $a_t \sim \pi_\theta(\cdot | s_t)$. Observe reward r_t , new state s_{t+1} .
 - 4: Add data to buffer: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$
 - 5: **for** G updates **do**
 - 6: Sample a mini-batch $B = \{(s, a, r, s')\}$ from $\mathcal{D}$
 - 7: Sample a set $\mathcal{M}$ of M distinct indices from $\{1, 2, \dots, N\}$
 - 8: Compute the Q target y (same for all of the N Q-functions):

$$y = r + \gamma \left(\min_{i \in \mathcal{M}} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}' | s') \right), \quad \tilde{a}' \sim \pi_\theta(\cdot | s')$$

- 9: **for** $i = 1, \dots, N$ **do**
- 10: Update ϕ_i with gradient descent using

$$\nabla_{\phi} \frac{1}{|B|} \sum_{(s,a,r,s') \in B} (Q_{\phi_i}(s, a) - y)^2$$

- 11: Update target networks with $\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i$
- 12: Update policy parameters θ with gradient ascent using

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} \left(\frac{1}{N} \sum_{i=1}^N Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s) | s) \right), \quad \tilde{a}_\theta(s) \sim \pi_\theta(\cdot | s)$$

2.1 EXPERIMENTAL RESULTS FOR REDQ

We now provide experimental results for REDQ and MBPO for the four most challenging MuJoCo environments, namely, Hopper, Walker2d, Ant, and Humanoid. We have taken great care to make a fair comparison of REDQ and MBPO. The MBPO results are reproduced using the author’s open source code, and we use the hyperparameters suggested in the MBPO paper, including $G = 20$. We obtain MBPO results similar to those reported in the MBPO paper. For REDQ, we use $G = 20$, $N = 10$, and $M = 2$ for all environments. We use the evaluation protocol proposed in the MBPO paper. Specifically, after every epoch we run one test episode with the current policy and record the performance as the undiscounted sum of all the rewards in the episode. A more detailed discussion on hyperparameters and implementation details is given in the Appendix.

Figure 1 shows the training curves for REDQ, MBPO, and SAC. For each algorithm, we plot the average return of 5 independent trials as the solid curve, and plot the standard deviation across 5 seeds as the transparent shaded region. For each environment, we train each algorithm for exactly the same number of environment interactions as done in the MBPO paper. Figure 1 shows that both REDQ and MBPO learn much faster than SAC, with REDQ performing somewhat better than MBPO on the whole. In particular, REDQ learns significantly faster for Hopper, and has somewhat better asymptotic performance for Hopper, Walker2d, and Humanoid. A more detailed performance comparison is given in the Appendix, where it is shown that, averaging across the environments, REDQ performs 1.4x better than MBPO half-way through training and 1.1x better at the end of training. These results taken together are perhaps counter-intuitive. They show that a simple model-free algorithm can achieve as good or better sample-efficiency performance as the state-of-the-art model-based algorithm for the MuJoCo environments.

Does REDQ achieve its sample efficiency using more computational resources than MBPO? We now compare the number of parameters used in REDQ and MBPO. With REDQ, for each Q network and the policy network, we use a multi-layer perceptron with two hidden layers, each with 256 units. For MBPO, we use the default network architectures for the Q networks, policy network, and model ensembles (Janner et al., 2019). The Appendix provides a table comparing the number of parameters: REDQ uses fewer parameters than MBPO for all four environments, specifically, between

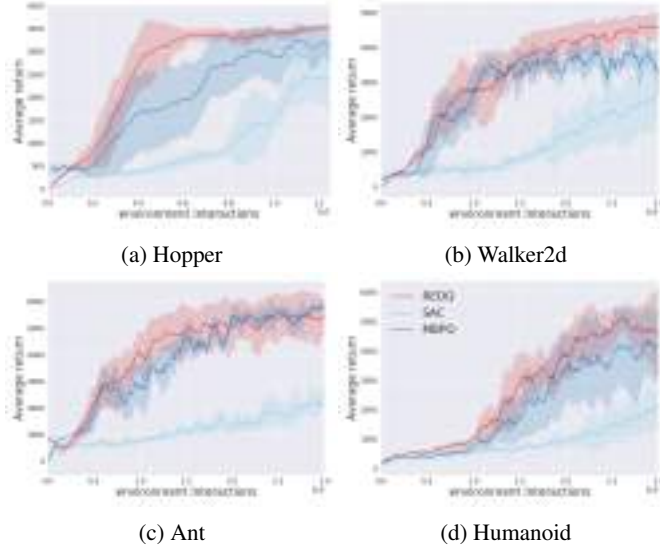


Figure 1: REDQ compared to MBPO and SAC. Both REDQ and MBPO use $G = 20$.

26% and 70% as many parameters depending on the environment. Additionally, we measured the runtime on a 2080-Ti GPU and found that MBPO roughly takes 75% longer. In summary, the results in this section show that the model-free algorithm REDQ is not only at least as sample efficient as MBPO, but also has fewer parameters and is significantly faster in terms of wall-clock time.

3 WHY DOES REDQ SUCCEED WHEREAS OTHERS FAIL?

REDQ is a simple model-free algorithm that matches the performance of a state-of-the-art model-based algorithm. Key to REDQ’s sample efficiency is using a $UTD \gg 1$. Why is it that SAC and ordinary ensemble averaging (AVG) cannot do as well as REDQ by simply increasing the UTD?

To address these questions, let $Q^\pi(s, a)$ be the action-value function for policy π using the standard infinite-horizon discounted return definition. Let $Q_\phi(s, a)$ be an estimate of $Q^\pi(s, a)$, which is defined as the average of $Q_{\phi_i}(s, a)$, $i = 1, \dots, N$, when using an ensemble. We define the bias of an estimate at state-action pair (s, a) to be $Q_\phi(s, a) - Q^\pi(s, a)$. We are primarily interested in the accuracy of $Q_\phi(s, a)$ over the state-action distribution of the current policy π . To quantitatively analyze how estimation error accumulates in the training process, we perform an analysis that is similar to previous work (Van Hasselt et al., 2016; Fujimoto et al., 2018), but not exactly the same. We run a number of analysis episodes from different random initial states using the current policy π . For each state-action pair visited, we obtain both the discounted Monte Carlo return and the estimated Q value using Q_ϕ , and then compute the difference to obtain an estimate of the bias for that state-action pair. We then calculate the average and std of these bias values. The average gives us an idea of whether Q_ϕ is in general overestimating or underestimating, and the std measures how uniform the bias is across different state-action pairs. We argue that the std is just as important as the average of the bias. As discussed in Van Hasselt et al. (2016), a uniform bias is not necessarily harmful as it does not change the action selection. Thus near-uniform bias can be preferable to a highly non-uniform bias with a small average value. Although average bias has been analyzed in several previous works (Van Hasselt et al., 2016; Fujimoto et al., 2018; Anschel et al., 2017), the std does not seem to have received much attention.

Since the MC return values can change significantly throughout training, to make comparisons more meaningful, we define the normalized bias of the estimate $Q_\phi(s, a)$ to be $(Q_\phi(s, a) - Q^\pi(s, a)) / |E_{\bar{s}, \bar{a} \sim \pi}[Q^\pi(\bar{s}, \bar{a})]|$, which is simply the bias divided by the absolute value of the expected discounted MC return for state-action pairs sampled from the current policy. We focus on the normalized bias in our analysis since it helps show how large the bias is, compared to the scale of the current MC return.

In this and the subsequent section, we compare REDQ with several algorithms and variants. We emphasize that all of the algorithms and variants use the same code base as used in the REDQ experiments (including using SAC as the underlying off-policy algorithm). The only difference is how the targets are calculated in lines 7 and 8 of Algorithm 1.

We first compare REDQ with two natural algorithms, which we call SAC-20 and ensemble averaging (AVG). SAC-20 is SAC but with G increased from 1 (as in standard SAC) to 20. For AVG, we use an ensemble of Q functions, and when computing the Q target, we take the average of all Q values without any in-target minimization. In these comparisons, all three algorithms use a UTD of $G = 20$. In the later ablation section we also have a detailed discussion on experimental results with Maxmin, and explain why it does not work well for the MuJoCo benchmark when using a large ensemble.

Figure 2 presents the results for Ant; the results for the other three environments are consistent with those for Ant and are shown in the Appendix. For each experiment we use 5 random seeds. We first note REDQ learns significantly faster than both SAC-20 and AVG. Strikingly, relative to the other two algorithms, REDQ has a very low normalized std of bias for most of training, indicating the bias across different in-distribution state-action pairs is about the same. Furthermore, throughout most of training, REDQ has a small and near-constant under-estimation bias. The shaded areas for mean and std of bias are also smaller, indicating that REDQ is robust to random initial conditions.

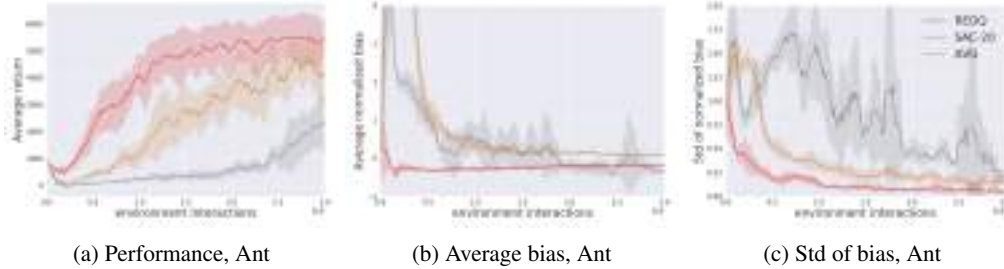


Figure 2: Performance, mean and std of normalized Q bias for REDQ, AVG, and SAC for Ant. Figures for the other three environments have similar trends and are shown in the Appendix.

SAC with a UTD ratio of 20 performs poorly for the most challenging environments Ant and Humanoid. For SAC-20, the high UTD ratio leads to an average bias that fluctuates during training. We also see a high normalized std of bias, indicating that the bias is highly non-uniform, which can be detrimental. The bias values also have large variance across random initial seeds, as indicated by the large shaded area, showing that the bias in SAC-20 is sensitive to initial conditions. Comparing AVG and SAC-20, we see AVG performs significantly better than SAC-20 in Ant and Humanoid. This can be explained again by the bias: due to ensemble averaging, AVG can achieve a lower std of bias; and when it does, its performance improves significantly faster than SAC-20.

REDQ has two critical components that allow it to maintain stable and near-uniform bias under high UTD ratios: an ensemble and in-target minimization. AVG and SAC-20 each has one of these components but neither has both. Thus the success of REDQ is largely due to a careful integration of both of these critical components. Additionally, as shown in the ablation study, the *random* selection of Q functions in the in-target minimization can give REDQ a further performance boost.

3.1 THEORETICAL ANALYSIS

We now characterize the relation between the estimation error, the in-target minimization parameter M and the size of the ensemble N . We use the theoretical framework introduced in [Thrun & Schwartz \(1993\)](#) and extended in [Lan et al. \(2020\)](#). We do this for the tabular version of REDQ, for which the target for $Q^i(s, a)$ for each $i = 1, \dots, N$ is:

$$r + \gamma \max_{a' \in \mathcal{A}} \min_{j \in \mathcal{M}} Q^j(s', a') \quad (1)$$

where $\mathcal{A}$ is the finite action space, (s, a, r, s') is a transition, and $\mathcal{M}$ is again a uniformly random subset from $\{1, \dots, N\}$ with $|\mathcal{M}| = M$. The complete pseudocode for tabular REDQ is provided in the Appendix.

Let $Q^i(s, a) - Q^\pi(s, a)$ be the *pre-update* estimation bias for the i th Q-function, where $Q^\pi(s, a)$ is once again the ground-truth Q-value for the current policy π . We are interested in how the bias changes after an update, and how this change is effected by M and N . Similar to [Thrun & Schwartz \(1993\)](#) and [Lan et al. \(2020\)](#), define the *post-update* estimation bias as the difference between the target [\(1\)](#) and the target when using the ground-truth:

$$\begin{aligned} Z_{M,N} &\triangleq r + \gamma \max_{a' \in \mathcal{A}} \min_{j \in \mathcal{M}} Q^j(s', a') - (r + \gamma \max_{a' \in \mathcal{A}} Q^\pi(s', a')) \\ &= \gamma (\max_{a' \in \mathcal{A}} \min_{j \in \mathcal{M}} Q^j(s', a') - \max_{a' \in \mathcal{A}} Q^\pi(s', a')) \end{aligned}$$

Here we write $Z_{M,N}$ to emphasize its dependence on both M and N . Following [Thrun & Schwartz \(1993\)](#) and [Lan et al. \(2020\)](#), fix s and assume each $Q^i(s, a)$ has a random approximation error e_{sa}^i :

$$Q^i(s, a) = Q^\pi(s, a) + e_{sa}^i$$

where for each fixed s , $\{e_{sa}^i\}$ are zero-mean independent random variables such that $\{e_{sa}^i\}$ are identically distributed across i for each fixed (s, a) pair. Note that due to the zero-mean assumption, the expected pre-update estimation bias is $\mathbb{E}[Q^i(s, a) - Q^\pi(s, a)] = 0$. Thus if $\mathbb{E}[Z_{M,N}] > 0$, then the expected post-update bias is positive and there is a tendency for over-estimation accumulation; and if $\mathbb{E}[Z_{M,N}] < 0$, then there is a tendency for under-estimation accumulation.

Theorem 1. 1. For any fixed M , $\mathbb{E}[Z_{M,N}]$ does not depend on N .

2. $\mathbb{E}[Z_{1,N}] \geq 0$ for all $N \geq 1$.

3. $\mathbb{E}[Z_{M+1,N}] \leq \mathbb{E}[Z_{M,N}]$ for any $M < N$.

4. Suppose that $e_{sa}^i \leq c$ for some $c > 0$ for all s, a and i . Then there exists an M such that for all $N \geq M$, $\mathbb{E}[Z_{M,N}] < 0$.

Points 2-4 of the Theorem indicate that we can control the expected post-update bias $\mathbb{E}[Z_{M,N}]$, bringing it from above zero (over estimation) to under zero (under estimation) by increasing M . Moreover, from the first point, the expected bias only depends on M , which implies that increasing N can reduce the variance of the ensemble average but does not change the expected post-update bias. Thus, we can control the post-update bias with M and separately control the variance of the average of the ensemble with N . Note that this is not possible for Maxmin Q-learning, for which $M = N$, and thus increasing the ensemble size N will also decrease the post-update bias, potentially making it more negative, which can be detrimental. Note that this also cannot be done for standard ensemble averaging, for which there is no in-target minimization parameter M .

Note we make very weak assumptions on the distribution of the error term. [Thrun & Schwartz \(1993\)](#) and [Lan et al. \(2020\)](#) make a strong assumption, namely, the error term is uniformly distributed. Because our assumption is much weaker, our proof methodology in the Appendix is very different.

We also consider a variant of REDQ where instead of choosing a random set of size M in the target, we calculate the target by taking the expected value over all possible subsets of size M . In this case, the target in the tabular version becomes

$$Y_{M,N} = r(s, a) + \gamma \frac{1}{\binom{N}{M}} \sum_{\substack{B \subset \mathcal{N} \\ |B|=M}} \max_{a'} \min_{j \in B} Q^j(s', a')$$

We write the target here as $Y_{M,N}$ to emphasize its dependence on both M and N . We refer to this variant of REDQ as “Weighted” since we can efficiently calculate the target as a weighted sum of a re-ordering of the N Q-functions, as described in the Appendix.

The following theorem shows that the variance of this target goes to zero as $N \rightarrow \infty$. We note, however, that in practice, some variance in the target may be beneficial in reducing overfitting or help exploration. We can retain some variance by keeping N finite or using the unweighted REDQ scheme.

Let $v_M := \text{Var}(\max_{a'} \min_{j \in B} Q^j(s', a'))$ for any subset $B \subset \mathcal{N}$ where $|B| = M$. (It is easily seen that v_M only depends on M and not only the specific elements of B .)

Theorem 2.

$$\text{Var}(Y_{M,N}) \leq G_M(N)$$

for some function $G_M(N)$ satisfying

$$\lim_{N \rightarrow \infty} \frac{G_M(N)}{M^2 v_M / N} = 1$$

Consequently,

$$\lim_{N \rightarrow \infty} \text{Var}(Y_{M,N}) = 0$$

Also in the Appendix we show that the tabular version of REDQ converges to the optimal Q function with probability one.

4 REDQ VARIANTS AND ABLATIONS

In this section, we use ablations to provide further insight into REDQ. We focus on the Ant environment. We first look at how the ensemble size N affects REDQ. The top row in Figure 3 shows REDQ with $N = 2, 3, 5, 10, 15$. We can see that when we increase the ensemble size, we generally get a more stable average bias, a lower std of bias, and stronger performance. The result shows that even a small ensemble (e.g., $N = 5$) can greatly help in stabilizing bias accumulation when training under high UTD.

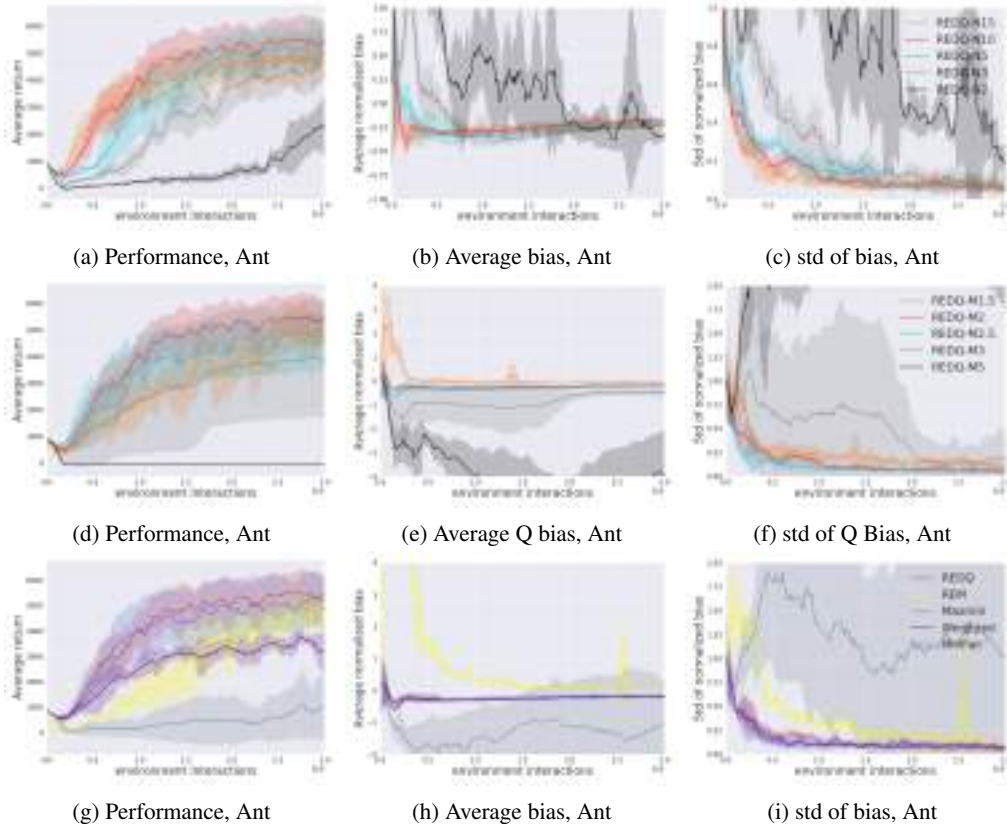


Figure 3: REDQ ablation results for Ant. The top row shows the effect of the ensemble size N . The middle row shows the effect of the in-target minimization parameter M . The bottom row compares REDQ to several variants.

The middle row of Figure 3 shows how M , the in-target minimization parameter, can affect performance. When M is not an integer, e.g., $M = 1.5$, for each update, with probability 0.5 only one

randomly-chosen Q function is used in the target, and with probability 0.5, two randomly-chosen functions are used. Similarly, for $M = 2.5$, for each update either two or three Q functions are used. Consistent with the theoretical result in Theorem 1, by increasing M we lower the average bias. When M gets too large, the Q estimate becomes too conservative and the large negative bias makes learning difficult.

$M = 2$, which has the overall best performance, strikes a good balance between average bias (small underestimation during most of training) and std of the bias (consistently small).

The bottom row of Figure 3 shows the results for different target computation methods. The Maxmin curve in the figures is a variant based on Maxmin Q-learning, where the min of *all* the Q networks in the ensemble is taken to compute the Q target. As the ensemble size increases, Maxmin Q-learning shifts from overestimation to underestimation (Lan et al., 2020); Figure 3 shows Maxmin with $N = 3$ instead of $N = 10$, since a large N value will cause even more divergence of the Q values. When varying the ensemble size of Maxmin, we see the same problem as shown in the middle row of Figure 3. When we increase the ensemble size to be larger than 3, Maxmin starts to reduce the bias so much that we get a highly negative Q bias, which accumulates quickly, leading to instability in the Q networks and poor performance. In the Maxmin paper, it was mainly tested on Atari environments with a small finite action space, in which case it provides good performance. Our results show that when using environments with high-dimensional continuous action spaces, such as MuJoCo, the rapid accumulation of (negative) bias becomes a problem. This result parallels some recent research in offline (i.e., batch) DRL. In Agarwal et al. (2020), it is shown that with small finite action spaces, naive offline training with deep Q-networks (DQN) only slightly reduces performance. However, continuous action Q-learning based methods such as Deep Deterministic Policy Gradient and SAC suffer much more from Q bias accumulation compared to discrete action methods. Recent work shows that offline training with these methods often lead to poor performance, and can even entirely diverge (Fujimoto et al., 2019; Kumar et al., 2019).

Random ensemble mixture (REM) is a method originally proposed to boost performance of DQN in the discrete-action setting. REM uses the random convex combination of Q values to compute the target: it is similar to ensemble average (AVG), but with more randomization (Agarwal et al., 2020).

For Weighted, the target is computed as the expectation of all the REDQ targets, where the expectation is taken over all N -choose-2 pairs of Q-functions. This leads to a formula that is a weighted sum of the ordered Q-functions, where the ordering is from the lowest to the highest Q value in the ensemble, as described in the Appendix. Our baseline REDQ in Algorithm 1 can be considered as a random-sample version of Weighted. For the MinPair REDQ variant, we divide the 10 Q networks into 5 fixed pairs, and during an update we sample a pair of Q networks from these 5 fixed pairs.

From Figure 3 we see that REDQ and MinPair are the best and their performance is similar. For Ant, the performance of Weighted is much lower than REDQ. However, as shown in the Appendix, Weighted and REDQ have similar performance for the other three environments. The randomization might help alleviate overfitting in the early stage, or improve exploration. REM has performance similar to AVG, studied in Section 3. In terms of the Q bias, REM has a positive average bias, while REDQ, MinPair, and Weighted all have a small negative average bias. Overall these results indicate that the REDQ algorithm is robust across different mechanisms for choosing the functions, and that randomly choosing the Q functions can sometimes boost performance. Additional results and discussions are provided in the appendix.

4.1 IMPROVING REDQ WITH AUXILIARY FEATURE LEARNING

We now investigate whether we can further improve the performance of REDQ by incorporating better representation learning? Ota et al. (2020) recently proposed the online feature extractor network (OFENet), which learns representation vectors from environment data, and provides them to the agent as additional input, giving significant performance improvement.

Is it possible to further improve the performance of REDQ with OFENet? We trained an OFENet together with REDQ to provide extra input, giving the algorithm REDQ-OFE. We found that OFENet did not help much for Hopper and Walker2d, which may be because REDQ already learns very fast, leaving little room for improvement. But as shown in Figure 4, online feature extraction can further improve REDQ performance for the more challenging environments Ant and Humanoid.

REDQ-OFE achieves 7x the sample efficiency of SAC to reach 5000 on Ant and Humanoid, and outperforms MBPO with 3.12x and 1.26x the performance of MBPO at 150K and 300K data, respectively. A more detailed performance comparison table can be found in the Appendix.

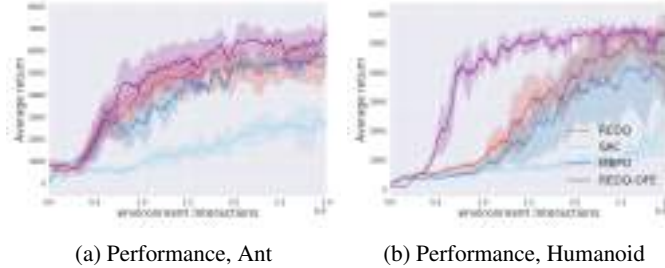


Figure 4: Performance of REDQ, REDQ with OFE, and SAC.

5 RELATED WORK

It has long been recognized that maximization bias in Q-learning can significantly impede learning. [Thrun & Schwartz \(1993\)](#) first highlighted the existence of maximization bias. [Van Hasselt \(2010\)](#) proposed Double Q-Learning to address maximization bias for the tabular case, and showed that in general it leads to an under-estimation bias. [Van Hasselt et al. \(2016\)](#) showed that adding Double Q-learning to deep Q networks (DQN) ([Mnih et al. 2013; 2015](#)) gives a major performance boost for the Atari games benchmark. For continuous-action spaces, [Fujimoto et al. \(2018\)](#) introduced clipped-double Q-learning (CDQ), which further reduces maximization bias and brings significant improvements over the deep deterministic policy gradient (DDPG) algorithm ([Lillicrap et al. 2015](#)). CDQ was later combined with entropy maximization in SAC to achieve even stronger performance ([Haarnoja et al. 2018a,b](#)). Other bias reduction techniques include using bias-correction terms ([Lee et al. 2013](#)), using weighted Q estimates ([Zhang et al. 2017; Li & Hou 2019](#)), penalizing deterministic policies at early stage of training ([Fox et al. 2015](#)), using multi-step methods ([Meng et al. 2020](#)), performing weighted Bellman updates to mitigate error propagation ([Lee et al. 2020](#)), and truncating sampled Q estimates with distributional networks ([Kuznetsov et al. 2020](#)).

It has also long been recognized that using ensembles can improve the performance of DRL algorithms ([Faußer & Schwenker 2015; Osband et al. 2016](#)). For Q-learning based methods, [Anschel et al. \(2017\)](#) use the average of multiple Q estimates to reduce variance. [Agarwal et al. \(2020\)](#) introduced Random Ensemble Mixture (REM), which enforces optimal Bellman consistency on random convex combinations of multiple Q estimates. [Lan et al. \(2020\)](#) introduced Maxmin Q-learning, as discussed in Sections 2-4. Although in discrete action domains it has been found that fine-tuning DQN variants, including the UTD, can boost performance, little experimental or theoretical analysis is given to explain how this improvement is obtained ([Kielak 2020; van Hasselt et al. 2019](#)).

To address some of the critical issues in model-based learning ([Langlois et al. 2019](#)), recent methods such as MBPO combine a model ensemble with a carefully controlled rollout horizon to obtain better performance ([Janner et al. 2019; Buckman et al. 2018](#)). These model-based methods can also be enhanced with advanced sampling ([Zhang et al. 2020](#)), bidirectional models ([Lai et al. 2020](#)), or backprop through the model ([Clavera et al. 2020](#)), and be analyzed through new theoretical frameworks ([Rajeswaran et al. 2020; Dong et al. 2020](#)).

6 CONCLUSION

The contributions of this paper are as follows. (1) We propose a simple model-free algorithm that attains sample efficiency that is as good as or better than state-of-the-art model-based algorithms for the MuJoCo benchmark. This result indicates that, at least for the MuJoCo benchmark, models may not be necessary for achieving high sample efficiency. (2) Using carefully designed experiments, we explain why REDQ succeeds when other model-free algorithms with high UTD ratios fail. (3) Finally, we combine REDQ with OFE, and show that REDQ-OFE can learn extremely fast for the challenging environments Ant and Humanoid.

A THEORETICAL RESULTS

A.1 TABULAR VERSION OF REDQ

In the tabular algorithm below, for clarity we use $G = 1$.

Algorithm 2 Tabular REDQ

```

1: Initialize  $\{Q^i(s, a), s \in \mathcal{S}, a \in \mathcal{A}\}_{i=1}^N$ ,
2: repeat
3:   Choose  $a \in \mathcal{A}$  based on  $\{Q^i(s, a)\}_{i=1}^N$ , observe  $r, s'$ 
4:   Randomly choose a subset  $\mathcal{M}$  of size  $M$  from  $\{1, \dots, N\}$ 
5:    $y = r + \gamma \max_{a' \in \mathcal{A}} \min_{j \in \mathcal{M}} Q^j(s', a')$ 
6:   for  $i = 1, \dots, N$  do:
7:      $Q^i(s, a) \leftarrow Q^i(s, a) + \alpha(y - Q^i(s, a))$ 
8:    $s \leftarrow s'$ 
9: until end

```

Alternatively in Algorithm 2, at each iteration we could just update one of the Q^i functions.

A.2 PROOF OF THEOREM 1

We first prove the following lemma:

Lemma 1. *Let $X_1, X_2, \dots$ be an infinite sequence of i.i.d. random variables. Let $F(x)$ be the cdf of X_m and let $\tau = \inf\{x : F(x) > 0\}$. Also let $Y_m = \min\{X_1, X_2, \dots, X_m\}$. Then $Y_1, Y_2, \dots$ converges to τ almost surely.*

Proof:

Let $F_m(x)$ be the cdf of Y_m . Since $X_1, \dots, X_m$ are independent,

$$F_m(x) = 1 - [1 - F(x)]^m$$

For $x < \tau$, $F_m(x) = 0$ since $F(x) = 0$. For $x > \tau$, $F_m(x) \xrightarrow{m \rightarrow \infty} 1$. Therefore, Y_m weakly converges to τ .

Moreover, for each $\omega \in \Omega$, $\{Y_m(\omega)\}$ is a decreasing sequence. So $\{Y_m(\omega)\}$ either converges to a real number or $-\infty$. Therefore $Y_m \rightarrow Y$ almost surely for some random variable Y . Combined with the result that $Y_m \xrightarrow{d} \tau$, we can conclude that

$$Y_m \xrightarrow{a.s.} \tau$$

Proof of Theorem 1:

1. Let B_1, B_2 be two subsets of $\mathcal{N} = \{1, \dots, N\}$ of size M . First of all, since $\{Q^j(s, a)\}_{j=1}^N$ are i.i.d for any $a \in \mathcal{A}$, $\min_{j \in B_1} Q^j(s, a)$ and $\min_{j \in B_2} Q^j(s, a)$ are identically distributed. Furthermore, since $Q^j(s, a)$ are independent for all $a \in \mathcal{A}$ and $1 \leq j \leq N$, $\{\min_{j \in B} Q^j(s, a)\}_{a \in \mathcal{A}}$ are independent for any $B \subset \mathcal{N}$. Denote the distribution function of $\max_a \min_{j \in B_1} Q^j(s, a)$ as $F_1(x)$ and the distribution function of $\max_a \min_{j \in B_2} Q^j(s, a)$ as $F_2(x)$. Then for any $x \in \mathbb{R}$,

$$\begin{aligned}
F_1(x) &= \mathbb{P}\left(\max_a \min_{j \in B_1} Q^j(s, a) \leq x\right) = \mathbb{P}\left(\cap_{a \in \mathcal{A}} \{\min_{j \in B_1} Q^j(s, a) \leq x\}\right) \\
&= \prod_{a \in \mathcal{A}} \mathbb{P}\left(\min_{j \in B_1} Q^j(s, a) \leq x\right) = \prod_{a \in \mathcal{A}} \mathbb{P}\left(\min_{j \in B_2} Q^j(s, a) \leq x\right) \\
&= \mathbb{P}\left(\max_a \min_{j \in B_2} Q^j(s, a) \leq x\right) = F_2(x)
\end{aligned}$$

Therefore, we have proved that $\max_a \min_{j \in B_1} Q^j(s, a)$ and $\max_a \min_{j \in B_2} Q^j(s, a)$ are identically distributed. Then

$$\begin{aligned} \mathbb{E}[Z_{M,N}] &= \gamma \mathbb{E}[(\max_a \min_{j \in \mathcal{M}} Q^j(s, a) - \max_a Q^\pi(s, a))] \\ &= \gamma \mathbb{E}\left[\frac{1}{\binom{N}{M}} \sum_{\substack{B \subset \mathcal{N} \\ |B|=M}} \max_a \min_{j \in B} Q^j(s, a)\right] - \gamma \max_a Q^\pi(s, a) \\ &= \gamma \left(\mathbb{E}\left[\max_a \min_{1 \leq j \leq M} Q^j(s, a)\right] - \max_a Q^\pi(s, a) \right) \end{aligned}$$

which does not depend on N .

2. It follows from 1 that

$$\mathbb{E}[Z_{1,N}] = \gamma \left(\mathbb{E}\left[\max_a Q^1(s, a)\right] - \max_a Q^\pi(s, a) \right)$$

Since $\max_a Q^1(s, a) \geq Q^1(s, a')$ for all $a' \in \mathcal{A}$, we have

$$\mathbb{E}\left[\max_a Q^1(s, a)\right] \geq \mathbb{E}[Q^1(s, a')]$$

for all $a' \in \mathcal{A}$. Consequently,

$$\mathbb{E}\left[\max_a Q^1(s, a)\right] \geq \max_a \mathbb{E}[Q^1(s, a)] = \max_a Q^\pi(s, a)$$

Hence

$$\mathbb{E}[Z_{1,N}] = \gamma \left(\mathbb{E}\left[\max_a Q^1(s, a)\right] - \max_a Q^\pi(s, a) \right) \geq 0$$

3. Since $\max_a \min_{1 \leq j \leq M} Q^j(s, a) \geq \max_a \min_{1 \leq j \leq M+1} Q^j(s, a)$,

$$\begin{aligned} \mathbb{E}[Z_{M,N}] &= \gamma \left(\mathbb{E}\left[\max_a \min_{1 \leq j \leq M} Q^j(s, a)\right] - \max_a Q^\pi(s, a) \right) \\ &\geq \gamma \left(\mathbb{E}\left[\max_a \min_{1 \leq j \leq M+1} Q^j(s, a)\right] - \max_a Q^\pi(s, a) \right) = \mathbb{E}[Z_{M+1,N}] \end{aligned}$$

4. Let $F_a(x)$ be the cdf of $Q^j(s, a)$ and let $\tau_a = \inf\{x : F_a(x) > 0\}$. Here we assume the approximation error e_{sa}^i is non-trivial, which implies $\tau_a < Q^\pi(s, a)$. Note that τ_a can be equal to $-\infty$. Let

$$Y_a^M = \min_{1 \leq i \leq M} Q^i(s, a)$$

From Lemma 1 we have Y_a^M converges to τ_a almost surely for each a . Because the action space is finite, it therefore follows that

$$Y^M = \max_a Y_a^M$$

converges almost surely to $\tau = \max_a \tau_a$. Furthermore, for each a we have

$$Y_a^M = \min_{1 \leq j \leq M} Q^j(s, a) \geq \min_{1 \leq j \leq M+1} Q^j(s, a) = Y_a^{M+1}$$

from which it follows that $Y^M \geq Y^{M+1}$. Thus $\{Y^M\}$ is a monotonically decreasing sequence. We also note that due to the assumption $e_{sa}^i \leq c$ for all a and i , and because $Q^\pi(s, a)$ is finite for all s and a , it follows that $Y^M \leq d$ for all M for a finite d . Thus $\{Y^M\}$ is a bounded-above, monotonically-decreasing sequence of random variables which converges almost surely to τ . We can therefore apply the monotone convergence theorem, giving

$$\begin{aligned} \mathbb{E}[Z_{M,N}] &= \gamma \left(\mathbb{E}\left[\max_a \min_{1 \leq j \leq M} Q^j(s, a)\right] - \max_a Q^\pi(s, a) \right) \\ &= \gamma \left(\mathbb{E}[\max_a Y_a^M] - \max_a Q^\pi(s, a) \right) \xrightarrow{M \rightarrow \infty} \gamma \left(\max_a \tau_a - \max_a Q^\pi(s, a) \right) < 0, \end{aligned}$$

where the last inequality follows from $\tau_a < Q^\pi(s, a)$ for all actions a .

A.3 PROOF OF THEOREM 2

For convenience, define $Y_B = \max_{a'} \min_{j \in B} Q^j(s', a')$. Suppose $N > 2M$.

$$\begin{aligned} \text{Var}(Y_{M,N}) &= \frac{\gamma^2}{\binom{N}{M}^2} \text{Var}\left(\sum_{\substack{B \subset \mathcal{N} \\ |B|=M}} Y_B\right) \\ &= \frac{\gamma^2(M!)^2}{(\Pi_{i=0}^{M-1}(N-i))^2} \left[\sum_{B \subset \mathcal{N}} \text{Var}(Y_B) + 2 \cdot \sum_{\substack{B_1, B_2 \subset \mathcal{N} \\ B_1 \neq B_2}} \text{Cov}(Y_{B_1}, Y_{B_2}) \right] \end{aligned}$$

Let $A = \sum_{\substack{B_1, B_2 \subset \mathcal{N} \\ B_1 \neq B_2}} \text{Cov}(Y_{B_1}, Y_{B_2})$, which consists of

$$\begin{aligned} \binom{\binom{N}{M}}{2} &= \frac{1}{2} \cdot \frac{N!}{(N-M)!M!} \cdot \left(\frac{N!}{(N-M)!M!} - 1 \right) \\ &= \frac{1}{2(M!)^2} \cdot \Pi_{i=0}^{M-1}(N-i)^2 - \frac{N!}{2 \cdot M!(N-M)!} \end{aligned}$$

terms. $\binom{\binom{N}{M}}{2}$ can be seen as a polynomial function of N with degree $2M$. The coefficient for the term N^{2M} is $\frac{1}{2(M!)^2}$. The coefficient for the term N^{2M-1} is $\frac{1}{2(M!)^2} \cdot (-2 \sum_{i=0}^{M-1} i)$.

Note that Y_{B_1} and Y_{B_2} are independent if $B_1 \cap B_2 = \emptyset$. The total number of different pairs (B_1, B_2) such that $B_1 \cap B_2 = \emptyset$ is

$$\binom{N}{2M} \cdot \binom{2M}{M} \cdot \frac{1}{2} = \frac{1}{2(M!)^2} \cdot \frac{N!}{(N-2M)!} = \frac{1}{2(M!)^2} \cdot \Pi_{i=0}^{2M-1}(N-i)$$

This is again a polynomial function of N with degree $2M$. The coefficient of the term N^{2M} is $\frac{1}{2(M!)^2}$. The coefficient of the term N^{2M-1} is $\frac{1}{2(M!)^2} \cdot (-\sum_{i=0}^{2M-1} i)$. So the number of non-zero terms in A is at most

$$\begin{aligned} &\frac{1}{2(M!)^2} \cdot \Pi_{i=0}^{M-1}(N-i)^2 - \frac{N!}{2 \cdot M!(N-M)!} - \frac{1}{2(M!)^2} \cdot \Pi_{i=0}^{2M-1}(N-i) \\ &= \frac{M^2}{2(M!)^2} \cdot N^{2M-1} + O(N^{2M-2}) \end{aligned}$$

Moreover, by Cauchy-Schwarz inequality, for any $B_1, B_2 \subset \mathcal{N}$

$$\text{Cov}(Y_{B_1}, Y_{B_2}) \leq \sqrt{\text{Var}(Y_{B_1}) \cdot \text{Var}(Y_{B_2})} = v_M$$

Therefore,

$$A \leq \left[\frac{M^2}{2(M!)^2} \cdot N^{2M-1} + O(N^{2M-2}) \right] v_M$$

which implies

$$\begin{aligned} \text{Var}(Y_{M,N}) &= \frac{\gamma^2(M!)^2}{(\Pi_{i=0}^{M-1}(N-i))^2} \left[\sum_{B \subset \mathcal{N}} \text{Var}(Y_B) + 2 \cdot \sum_{\substack{B_1, B_2 \subset \mathcal{N} \\ B_1 \neq B_2}} \text{Cov}(Y_{B_1}, Y_{B_2}) \right] \\ &= \frac{\gamma^2(M!)^2}{(\Pi_{i=0}^{M-1}(N-i))^2} \left[\sum_{B \subset \mathcal{N}} \text{Var}(Y_B) + 2A \right] \\ &\leq \gamma^2 \left[M^2 \cdot \frac{N^{2M-1}}{\Pi_{i=0}^{M-1}(N-i)^2} + O\left(\frac{1}{N^2}\right) \right] v_M \xrightarrow{N \rightarrow \infty} 0 \end{aligned}$$

Moreover,

$$\lim_{N \rightarrow \infty} \frac{M^2 v_M / N}{\left[M^2 \cdot \frac{N^{2M-1}}{\Pi_{i=0}^{M-1}(N-i)^2} + O\left(\frac{1}{N^2}\right) \right] v_M} = \lim_{N \rightarrow \infty} \frac{\Pi_{i=0}^{M-1}(N-i)^2}{N^{2M}} = 1$$

A.4 PROOF OF CONVERGENCE OF TABULAR REDQ

Assuming that the step size satisfies the standard Robbins-Monro conditions, it is easily seen that the tabular version of REDQ converges with probability 1 to the optimal Q function. In fact, for our Weighted scheme, where we take the expectation over all sets of size M , the convergence conditions in [Lan et al. \(2020\)](#) are fully satisfied.

For the randomized case, only very minor changes are needed in the proof in [Lan et al. \(2020\)](#). Note that in the case of REDQ, the underlying deterministic target is:

$$F(Q^1, Q^2, \dots, Q^N)(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) \sum_{\substack{B \subset \mathcal{N} \\ |B|=M}} \frac{1}{\binom{N}{M}} \max_{a'} \min_{j \in B} Q^j(s', a')$$

Let $\mathcal{T}$ be the operator that concatenates N identical copies of F , so that $\mathcal{T}: \mathbb{R}^{S \times A \times N} \rightarrow \mathbb{R}^{S \times A \times N}$ where S and A are the cardinalities of the state and action spaces, respectively. It is easy to show that the operator $\mathcal{T}$ is a contraction with the l_∞ norm. The stochastic approximation noise term is given by

$$\omega(s, a) = R - r(s, a) + \gamma \left[\max_{a'} \min_{j \in B} Q^j(s', a') - \sum_{s'} p(s' | s, a) \sum_{\substack{B \subset \mathcal{N} \\ |B|=M}} \frac{1}{\binom{N}{M}} \max_{a'} \min_{j \in B} Q^j(s', a') \right]$$

It is straightforward to show

$$\mathbb{E} \left[\omega^2(s, a) \mid \mathcal{F}_{\text{past}} \right] \leq \text{Var}(R \mid s, a) + \max_{1 \leq i \leq N} \max_{s', a'} (Q^i(s', a'))^2 \quad (2)$$

As in [Lan et al. \(2020\)](#), it follows from the contraction property and [\(2\)](#) that REDQ converges with probability 1 to the optimal Q function ([Tsitsiklis, 1994](#); [Bertsekas & Tsitsiklis, 1996](#)).

B HYPERPARAMETERS AND IMPLEMENTATION DETAILS

Since MBPO builds on top of a SAC agent, to make our comparisons fair, meaningful, and consistent with previous work, we make all SAC related hyperparameters exactly the same as used in the MBPO paper (Janner et al., 2019). Table 1 gives a list of hyperparameter used in the experiments. For all the REDQ curves reported in the results section, we use a Q network ensemble size N of 10. We use a UTD ratio G of 20 on the four MuJoCo environments, which is the same value that was used in the MBPO paper. Thus most of the hyperparameters are made to be the same as in the MBPO paper to ensure fairness and consistency in comparisons.

For all the algorithms and variants, we also first obtain 5000 data points by randomly sampling actions from the action space without making parameter updates. In our experiments we found that using a high UTD from the very beginning with a very small amount of data can easily lead to complete divergence on SAC-20. Sampling a number of random datapoints at the start of training is also a common technique that has been used in previous model-free as well as model-based works (Haarnoja et al., 2018a; Fujimoto et al., 2018; Janner et al., 2019).

Table 1: REDQ hyperparameters

Parameter	Value
<i>Shared</i>	
optimizer	Adam (Kingma & Ba, 2014)
learning rate	$3 \cdot 10^{-4}$
discount (γ)	0.99
target smoothing coefficient (ρ)	0.005
replay buffer size	10^6
number of hidden layers for all networks	2
number of hidden units per layer	256
mini-batch size	256
nonlinearity	ReLU
random starting data	5000
<i>REDQ</i>	
ensemble size N	10
in-target minimization parameter M	2
UTD ratio G	20
<i>OFENet</i>	
random starting data	20,000
OFENet number of pretraining updates	100,000
OFENet UTD ratio	4

For the REDQ-OFE experiments, we implemented a minimal version of the OFENet in the original paper, with no batchnorm layers. We use the recommended hyperparameters as described in the original paper (Ota et al., 2020). Compared to REDQ without OFENet, the main difference is we now first collect 20,000 random data points (which is accounted for in the training curves), and then pre-train the OFENet for 100,000 updates, with the same learning rate and batch size. We then train OFENet together with REDQ agent, and the OFENet uses a UTD ratio of 4. We tried a simple hyperparameter search on Ant with 200,000, 100,000 and 50,000 pre-train updates, and learning rates of $1e-4$, $3e-4$, $5e-4$, and a OFENet UTD of 1, 4 and 20. However, the results are not very different. It is possible that better results can be obtained through a more extensive hyperparameter search or other modifications.

B.1 EFFICIENT CALCULATION OF TARGET FOR WEIGHTED VERSION OF REDQ

In section 4 we provided experimental results for the Weighted version of REDQ. Recall that in this version, instead of sampling a random subset $\mathcal{M}$ in the target, we average over all subsets B in

$\{1, \dots, N\}$ of size M :

$$y = r + \gamma \frac{1}{\binom{N}{M}} \sum_B \left(\min_{i \in B} Q_{\phi_{\text{tar}}, i} (s', \tilde{a}') \right)$$

In practice, however, we do not need to sum over all N choose M subsets. Instead we can re-order the indices so that

$$Q_{\phi_{\text{tar}}, i} (s', \tilde{a}') \leq Q_{\phi_{\text{tar}}, i+1} (s', \tilde{a}')$$

for $i = 1, \dots, N - 1$. After the re-ordering, we can use the identity:

$$\frac{1}{\binom{N}{M}} \sum_B \left(\min_{i \in B} Q_{\phi_{\text{tar}}, i} (s', \tilde{a}') \right) = \frac{1}{\binom{N}{M}} \sum_{i=1}^{N-M+1} \binom{N-i}{M-1} Q_{\phi_{\text{tar}}, i} (s', \tilde{a}')$$

C SAMPLE EFFICIENCY COMPARISON FOR REDQ, SAC AND MBPO

The sample efficiency claims made in the main paper are based on Table 2 and Table 3. Table 2 shows that compared to naive SAC, REDQ is much more sample efficient. REDQ reaches 3500 on Hopper with 8x sample efficiency, and reaches 5000 for Ant and Humanoid with 5x and 3.7x sample efficiency. After adding OFE, this becomes more than 7x on Ant and Humanoid. If we average all the numbers for the four environments, then REDQ is 5.0x as sample efficient, and 6.4x after including OFE results.

Table 3 compares REDQ to SAC and MBPO. As in the MBPO paper, we train for 125K for Hopper, and 300K for the other three environments (Janner et al., 2019). The numbers in Table 3 show the performance when trained to half and to the full length of the MBPO training limits. When averaging the numbers, we see that REDQ reaches 4.5x and 2.1x the performance of SAC at 150K and 300K. REDQ is also stronger than MBPO, with 1.4x and 1.1x the performance of MBPO at 150K and 300K. If we include the results of REDQ-OFE, then the numbers become 5.5x and 2.3x the SAC performance at 150K and 300K, and 1.8x and 1.2x the MBPO performance at 150 and 300K.

Table 2: Sample efficiency comparison of SAC and REDQ. The numbers show the amount of data collected when the specified performance level is reached. The last two columns show how many times REDQ and REDQ-OFE are more sample efficient than SAC in reaching that performance.

Score	SAC	REDQ	REDQ-OFE	REDQ faster	REDQ-OFE faster
Hopper at 3500	933K	116K	-	8.04	-
Walker2d at 3500	440K	141K	-	3.12	-
Ant at 5000	771K	153K	106K	5.04	7.27
Humanoid at 5000	945K	255K	134K	3.71	7.05

Table 3: Performance comparison of REDQ, REDQ-OFE, MBPO and SAC. The numbers show the performance achieved when the specific amount of data is collected. The last two columns show the ratio of REDQ or REDQ-OFE performance compared to SAC and MBPO performance.

Amount of data	SAC	MBPO	REDQ	REDQ/SAC	REDQ/MBPO
Hopper at 62K	594	1919	3278	5.52	1.71
Hopper at 125K	2404	3131	3517	1.46	1.12
Walker2d at 150K	760	3308	3544	4.66	1.07
Walker2d at 300K	2556	3537	4589	1.80	1.30
Ant at 150K	1245	4388	4803	3.86	1.09
Ant at 300K	2485	5774	5369	2.16	0.93
Humanoid at 150K	674	1604	2641	3.92	1.65
Humanoid at 300K	1633	4199	4674	2.86	1.11
Amount of data	SAC	MBPO	REDQ-OFE	OFE/SAC	OFE/MBPO
Ant at 150K	1245	4388	5524	4.44	1.26
Ant at 300K	2485	5774	6578	2.65	1.14
Humanoid at 150K	674	1604	5011	7.43	3.12
Humanoid at 300K	1633	4199	5309	3.25	1.26

D NUMBER OF PARAMETERS COMPARISON

Table 4 gives the number of parameters for MBPO, REDQ and REDQ-OFE, for all four environments. As discussed in the main paper, REDQ uses fewer parameters than MBPO for all four environments: between 26% and 70% as many parameters depending on the environment. After adding OFENet, REDQ still uses fewer parameters than MBPO, with 80% and 35% as many parameters on Ant and Humanoid. In particular, it is surprising that REDQ-OFE can achieve a much stronger result on Humanoid with much fewer parameters.

Table 4: Number of parameters in millions. REDQ uses the same network structure and ensemble size for all four environments. The difference in the number of parameters comes from the fact that the environments have very different observation and action dimensions, which will affect the size of the input and output layers of the networks.

Algorithm	Hopper	Walker2d	Ant	Humanoid
MBPO	1.106M	1.144M	1.617M	7.087M
REDQ $N = 10$	0.769M	0.795M	1.066M	1.840M
REDQ-OFE $N = 10$	-	-	1.294M	2.460M

E ADDITIONAL RESULTS FOR REDQ, SAC-20, AND AVG

Due to lack of space, Figure 2 in Section 3 only compared REDQ with SAC-20 and AVG for the Ant environment. Figure 5 presents the results for all four environments. We can see that in all four environments, REDQ has much stronger performance and much lower std of bias compared to SAC-20 and AVG. Note in terms of average normalized bias, AVG is slightly closer to zero in Ant compared to REDQ, and SAC-20 is a bit closer to zero in Humanoid compared to REDQ; however, their std of normalized bias is consistently higher. This shows the importance of having a low std of the bias in addition to a close-to-zero average bias.

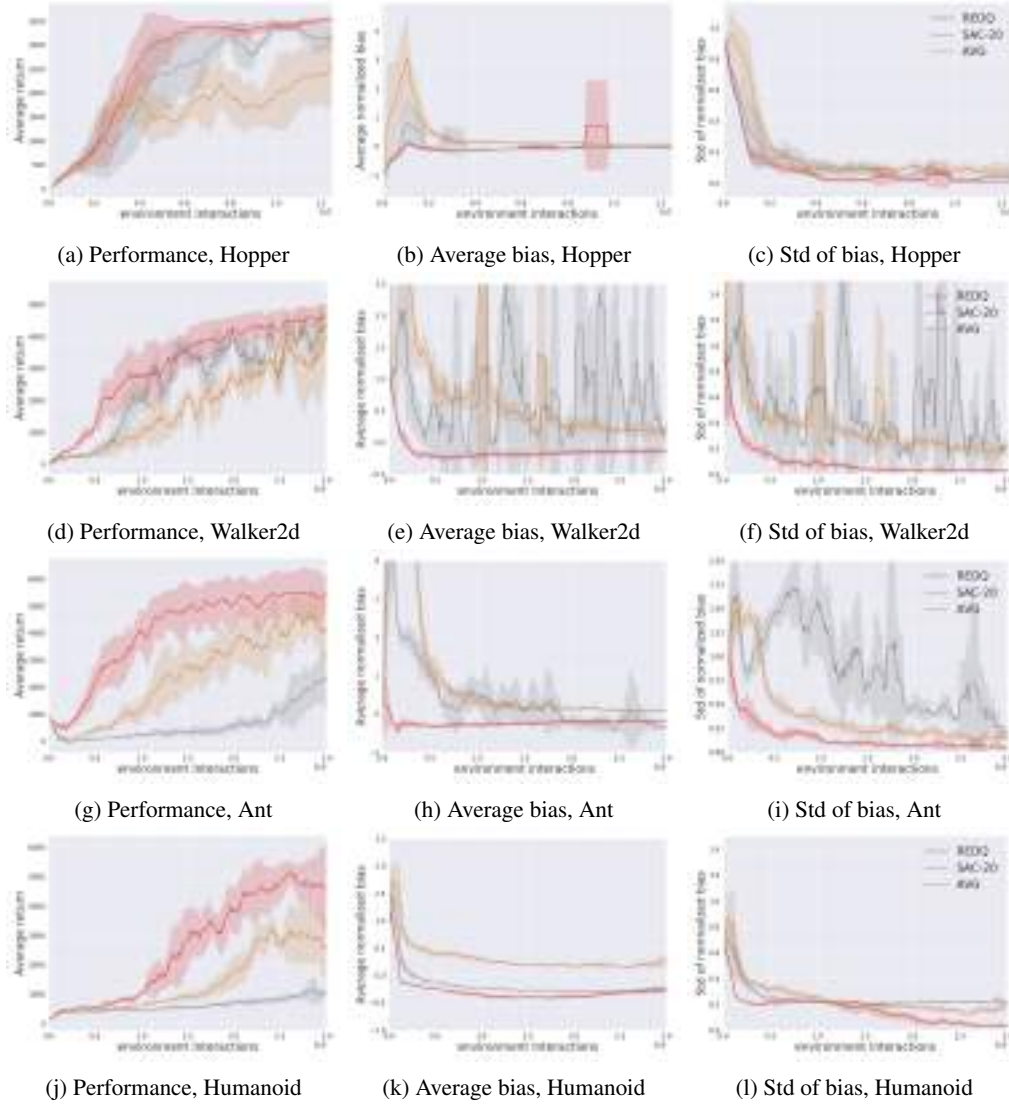


Figure 5: Performance, mean and std of normalized Q bias for REDQ, AVG, and SAC. All three algorithms have a UTD ratio of 20.

F REDQ AND SAC WITH AND WITHOUT POLICY DELAY

Note that in the REDQ pseudocode, the number of policy updates is always one for each data point collected. We set the UTD ratio for the policy update to always be one in order to isolate the effect of additional policy updates from Q updates. Note in this way, REDQ, SAC-20 and SAC-1 all take the same number of policy updates. This helps show that the performance gain mainly comes from the additional Q updates.

Having a lower number of policy updates can also be seen as a delayed policy update, or policy delay, and is a method that has been used in previous works to improve learning stability (Fujimoto et al., 2018). In this section we discuss how delayed policy update, or policy delay, impact the performance of REDQ and SAC (with UTD of 20). Figure 6 compares REDQ and SAC-20 with and without policy delay (NPD for no policy delay). We can see that having the policy delay consistently makes the bias and std of bias lower and more stable, although they have a smaller effect on REDQ than on SAC. Performance-wise SAC always gets a performance boost with policy delay, while REDQ sees improvement in Hopper and Humanoid, and becomes slightly worse in Walker2d and Ant. The results show that policy delay can be important under high UTD when the variance is not properly controlled. However, with enough variance reduction, the effect of policy delay is diminished, and in some cases having more policy update can give better performance.

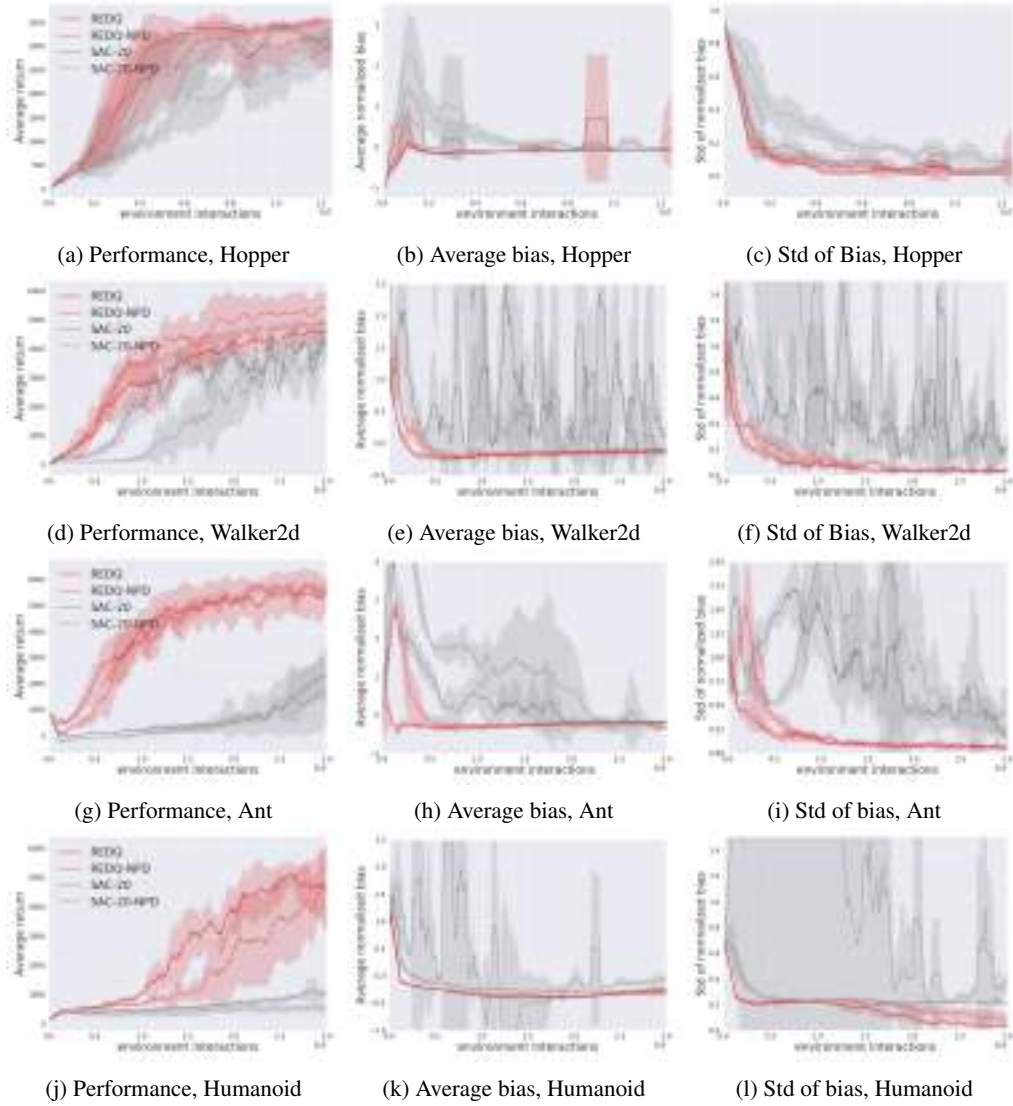


Figure 6: Performance, mean and std of normalized Q bias of REDQ and SAC, with and without policy delay.

G REDQ AND SAC WITH DIFFERENT UTD RATIOS

How do different UTD ratio values G impact the performance of REDQ and SAC? Figure 7 compares the two algorithms under UTD ratio values of 1, 5, 10 and 20 for the Ant environment. The results show that in the Ant environment, REDQ greatly benefits from larger UTD values, with UTD of 20 giving the best result. For SAC, performance improves slightly for UTD ratios of 5 and 10, but becomes much worse at 20. Looking at the normalized bias and the std of the bias, we see that changing the UTD ratio does not change the values very much for REDQ, while for SAC, we see that as the UTD ratio increases, both the mean and the std of the bias becomes larger and more unstable.

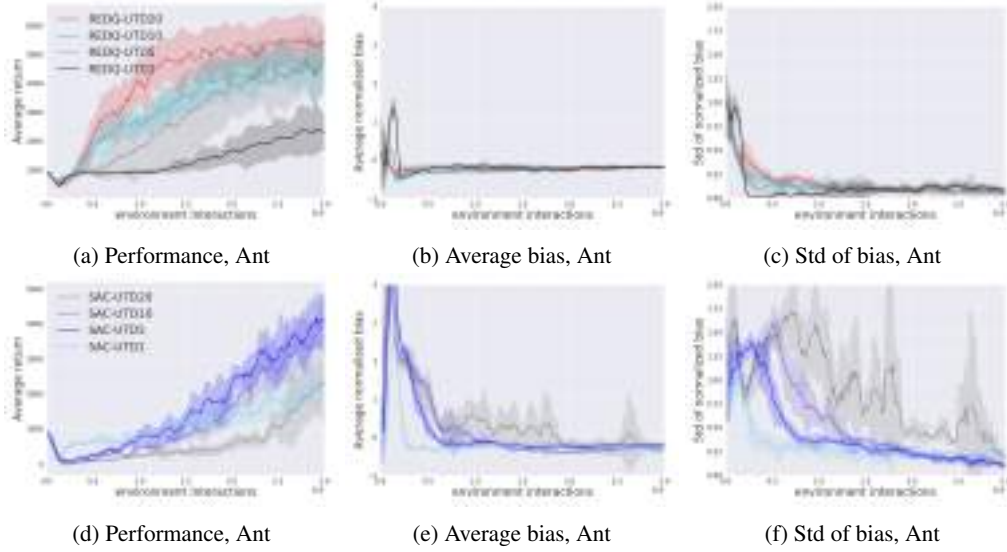


Figure 7: Performance, mean and std of normalized Q bias for REDQ, and SAC, with different UTD ratios, in Ant environment.

H ADDITIONAL RESULTS FOR WEIGHTED VARIANT

In this section we provide additional results for the Weighted variant. Figure 8 shows the performance and bias comparison on all four environments. Results show that Weighted and REDQ have similar average bias and std of bias. In terms of performance, Weighted is worse in Ant and Hopper, similar in Humanoid and slightly stronger in Walker2d. Overall REDQ seems to have stronger performance and is more robust. Randomness in the networks might help alleviate overfitting in the early stage, or improve exploration, as shown in previous studies (Osband et al., 2016; Fortunato et al., 2018). This can be important since positive bias in Q learning-based methods can sometimes help exploration. This is commonly referred to as optimistic initial values, or optimism in the face of uncertainty (Sutton & Barto, 2018; Brafman & Tennenholtz, 2002). Thus conservative Q estimates in recent algorithms can lead to the problem of pessimistic underexploration (Ciosek et al., 2019). An interesting future work direction is to study how robust and effective exploration can be achieved without relying on optimistic estimates.

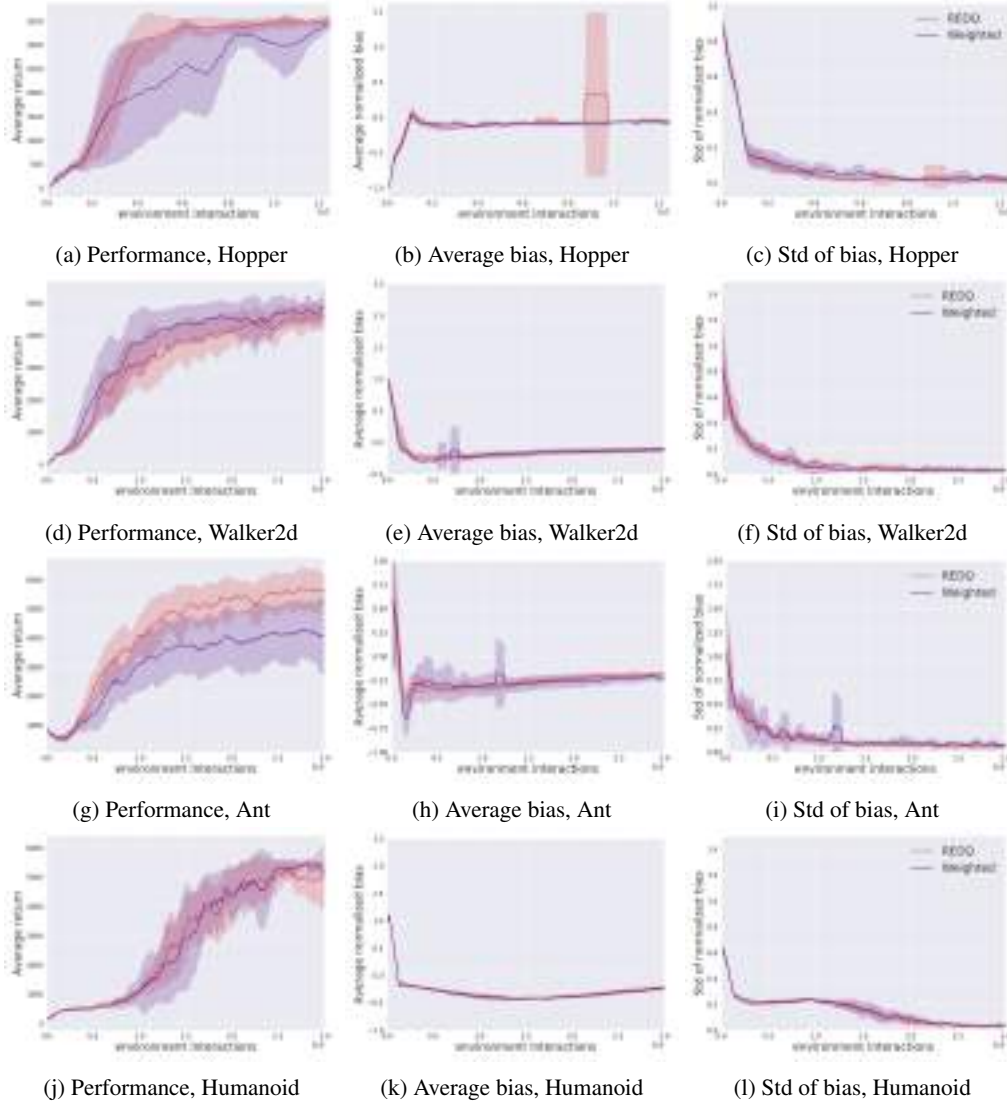


Figure 8: Performance, mean and std of normalized Q bias for REDQ and Weighted.

TD-MPC2: Scalable, Robust World Models for Continuous Control

Nicklas Hansen*, Hao Su^{*†}, Xiaolong Wang^{*†}

^{*}University of California San Diego, [†]Equal advising
{nihansen, haosu, xiw012}@ucsd.edu

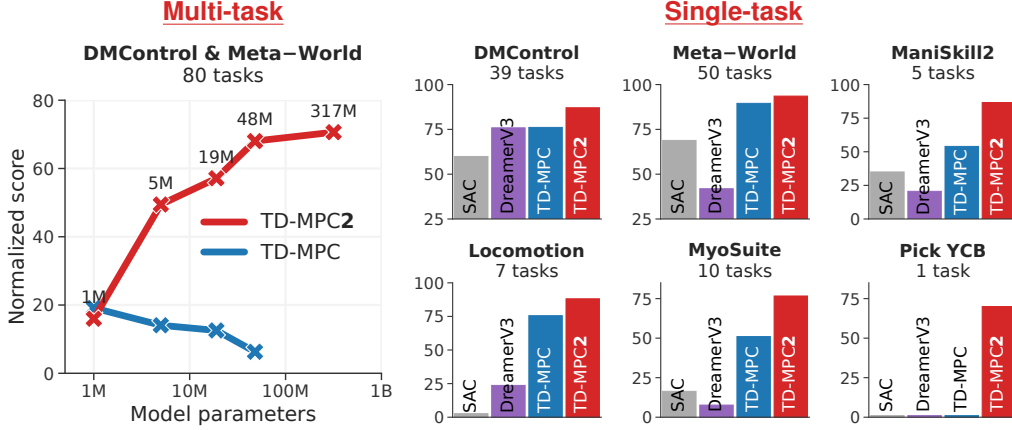


Figure 1. **Overview.** TD-MPC2 compares favorably to existing model-free and model-based RL methods across 104 continuous control tasks spanning multiple domains, with a *single* set of hyperparameters (*right*). We further demonstrate the scalability of TD-MPC2 by training a single 317M parameter agent to perform 80 tasks across multiple domains, embodiments, and action spaces (*left*).

ABSTRACT

TD-MPC is a model-based reinforcement learning (RL) algorithm that performs local trajectory optimization in the latent space of a learned implicit (decoder-free) world model. In this work, we present TD-MPC2: a series of improvements upon the TD-MPC algorithm. We demonstrate that TD-MPC2 improves significantly over baselines across 104 online RL tasks spanning 4 diverse task domains, achieving consistently strong results with a single set of hyperparameters. We further show that agent capabilities increase with model and data size, and successfully train a single 317M parameter agent to perform 80 tasks across multiple task domains, embodiments, and action spaces. We conclude with an account of lessons, opportunities, and risks associated with large TD-MPC2 agents.

Explore videos, models, data, code, and more at <https://tdmpc2.com>

1 INTRODUCTION

Training large models on internet-scale datasets has led to generalist models that perform a wide variety of language and vision tasks (Brown et al., 2020; He et al., 2022; Kirillov et al., 2023). The success of these models can largely be attributed to the availability of enormous datasets, and carefully designed architectures that reliably scale with model and data size. While researchers have recently extended this paradigm to robotics (Reed et al., 2022; Brohan et al., 2023), a generalist embodied agent that learns to perform diverse control tasks via low-level actions, across multiple embodiments, from large uncured (*i.e.*, mixed-quality) datasets remains an elusive goal. We argue that current approaches to generalist embodied agents suffer from (a) the assumption of near-expert trajectories for behavior cloning which severely limits the amount of available data (Reed et al., 2022; Lee et al., 2022; Kumar et al., 2022; Schubert et al., 2023; Driess et al., 2023; Brohan et al., 2023), and (b) a lack of scalable continuous control algorithms that are able to consume large uncured datasets.

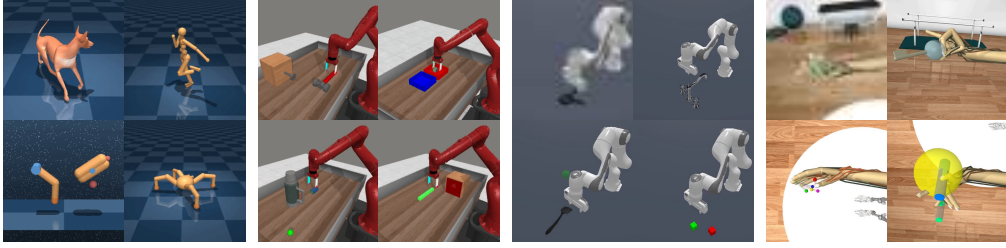


Figure 2. **Tasks.** TD-MPC2 performs 104 diverse tasks from (left to right) DMControl (Tassa et al., 2018), Meta-World (Yu et al., 2019), ManiSkill2 (Gu et al., 2023), and MyoSuite (Caggiano et al., 2022), with a *single* set of hyperparameters. See Appendix B for visualization of all tasks.

Reinforcement Learning (RL) is an ideal framework for extracting expert behavior from uncuration datasets. However, most existing RL algorithms (Lillicrap et al., 2016; Haarnoja et al., 2018) are designed for single-task learning and rely on per-task hyperparameters, with no principled method for selecting those hyperparameters (Zhang et al., 2021). An algorithm that can consume large multi-task datasets will invariably need to be robust to variation between different tasks (e.g., action space dimensionality, difficulty of exploration, and reward distribution). In this work, we present TD-MPC2: a significant step towards achieving this goal. TD-MPC2 is a model-based RL algorithm designed for learning generalist world models on large uncuration datasets composed of multiple task domains, embodiments, and action spaces, with data sourced from behavior policies that cover a wide range of skill levels, and without the need for hyperparameter-tuning.

Our algorithm, which builds upon TD-MPC (Hansen et al., 2022), performs local trajectory optimization in the latent space of a learned implicit (decoder-free) world model. While the TD-MPC family of algorithms has demonstrated strong empirical performance in prior work (Hansen et al., 2022; 2023; Yuan et al., 2022; Yang et al., 2023; Feng et al., 2023; Chitnis et al., 2023; Zhu et al., 2023; Lancaster et al., 2023), most successes have been limited to single-task learning with little emphasis on scaling. As shown in Figure 1, naively increasing model and data size of TD-MPC often leads to a net *decrease* in agent performance, as is commonly observed in RL literature (Kumar et al., 2023). In contrast, scaling TD-MPC2 leads to consistently improved capabilities. Our algorithmic contributions, which have been key to achieving this milestone, are two-fold: (1) improved algorithmic robustness by revisiting core design choices, and (2) careful design of an architecture that can accommodate datasets with multiple embodiments and action spaces without relying on domain knowledge. The resulting algorithm, TD-MPC2, is scalable, robust, and can be applied to a variety of single-task and multi-task continuous control problems using a *single* set of hyperparameters.

We evaluate TD-MPC2 across a total of 104 diverse continuous control tasks spanning 4 task domains: DMControl (Tassa et al., 2018), Meta-World (Yu et al., 2019), ManiSkill2 (Gu et al., 2023), and MyoSuite (Caggiano et al., 2022). We summarize our results in Figure 1, and visualize task domains in Figure 2. Tasks include high-dimensional state and action spaces (up to $\mathcal{A} \in \mathbb{R}^{39}$), image observations, sparse rewards, multi-object manipulation, physiologically accurate musculoskeletal motor control, complex locomotion (e.g. Dog and Humanoid embodiments), and cover a wide range of task difficulties. Our results demonstrate that TD-MPC2 consistently outperforms existing model-based and model-free methods, using the *same* hyperparameters across all tasks (Figure 1, *right*). Here, “Locomotion” and “Pick YCB” are particularly challenging subsets of DMControl and ManiSkill2, respectively. We further show that agent capabilities increase with model and data size, and successfully train a single 317M parameter world model to perform 80 tasks across multiple task domains, embodiments, and action spaces (Figure 1, *left*). In support of open-source science, **we publicly release 300+ model checkpoints, datasets, and code for training and evaluating TD-MPC2 agents, which is available at <https://tdmcp2.com>**. We conclude the paper with an account of lessons, opportunities, and risks associated with large TD-MPC2 agents.

2 BACKGROUND

Reinforcement Learning (RL) aims to learn a policy from interaction with an environment, formulated as a Markov Decision Process (MDP) (Bellman, 1957). We focus on infinite-horizon MDPs with continuous action spaces, which can be formalized as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma)$ where $s \in \mathcal{S}$ are states, $a \in \mathcal{A}$ are actions, $\mathcal{T}: \mathcal{S} \times \mathcal{A} \mapsto \mathcal{S}$ is the transition function, $R: \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ is a reward function associated with a particular task, and γ is a discount factor. The goal is to

derive a control policy $\pi: \mathcal{S} \mapsto \mathcal{A}$ such that the expected discounted sum of rewards (return) $\mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r_t]$, $r_t = R(\mathbf{s}_t, \pi(\mathbf{s}_t))$ is maximized. In this work, we obtain π by learning a *world model* (model of the environment) and then select actions by planning with the learned model.

Model Predictive Control (MPC) is a general framework for model-based control that optimizes action sequences $\mathbf{a}_{t:t+H}$ of finite length such that return is maximized (or cost is minimized) over the time horizon H , which corresponds to solving the following optimization problem:

$$\pi(\mathbf{s}_t) = \arg \max_{\mathbf{a}_{t:t+H}} \mathbb{E} \left[\sum_{i=0}^H \gamma^{t+i} R(\mathbf{s}_{t+i}, \mathbf{a}_{t+i}) \right]. \quad (1)$$

The return of a candidate trajectory is estimated by simulating it with the learned model (Negenborn et al., 2005). Thus, a policy obtained by Equation 1 will invariably be a (temporally) *locally* optimal policy and is not guaranteed (nor likely) to be a solution to the general reinforcement learning problem outlined above. As we discuss in the following, TD-MPC2 addresses this shortcoming of local trajectory optimization by bootstrapping return estimates beyond horizon H with a learned terminal value function.

3 TD-MPC2

Our work builds upon TD-MPC (Hansen et al., 2022), a model-based RL algorithm that performs local trajectory optimization (planning) in the latent space of a learned implicit world model. TD-MPC2 is a practical algorithm for training massively multitask world models. Specifically, we propose a series of improvements to the TD-MPC algorithm, which have been key to achieving strong algorithmic robustness (can use the same hyperparameters across all tasks) and scaling its world model to $300\times$ more parameters than previously. We introduce the TD-MPC2 algorithm in the following, and provide a full list of algorithmic improvements in Appendix A.

3.1 LEARNING AN IMPLICIT WORLD MODEL

Learning a generative model of the environment using a reconstruction (decoder) objective is tempting due to its rich learning signal. However, accurately predicting raw future observations (e.g., images or proprioceptive features) over long time horizons is a difficult problem, and does not necessarily lead to effective control (Lambert et al., 2020). Rather than explicitly modeling dynamics using reconstruction, TD-MPC2 aims to learn a *maximally useful* model: a model that accurately predicts *outcomes* (returns) conditioned on a sequence of actions. Specifically, TD-MPC2 learns an *implicit*, control-centric world model from environment interaction using a combination of joint-embedding prediction (Grill et al., 2020), reward prediction, and TD-learning (Sutton, 1998), *without* decoding observations. We argue that this alternative formulation of model-based RL is key to modeling large datasets with modest model sizes. The world model can subsequently be used for decision-making by performing local trajectory optimization (planning) following the MPC framework.

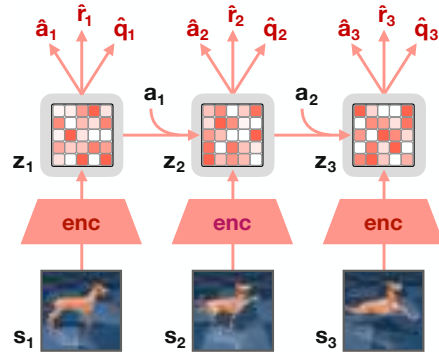


Figure 3. The TD-MPC2 architecture. Observations $\mathbf{s}$ are encoded into their (normalized) latent representation $\mathbf{z}$. The model then recurrently predicts actions $\hat{\mathbf{a}}$, rewards $\hat{r}$, and terminal values $\hat{q}$, *without* decoding future observations.

Components. The TD-MPC2 architecture is shown in Figure 3 and consists of five components:

Encoder	$\mathbf{z} = h(\mathbf{s}, \mathbf{e})$	▷ Maps observations to their latent representations
Latent dynamics	$\mathbf{z}' = d(\mathbf{z}, \mathbf{a}, \mathbf{e})$	▷ Models (latent) forward dynamics
Reward	$\hat{r} = R(\mathbf{z}, \mathbf{a}, \mathbf{e})$	▷ Predicts reward r of a transition
Terminal value	$\hat{q} = Q(\mathbf{z}, \mathbf{a}, \mathbf{e})$	▷ Predicts discounted sum of rewards (return)
Policy prior	$\hat{\mathbf{a}} = p(\mathbf{z}, \mathbf{e})$	▷ Predicts action $\mathbf{a}^*$ that maximizes Q

where $\mathbf{s}$ and $\mathbf{a}$ are states and actions, $\mathbf{z}$ is the latent representation, and $\mathbf{e}$ is a learnable task embedding for use in multitask world models. For visual clarity, we will omit $\mathbf{e}$ in the following unless it is

particularly relevant. The policy prior p serves to guide the sample-based trajectory optimizer (planner), and to reduce the computational cost of TD-learning. During online interaction, TD-MPC2 maintains a replay buffer $\mathcal{B}$ with trajectories, and iteratively (i) updates the world model using data sampled from $\mathcal{B}$, and (ii) collects new environment data by planning with the learned model.

Model objective. The h, d, R, Q components are jointly optimized to minimize the objective

$$\mathcal{L}(\theta) \doteq \mathbb{E}_{(s, \mathbf{a}, r, s')_{0:H} \sim \mathcal{B}} \left[\sum_{t=0}^H \lambda^t \left(\underbrace{\| \mathbf{z}'_t - \text{sg}(h(s'_t)) \|_2^2}_{\text{Joint-embedding prediction}} + \underbrace{\text{CE}(\hat{r}_t, r_t)}_{\text{Reward prediction}} + \underbrace{\text{CE}(\hat{q}_t, q_t)}_{\text{Value prediction}} \right) \right], \quad (3)$$

where sg is the `stop-grad` operator, $(\mathbf{z}'_t, \hat{r}_t, \hat{q}_t)$ are defined in Equation 2, $q_t \doteq r_t + \gamma \bar{Q}(\mathbf{z}'_t, p(\mathbf{z}'_t))$ is the TD-target at step t , $\lambda \in (0, 1]$ is a constant coefficient that weighs temporally farther time steps less, and CE is the cross-entropy. $\bar{Q}$ used to compute the TD-target is an exponential moving average (EMA) of Q (Lillicrap et al., 2016). As the magnitude of rewards may differ drastically between tasks, TD-MPC2 formulates reward and value prediction as a discrete regression (multi-class classification) problem in a log-transformed space, which is optimized by minimizing cross-entropy with r_t, q_t as soft targets (Bellemare et al., 2017; Kumar et al., 2023; Hafner et al., 2023).

Policy objective. The policy prior p is a stochastic maximum entropy (Ziebart et al., 2008; Haarnoja et al., 2018) policy that learns to maximize the objective

$$\mathcal{L}_p(\theta) \doteq \mathbb{E}_{(s, \mathbf{a})_{0:H} \sim \mathcal{B}} \left[\sum_{t=0}^H \lambda^t [\alpha Q(\mathbf{z}_t, p(\mathbf{z}_t)) - \beta \mathcal{H}(p(\cdot | \mathbf{z}_t))] \right], \quad \mathbf{z}_{t+1} = d(\mathbf{z}_t, \mathbf{a}_t), \quad \mathbf{z}_0 = h(s_0), \quad (4)$$

where $\mathcal{H}$ is the entropy of p which can be computed in closed form. Gradients of $\mathcal{L}_p(\theta)$ are taken wrt. p only. As magnitude of the value estimate $Q(\mathbf{z}_t, p(\mathbf{z}_t))$ and entropy $\mathcal{H}$ can vary greatly between datasets and different stages of training, it is necessary to balance the two losses to prevent premature entropy collapse (Yarats et al., 2021). A common choice for automatically tuning α, β is to keep one of them constant, and adjusting the other based on an entropy target (Haarnoja et al., 2018) or moving statistics (Hafner et al., 2023). In practice, we opt for tuning α via moving statistics, but empirically did not observe any significant difference in results between these two options.

Architecture. All components of TD-MPC2 are implemented as MLPs with intermediate linear layers followed by LayerNorm (Ba et al., 2016) and Mish (Misra, 2019) activations. To mitigate exploding gradients, we normalize the latent representation by projecting $\mathbf{z}$ into L fixed-dimensional simplices using a softmax operation (Lavoie et al., 2022). A key benefit of embedding $\mathbf{z}$ as simplices (as opposed to *e.g.* a discrete representation or squashing) is that it naturally biases the representation towards sparsity without enforcing hard constraints (see Appendix H for motivation and implementation). We dub this normalization scheme *SimNorm*. Let V be the dimensionality of each simplex $\mathbf{g}$ constructed from L partitions (groups) of $\mathbf{z}$. *SimNorm* then applies the following transformation:

$$\mathbf{z}^\circ \doteq [\mathbf{g}_1, \dots, \mathbf{g}_L], \quad \mathbf{g}_i = \frac{e^{\mathbf{z}_{i:i+V}/\tau}}{\sum_{j=1}^L e^{\mathbf{z}_{j:j+V}/\tau}}, \quad (5)$$

where $\mathbf{z}^\circ$ is the simplicial embedding of $\mathbf{z}$, $[\cdot]$ denotes concatenation, and $\tau > 0$ is a temperature parameter that modulates the “sparsity” of the representation. As we will demonstrate in our experiments, *SimNorm* is essential to the training stability of TD-MPC2. Finally, to reduce bias in TD-targets generated by $\bar{Q}$, we learn an *ensemble* of Q -functions using the objective from Equation 3 and maintain $\bar{Q}$ as an EMA of each Q -function. We use 5 Q -functions in practice. Targets are then computed as the minimum of two randomly sub-sampled $\bar{Q}$ -functions (Chen et al., 2021).

3.2 MODEL PREDICTIVE CONTROL WITH A POLICY PRIOR

TD-MPC2 derives its closed-loop control policy by planning with the learned world model. Specifically, our approach leverages the MPC framework for local trajectory optimization using Model Predictive Path Integral (MPPI) (Williams et al., 2015) as a derivative-free optimizer with sampled action sequences $(\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H})$ of length H evaluated by rolling out *latent* trajectories with the model. At each decision step, we estimate parameters μ^*, σ^* of a time-dependent multivariate Gaussian with diagonal covariance such that expected return is maximized, *i.e.*,

$$\mu^*, \sigma^* = \arg \max_{(\mu, \sigma)} \mathbb{E}_{(\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H}) \sim \mathcal{N}(\mu, \sigma^2)} \left[\gamma^H Q(\mathbf{z}_{t+H}, \mathbf{a}_{t+H}) + \sum_{h=t}^{H-1} \gamma^h R(\mathbf{z}_h, \mathbf{a}_h) \right], \quad (6)$$

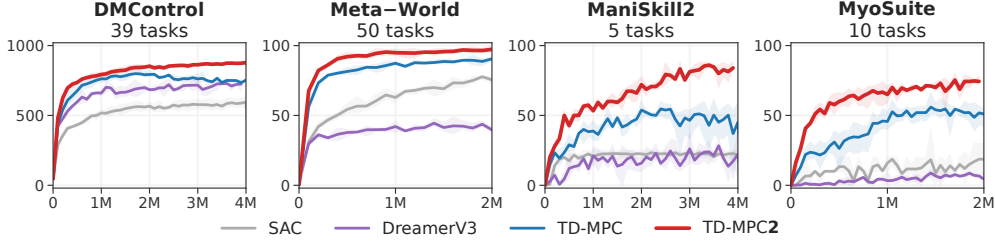


Figure 4. Single-task RL. Episode return (DMControl) and success rate (others) as a function of environment steps across **104** continuous control tasks spanning 4 diverse task domains. TD-MPC2 achieves higher data-efficiency and asymptotic performance than existing methods, while using the **same** hyperparameters across all tasks. Mean and 95% CIs over 3 seeds.

where $\mu, \sigma \in \mathbb{R}^{H \times m}$, $\mathcal{A} \in \mathbb{R}^m$. Equation 6 is solved by iteratively sampling action sequences from $\mathcal{N}(\mu, \sigma^2)$, evaluating their expected return, and updating μ, σ based on a weighted average. Notably, Equation 6 estimates the full RL objective introduced in Section 2 by bootstrapping with the learned terminal value function beyond horizon H . TD-MPC2 repeats this iterative planning process for a fixed number of iterations and executes the first action $\mathbf{a}_t \sim \mathcal{N}(\mu_t^*, \sigma_t^*)$ in the environment. To accelerate convergence of planning, a fraction of action sequences originate from the policy prior p , and we warm-start planning by initializing (μ, σ) as the solution to the previous decision step shifted by 1. Refer to Hansen et al. (2022) for more details about the planning procedure.

3.3 TRAINING GENERALIST TD-MPC2 AGENTS

The success of TD-MPC2 in diverse single-task problems can be attributed to the algorithm outlined above. However, learning a large generalist TD-MPC2 agent that performs a variety of tasks across multiple task domains, embodiments, and action spaces poses several unique challenges: (i) how to learn and represent task semantics? (ii) how to accommodate multiple observation and action spaces without specific domain knowledge? (iii) how to leverage the learned model for few-shot learning of new tasks? We describe our approach to multitask model learning in the following.

Learnable task embeddings. To succeed in a multitask setting, an agent needs to learn a common representation that takes advantage of task similarities, while still retaining the ability to differentiate between tasks at test-time. When task or domain knowledge is available, *e.g.* in the form of natural language instructions, the task embedding $\mathbf{e}$ from Equation 2 may encode such information. However, in the general case where domain knowledge cannot be assumed, we may instead choose to *learn* the task embeddings (and, implicitly, task relations) from data. TD-MPC2 conditions all of its five components with a learnable, fixed-dimensional task embedding $\mathbf{e}$, which is jointly trained together with other components of the model. To improve training stability, we constrain the ℓ_2 -norm of $\mathbf{e}$ to be ≤ 1 ; this also leads to more semantically coherent task embeddings in our experiments. When finetuning a multitask TD-MPC2 agent to a new task, we can choose to either initialize $\mathbf{e}$ as the embedding of a semantically similar task, or simply as a random vector.

Action masking. TD-MPC2 learns to perform tasks with a variety of observation and action spaces, without any domain knowledge. To do so, we zero-pad all model inputs and outputs to their largest respective dimensions, and mask out invalid action dimensions in predictions made by the policy prior p during both training and inference. This ensures that prediction errors in invalid dimensions do not influence TD-target estimation, and prevents p from falsely inflating its entropy for tasks with small action spaces. We similarly only sample actions along valid dimensions during planning.

4 EXPERIMENTS

We evaluate TD-MPC2 across a total of **104** diverse continuous control tasks spanning 4 task domains: DMControl (Tassa et al., 2018), Meta-World (Yu et al., 2019), ManiSkill2 (Gu et al., 2023), and MyoSuite (Caggiano et al., 2022). Tasks include high-dimensional state and action spaces (up to $\mathcal{A} \in \mathbb{R}^{39}$), sparse rewards, multi-object manipulation, physiologically accurate musculoskeletal motor control, complex locomotion (*e.g.* Dog and Humanoid embodiments), and cover a wide range of task difficulties. We also include **10** DMControl tasks with visual observations. In support of open-source science, **we publicly release 300+ model checkpoints, datasets, and code for training and evaluating TD-MPC2 agents**, which is available at <https://tdmpc2.com>.

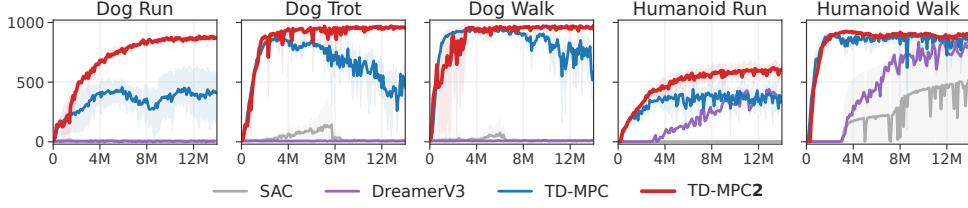


Figure 5. High-dimensional locomotion. Episode return as a function of environment steps in Humanoid ($\mathcal{A} \in \mathbb{R}^{21}$) and Dog ($\mathcal{A} \in \mathbb{R}^{38}$) locomotion tasks from DMControl. SAC and DreamerV3 are prone to numerical instabilities in Dog tasks, and are significantly less data-efficient than TD-MPC2 in Humanoid tasks. Mean and 95% CIs over 3 seeds. See Appendix D for more tasks.

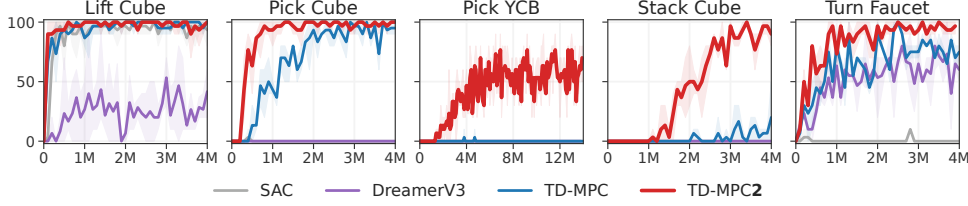


Figure 6. Object manipulation. Success rate (%) as a function of environment steps on 5 object manipulation tasks from ManiSkill2. *Pick YCB* considers manipulation of all 74 objects from the YCB (Calli et al., 2015) dataset. TD-MPC2 excels at hard tasks. Mean and 95% CIs over 3 seeds.

We seek to answer three core research questions through experimentation:

- **Comparison to existing methods.** How does TD-MPC2 compare to state-of-the-art model-free (SAC) and model-based (DreamerV3, TD-MPC) methods for data-efficient continuous control?
- **Scaling.** Do the algorithmic innovations of TD-MPC2 lead to improved agent capabilities as model and data size increases? Can a single agent learn to perform diverse skills across multiple task domains, embodiments, and action spaces?
- **Analysis.** How do the specific design choices introduced in TD-MPC2 influence downstream task performance? How much does planning contribute to its success? Are the learned task embeddings semantically meaningful? Can large multi-task agents be adapted to unseen tasks?

Baselines. Our primary baselines represent the state-of-the-art in data-efficient RL, and include (1) **Soft Actor-Critic** (SAC) (Haarnoja et al., 2018), a model-free actor-critic algorithm based on maximum entropy RL, (2) **DreamerV3** (Hafner et al., 2023), a model-based method that optimizes a model-free policy with rollouts from a learned generative model of the environment, and (3) the original version of **TD-MPC** (Hansen et al., 2022), a model-based RL algorithm that performs local trajectory optimization (planning) in the latent space of a learned *implicit* (non-generative) world model. Additionally, we also compare against current state-of-the-art visual RL methods (4) **CURL** (Srinivas et al., 2020), an extension of SAC that uses a contrastive auxiliary objective, and (5) **DrQ-v2**, a model-free RL algorithm that uses data augmentation. SAC and TD-MPC use task-specific hyperparameters, whereas TD-MPC2 uses the **same** hyperparameters across all tasks. Additionally, it is worth noting that both SAC and TD-MPC use a larger batch size of 512, while 256 is sufficient for stable learning with TD-MPC2. Similarly, DreamerV3 uses a high update-to-data (UTD) ratio of 512, whereas TD-MPC2 uses a UTD of 1 by default. We use a 5M parameter TD-MPC2 agent in all experiments (unless stated otherwise). For reference, the DreamerV3 baseline has approx. 20M learnable parameters. See Appendix H for more details.

4.1 RESULTS

Comparison to existing methods. We first compare the data-efficiency of TD-MPC2 to a set of strong baselines on 104 diverse tasks in an online RL setting. Aggregate results are shown in Figure 4. We find that TD-MPC2 outperforms prior methods across all task domains. The MyoSuite results are particularly noteworthy, as we did not run *any* TD-MPC2 experiments on this benchmark prior to the reported results. Individual task performances on some of the most difficult tasks (high-dimensional locomotion and multi-object manipulation) are shown in Figure 5 and Figure 6. TD-MPC2 outperforms baselines by a large margin on these tasks, despite using the same hyperparameters across all tasks. Notably, TD-MPC sometimes diverges due to exploding gradients,

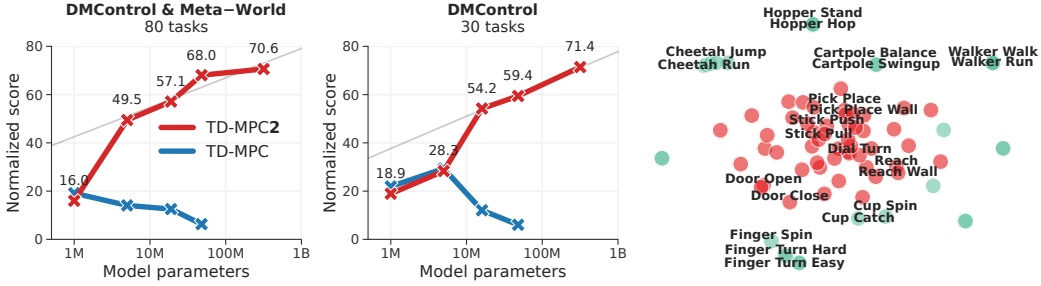


Figure 7. **Massively multi-task world models.** (Left) Normalized score as a function of model size on the two 80-task and 30-task datasets. TD-MPC2 capabilities scale with model size. (Right) T-SNE (van der Maaten & Hinton, 2008) visualization of task embeddings learned by a TD-MPC2 agent trained on 80 tasks from DMControl and Meta-World. A subset of labels are shown for clarity.

whereas TD-MPC2 remains stable. We provide per-task visualization of gradients in Appendix G. Similarly, we observe that DreamerV3 experiences occasional numerical instabilities (*Dog*) and generally struggles with tasks that require fine-grained object manipulation (*lift*, *pick*, *stack*). See Appendix D for the full single-task RL results.

Massively multitask world models. To demonstrate that our proposed improvements facilitate scaling of world models, we evaluate the performance of 5 multitask models ranging from 1M to 317M parameters on a collection of 80 diverse tasks that span multiple task domains and vary greatly in objective, embodiment, and action space. Models are trained on a dataset of 545M transitions obtained from the replay buffers of 240 single-task TD-MPC2 agents, and thus contain a wide variety of behaviors ranging from random to expert policies. The task set consists of all 50 Meta-World tasks, as well as 30 DMControl tasks. The DMControl task set includes 19 original DMControl tasks, as well as 11 new tasks. For completeness, we include a separate set of scaling results on the 30-task DMControl subset (345M transitions) as well. Due to our careful design of the TD-MPC2 algorithm, scaling up is straightforward: to improve rate of convergence we use a $4\times$ larger batch size (1024) compared to the single-task experiments, but make no other changes to hyperparameters.

Table 1. **Training cost.** Approximate TD-MPC2 training cost on the 80-task dataset, reported in GPU days on a single NVIDIA GeForce RTX 3090 GPU. We also list the normalized score achieved by each model at end of training.

Params (M)	GPU days	Score
1	3.7	16.0
5	4.2	49.5
19	5.3	57.1
48	12	68.0
317	33	70.6

Scaling TD-MPC2 to 317M parameters. Our scaling results are shown in Figure 7. To summarize agent performance with a single metric, we produce a normalized score that is an average of all individual task success rates (Meta-World) and episode returns normalized to the $[0, 100]$ range (DMControl). We observe that agent capabilities consistently increase with model size on both task sets. Notably, performance does not appear to have saturated for our largest models (317M parameters) on either dataset, and we can thus expect results to continue improving beyond our considered model sizes. We refrain from formulating a scaling law, but note that normalized score appears to scale linearly with the log of model parameters (gray line in Figure 7). We also report approximate training costs in Table 1. The 317M parameter model can be trained with limited computational resources. To better understand why multitask model learning is successful, we explore the task embeddings learned by TD-MPC2 (Figure 7, right). Intriguingly, tasks that are semantically similar (e.g., Door Open and Door Close) are close in the learned task embedding space. However, embedding similarity appears to align more closely with task *dynamics* (embodiment, objects) than objective (walk, run). This makes intuitive sense, as dynamics are tightly coupled with control.

Few-shot learning. While our work mainly focuses on the *scaling* and *robustness* of world models, we also explore the efficacy of finetuning pretrained world models for few-shot learning of unseen tasks. Specifically, we pretrain a 19M parameter TD-MPC2 agent on 70 tasks from DMControl and

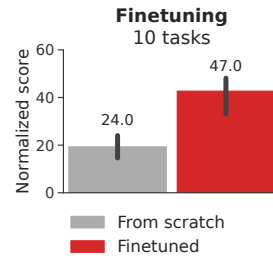


Figure 8. **Finetuning.** Score of a 19M parameter TD-MPC2 agent trained on 70 tasks and finetuned online to each of 10 held-out tasks for 20k environment steps. 3 seeds.

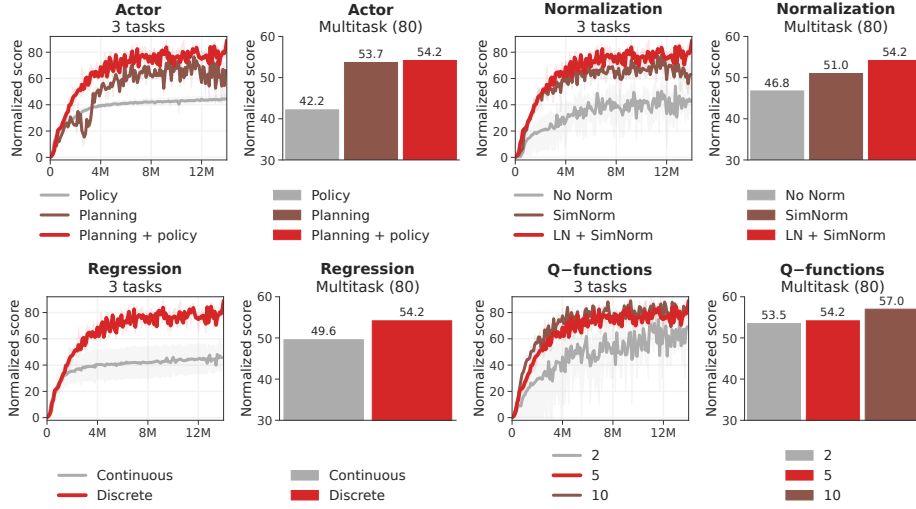


Figure 9. Ablations. (Curves) Normalized score as a function of environment steps, averaged across three of the most difficult tasks: *Dog Run*, *Humanoid Walk* (DMControl), and *Pick YCB* (ManiSkill2). Mean and 95% CIs over 3 random seeds. (Bars) Normalized score of 19M parameter multitask (80 tasks) TD-MPC2 agents. Our ablations highlight the relative importance of each design choice; red is the default formulation of TD-MPC2. See Appendix D for more ablations.

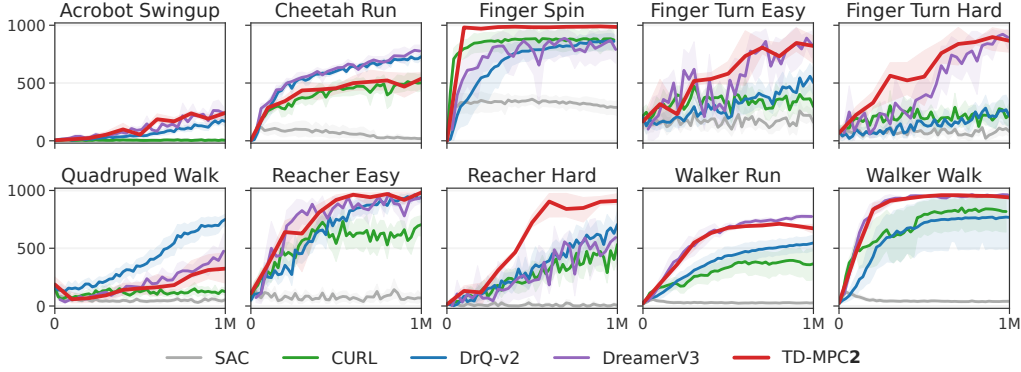


Figure 10. Visual RL. Episode return as a function of environment steps on 10 image-based DMControl tasks. Mean and 95% CIs over 3 random seeds. TD-MPC2 is comparable to state-of-the-art.

Meta-World, and naïvely finetune the full model to each of 10 held-out tasks (5 from each domain) via online RL with an initially empty replay buffer and no changes to hyperparameters. Aggregate results are shown in Figure 8. We find that TD-MPC2 improves $2\times$ over learning from scratch on new tasks in the low-data regime (20k environment steps¹). Although finetuning world models to new tasks is very much an open research problem, our exploratory results are promising. See Appendix E for experiment details and individual task curves.

Ablations. We ablate most of our design choices for TD-MPC2, including choice of actor, various normalization techniques, regression objective, and number of Q -functions. Our main ablations, shown in Figure 9, are conducted on three of the most difficult online RL tasks, as well as large-scale multitask training (80 tasks). We observe that all of our proposed improvements contribute meaningfully to the robustness and strong performance of TD-MPC2 in both single-task RL and multi-task RL. Interestingly, we find that the relative importance of each design choice is consistent across both settings. Lastly, we also ablate normalization of the learned task embeddings, shown in Appendix F. The results indicate that maintaining a normalized task embedding space (ℓ_2 -norm of 1) is moderately important for stable multitask training, and results in more meaningful task relations.

Visual RL. We mainly consider high-dimensional continuous control tasks with proprioceptive state observations in this work. However, TD-MPC2 can be readily applied to tasks with other input

¹20k environment steps corresponds to 20 episodes in DMControl and 100 episodes in Meta-World.

modalities as well. To demonstrate this, we replace the encoder of TD-MPC2 with a shallow convolutional encoder, and benchmark it against current state-of-the-art methods for visual RL on 10 DMControl tasks of varying difficulty. Results are shown in Figure 10. TD-MPC2 performs comparably to the two best baselines, DrQ-v2 and DreamerV3, without *any* changes to hyperparameters.

5 LESSONS, OPPORTUNITIES, AND RISKS

Lessons. Historically, RL algorithms have been notoriously sensitive to architecture, hyperparameters, characteristics of the task, and even random seed (Henderson et al., 2018), with no principled method for tuning the algorithms. As a result, successful application of deep RL often requires large teams of experts with significant computational resources (Berner et al., 2019; Schrittwieser et al., 2020; Ouyang et al., 2022). TD-MPC2 – along with several other contemporary RL methods (Yarats et al., 2021; Ye et al., 2021; Hafner et al., 2023) – seek to democratize use of RL (*i.e.*, lowering the barrier of entry for smaller teams of academics, practitioners, and individuals with fewer resources) by improving robustness of existing open-source algorithms. **We firmly believe that improving algorithmic robustness will continue to have profound impact on the field.** A key lesson from the development of TD-MPC2 is that the community has yet to discover an algorithm that truly masters *everything* out-of-the-box. While *e.g.* DreamerV3 (Hafner et al., 2023) has delivered strong results on challenging tasks with discrete action spaces (such as Atari games and Minecraft), we find that TD-MPC2 produces significantly better results on difficult continuous control tasks. At the same time, extending TD-MPC2 to discrete action spaces remains an open problem.

Opportunities. Our scaling results demonstrate a path for model-based RL in which massively multitask world models are leveraged as *generalist* world models. While multi-task world models remain relatively underexplored in literature, prior work suggests that the implicit world model of TD-MPC2 may be better suited than reconstruction-based approaches for tasks with large visual variation (Zhu et al., 2023). We envision a future in which implicit world models are used zero-shot to perform diverse tasks on *seen* embodiments (Xu et al., 2023; Yang et al., 2023), finetuned to quickly perform tasks on *new* embodiments, and combined with existing vision-language models to perform higher-level cognitive tasks in conjunction with low-level physical interaction. Our results are promising, but such level of generalization will likely require several orders of magnitude more tasks than currently available. Lastly, we want to remark that, while TD-MPC2 relies on rewards for task learning, it is useful to adopt a generalized notion of reward as simply a metric for task completion. Such metrics already exist in the wild, *e.g.*, success labels, human preferences or interventions (Ouyang et al., 2022), or the embedding distance between a current observation and a goal (Eysenbach et al., 2022; Ma et al., 2022) within a pre-existing learned representation. However, leveraging such rewards for large-scale pretraining is an open problem. To accelerate research in this area, we are releasing 300+ TD-MPC2 models, including 12 multitask models, as well as datasets and code, and we are beyond excited to see what the community will do with these resources.

Risks. While we are excited by the potential of generalist world models, several challenges remain: (i) misspecification of task rewards can lead to unintended outcomes (Clark & Amodei, 2016) that may be difficult to anticipate, (ii) handing over unconstrained autonomy of physical robots to a learned model can result in catastrophic failures if no additional safety checks are in place (Lancaster et al., 2023), and (iii) data for certain applications may be prohibitively expensive for small teams to obtain at the scale required for generalist behavior to emerge, leading to a concentration of power. Mitigating each of these challenges will require new research innovations, and we invite the community to join us in these efforts.

6 RELATED WORK

Multiple prior works have sought to build RL algorithms that are robust to hyperparameters, architecture, as well as variation in tasks and data. For example, (1) Double Q -learning (Hasselt et al., 2016), RED- Q (Chen et al., 2021), SVEA (Hansen et al., 2021), and SR-SPR (D’Oro et al., 2023) each improve the stability of Q -learning algorithms by adjusting the bias-variance trade-off in TD-target estimation, (2) C51 (Bellemare et al., 2017) and DreamerV3 (Hafner et al., 2023) improve robustness to the magnitude of rewards by performing discrete regression in a transformed space, and (3) model-free algorithms DrQ (Kostrikov et al., 2020) and DrQ-v2 (Yarats et al., 2021) improve training stability and exploration, respectively, through use of data augmentation and several other

minor but important implementation details. However, all of the aforementioned works strictly focus on improving data-efficiency and robustness in single-task online RL.

Existing literature that studies scaling of neural architectures for decision-making typically assume access to large datasets of near-expert demonstrations for behavior cloning (Reed et al., 2022; Lee et al., 2022; Kumar et al., 2022; Schubert et al., 2023; Driess et al., 2023; Brohan et al., 2023). Gato (Reed et al., 2022) learns to perform tasks across multiple domains by training a large Transformer-based sequence model (Vaswani et al., 2017) on an enormous dataset of expert demonstrations, and RT-1 (Brohan et al., 2023) similarly learns a sequence model for object manipulation on a single (real) robot embodiment by training on a large dataset collected by human teleoperation. While the empirical results of this line of work are impressive, the assumption of large demonstration datasets is impractical. Additionally, current sequence models rely on discretization of the action space (tokenization), which makes scaling to high-dimensional continuous control tasks difficult.

Most recently, researchers have explored scaling of RL algorithms as a solution to the aforementioned challenges (Baker et al., 2022; Jia et al., 2022; Xu et al., 2023; Kumar et al., 2023; Hafner et al., 2023). For example, VPT (Baker et al., 2022) learns to play Minecraft by first pretraining a behavior cloning policy on a large human play dataset, and then finetuning the policy with RL. GSL (Jia et al., 2022) requires no pre-existing data. Instead, GSL iteratively trains a population of “specialist” agents on individual task variations, distills them into a “generalist” policy via behavior cloning, and then uses the generalist as initialization for the next population of specialists. However, this work considers strictly single-task RL and assumes full control over the initial state in each episode. Lastly, DreamerV3 (Hafner et al., 2023) successfully scales its world model in terms of parameters and shows that larger models generally are more data-efficient in an online RL setting, but does not consider multitask RL.

ACKNOWLEDGEMENTS

This project was supported, in part, by grants from NSF CAREER Award (2240160), NSF TILOS AI Institute (2112665), NSF CCF-2112665 (TILOS), NSF 1730158 CI-New: Cognitive Hardware and Software Ecosystem Community Infrastructure (CHASE-CI), NSF ACI-1541349 CC*DNI Pacific Research Platform, and gifts from Qualcomm.

REFERENCES

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *Advances in Neural Information Processing Systems*, 2016.
- Bowen Baker, Ilge Akkaya, Peter Zhokov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *Advances in Neural Information Processing Systems*, 35:24639–24654, 2022.
- Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*, pp. 449–458. PMLR, 2017.
- Richard Bellman. A markovian decision process. *Indiana Univ. Math. J.*, 6:679–684, 1957. ISSN 0022-2518.
- Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

A SUMMARY OF IMPROVEMENTS

We summarize the main differences between TD-MPC and TD-MPC2 as follows:

- **Architectural design.** All components of TD-MPC2 are MLPs with LayerNorm (Ba et al., 2016) and Mish (Misra, 2019) activations after each layer. We apply SimNorm normalization to the latent state $\mathbf{z}$ which biases the representation towards sparsity and maintaining a small ℓ_2 -norm. We train an ensemble of Q -functions (5 by default) and additionally apply 1% Dropout (Srivastava et al., 2014) after the first linear layer in each Q -function. TD-targets are computed as the minimum of two randomly subsampled Q -functions (Chen et al., 2021). In contrast, TD-MPC is implemented as MLPs without LayerNorm, and instead uses ELU (Clevert et al., 2015) activations. TD-MPC does not constrain the latent state at all, which in some instances leads to exploding gradients (see Appendix G for experimental results). Lastly, TD-MPC learns only 2 Q -functions and does not use Dropout. The architectural differences in TD-MPC2 result in a 4M net increase in learnable parameters (5M total) for our default single-task model size compared to the 1M parameters of TD-MPC. However, as shown in Figure 7, naïvely increasing the model size of TD-MPC does not lead to consistently better performance, whereas it does for TD-MPC2.
- **Policy prior.** The policy prior of TD-MPC2 is trained with maximum entropy RL (Ziebart et al., 2008; Haarnoja et al., 2018), whereas the policy prior of TD-MPC is trained as a deterministic policy with Gaussian noise applied to actions. We find that a carefully tuned Gaussian noise schedule is comparable to a policy prior trained with maximum entropy. However, maximum entropy RL can more easily be applied with task-agnostic hyperparameters. We only compute policy entropy over valid action dimensions in multi-task learning with multiple action spaces.
- **Planning.** The planning procedure of TD-MPC2 closely follows that of TD-MPC. However, we simplify planning marginally by not leveraging momentum between iteration, as we find it to produce comparable results. We also improve the throughput of planning by approx. $2\times$ through a series of code-level optimizations.
- **Model objective.** We revisit the training objective of TD-MPC and improve its robustness to variation in tasks, such as the magnitude of rewards. TD-MPC2 uses discrete regression (soft cross-entropy) of rewards and values in a log-transformed space, which makes the magnitude of the two loss terms independent of the magnitude of the task rewards. TD-MPC uses continuous regression which leads to training instabilities in tasks where rewards are large. While this issue can be alleviated by, *e.g.*, normalizing task rewards based on moving statistics, in the single-task case, it is difficult to design robust reward normalization schemes for multi-task learning. TD-MPC2 retains the continuous regression term for joint-embedding prediction as the latent representation is already normalized by SimNorm, and discrete regression is computationally expensive for high-dimensional spaces (requires N bins for each dimension of $\mathbf{z}$).
- **Multi-task model.** TD-MPC2 introduces a framework for learning multi-task world models across multiple domains, embodiments, and action spaces. We introduce a normalized learnable task embedding space which all components of TD-MPC are conditioned on, and we accommodate multiple observation and action spaces by applying zero-padding and action masking during both training and inference. We train multi-task models on a large number of tasks, and finetune the model to held-out tasks (across embodiments) using online RL. TD-MPC only considers multi-task learning on a small number of tasks with shared observation and action space, and does not consider finetuning of the learned multi-task model.
- **Simplified algorithm and implementation.** TD-MPC2 removes momentum in MPPI (Williams et al., 2015), and replaces prioritized experience replay sampling from the replay buffer with uniform sampling, both of which simplify the implementation with no significant change in experimental results. Finally, we also use a faster replay buffer implementation that uses multiple workers for sampling, and we increase training and planning throughput through code-level optimizations such as Q -function ensemble vectorization, which makes the wall-time of TD-MPC2 comparable to that of TD-MPC despite a larger architecture (5M vs. 1M).

B TASK VISUALIZATIONS



Figure 11. Task visualizations. Visualization of a random initial state for each of the 104 tasks that we consider. Tasks vary greatly in objective, embodiment, and action space. Visit <https://tdmpc2.com> for videos of TD-MPC2 performing each task. See Appendix C for task details.

C TASK DOMAINS

We consider a total of 104 continuous control tasks from 4 task domains: DMControl (Tassa et al., 2018), Meta-World (Yu et al., 2019), ManiSkill2 (Gu et al., 2023), and MyoSuite (Caggiano et al., 2022). This section provides an exhaustive list of all tasks considered, as well as their observation and action dimensions. Environment details are listed at the end of the section. We provide (static) task visualizations in Appendix B and videos of TD-MPC2 agents performing each task at <https://www.tdmcp2.com>.

Table 2. DMControl. We consider a total of 39 continuous control tasks in the DMControl domain, including 19 original DMControl tasks and 11 new (custom) tasks created specifically for TD-MPC2 benchmarking and multitask training. We list all considered DMControl tasks below. The *Locomotion* task set shown in Figure 1 corresponds to the *Humanoid* and *Dog* embodiments of DMControl, with performance reported at 14M environment steps.

Task	Observation dim	Action dim	Sparse?	New?
Acrobot Swingup	6	1	N	N
Cartpole Balance	5	1	N	N
Cartpole Balance Sparse	5	1	Y	N
Cartpole Swingup	5	1	N	N
Cartpole Swingup Sparse	5	1	Y	N
Cheetah Jump	17	6	N	Y
Cheetah Run	17	6	N	N
Cheetah Run Back	17	6	N	Y
Cheetah Run Backwards	17	6	N	Y
Cheetah Run Front	17	6	N	Y
Cup Catch	8	2	Y	N
Cup Spin	8	2	N	Y
Dog Run	223	38	N	N
Dog Trot	223	38	N	N
Dog Stand	223	38	N	N
Dog Walk	223	38	N	N
Finger Spin	9	2	Y	N
Finger Turn Easy	12	2	Y	N
Finger Turn Hard	12	2	Y	N
Fish Swim	24	5	N	N
Hopper Hop	15	4	N	N
Hopper Hop Backwards	15	4	N	Y
Hopper Stand	15	4	N	N
Humanoid Run	67	24	N	N
Humanoid Stand	67	24	N	N
Humanoid Walk	67	24	N	N
Pendulum Spin	3	1	N	Y
Pendulum Swingup	3	1	N	N
Quadruped Run	78	12	N	N
Quadruped Walk	78	12	N	N
Reacher Easy	6	2	Y	N
Reacher Hard	6	2	Y	N
Reacher Three Easy	8	3	Y	Y
Reacher Three Hard	8	3	Y	Y
Walker Run	24	6	N	N
Walker Run Backwards	24	6	N	Y
Walker Stand	24	6	N	N
Walker Walk	24	6	N	N
Walker Walk Backwards	24	6	N	Y

Table 3. Meta-World. We consider a total of 50 continuous control tasks from the Meta-World domain. The Meta-World benchmark is designed for multitask and meta-learning research and all tasks thus share embodiment, observation space, and action space.

Task	Observation dim	Action dim
Assembly	39	4
Basketball	39	4
Bin Picking	39	4
...	...	...
Window Open	39	4

Table 4. ManiSkill2. We consider a total of 5 continuous control tasks from the ManiSkill2 domain. The ManiSkill2 benchmark is designed for large-scale robot learning and contains a high degree of randomization and task variations. The *Pick YCB* task shown in Figure 1 corresponds to the ManiSkill2 task of the same name, with performance reported at 14M environment steps.

Task	Observation dim	Action dim
Lift Cube	42	4
Pick Cube	51	4
Pick YCB	51	7
Stack Cube	55	4
Turn Faucet	40	7

Table 5. MyoSuite. We consider a total of 10 continuous control tasks from the MyoSuite domain. The MyoSuite benchmark is designed for high-dimensional physiologically accurate musculoskeletal motor control and involves particularly complex object manipulation with a dexterous hand. The MyoSuite domain consists of tasks with and without goal randomization. We consider both settings, and refer to them as *Easy* (fixed goal) and *Hard* (random goal), respectively.

Task	Observation dim	Action dim
Reach Easy	115	39
Reach Hard	115	39
Pose Easy	108	39
Pose Hard	108	39
Pen Twirl Easy	83	39
Pen Twirl Hard	83	39
Object Hold Easy	91	39
Object Hold Hard	91	39
Key Turn Easy	93	39
Key Turn Hard	93	39

Environment details. We benchmark algorithms on DMControl, Meta-World, ManiSkill2, and MyoSuite without modification. All four domains are infinite-horizon continuous control environments for which we use a fixed episode length and no termination conditions. We list episode lengths, action repeats, total number of environment steps, and the performance metric used for each domain in Table 6. In all experiments, we only consider an episode successful if the final step of an episode is successful. This is a stricter definition of success than used in some of the related literature, which *e.g.* may consider an episode successful if success is achieved at *any* step within a given episode. In tasks that require manipulation of objects, such as picking up an object, our definition of success ensures that an episode in which an object is picked up but then dropped again is not considered successful.

Table 6. Environment details. We list the episode length and action repeat used for each task domain, as well as the total number of environment steps and performance metrics that we use for benchmarking methods. All methods use the same values for all tasks.

	DMControl	Meta-World	ManiSkill2	MyoSuite
Episode length	1,000	200	200	100
Action repeat	2	2	2	1
Effective length	500	100	100	100
Total env. steps	4M - 14M	2M	4M - 14M	2M
Performance metric	Reward	Success	Success	Success

— Appendices continue on next page —

D SINGLE-TASK EXPERIMENTAL RESULTS

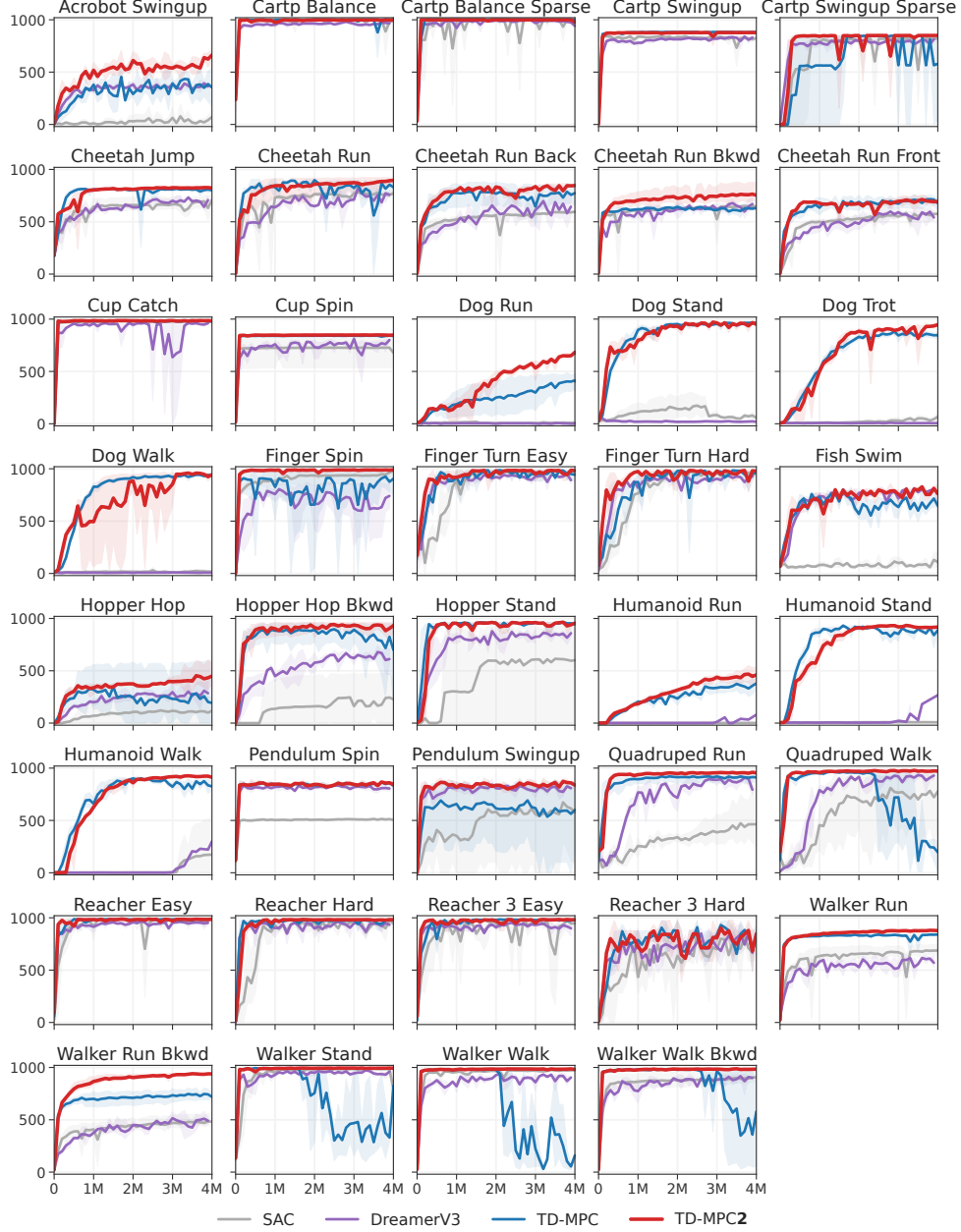


Figure 12. Single-task DMControl results. Episode return as a function of environment steps. The first 4M environment steps are shown for each task, although the Humanoid and Dog tasks are run for 14M environment steps; we provide those curves in Figure 15 as part of the “Locomotion” benchmark. Note that TD-MPC diverges on tasks like *Walker Stand* and *Walker Walk* whereas TD-MPC2 remains stable. We visualize gradients on these tasks in Appendix G. Mean and 95% CIs over 3 seeds.

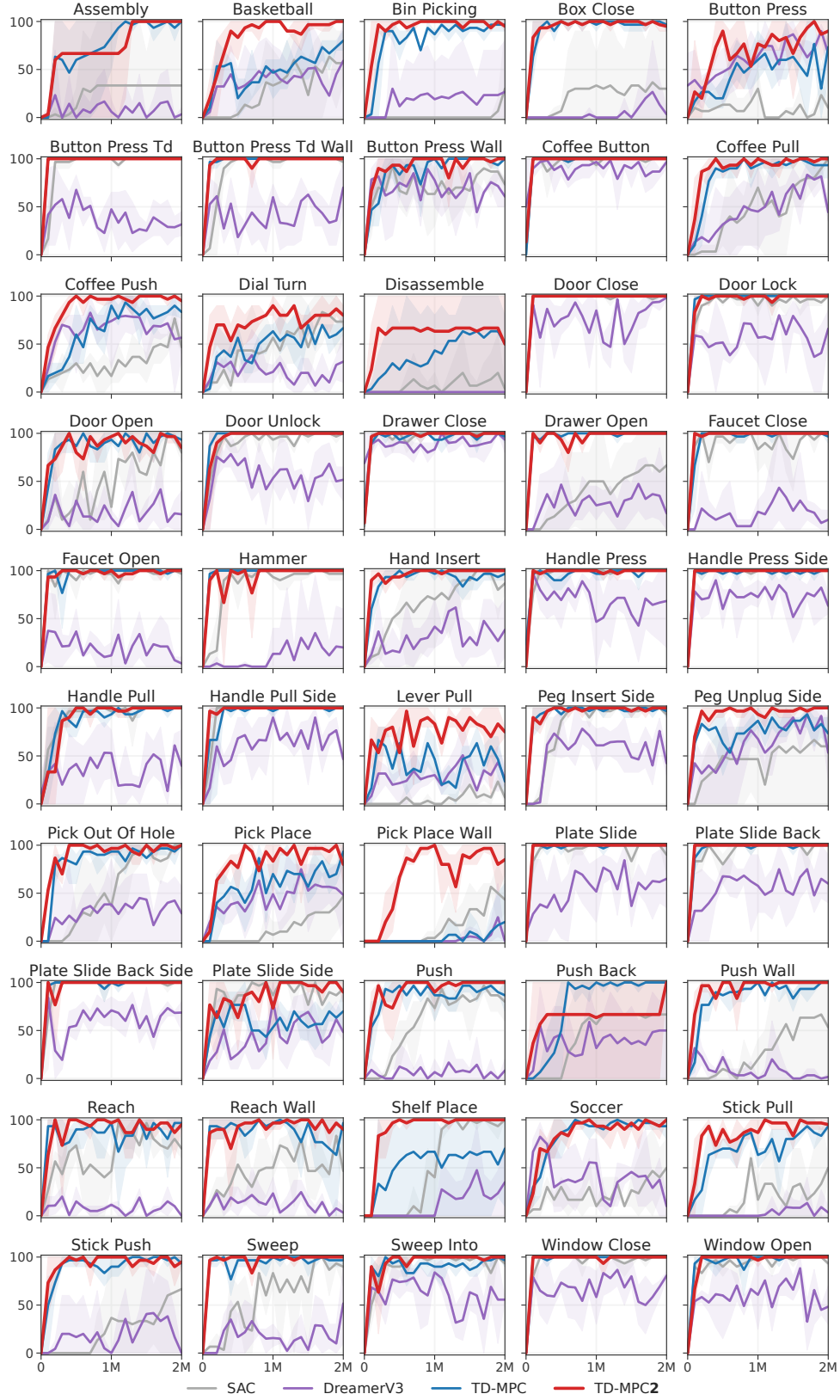


Figure 13. **Single-task Meta-World results.** Success rate (%) as a function of environment steps. TD-MPC2 performance is comparable to existing methods on easy tasks, while outperforming other methods on hard tasks such as *Pick Place Wall* and *Shelf Place*. DreamerV3 often fails to converge.

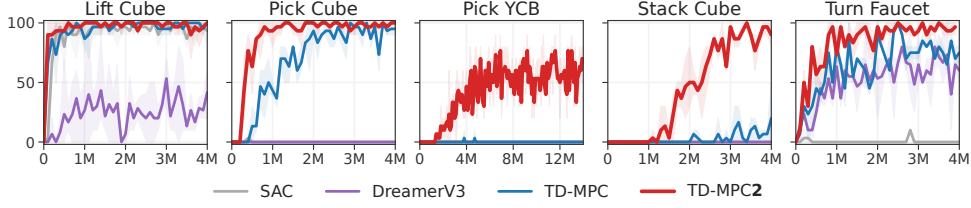


Figure 14. Single-task ManiSkill2 results. Success rate (%) as a function of environment steps on 5 object manipulation tasks from ManiSkill2. *Pick YCB* is the hardest task and considers manipulation of all 74 objects from the YCB (Calli et al., 2015) dataset. We report results for this task at 14M environment steps, and 4M environment steps for other tasks. TD-MPC2 achieves a $> 60\%$ success rate on the Pick YCB task, whereas other methods fail to learn within the given budget. Mean and 95% CIs over 3 seeds.

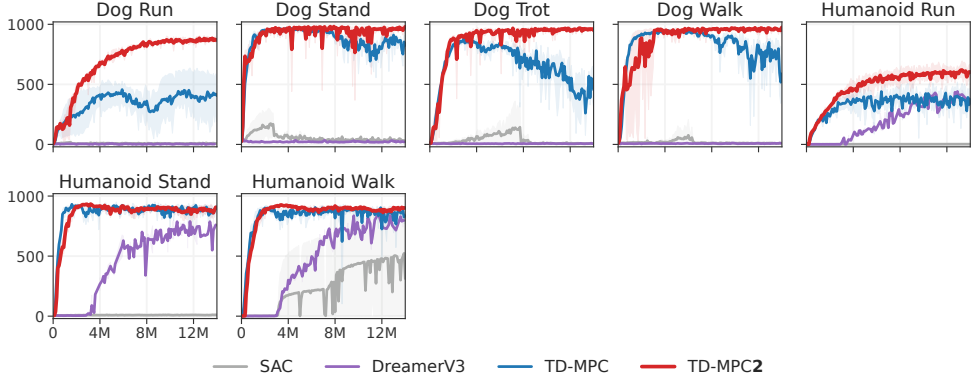


Figure 15. Single-task high-dimensional locomotion results. Episode return as a function of environment steps on all 7 "Locomotion" benchmark tasks. This domain includes high-dimensional Humanoid ($\mathcal{A} \in \mathbb{R}^{21}$) and Dog ($\mathcal{A} \in \mathbb{R}^{38}$) embodiments. Mean and 95% CIs over 3 seeds.

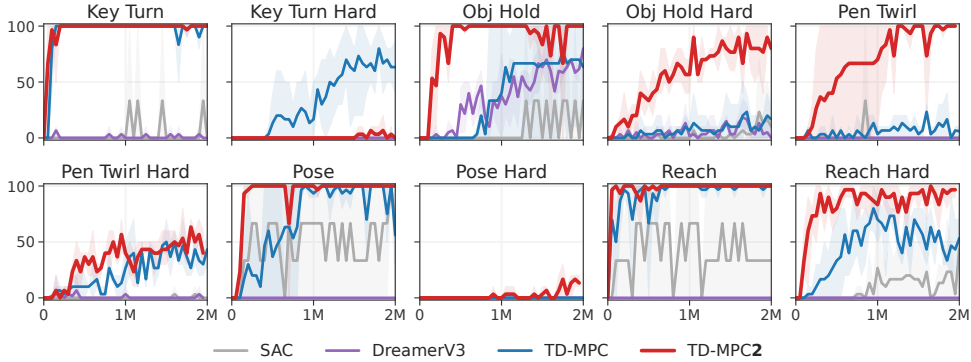


Figure 16. Single-task MyoSuite results. Success rate (%) as a function of environment steps. This task domain includes high-dimensional contact-rich musculoskeletal motor control ($\mathcal{A} \in \mathbb{R}^{39}$) with a physiologically accurate robot hand. Goals are randomized in tasks designated as "Hard". TD-MPC2 achieves comparable or better performance than existing methods on all tasks from this benchmark, except for *Key Turn Hard* in which TD-MPC succeeds early in training.

E FEW-SHOT EXPERIMENTAL RESULTS

We finetune a 19M parameter TD-MPC2 agent trained on 70 tasks to each of 10 held-out tasks. Individual task curves are shown in Figure 17. We compare data-efficiency of the finetuned model to a baseline agent of similar model capacity trained from scratch. However, we find that performance of our 19M parameter baselines trained from scratch are comparable to our 5M parameter agents also trained from scratch. Our few-shot finetuning results suggest that the efficacy of finetuning is somewhat task-dependent. However, more research is needed to conclude whether this is due to task similarity (or rather lack thereof) or due to subpar task performance of the pretrained agent on the source task. We conjecture that both likely influence results.

When finetuning to an unseen task, we initialize the learnable task embedding for the new task as the embedding of a semantically similar task from the pretraining dataset. We list the source task embedding used as initialization for each experiment in Table 7. We did not experiment with other initialization schemes, nor other task pairings.

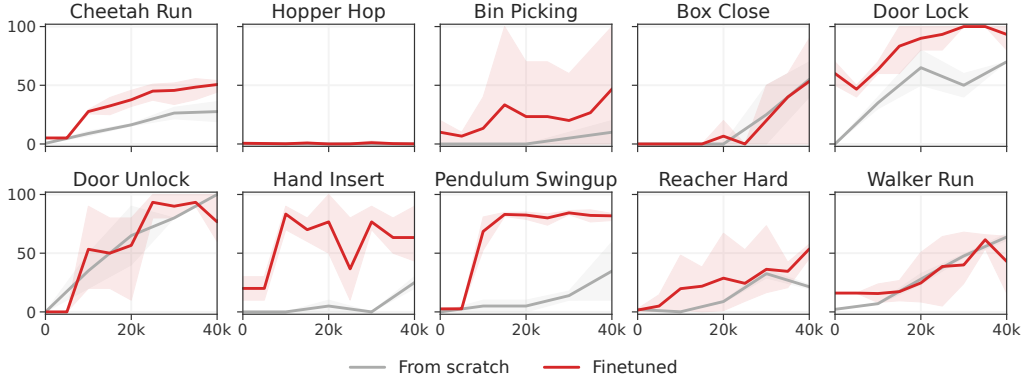


Figure 17. Few-shot learning. Normalized episode return (DMControl) and success rate (Meta-World) as a function of environment steps while finetuning a 19M parameter TD-MPC2 agent trained on 70 tasks to each of 10 held-out tasks. 40k steps corresponds to 40 episodes in DM-Control and 200 in Meta-World. Mean and 95% CIs over 3 seeds.

Table 7. Initialization of task embeddings for few-shot learning. We list the task embeddings used as initialization when finetuning our 19M parameter TD-MPC2 agent to held-out tasks. We did not experiment with other initialization schemes, nor other task pairings.

Target task	Source task
Walker Run	Walker Walk
Cheetah Run	Cheetah Run Backwards
Hopper Hop	Hopper Stand
Pendulum Swingup	Pendulum Spin
Reacher Hard	Reacher Easy
Bin Picking	Pick Place
Box Close	Assembly
Door Lock	Door Open
Door Unlock	Door Open
Hand Insert	Sweep Into

F ADDITIONAL ABLATIONS

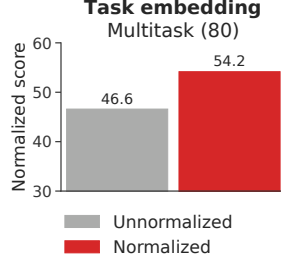


Figure 18. **Normalized task embeddings.** Normalized score of 19M parameter multitask (80 tasks) TD-MPC2 agents, with and without normalized task embeddings e as described in Section 3.1. We find that normalizing e to have a maximum ℓ_2 -norm of 1 improves multitask performance.

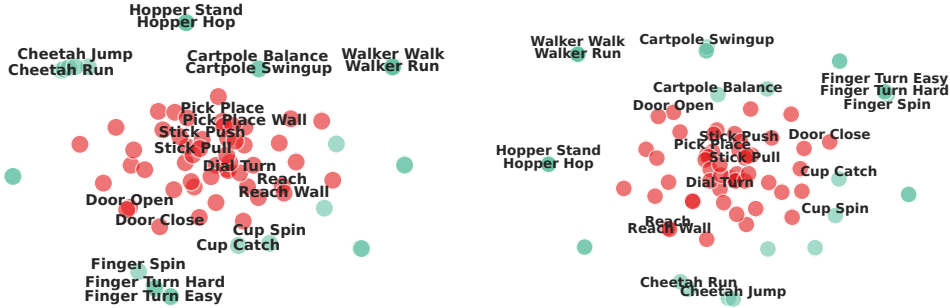


Figure 19. **T-SNE of task embeddings with and without normalization.** T-SNE (van der Maaten & Hinton, 2008) visualizations of task embeddings learned by TD-MPC2 agent trained on 80 tasks from DMControl and Meta-World. (Left) with normalization. (Right) without normalization. A subset of labels are shown for clarity. We observe that task embeddings are more semantically meaningful when normalized during training, e.g., “Door Open” and “Door Close” are close in embedding space on the left, but far apart on the right.

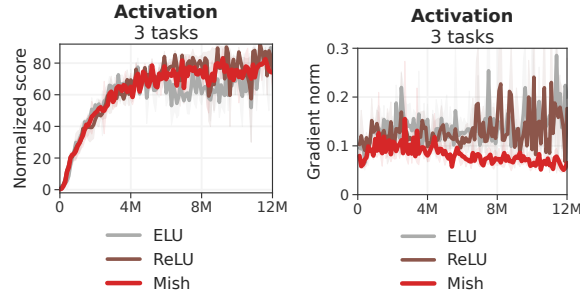


Figure 20. **Activation function.** Normalized score as a function of environment steps, averaged across three of the most difficult tasks: *Dog Run*, *Humanoid Walk* (DMControl), and *Pick YCB* (ManiSkill2). Mean and 95% CIs over 3 random seeds. We find that TD-MPC2 achieves comparable asymptotic performance and data-efficiency with either activation function, but that Mish (Misra, 2019) leads to smoother gradients overall.

G GRADIENT NORM AND TRAINING STABILITY

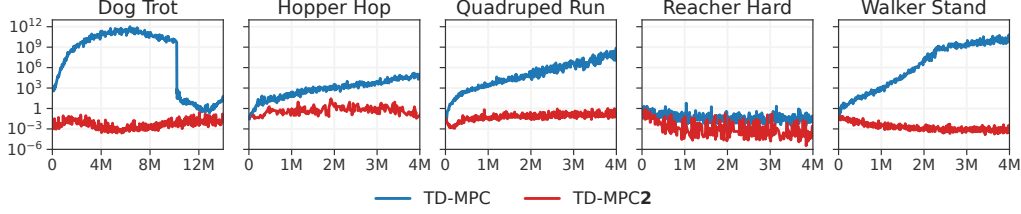


Figure 21. Gradient norm during training. We compare the gradient norm (log-scale) of TD-MPC and TD-MPC2 as a function of environment steps on five tasks from DMControl. TD-MPC is prone to exploding gradients, which can cause learning to diverge on some tasks (e.g., Walker Stand in Figure 12). In comparison, the gradients of TD-MPC2 remain stable throughout training. We only display 1 seed per task for visual clarity.

H IMPLEMENTATION DETAILS

Architectural details. All components of TD-MPC2 are implemented as MLPs. The encoder h contains a variable number of layers (2 – 5) depending on the architecture size; all other components are 3-layer MLPs. Intermediate layers consist of a linear layer followed by LayerNorm and a Mish activation function. The latent representation is normalized as a simplicial embedding. Q -functions additionally use Dropout. We summarize the TD-MPC2 architecture for the 5M parameter base (default for online RL) model size using PyTorch-like notation:

```
Encoder parameters: 167,936
Dynamics parameters: 843,264
Reward parameters: 631,397
Policy parameters: 582,668
Q parameters: 3,156,985
Task parameters: 7,680
Total parameters: 5,389,930

Architecture: TD-MPC2 base 5M(
  (task_embedding): Embedding(T, 96, max_norm=1)
  (encoder): ModuleDict(
    (state): Sequential(
      (0): NormedLinear(in_features=S+T, out_features=256, act=Mish)
      (1): NormedLinear(in_features=256, out_features=512, act=SimNorm)
    )
  )
  (dynamics): Sequential(
    (0): NormedLinear(in_features=512+T+A, out_features=512, act=Mish)
    (1): NormedLinear(in_features=512, out_features=512, act=Mish)
    (2): NormedLinear(in_features=512, out_features=512, act=SimNorm)
  )
  (reward): Sequential(
    (0): NormedLinear(in_features=512+T+A, out_features=512, act=Mish)
    (1): NormedLinear(in_features=512, out_features=512, act=Mish)
    (2): Linear(in_features=512, out_features=101,)
  )
  (pi): Sequential(
    (0): NormedLinear(in_features=512+T, out_features=512, act=Mish)
    (1): NormedLinear(in_features=512, out_features=512, act=Mish)
    (2): Linear(in_features=512, out_features=2A, bias=True)
  )
  (Qs): Vectorized ModuleList(
    (0-4): 5 x Sequential(
      (0): NormedLinear(in_features=512+T+A, out_features=512, dropout=0.01, act=Mish)
      (1): NormedLinear(in_features=512, out_features=512, act=Mish)
      (2): Linear(in_features=512, out_features=101, bias=True)
    )
  )
)
```

where S is the input dimensionality, T is the number of tasks, and A is the action space. We exclude the task embedding T from single-task experiments. The exact parameter counts listed above are for $S=39$, $T=80$, and $A=6$.

Hyperparameters. We use the same hyperparameters across all tasks. Our hyperparameters are listed in Table 8. We use the same hyperparameters for TD-MPC and SAC as in Hansen et al. (2022). DreamerV3 (Hafner et al., 2023) uses a fixed set of hyperparameters.

Table 8. **TD-MPC2 hyperparameters.** We use the same hyperparameters across all tasks. Certain hyperparameters are set automatically using heuristics.

Hyperparameter	Value
Planning	
Horizon (H)	3
Iterations	6 (+2 if $\ \mathcal{A}\ \geq 20$)
Population size	512
Policy prior samples	24
Number of elites	64
Minimum std.	0.05
Maximum std.	2
Temperature	0.5
Momentum	No
Policy prior	
Log std. min.	-10
Log std. max.	2
Replay buffer	
Capacity	1,000,000
Sampling	Uniform
Architecture (5M)	
Encoder dim	256
MLP dim	512
Latent state dim	512
Task embedding dim	96
Task embedding norm	1
Activation	LayerNorm + Mish
Q -function dropout rate	1%
Number of Q -functions	5
Number of reward/value bins	101
SimNorm dim (V)	8
SimNorm temperature (τ)	1
Optimization	
Update-to-data ratio	1
Batch size	256
Joint-embedding coef.	20
Reward prediction coef.	0.1
Value prediction coef.	0.1
Temporal coef. (λ)	0.5
Q -fn. momentum coef.	0.99
Policy prior entropy coef.	1×10^{-4}
Policy prior loss norm.	Moving (5%, 95%) percentiles
Optimizer	Adam
Learning rate	3×10^{-4}
Encoder learning rate	1×10^{-4}
Gradient clip norm	20
Discount factor	Heuristic
Seed steps	Heuristic

We set the discount factor γ for a task using the heuristic

$$\gamma = \text{clip}\left(\frac{\frac{T}{5} - 1}{\frac{T}{5}}, [0.95, 0.995]\right) \quad (7)$$

where T is the expected length of an episode *after* applying action repeat, and clip constrains the discount factor to the interval $[0.95, 0.995]$. Using this heuristic, we obtain $\gamma = 0.99$ for DMControl ($T = 500$), which is the most widely used discount factor for this task domain. Tasks with shorter episodes are assigned a lower discount factor, whereas tasks with longer episodes are assigned a higher discount factor. All of the tasks that we consider are infinite-horizon MDPs with fixed episode lengths. We use individual discount factors (set using the above heuristic) for each task in our multitask experiments. For tasks with variable or unknown episode lengths, we suggest using an empirical mean length, a qualified guess, or simply $\gamma = 0.99$. While this heuristic is introduced in TD-MPC2, we apply the same discount factor for the TD-MPC and SAC baselines to ensure that comparison is fair across all task domains.

We set the seed steps S (number of environment steps before any gradient updates) for a task using the heuristic

$$S = \max(5T, 1000) \quad (8)$$

where T again is the expected episode length of the task after applying action repeat. We did not experiment with other heuristics nor constant values, but conjecture that Equation 8 will ensure that the replay buffer $\mathcal{B}$ has sufficient data for model learning regardless of episode lengths.

Model configurations. Our multitask experiments consider TD-MPC2 agents with model sizes ranging from 1M parameters to 317M parameters. Table 9 lists the exact specifications for each of our model sizes. We scale the model size by varying dimensions of fully-connected layers, the latent state dimension $\mathbf{z}$, the number of encoder layers, and the number of Q -functions. We make no other modifications to the architecture nor hyperparameters across model sizes.

Table 9. Model configurations. We list the specifications for each model configuration (size) of our multitask experiments. *Encoder dim* is the dimensionality of fully connected layers in the encoder h , *MLP dim* is the dimensionality of layers in all other components, *Latent state dim* is the dimensionality of the latent representation $\mathbf{z}$, *# encoder layers* is the number of layers in the encoder h , *# Q -functions* is the number of learned Q -functions, and *Task embedding dim* is the dimensionality of $\mathbf{e}$ from Equation 2. TD-targets are always computed by randomly subsampling two Q -functions, regardless of the number of Q -functions in the ensemble. We did not experiment with other model configurations. *The default (base) configuration used in our single-task RL experiments has 5M parameters.

	1M	5M*	19M	48M	317M
Encoder dim	256	256	1024	1792	4096
MLP dim	384	512	1024	1792	4096
Latent state dim	128	512	768	768	1376
# encoder layers	2	2	3	4	5
# Q -functions	2	5	5	5	8
Task embedding dim	96	96	96	96	96

Simplicial Normalization (SimNorm). SimNorm is a simple method for normalizing the latent representation $\mathbf{z}$ by projecting it into L fixed-dimensional simplices using a softmax operation (Lavoie et al., 2022). A key benefit of embedding $\mathbf{z}$ as simplices (as opposed to *e.g.* a discrete representation or squashing) is that it naturally biases the representation towards sparsity without enforcing hard constraints. Intuitively, SimNorm can be thought of as a “*soft*” variant of the vector-of-categoricals approach to representation learning proposed by Oord et al. (2017) (VQ-VAE). Whereas VQ-VAE represents latent codes using a set of discrete codes (L vector partitions each consisting of a one-hot encoding), SimNorm partitions the latent state into L vector partitions of continuous values that each sum to 1 due to the softmax operator. This relaxation of the latent representation is akin to softmax being a relaxation of the arg max operator. While we do not adjust the temperature $\tau \in [0, \infty)$ of the softmax used in SimNorm in our experiments, it is useful to note that it provides a mechanism

for interpolating between two extremes. For example, $\tau \rightarrow \infty$ would force all probability mass onto single categories, resulting in the discrete codes (one-hot encodings) of VQ-VAE. The alternative of $\tau = 0$ would result in trivial codes (constant vectors; uniform probability mass) and prohibit propagation of information. SimNorm thus biases representations towards sparsity without enforcing discrete codes or other hard constraints. We implement the SimNorm normalization layer (Lavoie et al., 2022) using PyTorch-like notation as follows:

```
def simnorm(self, z, V=8):
    shape = z.shape
    z = z.view(*shape[:-1], -1, V)
    z = softmax(z, dim=-1)
    return z.view(*shape)
```

Here, $\mathbf{z}$ is the latent representation $\mathbf{z}$, and V is the dimensionality of each simplex. The number of simplices L can be inferred from V and the dimensionality of $\mathbf{z}$. We apply a softmax (optionally modulated by a temperature τ) to each of L partitions of $\mathbf{z}$ to form simplices, and then reshape to the original shape of $\mathbf{z}$.

Visual RL. TD-MPC2 can be readily applied to tasks with other input modalities. In our visual RL experiments shown in Figure 10, we replace the default MLP encoder of TD-MPC2 with a shallow (4 layers) convolutional encoder and use an image resolution of 64×64 . We additionally apply random shift augmentation as in previous work (Kostrikov et al., 2020; Yarats et al., 2021; Hansen et al., 2022). We remark that, while our experiments use a relatively small image resolution, we do so to reduce computational cost and to ensure a fair comparison to baselines. However, prior work has demonstrated that TD-MPC supports inputs as large as $9 \times 224 \times 224$ (Lancaster et al., 2023).

TD-MPC baseline implementation. We benchmark against the official implementation of TD-MPC available at <https://github.com/nicklashansen/tdmpc>. The default TD-MPC world model has approx. 1M trainable parameters, and uses per-task hyperparameters. We use the suggested hyperparameters where available (DMControl and Meta-World). For example, TD-MPC requires tuning of the number of planning iterations, latent state dimensionality, batch size, and learning rate in order to solve the challenging Dog and Humanoid tasks. Refer to their paper for a complete list of hyperparameters.

DreamerV3 baseline implementation. We benchmark against the official reimplementation of DreamerV3 available at <https://github.com/danijar/dreamerv3>. We follow the authors’ suggested hyperparameters for proprioceptive control (DMControl) and use the S model size (20M parameters), as well as an update-to-data (UTD) ratio of 512. We use this model size and UTD for all tasks. Refer to their paper for a complete list of hyperparameters.

SAC baseline implementation. We follow the TD-MPC (Hansen et al., 2022) paper in their decision to benchmark against the SAC implementation from https://github.com/denisyarats/pytorch_sac, and we use the hyperparameters suggested by the authors (when available). For example, this includes tuning the latent dimension, learning rate, and batch size for the Dog and Humanoid tasks. Refer to their paper for a complete list of hyperparameters.

I EXTENDING TD-MPC2 TO DISCRETE ACTION SPACES

It is desirable to develop a single algorithm that excels at tasks with continuous and discrete action spaces alike. However, the community has yet to discover such an algorithm. While *e.g.* DreamerV3 (Hafner et al., 2023) has delivered strong results on challenging tasks with discrete action spaces (such as Atari and Minecraft), we find that TD-MPC2 produces significantly better results on difficult continuous control tasks. At the same time, extending TD-MPC2 to discrete action spaces remains an open problem. While we do not consider discrete action spaces in this work, we acknowledge the value of such an extension. At present, the main challenge in applying TD-MPC2 to discrete actions lies in the choice of planning algorithm. TD-MPC2 relies on the MPC framework for planning, which is designed for continuous action spaces. We believe that MPC could be replaced with a planning algorithm designed for discrete action spaces, such as MCTS (Coulom, 2007) as used in MuZero (Schrittwieser et al., 2020). It is also possible that there exists a way to apply MPC to discrete action spaces that is yet to be discovered (to the best of our knowledge), similar to how recent work (Hubert et al., 2021) has discovered ways to apply MCTS to continuous action spaces through sampling.

J TEST-TIME REGULARIZATION FOR OFFLINE RL

Our multi-task experiments revolve around training massively multi-task world models on fixed datasets that consist of a variety of behaviors, which is an offline RL problem. We do not consider any special treatment of the offline RL problem in the main paper, and simply train TD-MPC2 agents without any additional regularization nor hyperparameter-tuning. However, we recognize that models may benefit from such regularization (conservative estimations) due to extrapolation errors when the dataset has limited state-action coverage and/or is highly skewed. Current offline RL algorithms are ill-suited for our problem setting, given that we aim to develop an algorithm that can seamlessly transition from massively multi-task offline pretraining to single-task online fine-tuning, without any changes in hyperparameters. Current offline RL techniques rely on (1) explicit or implicit conservatism in Q -value estimation which requires modifications to the training objective and empirically hampers online RL performance (Nakamoto et al., 2023), and (2) relies on a task-specific coefficient that balances value estimation and the regularizer. Instead, we propose to regularize the *planning procedure* of TD-MPC2, which can be done at test-time without any additional model updates. Concretely, we apply the test-time regularizer proposed by Feng et al. (2023), which penalizes trajectories with large uncertainty (as estimated by the variance in Q -value predictions) during planning. While this approach eliminates the need for training-time regularization, it still requires users to specify a coefficient that weighs estimated value relative to uncertainty for a trajectory, which is infeasible in a multi-task scenario where estimated values may differ drastically between tasks. To circumvent this issue, we propose a simple heuristic for automatically scaling the regularization strength at each timestep based on the (magnitude of) mean value predictions for a given latent state. Specifically, we estimate the uncertainty penalty at latent state $\mathbf{z}_t$ of a sampled trajectory as

$$u_t = c \cdot \text{avg}([\hat{q}_1, \hat{q}_2, \dots, \hat{q}_N]) \cdot \text{std}([\hat{q}_1, \hat{q}_2, \dots, \hat{q}_N]) \quad (9)$$

where $\hat{q}_n$ is a value prediction from Q -function n in an ensemble of N Q -functions, and c is now a *task-agnostic* coefficient that balances return maximization and uncertainty minimization. The planning objective in Equation 6 is then redefined as

$$\mu^*, \sigma^* = \arg \max_{(\mu, \sigma)} \mathbb{E}_{(\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H}) \sim \mathcal{N}(\mu, \sigma^2)} \quad (10)$$

$$\left[\gamma^H Q(\mathbf{z}_{t+H}, \mathbf{a}_{t+H}) - \textcolor{red}{u_{t+H}} \right] \quad (11)$$

$$+ \sum_{h=t}^{H-1} \left(\gamma^h R(\mathbf{z}_h, \mathbf{a}_h) - \textcolor{red}{u_h} \right) \quad (12)$$

using the definition of task-agnostic uncertainty in Equation 9. We conduct an experiment in which we apply our proposed test-time regularization to a 19M parameter TD-MPC2 agent trained on the 80-task dataset, varying the regularization strength c . Results are shown in Table 10. Our results indicate that additional regularization (using our heuristic for automatic tuning) can indeed improve the average model performance for some values of c . Similar to what one would expect in a single-task setting, we find that large values of c (strong regularization) decrease performance, whereas small (but > 0) values of c tend to improve performance compared to TD-MPC2 without regularization. Given that our heuristic with $c = 0.01$ leads to meaningful improvements across 80 tasks, we expect it to work reasonably well for other datasets as well, but leave this for future work.

Table 10. Test-time regularization. Normalized score of a 19M parameter TD-MPC2 agent trained on the 80-task dataset, varying the regularization strength c of our proposed test-time regularizer. We do not apply this regularization in any of our other experiments, and only include these results to inspire future research directions.

	No reg.	$c = 0.001$	$c = 0.01$	$c = 0.1$
Normalized score	56.54	58.14	62.01	44.13

K ADDITIONAL MULTI-TASK RESULTS

To provide further insights into the effect of data size and task diversity on TD-MPC2 performance in a multi-task setting, we provide additional experiments on a 15-task subset of DMControl, selected at random. Results for TD-MPC2 agents trained on 15 tasks, 30 tasks, and 80 tasks are shown in Figure 22. We observe that performance scales with model size across all three task suites, but numbers are higher across the board on the smallest dataset compared to similar capacity models trained on larger datasets. This makes intuitive sense, since model capacity remains the same while there are comparably fewer tasks to learn.

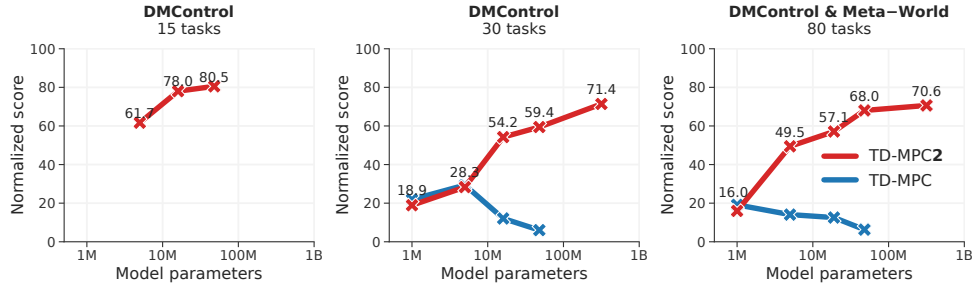


Figure 22. **Additional results on massively multi-task world models.** Normalized score as a function of model size on the 15-task, 30-task, and 80-task datasets. TD-MPC2 capabilities scale with model size.

MENTOR: Mixture-of-Experts Network with Task-Oriented Perturbation for Visual Reinforcement Learning

Suning Huang^{*1,2} Zheyu Zhang^{*1,3} Tianhai Liang¹ Yihan Xu¹ Zhehao Kou¹ Chenhao Lu¹ Guowei Xu¹
Zhengrong Xue^{†1} Huazhe Xu¹

Abstract

Visual deep reinforcement learning (RL) enables robots to acquire skills from visual input for unstructured tasks. However, current algorithms suffer from low sample efficiency, limiting their practical applicability. In this work, we present MENTOR, a method that improves both the *architecture* and *optimization* of RL agents. Specifically, MENTOR replaces the standard multi-layer perceptron (MLP) with a mixture-of-experts (MoE) backbone and introduces a task-oriented perturbation mechanism. MENTOR outperforms state-of-the-art methods across three simulation benchmarks and achieves an average of 83% success rate on three challenging real-world robotic manipulation tasks, significantly surpassing the 32% success rate of the strongest existing model-free visual RL algorithm. These results underscore the importance of sample efficiency in advancing visual RL for real-world robotics. Experimental videos are available at [mentor-vrl](https://mentor-vrl.github.io).



Figure 1. **MENTOR validation on real-world tasks.** We design three challenging robotic learning tasks where agents are **trained from scratch in the real world**. MENTOR demonstrates the most efficient and robust policies compared to baseline methods.

^{*}Equal contribution [†] Student advisor ¹Tsinghua University ²Stanford University ³University of Illinois Urbana-Champaign. Correspondence to: Suning Huang <suning@stanford.edu>, Zheyu Zhang <zheyuz5@illinois.edu>.

1. Introduction

Visual deep reinforcement learning (RL) focuses on agents that perceive their environment through high-dimensional image data, closely aligning with robot control scenarios where vision is the primary modality. Despite substantial progress in this field (Kostrikov et al., 2020; Yarats et al., 2021; Schwarzer et al., 2020; Stooke et al., 2021; Laskin et al., 2020b), these methods still suffer from low sample efficiency. As a result, most visual RL pipelines have to be first trained in the simulator and then deployed to the real world, inevitably leading to the problem of sim-to-real gap (Zhao et al., 2020; Salvato et al., 2021).

To bypass this difficulty, one approach is to train visual RL agents from scratch on physical robots, which is known as real-world RL (Dulac-Arnold et al., 2019; Luo et al., 2024; Zhu et al., 2020). Given the numerous challenges of real-world RL, we argue that the fundamental solution lies not in task-specific tweaks, but in developing substantially more sample-efficient RL algorithms. In this paper, we introduce **MENTOR: Mixture-of-Experts Network with Task-Oriented perturbation for visual Reinforcement learning**, which significantly boosts the sample efficiency of visual RL through improvements in both agent network *architecture* and *optimization*.

In terms of architecture, visual RL agents typically use convolutional neural networks (CNNs) for feature extraction from high-dimensional images, followed by multi-layer perceptrons (MLPs) for action output (Yarats et al., 2021; Zheng et al., 2023; Cetin et al., 2022; Xu et al., 2023). However, the learning efficiency of standard MLPs is hindered by intrinsic *gradient conflicts* in challenging robotic tasks (Yu et al., 2020a; Liu et al., 2023; Zhou et al., 2022; Liu et al., 2021), where the gradient directions for optimizing neural parameters across different stages of the task trajectory or between tasks may conflict. In this work, we propose to alleviate gradient conflicts by integrating mixture-of-experts (MoE) architectures (Jacobs et al., 1991; Shazeer et al., 2017; Masoudnia & Ebrahimpour, 2014) as the backbone to the visual RL framework. Intuitively, MoE architectures can alleviate gradient conflicts due to their ability to dynamically allocate gradients to specialized experts for

each input through the sparse routing mechanism (Akbari et al., 2023; Yang et al., 2024).

In terms of optimization, visual RL agents often struggle with local minima due to the unstructured nature of robotic tasks. Recent works have shown that periodically perturbing the agent’s weights with random noise can help escape local minima (Nikishin et al., 2022; Sokar et al., 2023; Xu et al., 2023; Ji et al., 2024). However, the choice of perturbation candidates (i.e., the network weights used to perturb the current agent’s weights) has not been thoroughly explored. Building on this idea, we propose a task-oriented perturbation mechanism. Instead of sampling from a fixed distribution, we maintain a heuristically shifted distribution based on the top-performing agents from the RL history. The intuition is that the distribution gradually formed by the weights of previous top-performing agents may accumulate task-relevant information, leading to more promising optimization directions than purely random noise.

Empirically, we find MENTOR outperforms current state-of-the-art methods (Xu et al., 2023; Yarats et al., 2021; Cetin et al., 2022; Zheng et al., 2023) across all tested scenarios in *DeepMind Control Suite* (Tassa et al., 2018), *Meta-World* (Yu et al., 2020b), and *Adroit* (Rajeswaran et al., 2017). Furthermore, we present three challenging real-world robotic manipulation tasks, shown in Figure 1: *Peg Insertion* – inserting three kinds of pegs into the corresponding sockets; *Cable Routing* – maneuvering one end of a rope to make it fit into two non-parallel slots; and *Tabletop Golf* – striking a golf ball into the target hole while avoiding getting stuck into the trap. In these experiments, MENTOR demonstrates significantly higher learning efficiency, achieving an average success rate of 83%, compared to 32% for the state-of-the-art counterpart (Xu et al., 2023) within the same training time. This confirms the effectiveness of our approach and underscores the importance of improving sample efficiency for making RL algorithms more practical in robotics applications.

Our key contributions are threefold. First, we introduce the MoE architecture to replace the MLP as the agent backbone in model-free visual RL, improving the agent’s learning ability to handle complex robotic environments and reducing gradient conflicts. Second, we propose a task-oriented perturbation mechanism which samples candidates from a heuristically updated distribution, making network perturbation a more efficient and targeted optimization process compared to the random parameter exploration used in previous RL perturbation methods. Third, we achieve state-of-the-art performance in both simulated environments and three challenging real-world tasks, highlighting the sample efficiency and practical value of MENTOR.

2. Preliminary

Mixture-of-Experts (MoE). Mixture-of-experts (MoE), introduced by Jacobs et al. (1991) and Jordan & Jacobs (1994), is a framework where specialized model components, called experts, handle different tasks or aspects of a task. A sparse MoE layer comprises multiple experts and a router. The router predicts a probability distribution over the experts, activating only the top- k for each input (Shazeer et al., 2017). With N experts, each being a feed-forward network (FFN), the MoE output is:

$$w(i; \mathbf{x}) = \text{softmax}(\text{topk}(h(\mathbf{x}))) [i], \quad (1)$$

$$F^{\text{MoE}}(\mathbf{x}) = \sum_{i=1}^N w(i; \mathbf{x}) \text{FFN}_i(\mathbf{x}), \quad (2)$$

where $w(i; \mathbf{x})$ is the gating function determining the weight of the i -th expert for input $\mathbf{x}$, $h(\mathbf{x})$ provides logits for expert selection, and $\text{topk}(h(\mathbf{x}))$ selects the top k .

Visual Reinforcement Learning. We employ visual reinforcement learning (RL) to train policies for robotic systems, modeled as a Partially Observable Markov Decision Process (POMDP) defined by the tuple (S, O, A, P, r, γ) . Here, S is the true state space, O represents visual observations, A is the robot’s action space, $P : S \times A \rightarrow S$ defines the transition dynamics, $r(s, a) : S \times A \rightarrow \mathbb{R}$ specifies the reward, and $\gamma \in (0, 1]$ is the discount factor. The goal is to learn an optimal policy $\pi_\theta(a_t | o_t)$ that maximizes the expected cumulative reward $E_\pi [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$.

Dormant-Ratio-based Perturbation in RL. The concept of dormant neurons, introduced by Sokar et al. (2023), refers to neurons that have become nearly inactive. It is formally defined as follows:

Definition 2.1. Consider a fully connected layer l with N^l neurons. Let $\text{linear}_i^l(\mathbf{x})$ denote the output of neuron i in layer l for an input distribution $\mathbf{x} \in \mathcal{I}$. The **score** of neuron i is given by:

$$s_i^l = \frac{\mathbb{E}_{\mathbf{x} \in \mathcal{I}} |\text{linear}_i^l(\mathbf{x})|}{\frac{1}{N^l} \sum_{k \in l} \mathbb{E}_{\mathbf{x} \in \mathcal{I}} |\text{linear}_k^l(\mathbf{x})|}. \quad (3)$$

This neuron is considered τ -dormant if its score $s_i^l \leq \tau$.

Definition 2.2. In layer l , the total number of τ -dormant neurons is denoted by D_τ^l . The **τ -dormant ratio** of a neural network θ is defined as:

$$\beta_\tau = \frac{\sum_{l \in \theta} D_\tau^l}{\sum_{l \in \theta} N^l}. \quad (4)$$

As shown by Xu et al. (2023); Ji et al. (2024), the dormant ratio is a key metric in neural network behavior and enhances

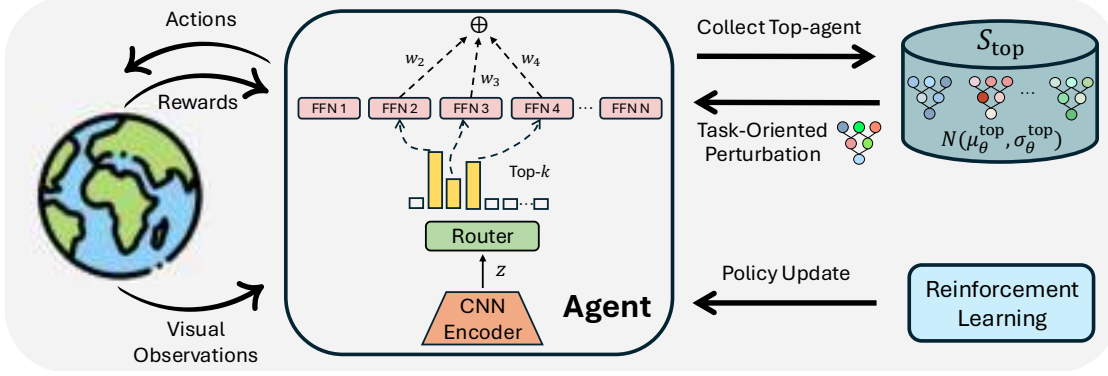


Figure 2. **Overview.** MENTOR uses an MoE backbone with a CNN encoder to process visual inputs. A router selects and weights the relevant experts based on the inputs to generate the final actions. In addition to regular reinforcement learning updates, periodic task-oriented perturbations are applied during training by sampling from top-performing agents to adjust the current agent’s weights.

RL efficiency via parameter perturbation. This periodically resets network weights by interpolating between current parameters and random initialization (Ash & Adams, 2020; D’Oro et al., 2022):

$$\theta_k = \alpha\theta_{k-1} + (1 - \alpha)\phi, \quad \phi \sim \text{initializer}. \quad (5)$$

Here, α is the perturbation factor, θ_{k-1} and θ_k are the weights pre- and post-reset, and ϕ denotes randomly initialized weights (e.g., Gaussian noise). The value of α dynamically adjusts as $\alpha = \text{clip}(1 - \mu\beta, \alpha_{\min}, \alpha_{\max})$, where μ is the perturbation rate and β the dormant ratio.

3. Method

In this section, we introduce MENTOR, which includes two key enhancements to the *architecture* and *optimization* of agents, aimed at improving sample efficiency and overall performance in visual RL tasks. The first enhancement addresses the issue of low sample efficiency caused by gradient conflicts in challenging scenarios, achieved by adopting an MoE structure in place of the traditional MLP as the agent backbone, as detailed in Section 3.1. The second enhancement introduces a task-oriented perturbation mechanism that optimizes the agent’s training through targeted perturbations, effectively balancing exploration and exploitation, as outlined in Section 3.2. The framework of our method is illustrated in Figure 2.

3.1. Mixture-of-Experts as the Policy Backbone

In challenging robotic tasks, RL agents often assigned $K \geq 2$ different tasks or subgoals, each with a loss $L_i(\theta)$. The goal is to optimize shared weights $\theta \in \mathbb{R}^m$ by minimizing the average loss:

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^m} \left\{ L_0(\theta) \triangleq \frac{1}{K} \sum_{i=1}^K L_i(\theta) \right\}.$$

When using shared parameters θ (e.g., MLP), meaning all parameters must be simultaneously active to function, the optimization process using gradient descent may compromise individual loss optimization. This issue, known as conflicting gradients (Yu et al., 2020a; Liu et al., 2021), hinders the agent’s ability to optimize its behavior when facing complex scenarios effectively.

We propose replacing the MLP with an MoE backbone. The MoE, composed of modular experts $\theta_{\text{MoE}} = \{\theta_1, \theta_2, \dots, \theta_N\}$, which allows the agent to activate different experts via a dynamic routing mechanism flexibly. This enables gradients dynamically route from different tasks to specific experts, reducing gradient conflicts. Each expert is updated using gradients from related tasks, addressing conflicts effectively. As shown in Figure 2, the MoE agent uses a CNN encoder to map visual inputs to latent space Z . The router h computes a probability $h(i | z)$ over experts for latent $z \in Z$. The top- k experts are selected, and their outputs a_i are combined using softmax weights w_i (Equations 1 and 2). This structure routes inputs to specialized experts, improving multi-task performance.

To illustrate the important role of dynamic modular expert learning for RL agents, we conduct a multi-task experiment (MT5) in Meta-World, training an agent (#Experts = 16, $k = 4$) for five opposing tasks: Open (Door-Open, Drawer-Open, Window-Open) and Close (Drawer-Close, Window-Close). Figure 3a shows Open and Close tasks share some experts but also utilize dedicated ones. To quantitatively demonstrate how much the MoE alleviates the gradient conflict issue, we evaluate the cosine similarities (Yu et al., 2020a) for both MLP and MoE agents in Figure 3b. The MLP’s gradients show significant conflicts between opposing tasks, while the MoE model demonstrates higher gradient compatibility. As a result, there is a performance gap, with the MLP achieving 100% success in Close tasks

but only 82% in Open tasks, whereas the MoE achieves 100% success in **both** task types.

This structural advantage can also be propagated to challenging single tasks, as the dynamic routing mechanism automatically activates different experts to adjust the agent’s behavior throughout the task, alleviating the burden on shared parameters. We illustrate this through training a same-structure MoE agent on a single, highly challenging Assembly task from Meta-World (MW). Figure 4 shows the engagement of the $k = 4$ most active experts during task execution, with **Expert 15** serving as the shared module throughout the entire policy execution. The other experts vary and automatically divide the task into four distinct stages: **Expert 9** handles gripper control for grasping and releasing; **Expert 13** manages arm movement while maneuvering the ring; and **Expert 14** oversees the assembly process as the ring approaches its fitting location. More detailed results about how MoE alleviates gradient conflicts in the single task are shown in Appendix F.

Algorithm 1 Task-Oriented Perturbation Mechanism

```

Initialize the set  $S_{\text{top}} = \emptyset$ , perturb interval  $T_p$ 
for each episode  $t = 1, 2, \dots$  do
    Execute policy  $\pi_{\theta_t}$  and obtain episode reward  $R_t$  and dormant
    ratio  $\beta$ 
    if  $|S_{\text{top}}| < N$  then
        Add  $(\theta_t, R_t)$  to  $S_{\text{top}}$ 
    else
        if  $R_t > \min\{R_i \mid (\theta_i, R_i) \in S_{\text{top}}\}$  then
            Replace  $(\theta_j, R_j)$  with  $(\theta_t, R_t)$ , where  $j = \text{argmin} R_j$ 
        end if
    end if
    if (Number of steps since last perturb)  $\geq T_p$  then
        Compute mean  $\mu_{\theta}^{\text{top}}$  and standard deviation  $\sigma_{\theta}^{\text{top}}$  from  $S_{\text{top}}$ 
        Sample perturbation weight  $\phi \sim \Phi_{\text{oriented}} = \mathcal{N}(\mu_{\theta}^{\text{top}}, \sigma_{\theta}^{\text{top}})$ 
        Calculate perturb factor as in Sokar et al. (2023):  $\alpha = \text{clip}(1 - \mu\beta, \alpha_{\min}, \alpha_{\max})$ 
        Update agent weights:  $\theta_t = \alpha\theta_t + (1 - \alpha)\phi$ 
    end if
end for
    
```

3.2. Task-oriented Perturbation Mechanism

Neural network perturbation is employed to enhance the exploration capabilities in RL. Two key factors influence the effectiveness of this process $\theta_k = \alpha\theta_{k-1} + (1 - \alpha)\phi$, $\phi \sim \Phi$. α is the perturbation factor controlling the mix between current agent and perturbation candidate weights. ϕ represents the perturbation candidate sampled from a distribution Φ , which typically is a fixed Gaussian noise $\mathcal{N}(\mu, \sigma)$. Previous works (Xu et al., 2023; Ji et al., 2024) have investigated the use of the dormant ratio to determine α , resulting in improved exploration efficiency (see Section 2). However, the selection of perturbation candidates has not been thoroughly examined. In this work, we propose sampling ϕ from a heuristically updated distribution Φ_{oriented} , generated from

past high-performing agents, to provide more task-oriented candidates that better facilitate optimization.

We define Φ_{oriented} as a distribution from which high-performing agent weights can be sampled. This distribution is obtained by maintaining a fixed-size set $S_{\text{top}} = \{(\theta, R)\}$, where (θ, R) represents an agent with weights θ and episode reward R . The distribution is approximated as $\Phi_{\text{oriented}} = \mathcal{N}(\mu_{\theta}^{\text{top}}, \sigma_{\theta}^{\text{top}})$, where $\mu_{\theta}^{\text{top}}$ and $\sigma_{\theta}^{\text{top}}$ are the mean and standard deviation of weights in S_{top} . As shown in Figure 2, S_{top} is updated during training: at episode t , if an agent with weights θ_t achieves reward R_t exceeding the lowest in S_{top} , (θ_t, R_t) replaces the lowest-reward tuple. This ensures Φ_{oriented} reflects current high-performing agents, improving perturbation candidates ϕ for subsequent iterations. The pseudocode is in Algorithm 1.

For illustration, we conduct experiments on the Hopper Hop task from the DeepMind Control Suite (DMC), comparing task-oriented perturbation approach to leading model-free visual RL baselines (DrM (Xu et al., 2023) and DrQ-v2 (Yarats et al., 2021)). Our approach solely replaces DrM’s perturbation mechanism with task-oriented perturbations. Both our method and DrM outperform DrQ-v2 due to dormant-ratio-based perturbation, but our method achieves faster skill acquisition and maintains a lower, smoother dormant ratio throughout training (Figure 5a and 5b). By directly testing perturbation candidates as agents in the task (Figure 5c), we observe that candidates sampled from Φ_{oriented} steadily improve throughout training, sometimes even surpassing the performance of the agent they perturb. This demonstrates that Φ_{oriented} progressively captures the optimal weight distribution, rather than simply interpolating from past agents, leading to more targeted optimization. In contrast, perturbation candidates from DrM (initialized with Gaussian noise) consistently yield zero reward, indicating the lack of task-relevant information.

4. Experiments

In this section, we present a comprehensive empirical evaluation of MENTOR. Section 4.1 showcases its effectiveness on three simulation benchmarks: DeepMind Control Suite (DMC) (Tassa et al., 2018), Meta-World (MW) (Yu et al., 2020b), and Adroit (Rajeswaran et al., 2017), which feature rich visuals and complex dynamics. MENTOR consistently outperforms leading visual RL algorithms. However, a critical limitation in visual RL research is the over-reliance on simulated environments, raising concerns about practical applicability. To bridge this gap, Section 4.2 validates MENTOR on three challenging real-world robotic tasks, highlighting the importance of real-world testing.

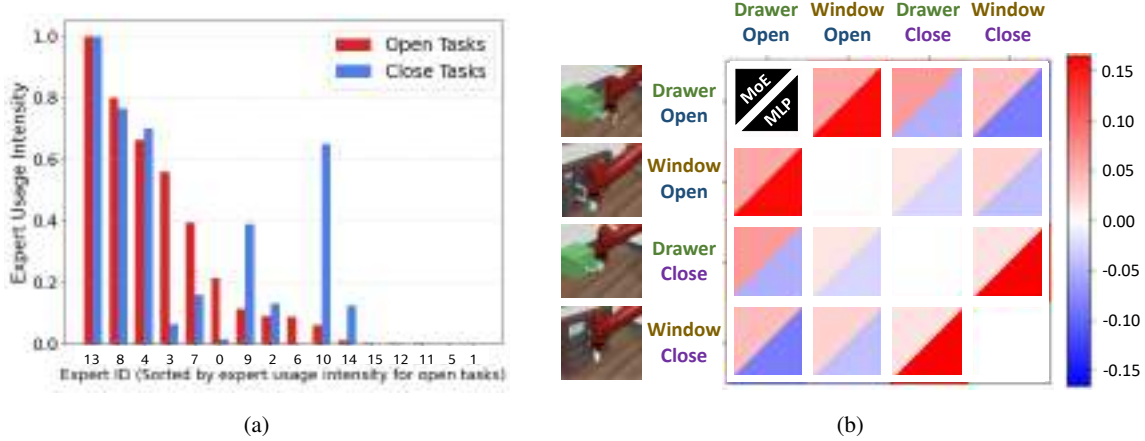


Figure 3. **MoE in multi-task scenarios.** Left: Expert usage intensity distribution of the MoE agent in opposing tasks. Right: Gradient conflict among opposing tasks for both MLP and MoE agents. The MLP agent frequently encounters gradient conflicts (indicated by negative cosine similarity) when learning multiple skills, while the MoE agent avoids these conflicts (indicated by positive values). We also provide a comparison of gradient conflicts for MLP and MoE agents in single-task settings, as detailed in Appendix F.

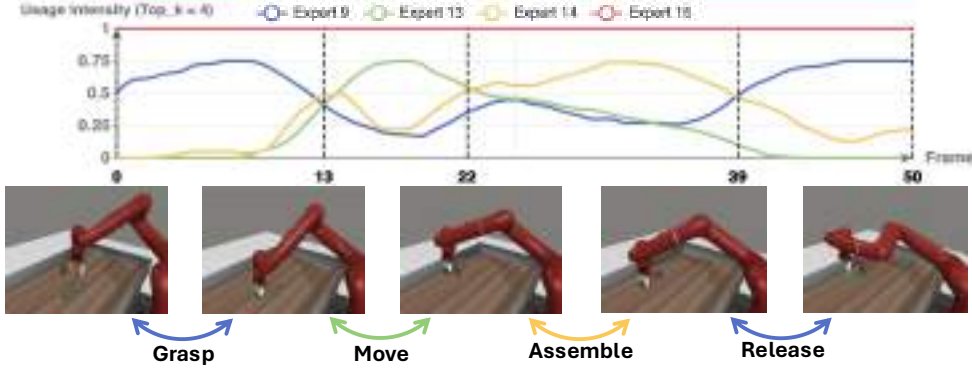


Figure 4. **MoE in multi-stage scenarios.** We present the expert usage intensity during the Assembly task in Meta-World. While Expert 15 remains highly active throughout the entire process, other experts are activated with varying intensity over time, automatically dividing the task into four distinct stages.

4.1. Simulation Experiments

Baselines: We compare MENTOR against four leading model-free visual RL methods: DrM (Xu et al., 2023), ALIX (Cetin et al., 2022), TACO (Zheng et al., 2023), and DrQ-v2 (Yarats et al., 2021). DrM, ALIX, and TACO all use DrQ-v2 as their backbone. DrM periodically perturbs the agent’s weights with random noise based on the proportion of dormant neurons in the neural network; ALIX adds regularization to the encoder gradients to mitigate overfitting; and TACO employs contrastive learning to improve latent state and action representations.

Experimental Settings: We evaluate MENTOR on a diverse set of tasks across three simulation environments with complex dynamics and even sparse reward. The DMC includes challenging tasks like Dog Stand, Dog Walk, Manipulator Bring Ball, and Acrobot Swingup (Sparse), focusing

on long-horizon continuous locomotion and manipulation challenges. The MW environment provides a suite of robotic tasks including Assembly, Disassembly, Pick Place, Coffee Push (Sparse), Soccer (Sparse), and Hammer (Sparse), which test the agent’s manipulation abilities and require sequential reasoning. The Adroit environment includes complex robotic manipulation tasks such as Door and Hammer, which involve controlling dexterous hands to interact with articulated objects. Notably, DMC tasks are evaluated using episode reward, while tasks in MW and Adroit are assessed based on success rate. We conducted experiments with four random seeds on each task, with detailed hyperparameters and training settings provided in Appendix B.

Results: Figure 6 presents performance comparisons between MENTOR and the baselines. In the DMC tasks, Dog Stand and Dog Walk feature high action dimensionality with a 38-dimensional action space representing joint controls for

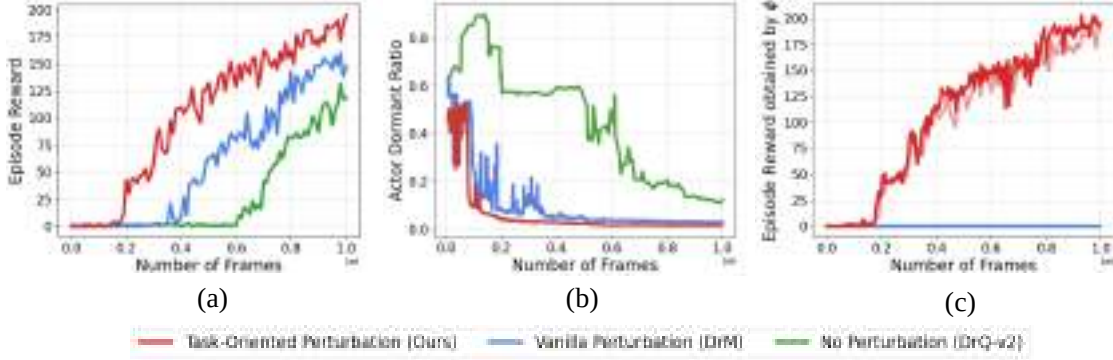


Figure 5. Validation of task-oriented perturbation on Hopper Hop (a MENTOR, DrM, and DrQ-v2 agent trained on the Hopper Hop task during the first 1M frames). Our method consistently achieves higher episode rewards with a consistently lower dormant ratio. (c) shows the episode reward obtained by perturbation candidate ϕ sampled from Φ_{oriented} steadily increases and occasionally surpasses that of the corresponding RL agent (replotted as the light red line), whereas in DrM, the reward remains at zero due to the use of randomly generated perturbation parameters.

the dog model. These tasks also have complex kinematics involving intricate joint coordination, muscle dynamics, and collision handling, making them challenging to optimize. Our method outperforms the top baseline, achieving approximately 17% and 10% higher episode rewards, respectively. In the MW tasks, the Hammer (Sparse) task stands out. It requires a robotic arm to hammer a nail into a wall, with highly sparse rewards: success yields significantly larger rewards than merely touching or missing the nail. In fact, the reward for failure is only one-thousandth of the success reward, making the task extremely sparse. However, our task-oriented perturbation effectively captures these sparse rewards, reducing the required training frames by 70% compared to the best baseline. In the Adroit tasks, our method achieves nearly 100% success with significantly less training time, while the most competitive counterpart (DrM) requires more frames, and other baselines fail to match performance even after 6 million frames. A key highlight is the Door task, which involves multiple stages of dexterous hand manipulation—grasping, turning, and opening the door. Leveraging the MoE architecture, our method reduces training time to achieve over 80% success by approximately 23% compared to the best baseline. In summary, MENTOR demonstrates superior efficiency and performance compared to the strongest existing model-free visual RL baselines across all 12 tasks. For the robustness against disturbances of agent trained by MENTOR, please see Appendix I.

Ablation Study: We conduct a detailed ablation study to demonstrate the significance contribution of MoE and Task-oriented Perturbation separately in Appendix C.

4.2. Real-World Experiments

Our real-world RL experiments evaluate MENTOR on three key challenges: *multi-task learning*, *multi-stage deformable object manipulation*, and *dynamic skill acquisition* with

policies **trained from scratch in the real world**.

Experimental Settings: We use a Franka Panda arm for execution and RealSense D435 cameras for RGB observations, capturing both global and local views. Rewards are based on the absolute distance between current and desired states. To prevent trajectory overfitting, the end-effector’s initial position is randomly sampled within a predefined region at the start of each episode. Tasks are detailed below, with illustrations in Figure 7. Additional details are in Appendix D.

Peg Insertion: This task mimics assembly-line scenarios requiring the agent to insert pegs of three shapes (Star, Triangle, Arrow) into corresponding sockets. Simulating contact-rich interactions in these multi-task settings is highly challenging, making real-world evaluation essential.

Cable Routing: Manipulating deformable cables presents significant challenges due to the complexities of modeling and simulating their physical dynamics, making this task ideal for direct, model-free visual RL training in real-world environments. The robot must guide a deformable cable sequentially into two parallel slots. Since both slots cannot be filled simultaneously, the agent must perform the task sequentially, requiring long-horizon, multi-stage planning to successfully accomplish the task.

Tabletop Golf: In this task, the robot uses a golf club to strike a ball on a grass-like surface, aiming to land it in a target hole. An automated reset system retrieves the ball when it reaches the hole, enters a mock water hazard, or rolls out of bounds, and randomly repositions it. The agent must learn to approach the ball, control the club’s striking force and direction to guide the ball toward the hole while avoiding obstacles through real-world interaction.

Results: Our policies exhibit strong performance during

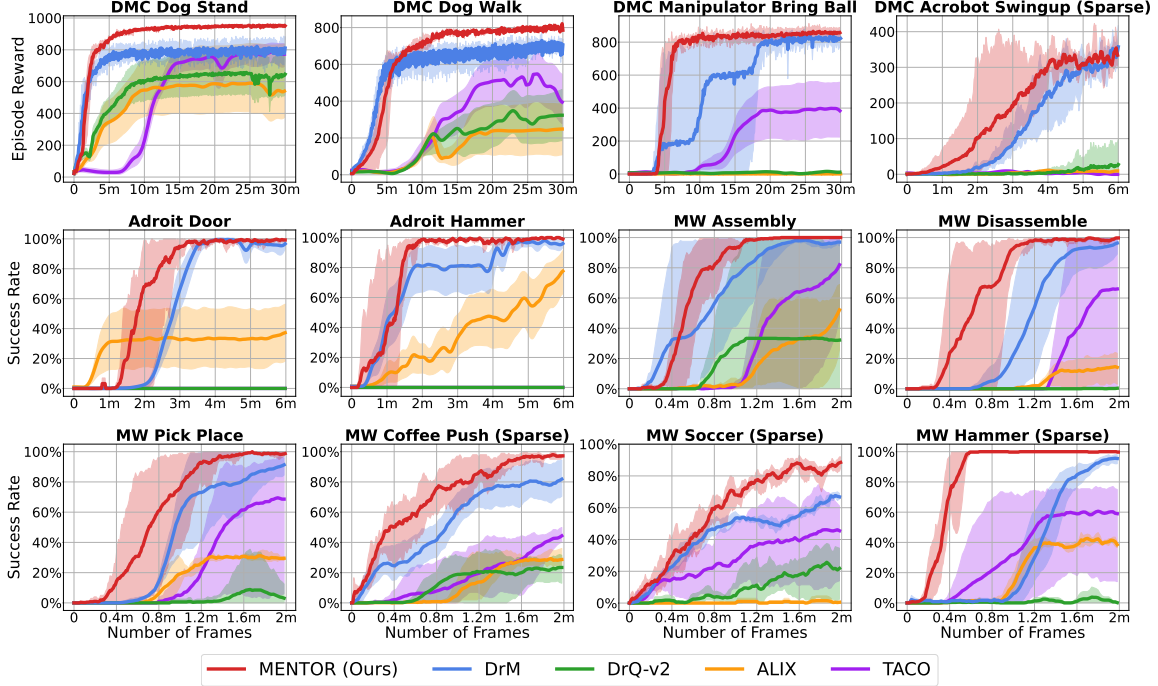


Figure 6. Performance comparisons in simulations. This figure compares the performance of our method to DrM, DrQ-v2, ALIX, and TACO across 12 tasks with four random seeds in three different benchmarks (DMC, MW, and Adroit). The shaded region indicates standard deviation in DMC and the range of success rates in MW and Adroit.

evaluation, as shown in Figure 7. In Peg Insertion, the agent randomly picks a peg and inserts it from varying initial positions, learning to align the peg with the hole and adjust the angle for accurate insertion. During one execution, as the peg nears the hole, we manually disturb by altering the robot arm’s pose significantly. Despite this interference, the agent successfully completes the task relying solely on visual observations. In Cable Routing, the agent learns to prioritize routing it into the farther slot first, then into the closer one. This second step requires careful handling to avoid dislodging the cable from the first slot. During execution, if the cable is randomly removed from the slot, the agent can visually detect this issue and re-route it back into position. In Tabletop Golf, the agent must master two key skills: striking the ball with the correct direction and force, and repositioning the club to follow the ball after the strike. Due to a “water hazard”, the ball cannot be struck directly toward the target hole from its starting position. The agent learns to angle its shots to bypass hazards and guide the ball into the hole, even as the ball’s rolling on the grass-like surface naturally introduces significant variability.

Ablation Study: We conduct a detailed ablation study to demonstrate the effectiveness of MENTOR in improving sample efficiency and performance, as shown in Table 1.

The first two rows reveal that utilizing the pretrained visual encoder (Lin et al., 2024) instead of a CNN trained

from scratch results in an average performance improvement of 9%. However, no significant performance gain is observed in simulation benchmarks with this substitution. This discrepancy may arise from the gap between simulation and real-world environments, where real scenes offer richer textures more aligned with the pretraining domain.

Furthermore, the results confirm the effectiveness of our technical contributions. When the MoE structure is removed from the agent (i.e., replaced with an MLP, as in MENTOR w/o MoE), overall performance drops by nearly 30%. Additionally, further switching the task-oriented perturbation mechanism to basic random perturbation (as in DrM) leads to an additional performance decline of approximately 30%.

5. Related work

Visual Reinforcement Learning. Visual reinforcement learning (RL), which operates on pixel observations rather than ground-truth state vectors, faces significant challenges in decision-making due to the high-dimensional nature of visual inputs and the difficulty in extracting meaningful features for policy optimization (Ma et al., 2022; Choi et al., 2023; Ma et al., 2022). Despite these challenges, there has been considerable progress in this area. Methods such as Hafner et al. (2019; 2020; 2023); Hansen et al. (2022) improve visual RL by building world models. Other approaches (Yarats et al., 2021; Kostrikov et al., 2020; Laskin

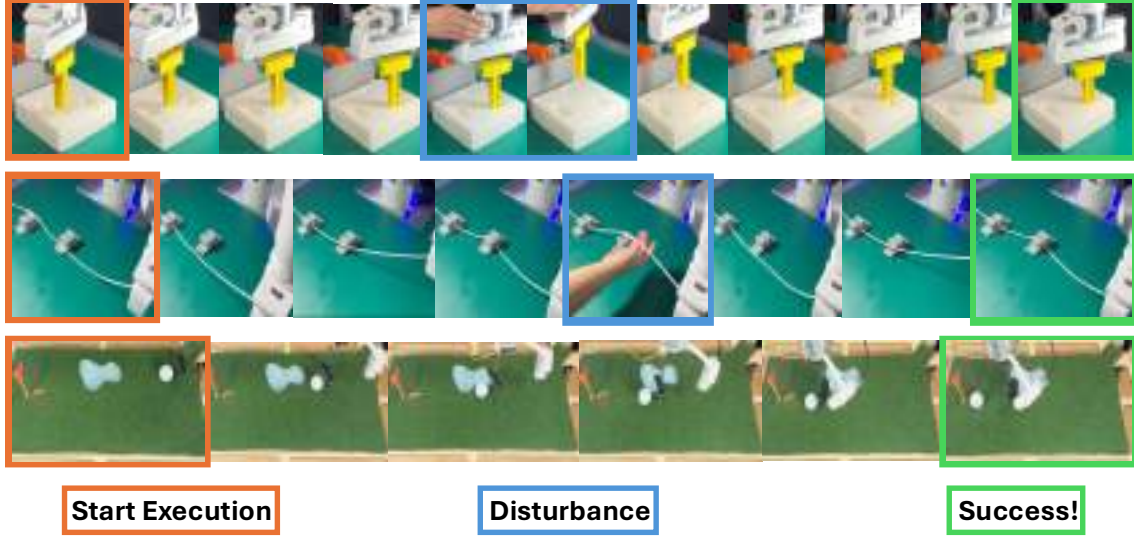


Figure 7. Real-world experiments (up to down rows: Peg Insertion, Cable Routing, and Tabletop Golf). This set of images illustrates the execution of the learned visual policy trained using MENTOR. The agent consistently and accurately accomplishes tasks even in the presence of human disturbances.

Table 1. Comparison of success ratios between MENTOR and ablations with equal training times. Peg Insertion and Cable Routing are trained for 3 hours, and Tabletop Golf for 2 hours. During evaluation, each subtask in Peg Insertion is rolled out 10 times, while Cable Routing and Tabletop Golf are rolled out 20 times.

Method	Peg Insertion (Subtasks)			Cable Routing	Tabletop Golf
	Star	Triangle	Arrow		
MENTOR w/ pretrained encoder	1.0	1.0	1.0	0.9	0.8
MENTOR	1.0	1.0	1.0	0.8	0.7
MENTOR w/o MoE	1.0	0.7	0.6	0.45	0.55
DrM	0.5	0.2	0.1	0.2	0.5

et al., 2020a), use data augmentation to enhance learning robustness from pixel inputs. Contrastive learning, as in Laskin et al. (2020b); Zheng et al. (2023), aids in learning more informative state and action representations. Additionally, Cetin et al. (2022) applies regularization to prevent catastrophic self-overfitting, while DrM (Xu et al., 2023) enhances exploration by periodically perturbing the agent’s parameters. Despite recent progress, these methods still suffer from low sample efficiency in complex robotic tasks. In this paper, we propose enhancing the agent’s learning capability by replacing the standard MLP backbone with an MoE architecture. This dynamic expert learning mechanism helps mitigate gradient conflicts in complex scenarios.

Neural Network Perturbation in RL. Perturbation theory has been explored in machine learning to escape local minima during gradient descent (Jin et al., 2017; Neelakantan et al., 2015). In deep RL, agents often overfit and lose expressiveness during training (Song et al., 2019; Zhang et al., 2018; Schilling, 2021). To address this issue, Sokar

et al. (2023) identified a correlation where improved learning capability is often accompanied by a decline in the dormant neural ratio in agent networks. Building on this insight, Xu et al. (2023); Ji et al. (2024) introduced parameter perturbation mechanisms that softly blend randomly initialized perturbation candidates with the current ones, aiming to reduce the agent’s dormant ratio and encourage exploration. However, previous works have not fully explored the choice of perturbation candidates. In this work, we uncover the potential of targeted perturbation for more efficient policy optimization by introducing a simple yet effective task-oriented perturbation mechanism. This mechanism samples perturbation candidates from a time-variant distribution formed by the top-performing agents collected throughout RL history.

6. Conclusion

In this paper, we present MENTOR, a state-of-the-art model-free visual RL framework that achieves superior performance in challenging robotic control tasks. MENTOR en-

hances learning efficiency through two key improvements in both agent network *architecture* and *optimization*. MENTOR consistently outperforms the strongest baselines across 12 tasks in three simulation benchmark environments. Furthermore, we extend our evaluation beyond simulations, demonstrating the effectiveness of MENTOR in *real-world* settings on three challenging robotic manipulation tasks. We believe MENTOR is a capable visual RL algorithm with the potential to push the boundaries of RL application in real-world robotic tasks. While MENTOR has proven effective, it has been evaluated on single tasks with a single robot embodiment. Future work could scale our method to handle hundreds of tasks or diverse robot embodiments, enabling broader real-world applications.

Impact Statement

This paper presents work whose goal is to advance the field of Model-free Visual Reinforcement Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Akbari, H., Kondratyuk, D., Cui, Y., Hornung, R., Wang, H., and Adam, H. Alternating gradient descent and mixture-of-experts for integrated multimodal perception. *Advances in Neural Information Processing Systems*, 36: 79142–79154, 2023.
- Ash, J. and Adams, R. P. On warm-starting neural network training. *Advances in neural information processing systems*, 33:3884–3894, 2020.
- Cetin, E., Ball, P. J., Roberts, S., and Celiktutan, O. Stabilizing off-policy deep reinforcement learning from pixels. *arXiv preprint arXiv:2207.00986*, 2022.
- Chen, Z., Shen, Y., Ding, M., Chen, Z., Zhao, H., Learned-Miller, E. G., and Gan, C. Mod-squad: Designing mixtures of experts as modular multi-task learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11828–11837, 2023.
- Choi, H., Lee, H., Jeong, S., and Min, D. Environment agnostic representation for visual reinforcement learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 263–273, 2023.
- D’Oro, P., Schwarzer, M., Nikishin, E., Bacon, P.-L., Bellemare, M. G., and Courville, A. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *Deep Reinforcement Learning Workshop NeurIPS 2022*, 2022.
- Dulac-Arnold, G., Mankowitz, D., and Hester, T. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*, 2019.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- Hafner, D., Lillicrap, T., Norouzi, M., and Ba, J. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.
- Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- Ji, T., Liang, Y., Zeng, Y., Luo, Y., Xu, G., Guo, J., Zheng, R., Huang, F., Sun, F., and Xu, H. Ace: Off-policy actor-critic with causality-aware entropy regularization. *arXiv preprint arXiv:2402.14528*, 2024.
- Jin, C., Ge, R., Netrapalli, P., Kakade, S. M., and Jordan, M. I. How to escape saddle points efficiently. In *International conference on machine learning*, pp. 1724–1732. PMLR, 2017.
- Jordan, M. I. and Jacobs, R. A. Hierarchical mixtures of experts and the em algorithm. *Neural computation*, 6(2): 181–214, 1994.
- Kostrikov, I., Yarats, D., and Fergus, R. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. *arXiv preprint arXiv:2004.13649*, 2020.
- Laskin, M., Lee, K., Stooke, A., Pinto, L., Abbeel, P., and Srinivas, A. Reinforcement learning with augmented data. *Advances in neural information processing systems*, 33: 19884–19895, 2020a.
- Laskin, M., Srinivas, A., and Abbeel, P. Curl: Contrastive unsupervised representations for reinforcement learning. In *International Conference on Machine Learning*, pp. 5639–5650. PMLR, 2020b.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

Appendix

A. Algorithm Details

We illustrate the overview framework of MENTOR in Section 3, where we employ two enhancements in terms of agent *structure* and *optimization*: substituting the MLP backbone with MoE to alleviate gradient conflicts when learning complex tasks, and implementing a task-oriented perturbation mechanism to update the agent’s weights in a more targeted direction by sampling from a distribution formed by the top-performing agents in training history. The detailed implementation of task-oriented perturbation is shown in Algorithm 1, and the implementation of using MoE as the policy backbone is described as follows:

Algorithm 2 illustrates how MENTOR employs the MoE architecture as the backbone of its policy network. In addition to the regular training process, using MoE as the policy agent requires adding an additional loss to prevent MoE degradation during training—where a fixed subset of experts is consistently activated. The MoE layer computes the output action while simultaneously calculating an auxiliary loss for load balancing (Lepikhin et al., 2020; Fedus et al., 2022). Specifically, we extract the distribution over experts produced by the router for each input. By averaging these distributions over a large batch, we obtain an overall expert distribution, which we aim to keep uniform across all experts. To achieve this, we introduce an auxiliary loss term—the negative entropy of the overall expert distribution (Chen et al., 2023; Shen et al., 2023). This loss reaches its minimum value of $-\log(N_e)$, where N_e is the number of experts in the MoE, when all experts are equally utilized, thus preventing degradation. This auxiliary loss is added to the actor loss and used to update the actor during the RL process.

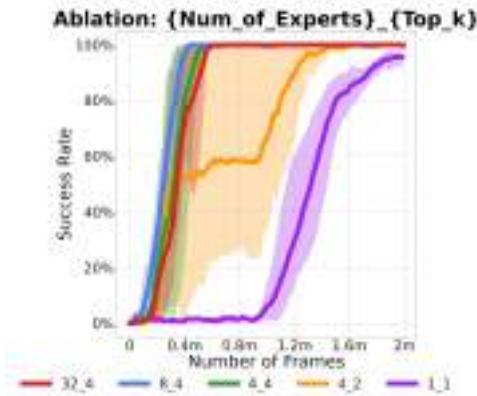


Figure 8. Ablation study on the hyperparameter of MoE. We evaluated MENTOR on MW Hammer (Sparse) with different MoE settings.

Algorithm 2 Mixture-of-Experts as the Policy Backbone

Require: Batch of visual inputs $\{\mathbf{x}_b\}_{b=1}^B$
Ensure: Final actions $\{a_b\}_{b=1}^B$, Load balancing loss $\mathcal{L}_{LB}$
 Initialize experts $\{\text{FFN}_1, \text{FFN}_2, \dots, \text{FFN}_N\}$
 $\{\mathbf{z}_b\}_{b=1}^B \leftarrow \text{Encoder}(\{\mathbf{x}_b\}_{b=1}^B)$
 $\{\mathbf{h}_b\}_{b=1}^B \leftarrow h(\{\mathbf{z}_b\}_{b=1}^B)$
for $b = 1$ to B **do**
 $\mathcal{E}_b \leftarrow \text{topk}(\mathbf{h}_b, k)$
 $w_b(i) \leftarrow \text{softmax}(\mathbf{h}_{b, \mathcal{E}_b})[i], \forall i \in \mathcal{E}_b$
 for each $i \in \mathcal{E}_b$ **do**
 $\mathbf{f}_{b,i} \leftarrow \text{FFN}_i(\mathbf{z}_b)$
 end for
 $a_b \leftarrow \text{ActionProjector}(\sum_{i \in \mathcal{E}_b} w_b(i) \mathbf{f}_{b,i})$
end for
 // Compute load balancing loss
 $\{\mathbf{pr}_b\}_{b=1}^B \leftarrow \text{softmax}(\{\mathbf{h}_b\}_{b=1}^B)$
 $p(i) \leftarrow \frac{1}{B} \sum_{b=1}^B pr_b(i), \forall i$
 $\mathcal{L}_{LB} \leftarrow -H(p) = -\sum_{i=1}^N p(i) \log(p(i))$

B. Simulation Experimental Settings

The hyperparameters employed in our experiments are detailed in Table 3. In alignment with previous work, we predominantly followed the hyperparameters utilized in DrM (Xu et al., 2023).

For the hyperparameters used in MoE module, Figure 8 shows the ablation study on the number of experts and top_k chosen experts. The results indicate the optimal setting for the Hammer task is MoE has around 8 experts, performance remains consistent across 4, 8, and 32 experts as long as top_k = 4. That is, given a proper top_k, the performance is not sensitive to the number of experts.

C. Ablation Study on Key Contributions

We conducted additional ablation studies on four diverse tasks: Hopper Hop, Disassemble, Coffee-Push (Sparse), and Hammer (Sparse). These studies aim to decouple the effects of the MoE architecture and the Task-oriented Perturbation (TP) mechanism proposed in our paper.

For the experiments, we evaluate four ablated versions of MENTOR using the same four random seeds as in the original experiments, as shown in Figure 9:

- **MENTOR:** Full model with both MoE and Task-oriented Perturbation.
- **MENTOR.w/o_TP:** Task-oriented Perturbation is replaced with random perturbation.
- **MENTOR.w/o_MoE:** The policy backbone uses an MLP architecture instead of MoE.

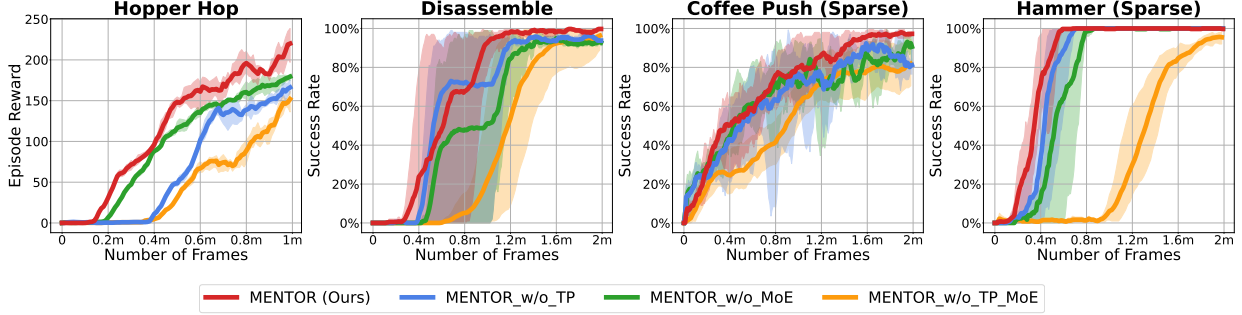


Figure 9. **Ablation study on key contributions.** This figure shows the experiment result of four ablated versions of MENTOR using the same four random seeds as in the original experiments, illustrating our Mixture-of-Experts (MoE) and Task-oriented Perturbation (TP) are both significant to improving performance.

Table 2. **Sample efficiency comparison across tasks.**

Sample Efficiency (Training Time)	Hopper Hop	Disassemble	Coffee Push	Hammer
MENTOR (Ours)	0.6167	0.7056	0.8066	0.7167
MENTOR_w/o_TP	1	0.8505	0.9481	0.875
MENTOR_w/o_MoE	0.85	1	1	1

- **MENTOR_w/o_TP_MoE:** Neither MoE nor Task-oriented Perturbation is used.

The results, summarized below, demonstrate the individual contributions of each component:

The overall sample efficiency and performance of **MENTOR_w/o_TP** and **MENTOR_w/o_MoE** remain lower than the full **MENTOR** model. This underscores the complementary nature of these two components in enhancing the overall learning efficiency and robustness of **MENTOR**.

Considering the performance, **MENTOR_w/o_MoE** and **MENTOR_w/o_TP** also consistently outperform **MENTOR_w/o_TP_MoE**, indicating that both the MoE architecture and Task-oriented Perturbation independently contribute to improved policy learning.

Considering efficiency, the following part shows a quantitative result based on standard training time.

Standard Training Time: Let T_{MENTOR} , $T_{\text{MENTOR}_w/o_MoE}$, and T_{MENTOR_w/o_TP} denote the time required for the three different methods to reach the same performance (the final performance of the worst method). The standard training time T_{standard} is defined as the training time for the worst method to achieve this performance:

$$T_{\text{standard}} = \max(T_{\text{MENTOR}}, T_{\text{MENTOR}_w/o_MoE}, T_{\text{MENTOR}_w/o_TP})$$

We define normalized sample efficiency as $\frac{T_{*}}{T_{\text{standard}}}$ (lower is

better).

Table 2 shows the normalized sample efficiency of each setting. MENTOR (Ours) achieves an average of **22.6%** and **26.1%** less training time over the 4 tasks compared with MENTOR_w/o_TP and MENTOR_w/o_MoE.

D. Real-World Experimental Settings

The training and testing videos are available at [mentor-vrl](https://mentor-vrl.github.io). The hyperparameters for the real-world experiments are the same as those used in the simulator, as shown in Table 3. We use 16 experts, with the top 4 experts activated.

D.1. Observation Space

The observation space for all real-world tasks is constructed from information **only** provided by several cameras. Each camera delivers three 84x84x3 images (3-channel RGB, with a resolution of 84x84), which capture frames from the beginning, midpoint, and end of the previous action.

For the Peg Insertion and Tabletop Golf tasks, the observation space is provided by two cameras: a wrist camera and a side camera. As shown in Figure 10, these two cameras in Tabletop Golf offer different perspectives. The wrist camera is attached to the robot arm’s wrist, capturing close-up images of the end-effector, while the side camera provides a more global view. As previously mentioned, each camera provides three images, resulting in a total of six 3-channel 84x84 images.

Table 3. Hyper-parameters used in our experiments.

	Parameter	Setting
Architecture	Features dimension	100 (Dog) 50 (Others)
	Hidden dimension	1024
	Number of MoE experts	4 or 16 or 32
	Activated MoE experts (top- k)	2 or 4
	MoE experts hidden dimension	256
Optimization	Optimizer	Adam
	Learning rate	8×10^{-5} (DMC) 10^{-4} (MW & Adroit)
	Learning rate of policy network	0.5 lr or lr
	Agent update frequency	2
	Soft update rate	0.01
	MoE load balancing loss weight	0.002
Perturb	Minimum perturb factor $\alpha_{\min}$	0.2
	Maximum perturb factor $\alpha_{\max}$	0.6 (Dog, Coffee Push & Soccer) 0.9 (Others)
	Perturb rate α_{rate}	2
	Perturb frames	200000
	Task-oriented perturb buffer size	10
Replay Buffer	Replay buffer capacity	10^6
	Action repeat	2
	Seed frames	4000
	n -step returns	3
	Mini-batch size	256
	Discount γ	0.99
Exploration	Exploration steps	2000
	Linear exploration stddev. clip	0.3
	Linear exploration stddev. schedule	linear(1.0, 0.1, 2000000) (DMC) linear(1.0, 0.1, 3000000) (MW & Adroit)
	Awaken exploration temperature T	0.1
	Target exploitation parameter $\hat{\lambda}$	0.6
	Exploitation temperature T'	0.02
	Exploitation expectile	0.9

In the Cable Routing task, the observation space is constructed using three cameras: a side camera for an overview, and two dedicated cameras for each slot to capture detailed views of the spatial relationship between the slots and the cable. This setup results in a total of nine 3-channel 84x84 images.

D.2. Action Space

The policy outputs an end-effector delta pose from the current pose tracked by the low-level controller equipped in robot arm. Typically, the end-effector of a robotic arm has six degrees of freedom (DOF); however, in our tasks, the action space is constrained to be fewer. The reason for this

restriction in DOF is specific to our setting: in our case, we train model-free visual reinforcement learning algorithms directly in the real-world environment from scratch, without any initial demonstrations and prior knowledge toward the tasks. As a result, the exploration process is highly random, and limiting the degrees of freedom is crucial for safeguarding both the robotic arm and the experimental equipment. For instance, in the Peg Insertion task, the use of rigid 3D-printed materials means allowing the end-effector to attempt insertion at arbitrary angles could easily cause damage. Similarly, in the Cable Routing task, an unrestricted end-effector might collide with the slot, posing a risk to the equipment.

Peg Insertion: The end-effector in this task has four degrees



Figure 10. **Agent’s visual observation example in tabletop golf.** MENTOR only uses visual data as policy input. At every step, we capture and stack the frames at the beginning, midpoint, and end of the actuation process. The images captured from cameras are resized to a resolution of 84x84 before being input to the agent.

of freedom: x , y , z , and r . Here, x and y represent the planar coordinates, z represents the height, and r denotes the rotation around the z -axis. The x , y , and z dimensions are normalized based on the environment’s size, ranging from -1 to 1, while r is normalized over a feasible rotation range of 0.6π .

The action space is a 4-dimensional continuous space $(\Delta x, \Delta y, \Delta z, \Delta r)$, where each action updates the end-effector’s state as:

$$(x, y, z, r) \rightarrow \left(x + \frac{\Delta x}{8}, y + \frac{\Delta y}{8}, z + \frac{\Delta z}{10}, r + \frac{\Delta r}{8} \right).$$

Cable Routing: In this task, the end-effector is constrained to two degrees of freedom: x and z . The x -axis controls movement almost perpendicular to the cable, while the z -axis controls the height. Both dimensions are normalized based on the environment’s size, with values ranging from -1 to 1. Although we restrict the action space to two dimensions, this task remains extremely challenging for the RL agent to master, as it requires inserting cable in both slots sequentially, making it the most time-consuming task among the three, as shown in Figure 14. The difficulty stems largely from the structure and parallel configuration of the two slots: the agent cannot route the cable into both slots simultaneously and must insert one first. However, as shown in Figure 11b, without a hook-like structure to secure the cable in the slot, the cable easily slips out when the agent attempts to route it into the second slot. This task therefore requires highly precise movements, forcing the agent to learn the complex dynamics of soft cables.

The action space is a 2-dimensional continuous space $(\Delta x, \Delta z)$, where each action updates the end-effector’s position as:

$$(x, z) \rightarrow \left(x + \frac{\Delta x}{5}, z + \frac{\Delta z}{5} \right).$$

Tabletop Golf: The end-effector in this task has three degrees of freedom: x , y , and r . Here, x and y represent the planar coordinates, and r denotes the angle around the normal vector to the xy plane. The x and y dimensions are normalized based on the environment’s size, ranging from -1 to 1, while r is normalized over a feasible rotation range of 0.5π .

The action space has four dimensions: three spatial dimensions $(\Delta x, \Delta y, \Delta r)$ and a strike dimension, where the values range from -1 to 1. The end-effector’s state is updated as:

$$(x, y, r) \rightarrow \left(x + \frac{\Delta x}{10}, y + \frac{\Delta y}{10}, r + \frac{\Delta r}{8} \right),$$

and if $\text{strike} > 0$, the end-effector performs a swing with strength proportional to the value of strike.

D.3. Reward Design

In this section, we describe the reward functions for the three real-world robotic tasks used in our work: Peg Insertion, Cable Routing, and Tabletop Golf. The basic principle behind these functions is to measure the distance between the current state and the target state. These reward functions are designed to provide continuous feedback—though they can be extremely sparse, as seen in Cable Routing—based on the task’s progress, enabling the agent to learn efficient strategies to achieve the goal. Notably, we trained two visual classifiers for the Cable Routing task to determine the relationship between the cables and the slots for reward calculation. Other positional information is obtained through feedback from the robot arm or image processing algorithms. The lower and upper bounds of each dimension in the pose are normalized to -1 and 1, respectively. The coefficients used in the reward functions are listed in Table 4.

Peg Insertion: The reward is computed as the negative

absolute difference between the current robot arm pose and the target insertion pose, which varies for each peg.

$$R_{\text{peg}} = \frac{1}{2} \left(\left(\sqrt{2} - \|\mathbf{x}_g - \mathbf{x}_c\| \right) \cdot C_1 + (2 - |\Delta z|) \cdot C_2 + \left(\frac{\pi}{2} - |\theta_c - \theta_g| \right) \cdot C_3 - C_4 \right)$$

Where:

- $\mathbf{x}_g$ and $\mathbf{x}_c$: Represent the goal position and the current position of the robot's end-effector in the x-y plane.
- $\|\mathbf{x}_g - \mathbf{x}_c\|$: Euclidean distance between the goal and current positions of the end-effector.
- Δz : The height difference between the current and target z positions.
- θ_c and θ_g : Current and goal angles of the end-effector, respectively.

Cable Routing: To provide continuous reward feedback, we trained a simple CNN classifier to detect whether the cable is correctly positioned in the slot, awarding full reward when the cable is in the slot and zero when it is far outside. The CNN classifier was trained by labeling images to classify the spatial relationship between the cable and the slot into several categories, with different rewards assigned based on the classification. However, when the cable remains in a particular category without progressing to different stages, the agent receives constant rewards, making it difficult for the agent to learn more refined cable manipulation skills.

$$R_{\text{cable}} = r_{\text{slot}_1} + \mathbb{I}(r_{\text{slot}_1} \geq 2) \cdot (r_{\text{slot}_2} + C_5)$$

Where:

- r_{slot_1} : Reward for the first slot, determined by the position of the cable relative to the slot. The possible rewards are:
 - Outside the slot: $r_{\text{slot}_1} = -3$
 - On the side of the slot: $r_{\text{slot}_1} = -1$
 - Above the slot: $r_{\text{slot}_1} = 1$
 - Inside the slot: $r_{\text{slot}_1} = 5$
- r_{slot_2} : Reward for the second slot, with more detailed classifications:
 - Outside the slot: $r_{\text{slot}_2} = -3$
 - On the side of the slot: $r_{\text{slot}_2} = -1$

- Partially above the slot: $r_{\text{slot}_2} = 1$
- Above the slot and at the edge: $r_{\text{slot}_2} = 3$
- Above the slot and close to the middle: $r_{\text{slot}_2} = 5$
- Partially inside the slot: $r_{\text{slot}_2} = 10$
- Fully inside the slot: $r_{\text{slot}_2} = 15$

- $\mathbb{I}(r_{\text{slot}_1} \geq 2)$: Indicator function that activates only if the cable is inserted correctly in the first slot, allowing the agent to receive rewards for the second slot.

Tabletop Golf: The reward consists of two components: the negative absolute distance between the robot arm and the ball, and the negative absolute distance between the ball and the target hole. This encourages the agent to learn how to move the robot arm toward the ball and control the striking force and direction to guide the ball toward the hole while avoiding obstacles. Additional rewards include: $R_{\text{golf}+} = C_6$ (if the ball reaches the hole) and $R_{\text{golf}-} = C_7$ (if the ball goes out of bounds). In this experiment, we deploy two cameras at the middle of two adjacent sides of the golf court. The pixel locations of the ball in both cameras are used to roughly estimate its location to calculate the reward function. Despite using an approximate estimation for the reward, MENTOR still quickly learns to follow the ball and strike it with the appropriate angle and force, demonstrating the effectiveness of our proposed method.

$$R_{\text{golf}} = (2 - \|\mathbf{p}_{\text{club}} - \mathbf{p}_{\text{ball}}\|) \cdot C_8 + (2 - \|\mathbf{p}_{\text{ball}} - \mathbf{p}_{\text{hole}}\|) \cdot C_9 - \mathbb{I}(\text{strike}) + (2 - |\theta_{\text{best}} - \theta_{\text{current}}|) \cdot C_{10} - \max(0, \mathbf{p}_{\text{ball}}[y] - \mathbf{p}_{\text{club}}[y] + 0.05) \cdot C_{11}$$

Table 4. Coefficients used in the reward functions over three real-world robotic tasks.

Symbol	Value
C_1	16
C_2	6
C_3	8
C_4	17
C_5	3
C_6	20
C_7	5
C_8	4
C_9	8
C_{10}	2
C_{11}	10

Where:

- $\mathbf{p}_{\text{club}}$ and $\mathbf{p}_{\text{ball}}$: Positions of the robot's golf club and the ball, respectively.

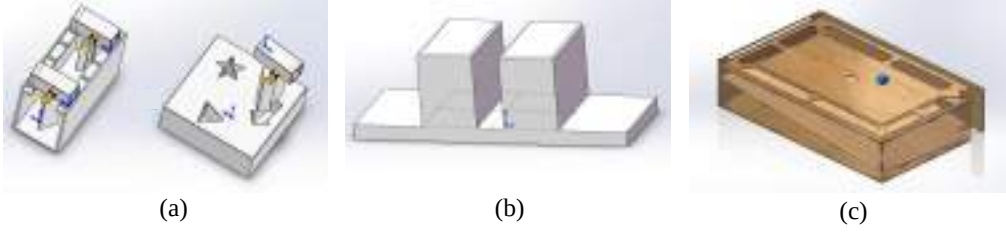


Figure 11. Blueprints of the self-designed mechanisms for the three real-world robotic manipulation tasks (from left to right: Peg Insertion, Cable Routing, and Tabletop Golf).

Table 5. Comparison of time efficiency in the simulation task (FPS).

Task Name	MENTOR	DrM	DrQ-v2	ALIX	TACO
Hopper	37	55	78	49	23

- $\mathbf{p}_{\text{hole}}$: Position of the target hole.
- $\|\mathbf{p}_{\text{club}} - \mathbf{p}_{\text{ball}}\|$: Distance between the club and the ball.
- $\|\mathbf{p}_{\text{ball}} - \mathbf{p}_{\text{hole}}\|$: Distance between the ball and the hole.
- θ_{best} and θ_{current} : Best calculated angle and current angle of the robot’s arm for optimal striking.
- $\mathbb{I}(\text{strike})$: Indicator function that penalizes unnecessary strikes.
- $\mathbf{p}_{\text{ball}}[y]$ and $\mathbf{p}_{\text{club}}[y]$: The y-axis is the long side of the golf course. The ball should be hit from the positive to the negative y-axis, so the club should always be on the positive y-side of the ball.

D.4. Auto-Reset Mechanisms

One major challenge in real-world RL is the burden of frequent manual resets during training. To address this, we designed auto-reset mechanisms to make the training process more feasible and efficient.

In the Peg Insertion task, the robot arm is set to frequently switch among different pegs to help the agent acquire multi-tasking skills. To facilitate this, we design a shelf to hold spare pegs while the robot arm is handling one. With the fixed position of the shelf, we pre-programmed a peg-switching routine, eliminating the need for manual peg replacement. After switching, the robot arm automatically moves the peg to the workspace and randomizes its initial position for training.

In the Cable Routing task, manual resets are unnecessary, as the robot arm can auto-reset the cable by simply moving back to its initial position with added randomness.

In the Tabletop Golf task, we design an auto-collection mechanism to reset the task. As shown in Figure 11c, the

tabletop golf device has two layers: the top golf court surface and a lower inclined floor. When the ball is hit into the hole or out of bounds, it rolls down to the corner of the lower layer, where a light sensor triggers a motor to return the ball to the court. The variability in the ball’s initial velocity during reset introduces randomness to its starting position.

E. Time Efficiency of MENTOR

We run all simulation and real-world experiments on an Nvidia RTX 3090 GPU and assess the speed of the algorithms compared to baselines. Frames per second (FPS) is used as the evaluation metric for time efficiency.

For simulation, we use the Hopper Hop task to compare time efficiency, as shown in Table 5. While MENTOR demonstrates significant sample efficiency, its time efficiency is relatively lower. This is primarily due to the implementation of a plain MoE version in this work, where input feature vectors are passed to all experts, and only the top- k outputs are weighted and combined to generate the final output. In most tasks, the active expert ratio (i.e., top- k /total number of experts) is equal to or below 25%. More efficient implementations of MoE could significantly improve time efficiency, which we leave for future exploration.

We also evaluate time efficiency on three real-world tasks, as shown in Table 6. In real-world applications, the primary bottlenecks in improving time efficiency are data collection efficiency and reset speed. Additionally, the sample efficiency of the RL algorithm plays a crucial role. If the algorithm has low sample efficiency, it may take many poor actions over a long training period, leading to frequent auto-resets and ultimately lowering the overall FPS.

As a result, MENTOR and DrM achieve similar levels of

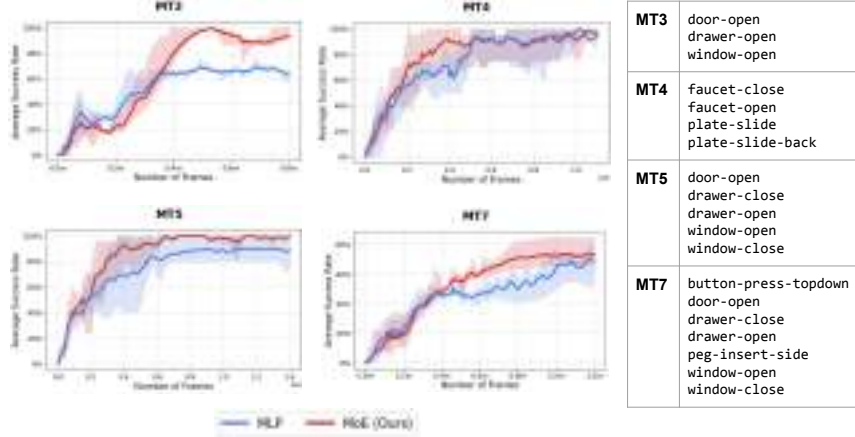


Figure 12. Multi-task performance of MoE on Meta-World Simulator. Evaluation accuracy during training for 3, 4, 5, and 7-task combinations, highlighting MoE’s advantage over MLP in multitask settings.

Table 6. Comparison of time efficiency in real-world tasks (FPS).

Task Name	MENTOR	DrM
Peg Insertion	0.46	0.40
Cable Routing	0.67	0.62
Tabletop Golf	0.52	0.47

efficiency. However, due to its superior learning capability, MENTOR quickly acquires skills and transitions out of the initial frequent-reset phase faster than DrM, leading to slightly better overall time efficiency during training.

We further extend the training process of the DrM baseline to reach the same performance level as MENTOR in three real-world experiments with the training time comparison shown in Figure 14, which demonstrates an average 37% improvement in time efficiency for our method. These findings underscore the importance of each component in achieving superior results.

F. Mixture-of-Experts Alleviation Gradient Conflicts in Single Task

In Meta-World, manipulation tasks are associated with compound reward functions that typically include components such as reaching, grasping, and placing. Conflicts between these objectives can arise, creating a burden for shared parameters.

To validate this, we analyze the gradient cosine similarities for the Assembly task. The Assembly task, as shown in Figure 4 can naturally be divided into four stages: Grasp, Move, Assemble, and Release.

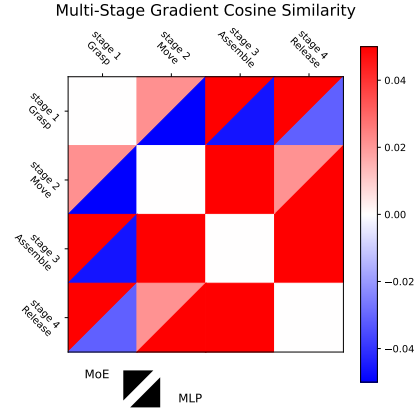


Figure 13. Cosine similarity of multistage in a single task. This figure shows the cosine similarities of gradients on the corresponding four stages (Grasp, Move, Assemble, and Release) for both MLP and MoE agents.

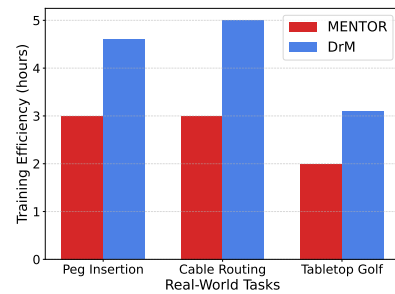


Figure 14. Time efficiency comparison. This figure compares the training time required for DrM to reach the performance level of MENTOR, as shown in Table 1.

To illustrate how Mixture-of-Experts alleviates gradient conflicts in a single task, we evaluate the cosine similarities of

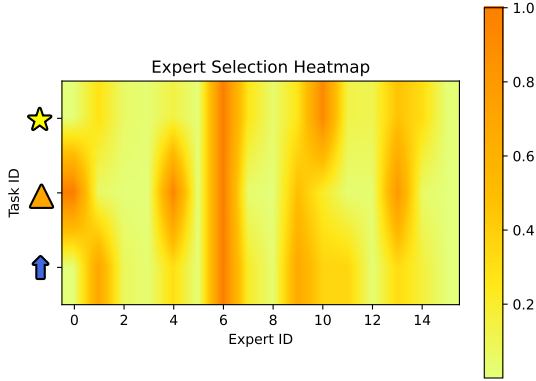


Figure 15. **Expert utilization on Peg Insertion task.** This figure shows the usage intensity of the 16 experts in MENTOR during the Peg Insertion task for three different plug shapes.

gradients on the corresponding four stages for both MLP and MoE agents, as shown in Figure 13. The result show that the MLP agent experiences gradient conflicts between grasping and the other stages. This can occur because the procedure of reaching to grasp objects could increase the distance between the robot and the target pillar, leading to competing optimization signals. In contrast, the MoE agent mitigates these conflicts, achieving consistently positive gradient cosine similarities across all stage pairs. This validates the ability of the MoE architecture to alleviate the burden of shared parameters and facilitate more efficient optimization, even in single-task scenarios.

G. MENTOR in Simulator Multi-Tasking Process

To further demonstrate the multi-task capabilities of MoE, we conducted additional multitask learning experiments on Meta-World. In these experiments, we used four task combinations, consisting of 3, 4, 5, and 7 tasks, respectively. Figure 12 shows the evaluation accuracy during training and the detailed composition of the multi-tasks. In this experiment, neither MENTOR nor DrM used perturbation, with the only difference being the use of MoE versus MLP, indicating the effectiveness of MoE in multi-tasking process.

H. MENTOR in Real-World Multi-Tasking Process

Figure 15 shows the utilization of experts in the Peg Insertion task for various plug shapes. Each shape is handled by some specialized experts, which aids in multi-task learning. This specialization helps mitigate gradient conflict by directing gradients from different tasks to specific experts, improving learning efficiency, as discussed in the main text.

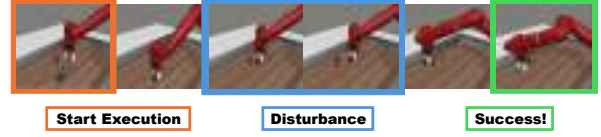


Figure 16. **Random disturbances in simulation.** This figure shows the execution of the learned agent using MENTOR. The agent consistently accomplishes Assembly task even with the disturbance.

I. Random Disturbances in Simulation

To demonstrate the generalization capabilities of the agents trained by MENTOR, we have introduced random disturbances in the real-world experiments presented in Section 4.2. Additionally, we make evaluation of Meta-World Assembly task with random disturbance. In detail, the training phase remains unchanged, but during evaluation, we introduce a random disturbance: after the robot grasps the ring and moves toward the fitting area, the fitting pillar randomly changes its location (Disturbance). This forces the robot agent to adjust its trajectory to the new target position. Figure 16 shows the agent consistently accomplishes Assembly task even with the disturbance, showing the policies learned by MENTOR exhibit strong robustness against disturbances.